

修改记录

修改摘要	修改记录戳	修改人
新增《线程的调度》	2019 年 7 月 7 日	Mark

1、线程基础、线程之间的共享和协作

基础概念

什么是进程和线程

进程是程序运行资源分配的最小单位

进程是操作系统进行资源分配的最小单位,其中资源包括:CPU、内存空间、磁盘 IO 等,同一进程中的多条线程共享该进程中的全部系统资源,而进程和进程之间是相互独立的。进程是具有一定独立功能的程序关于某个数据集合上的一次运行活动,进程是系统进行资源分配和调度的一个独立单位。

进程是程序在计算机上的一次执行活动。当你运行一个程序,你就启动了一个进程。显然,程序是死的、静态的,进程是活的、动态的。进程可以分为系统进程和用户进程。凡是用于完成操作系统的各种功能的进程就是系统进程,它们就是处于运行状态下的操作系统本身,用户进程就是所有由你启动的进程。

线程是 CPU 调度的最小单位,必须依赖于进程而存在

线程是进程的一个实体,是 CPU 调度和分派的基本单位,它是比进程更小的、能独立运行的基本单位。线程自己基本上不拥有系统资源,只拥有一点在运行中必不可少的资源(如程序计数器,一组寄存器和栈),但是它可与同属一个进程的其他线程共享进程所拥有的全部资源。

线程无处不在

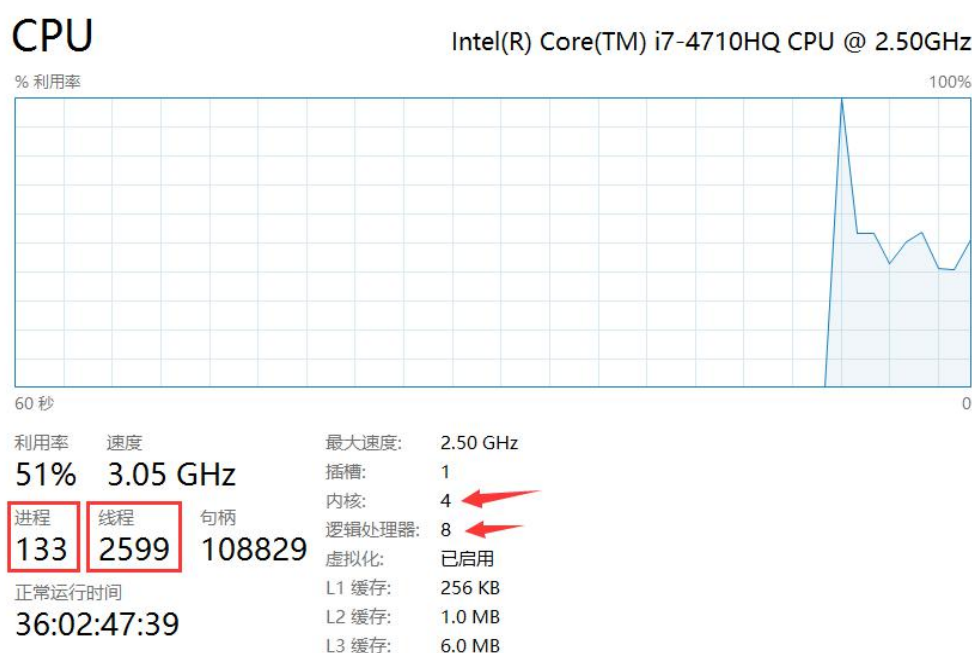
任何一个程序都必须创建线程,特别是 Java 不管任何程序都必须启动一个 main 函数的主线程; Java Web 开发里面的定时任务、定时器、JSP 和 Servlet、异步消息处理机制,远程访问接口 RM 等,任何一个监听事件, onclick 的触发事件等都离不开线程和并发的知识。

CPU 核心数和线程数的关系

多核心:也指单芯片多处理器(Chip Multiprocessors,简称 CMP),CMP 是由美国斯坦福大学提出的,其思想是将大规模并行处理器中的 SMP(对称多处理器)集成到同一芯片内,各个处理器并行执行不同的进程。这种依靠多个 CPU 同时并行地运行程序是实现超高速计算的一个重要方向,称为并行处理

多线程: Simultaneous Multithreading.简称 SMT.让同一个处理器上的多个线程同步执行并共享处理器的执行资源。

核心数、线程数:目前主流 CPU 都是多核的。增加核心数目就是为了增加线程数,因为操作系统是通过线程来执行任务的,一般情况下它们是 1:1 对应关系,也就是说四核 CPU 一般拥有四个线程。但 Intel 引入超线程技术后,使核心数与线程数形成 1:2 的关系



CPU 时间片轮转机制

我们平时在开发的时候,感觉并没有受 cpu 核心数的限制,想启动线程就启动线程,哪怕是在单核 CPU 上,为什么?这是因为操作系统提供了一种 CPU 时间片轮转机制。

时间片轮转调度是一种最古老、最简单、最公平且使用最广的算法,又称 RR 调度。每个进程被分配一个时间段,称作它的时间片,即该进程允许运行的时间。

百度百科对 CPU 时间片轮转机制原理解释如下:

如果在时间片结束时进程还在运行,则 CPU 将被剥夺并分配给另一个进程。如果进程在时间片结束前阻塞或结束,则 CPU 当即进行切换。调度程序所要做的就是维护一张就绪进程列表,当进程用完它的时间片后,它被移到队列的末尾

时间片轮转调度中唯一有趣的一点是时间片的长度。从一个进程切换到另一个进程是需要定时间的,包括保存和装入寄存器值及内存映像,更新各种表格和队

列等。假如进程切(processwitch),有时称为上下文切换(context switch),需要 5ms,再假设时间片设为 20ms,则在做完 20ms 有用的工作之后,CPU 将花费 5ms 来进行进程切换。CPU 时间的 20%被浪费在了管理开销上了。

为了提高 CPU 效率,我们可以将时间片设为 5000ms。这时浪费的时间只有 0.1%。但考虑到在一个分时系统中,如果有 10 个交互用户几乎同时按下回车键,将发生什么情况?假设所有其他进程都用足它们的时间片的话,最后一个不幸的进程不得不等待 5s 才能获得运行机会。多数用户无法忍受一条简短命令要 5 才能做出响应,同样的问题在一台支持多道程序的个人计算机上也会发

结论可以归结如下:时间片设得太短会导致过多的进程切换,降低了 CPU 效率:而设得太长又可能引起对短的交互请求的响应变差。将时间片设为 100ms 通常是一个比较合理的折衷。

在 CPU 死机的情况下,其实大家不难发现当运行一个程序的时候把 CPU 给弄到了 100%再不重启电脑的情况下,其实我们还是有机会把它 KIII 掉的,我想也正是因为这种机制的缘故。

澄清并行和并发

我们举个例子,如果有条高速公路 A 上面并排有 8 条车道,那么最大的并行车车辆就是 8 辆此条高速公路 A 同时并排行走的车辆小于等于 8 辆的时候,车辆就可以并行运行。CPU 也是这个原理,一个 CPU 相当于一个高速公路 A,核心数或者线程数就相当于并排可以通行的车道;而多个 CPU 就相当于并排有多条高速公路,而每个高速公路并排有多个车道。

当谈论**并发**的时候一定要加个单位时间,也就是说单位时间内并发量是多少?离开了单位时间其实是没有意义的。

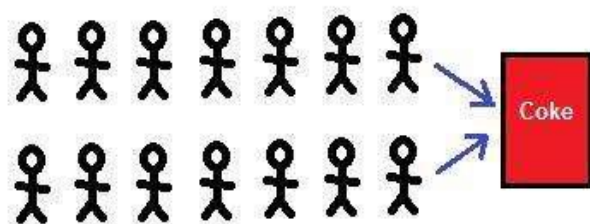
俗话说,一心不能二用,这对计算机也一样,原则上一个 CPU 只能分配给一个进程,以便运行这个进程。我们通常使用的计算机中只有一个 CPU,也就是说只有一颗心,要让它一心多用同时运行多个进程,就必须使用并发技术。实现并发技术相当复杂,最容易理解的是“时间片轮转进程调度算法”。

综合来说:

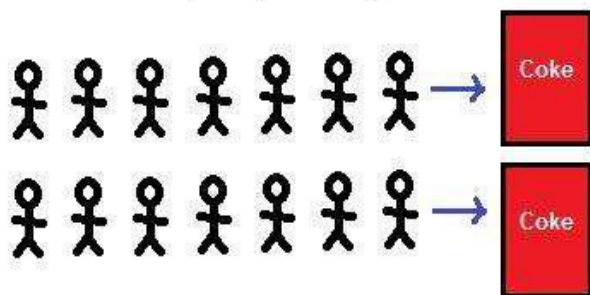
并发:指应用能够交替执行不同的任务,比如单 CPU 核心下执行多线程并非是同时执行多个任务,如果你开两个线程执行,就是在你几乎不可能察觉到的速度不断去切换这两个任务,已达到"同时执行效果",其实并不是的,只是计算机的速度太快,我们无法察觉到而已。

并行:指应用能够同时执行不同的任务,例:吃饭的时候可以边吃饭边打电话,这两件事情可以同时执行

两者区别:一个是交替执行,一个是同时执行。



Concurrent: 2 queues, 1 vending machine



Parallel: 2 queues, 2 vending machines

高并发编程的意义、好处和注意事项

由于多核多线程的 CPU 的诞生,多线程、高并发的编程越来越受重视和关注。多线程可以给程序带来如下好处。

(1)充分利用 CPU 的资源

从上面的 CPU 的介绍,可以看的出来,现在市面上没有 CPU 的内核不使用多线程并发机制的,特别是服务器还不止一个 CPU,如果还是使用单线程的技术做思路,明显就 out 了。因为程序的基本调度单元是线程,并且一个线程也只能在一个 CPU 的一个核的一个线程跑,如果你是个 i3 的 CPU 的话,最差也是双核心 4 线程的运算能力:如果是一个线程的程序的话,那是要浪费 3/4 的 CPU 性能:如果设计一个多线程的程序的话,那它就可以同时在多个 CPU 的多个核的多个线程上跑,可以充分地利用 CPU,减少 CPU 的空闲时间,发挥它的运算能力,提高并发量。

就像我们平时坐地铁一样,很多人坐长线地铁的时候都在认真看书,而不是为了坐地铁而坐地铁,到家了再去看书,这样你的时间就相当于有了两倍。这就是为什么有些人时间很充裕,而有些人老是说没时间的一个原因,工作也是这样,有的时候可以并发地去做几件事情,充分利用我们的时间,CPU 也是一样,也要充分利用。

(2)加快响应用户的时间

比如我们经常用的迅雷下载,都喜欢多开几个线程去下载,谁都不愿意用一个线程去下载,为什么呢?答案很简单,就是多个线程下载快啊。

我们在做程序开发的时候更应该如此,特别是我们做互联网项目,网页的响应时间若提升 1s,如果流量大的话,就能增加不少转换量。做过高性能 web 前端调优的都知道,要将静态资源地址用两三个子域名去加载,为什么?因为每多一个子域名,浏览器在加载你的页面的时候就会多开几个线程去加载你的页面资源,提升网站的响应速度。多线程,高并发真的是无处不在。

(3)可以使你的代码模块化,异步化,简单化

例如我们实现电商系统,下订单和给用户发送短信、邮件就可以进行拆分,将给用户发送短信、邮件这两个步骤独立为单独的模块,并交给其他线程去执行。这样既增加了异步的操作,提升了系统性能,又使程序模块化,清晰化和简单化。

多线程应用开发的好处还有很多,大家在日后的代码编写过程中可以慢慢体会它的魅力。

多线程程序需要注意事项

(1)线程之间的安全性

从前面的章节中我们都知道,在同一个进程里面的多线程是资源共享的,也就是都可以访问同一个内存地址其中的一个变量。例如:若每个线程中对全局变量、静态变量只有读操作,而无写操作,一般来说,这个全局变量是线程安全的:若有多个线程同时执行写操作,一般都需要考虑线程同步,否则就可能影响线程安全。

(2)线程之间的死锁

为了解决线程之间的安全性引入了 Java 的锁机制,而一不小心就会产生 Java 线程死锁的多线程问题,因为不同的线程都在等待那些根本不可能被释放的锁,从而导致所有的工作都无法完成。假设有两个线程,分别代表两个饥饿的人,他们必须共享刀叉并轮流吃饭。他们都需要获得两个锁:共享刀和共享叉的锁。

假如线程 A 获得了刀,而线程 B 获得了叉。线程 A 就会进入阻塞状态来等待获得叉,而线程 B 则阻塞来等待线程 A 所拥有的刀。这只是人为设计的例子,但尽管在运行时很难探测到,这类情况却时常发生

(3)线程太多了会将服务器资源耗尽形成死机当机

线程数太多有可能造成系统创建大量线程而导致消耗完系统内存以及 CPU 的“过渡切换”,造成系统的死机,那么我们该如何解决这类问题呢?

某些系统资源是有限的,如文件描述符。多线程程序可能耗尽资源,因为每个线程都可能希望有一个这样的资源。如果线程数相当大,或者某个资源的候选线程数远远超过了可用的资源数则最好使用资源池。一个最好的示例是数据库连接池。只要线程需要使用一个数据库连接,它就从池中取出一个,使用以后再将它返回池中。资源池也称为资源库。

多线程应用开发的注意事项很多,希望大家在日后的工作中可以慢慢体会它的危险所在。

认识 Java 里的线程

Java 程序天生就是多线程的

一个 Java 程序从 `main()` 方法开始执行,然后按照既定的代码逻辑执行,看似没有其他线程参与,但实际上 Java 程序天生就是多线程程序,因为执行 `main()` 方法的是一个名称为 `main` 的线程。

[6] `Monitor Ctrl-Break` //监控 `Ctrl-Break` 中断信号的

[5] `Attach Listener` //内存 dump, 线程 dump, 类信息统计, 获取系统属性等

[4] `Signal Dispatcher` // 分发处理发送给 JVM 信号的线程

-
- [3] Finalizer // 调用对象 finalize 方法的线程
 - [2] Reference Handler//清除 Reference 的线程
 - [1] main //main 线程，用户程序入口

线程的启动与中止

启动

启动线程的方式有：

- 1、X extends Thread;，然后 X.start
 - 2、X implements Runnable; 然后交给 Thread 运行
- 参见代码：cn.enjoyedu.ch1.base.NewThread

Thread 和 Runnable 的区别

Thread 才是 Java 里对线程的唯一抽象，Runnable 只是对任务（业务逻辑）的抽象。Thread 可以接受任意一个 Runnable 的实例并执行。

中止

线程自然终止

要么是 run 执行完成了，要么是抛出了一个未处理的异常导致线程提前结束。

stop

暂停、恢复和停止操作对应在线程 Thread 的 API 就是 **suspend()**、**resume()** 和 **stop()**。但是这些 API 是过期的，也就是不建议使用的。不建议使用的原因主要有：以 **suspend()** 方法为例，在调用后，线程不会释放已经占有的资源（比如锁），而是占有着资源进入睡眠状态，这样容易引发死锁问题。同样，**stop()** 方法在终结一个线程时不会保证线程的资源正常释放，通常是没有给予线程完成资源释放工作的机会，因此会导致程序可能工作在不确定状态下。正因为 **suspend()**、**resume()** 和 **stop()** 方法带来的副作用，这些方法才被标注为不建议使用的过期方法。

中断

安全的中止则是其他线程通过调用某个线程 A 的 **interrupt()** 方法对其进行中断操作，中断好比其他线程对该线程打了个招呼，“A，你要中断了”，不代表线程 A 会立即停止自己的工作，同样的 A 线程完全可以不理睬这种中断请求。线程通过检查自身的中断标志位是否被置为 **true** 来进行响应，

线程通过方法 **isInterrupted()** 来进行判断是否被中断，也可以调用静态方法 **Thread.interrupted()** 来进行判断当前线程是否被中断，不过 **Thread.interrupted()** 会同时将中断标识位改写为 **false**。

如果一个线程处于了阻塞状态（如线程调用了 **thread.sleep**、**thread.join**、**thread.wait** 等），则在线程在检查中断标示时如果发现中断标示为 **true**，则会在

这些阻塞方法调用处抛出 `InterruptedException` 异常，并且在抛出异常后会立即将线程的中断标示位清除，即重新设置为 `false`。

不建议自定义一个取消标志位来中止线程的运行。因为 `run` 方法里有阻塞调用时会无法很快检测到取消标志，线程必须从阻塞调用返回后，才会检查这个取消标志。这种情况下，使用中断会更好，因为，

一、一般的阻塞方法，如 `sleep` 等本身就支持中断的检查，

二、检查中断位的状态和检查取消标志位没什么区别，用中断位的状态还可以避免声明取消标志位，减少资源的消耗。

注意：处于死锁状态的线程无法被中断

对 Java 里的线程再多一点点认识

深入理解 `run()` 和 `start()`

`Thread` 类是 Java 里对线程概念的抽象，可以这样理解：我们通过 `new Thread()` 其实只是 `new` 出一个 `Thread` 的实例，还没有操作系统中真正的线程挂起钩来。只有执行了 `start()` 方法后，才实现了真正意义上的启动线程。

`start()` 方法让一个线程进入就绪队列等待分配 `cpu`，分到 `cpu` 后才调用实现的 `run()` 方法，`start()` 方法不能重复调用，如果重复调用会抛出异常。

而 `run` 方法是业务逻辑实现的地方，本质上和任意一个类的任意一个成员方法并没有任何区别，可以重复执行，也可以被单独调用。

其他的线程相关方法

`yield()` 方法：使当前线程让出 `CPU` 占有权，但让出的时间是不可设定的。也不会释放锁资源。注意：并不是每个线程都需要这个锁的，而且执行 `yield()` 的线程不一定就会持有锁，我们完全可以在释放锁后再调用 `yield` 方法。

所有执行 `yield()` 的线程有可能在进入到就绪状态后会被操作系统再次选中马上又被执行。

`wait()/notify()/notifyAll()`：后面会单独讲述

join 方法

把指定的线程加入到当前线程，可以将两个交替执行的线程合并为顺序执行。比如在线程 B 中调用了线程 A 的 `Join()` 方法，直到线程 A 执行完毕后，才会继续执行线程 B。(此处为常见面试考点)

线程的优先级

在 Java 线程中，通过一个整型成员变量 `priority` 来控制优先级，优先级的范围从 1~10，在线程构建的时候可以通过 `setPriority(int)` 方法来修改优先级，默认优先级是 5，优先级高的线程分配时间片的数量要多于优先级低的线程。

设置线程优先级时，针对频繁阻塞（休眠或者 I/O 操作）的线程需要设置较高优先级，而偏重计算（需要较多 CPU 时间或者偏运算）的线程则设置较低的优先级，确保处理器不会被独占。在不同的 JVM 以及操作系统上，线程规划会存在差异，有些操作系统甚至会忽略对线程优先级的设定。

线程的调度

线程调度是指系统为线程分配 CPU 使用权的过程，主要调度方式有两种：

协同式线程调度(Cooperative Threads-Scheduling)

抢占式线程调度(Preemptive Threads-Scheduling)

使用协同式线程调度的多线程系统，线程执行的时间由线程本身来控制，线程把自己的工作执行完之后，要主动通知系统切换到另外一个线程上。使用协同式线程调度的最大好处是实现简单，由于线程要把自己的事情做完后才会通知系统进行线程切换，所以没有线程同步的问题，但是坏处也很明显，如果一个线程出了问题，则程序就会一直阻塞。

使用抢占式线程调度的多线程系统，每个线程执行的时间以及是否切换都由系统决定。在这种情况下，线程的执行时间不可控，所以不会有「一个线程导致整个进程阻塞」的问题出现。

在 Java 中，`Thread.yield()`可以让出 CPU 执行时间，但是对于获取，线程本身是没有办法的。对于获取 CPU 执行时间，线程唯一可以使用的手段是设置线程优先级，Java 设置了 10 个级别的程序优先级，当两个线程同时处于 Ready 状态时，优先级越高的线程越容易被系统选择执行。

Java 中的线程优先级是通过映射到操作系统的原生线程上实现的，所以线程的调度最终取决于操作系统，操作系统中线程的优先级有时并不能和 Java 中的一一对应，所以 Java 优先级并不是特别靠谱。

所以在面试中如果遇到相关的问题，可以这样回答：Java 中的线程是通过映射到操作系统的原生线程上实现的，所以线程的调度最终取决于操作系统，而操作系统级别，OS 是以抢占式调度线程，我们可以认为线程是抢占式的。Java 虚拟机采用抢占式调度模型，是指优先让可运行池中优先级高的线程占用 CPU，如果可运行池中的线程优先级相同，那么就随机选择一个线程，使其占用 CPU。处于运行状态的线程会一直运行，直至它不得不放弃 CPU。而且操作系统中线程的优先级有时并不能和 Java 中的一一对应，所以 Java 优先级并不是特别靠谱。但是在 Java 中，因为 Java 没有提供安全的抢占式方法来停止线程，要安全的停止线程只能以协作式的方式。

守护线程

Daemon（守护）线程是一种支持型线程，因为它主要被用作程序中后台调度以及支持性工作。这意味着，当一个 Java 虚拟机中不存在非 Daemon 线程的时候，Java 虚拟机将会退出。可以通过调用 `Thread.setDaemon(true)`将线程设置为 Daemon 线程。我们一般用不上，比如垃圾回收线程就是 Daemon 线程。

Daemon 线程被用作完成支持性工作，但是在 Java 虚拟机退出时 Daemon 线程中的 finally 块并不一定会执行。在构建 Daemon 线程时，不能依靠 finally 块中的内容来确保执行关闭或清理资源的逻辑。

线程间的共享和协作

线程间的共享

synchronized 内置锁

线程开始运行，拥有自己的栈空间，就如同一个脚本一样，按照既定的代码一步一步地执行，直到终止。但是，每个运行中的线程，如果仅仅是孤立地运行，那么没有一点儿价值，或者说价值很少，如果多个线程能够相互配合完成工作，包括数据之间的共享，协同处理事情。这将会带来巨大的价值。

Java 支持多个线程同时访问一个对象或者对象的成员变量，关键字 `synchronized` 可以修饰方法或者以同步块的形式来进行使用，它主要确保多个线程在同一个时刻，只能有一个线程处于方法或者同步块中，它保证了线程对变量访问的可见性和排他性，又称为内置锁机制。

对象锁和类锁：

对象锁是用于对象实例方法，或者一个对象实例上的，类锁是用于类的静态方法或者一个类的 `class` 对象上的。我们知道，类的对象实例可以有很多个，但是每个类只有一个 `class` 对象，所以不同对象实例的对象锁是互不干扰的，但是每个类只有一个类锁。

但是有一点必须注意的是，其实类锁只是一个概念上的东西，并不是真实存在的，类锁其实锁的是每个类的对应的 `class` 对象。类锁和对象锁之间也是互不干扰的。

对象锁和类锁，以及锁 `static` 变量之间的运行情况，请参考包 `cn.enjoyedu.ch1.syn` 下的代码。

错误的加锁和原因分析

参见代码 `cn.enjoyedu.ch1.syn.TestIntegerSyn`

原因：虽然我们对 i 进行了加锁，但是

```
private Integer i;

public Worker(Integer i) {
    this.i=i;
}

@Override
public void run() {
    synchronized (i) {
        Thread thread=Thread.currentThread();
        System.out.println(thread.getName()+"--@"+System
        i++;
        System.out.println(thread.getName()+"-----"+i+
        try {
            Thread.sleep( millis: 3000);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }

        System.out.println(thread.getName()+"-----"+i+
    }
```

但是当我们反编译这个类的 class 文件后，可以看到 i++ 实际是，

```
= Thread.currentThread();
ntln(thread.getName() + "--@" + System.identityHashCode(this.i));
nteger1 = this.i; Integer localInteger2 = this.i = Integer.valueOf(this.i.intValue() + 1);
ntln(thread.getName() + "-----" + this.i + "--@" + System.identityHashCode(this.i));
```

```
public static Integer valueOf(int i) {
    if (i >= IntegerCache.low && i <= IntegerCache.high)
        return IntegerCache.cache[i + (-IntegerCache.low)];
    return new Integer(i);
}
```

本质上是返回了一个新的 Integer 对象。也就是每个线程实际加锁的是不同的 Integer 对象。

volatile , 最轻量的同步机制

volatile 保证了不同线程对这个变量进行操作时的可见性，即一个线程修改了某个变量的值，这新值对其他线程来说是立即可见的。参见代码：

cn.enjoyedu.ch1.vola.VolatileCase

```
public class VolatileCase {
    private static boolean ready;
    private static int number;
```

不加 volatile 时，子线程无法感知主线程修改了 ready 的值，从而不会退出循环，而加了 volatile 后，子线程可以感知主线程修改了 ready 的值，迅速退出循环。

但是 volatile 不能保证数据在多个线程下同时写时的线程安全，参见代码：

cn.enjoyedu.ch1.vola.NotSafe

volatile 最适用的场景：一个线程写，多个线程读。

ThreadLocal 辨析

与 Synchronized 的比较

ThreadLocal 和 Synchronized 都用于解决多线程并发访问。可是 ThreadLocal 与 synchronized 有本质的差别。synchronized 是利用锁的机制，使变量或代码块在某一时刻仅仅能被一个线程访问。而 ThreadLocal 为每个线程都提供了变量的副本，使得每个线程在某一时间访问到的并非同一个对象，这样就隔离了多个线程对数据的数据共享。

Spring 的事务就借助了 ThreadLocal 类。Spring 会从数据库连接池中获得一个 connection，然会把 connection 放进 ThreadLocal 中，也就和线程绑定了，事务需要提交或者回滚，只要从 ThreadLocal 中拿到 connection 进行操作。为何 Spring 的事务要借助 ThreadLocal 类？

以 JDBC 为例，正常的事务代码可能如下：

```
dbc = new DataBaseConnection();//第 1 行
Connection con = dbc.getConnection();//第 2 行
con.setAutoCommit(false);//第 3 行
con.executeUpdate(...);//第 4 行
con.executeUpdate(...);//第 5 行
con.executeUpdate(...);//第 6 行
con.commit();//第 7 行
```

上述代码，可以分成三个部分：

事务准备阶段：第 1～3 行

业务处理阶段：第 4～6 行

事务提交阶段：第 7 行

可以很明显的看到，不管我们开启事务还是执行具体的 sql 都需要一个具体的数据库连接。

现在我们开发应用一般都采用三层结构，如果我们控制事务的代码都放在 DAO(DataAccessObject)对象中，在 DAO 对象的每个方法当中去打开事务和关闭事务，当 Service 对象在调用 DAO 时，如果只调用一个 DAO，那我们这样实现则效果不错，但往往我们的 Service 会调用一系列的 DAO 对数据库进行多次操作，那么，这个时候我们就无法控制事务的边界了，因为实际应用当中，我们的 Service 调用的 DAO 的个数是不确定的，可根据需求而变化，而且还可能出现 Service 调用 Service 的情况。

如果不使用 ThreadLocal，代码大概就会是这个样子：

```
public void serviceMethod(){
    Connection conn=null;
    try{
        Connection conn=getConnection();
        conn.setAutoCommit(false);
        Dao1 dao1=new Dao1(conn);
        dao1.doSomething();
        Dao2 dao2=new Dao2(conn);
        dao2.doSomething();
        Dao3 dao3=new Dao3(conn);
        dao3.doSomething();
        conn.commit();
    }
}

Class Dao1{
    private Connection conn=null;
    public Dao1(Connection conn){
        this.conn=conn;
    }
    public void doSomething(){
        PreparedStatement pstmt=null;
        try{
            pstmt=conn.prepareStatement(sql);
            pstmt.execute...
        }
    }
}
```

但是需要注意一个问题，如何让三个 DAO 使用同一个数据源连接呢？我们就必须为每个 DAO 传递同一个数据库连接，要么就是在 DAO 实例化的时候作为构造方法的参数传递，要么在每个 DAO 的实例方法中作为方法的参数传递。这两种方式无疑对我们的 Spring 框架或者开发人员来说都不合适。为了让这个数据库连接可以跨阶段传递，又不显示的进行参数传递，就必须使用别的办法。

Web 容器中，每个完整的请求周期会由一个线程来处理。因此，如果我们将一些参数绑定到线程的话，就可以实现在软件架构中跨层次参数共享（是隐式的共享）。而 JAVA 中恰好提供了绑定的方法--使用 ThreadLocal。

结合使用 Spring 里的 IOC 和 AOP，就可以很好的解决这一点。

只要将一个数据库连接放入 ThreadLocal 中，当前线程执行时只要有使用数据库连接的地方就从 ThreadLocal 获得就行了。

ThreadLocal 的使用

ThreadLocal 类接口很简单，只有 4 个方法，我们先来了解一下：

- void set(Object value)

设置当前线程的线程局部变量的值。

- public Object get()

该方法返回当前线程所对应的线程局部变量。

- public void remove()

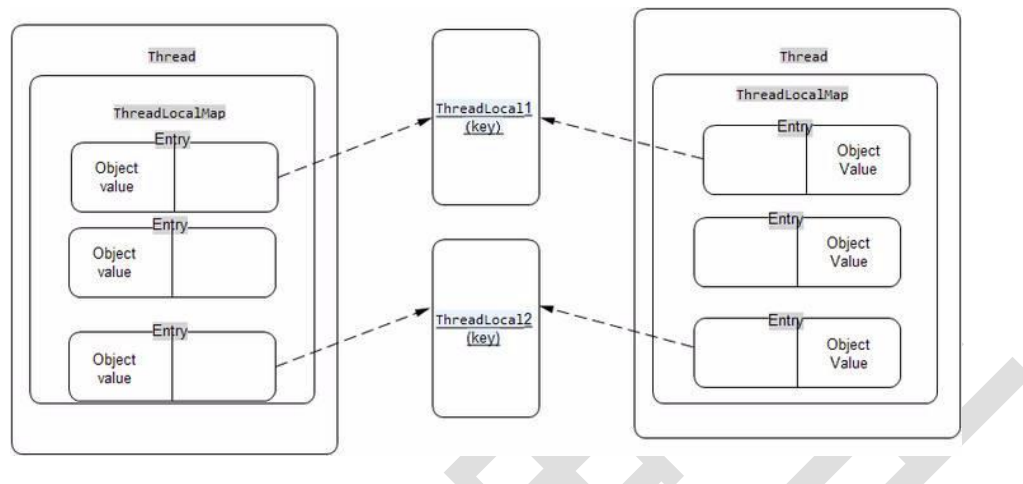
将当前线程局部变量的值删除，目的是为了减少内存的占用，该方法是 JDK 5.0 新增的方法。需要指出的是，当线程结束后，对应该线程的局部变量将自动被垃圾回收，所以显式调用该方法清除线程的局部变量并不是必须的操作，但可以加快内存回收的速度。

- protected Object initialValue()

返回该线程局部变量的初始值，该方法是一个 protected 的方法，显然是为了让子类覆盖而设计的。这个方法是一个延迟调用方法，在线程第 1 次调用 get() 或 set(Object) 时才执行，并且仅执行 1 次。ThreadLocal 中的缺省实现直接返回一个 null。

`public final static ThreadLocal<String> RESOURCE = new ThreadLocal<String>();`RESOURCE 代表一个能够存放 String 类型的 ThreadLocal 对象。此时不论什么一个线程能够并发访问这个变量，对它进行写入、读取操作，都是线程安全的。

实现解析



```
public T get() {
    Thread t = Thread.currentThread();
    ThreadLocalMap map = getMap(t);
    if (map != null) {
        ThreadLocalMap.Entry e = map.getEntry(key: this);
        if (e != null) {
            /unchecked/
            T result = (T)e.value;
            return result;
        }
    }
    return setInitialValue();
}
```

```
ThreadLocalMap getMap(Thread t) {
    return t.threadLocals;
}
```

java × OnlyMain.java × ThreadPoolExecutor.java × ThreadLocal.java × Thread.java ×

`ThreadLocal.ThreadLocalMap threadLocals = null;`

上面先取到当前线程，然后调用 `getMap` 方法获取对应的 `ThreadLocalMap`，`ThreadLocalMap` 是 `ThreadLocal` 的静态内部类，然后 `Thread` 类中有一个这样类型成员，所以 `getMap` 是直接返回 `Thread` 的成员。

看下 `ThreadLocal` 的内部类 `ThreadLocalMap` 源码：


```

static class ThreadLocalMap {
    /**
     * The entries in this hash map extend WeakReference, using
     * its main ref field as the key (which is always a
     * ThreadLocal object). Note that null keys (i.e. entry.get()
     * == null) mean that the key is no longer referenced, so the
     * entry can be expunged from table. Such entries are referred to
     * as "stale entries" in the code that follows.
     */
    static class Entry extends WeakReference<ThreadLocal<?>> {
        /** The value associated with this ThreadLocal. */
        Object value;

        Entry(ThreadLocal<?> k, Object v) {
            super(k);
            value = v;
        }
    }

    /**
     * The initial capacity -- MUST be a power of two.
     */
    private static final int INITIAL_CAPACITY = 16;

    /**
     * The table, resized as necessary.
     * table.length MUST always be a power of two.
     */
    private Entry[] table;
}

```

类似于map的key, value结构
key就是ThreadLocal, Value就
需要隔离访问的变量

用数组保存了Entry, 因为可能有多个变量
需要线程隔离访问

可以看到有个 Entry 内部静态类, 它继承了 WeakReference, 总之它记录了两个信息, 一个是 ThreadLocal<?>类型, 一个是 Object 类型的值。getEntry 方法则是获取某个 ThreadLocal 对应的值, set 方法就是更新或赋值相应的 ThreadLocal 对应的值。

```

private Entry getEntry(ThreadLocal<?> key) {
    int i = key.threadLocalHashCode & (table.length - 1);
    Entry e = table[i];

    private void set(ThreadLocal<?> key, Object value) {
        // We don't use a fast path as with get() because

```

回顾我们的 get 方法, 其实就是拿到**每个线程独有的 ThreadLocalMap**

然后再用 ThreadLocal 的当前实例, 拿到 Map 中的相应的 Entry, 然后就可以拿到相应的值返回出去。当然, 如果 Map 为空, 还会先进行 map 的创建, 初始化等工作。

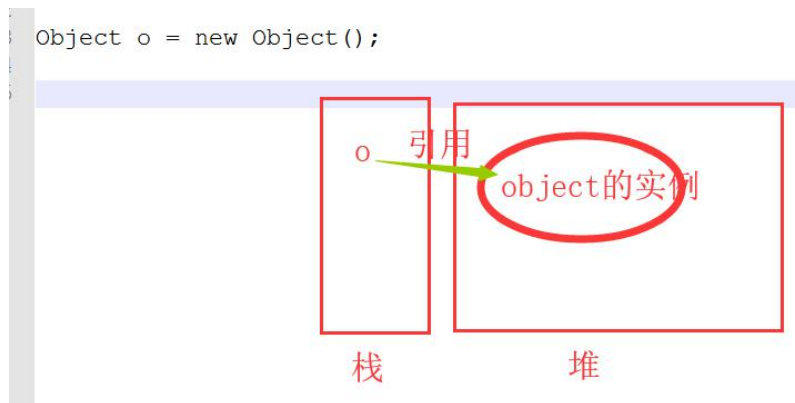
引发的内存泄漏分析

预备知识

引用

Object o = new Object();

这个 o，我们可以称之为对象引用，而 new Object()我们可以称之为在内存中产生了一个对象实例。



当写下 **o=null** 时，只是表示 o 不再指向堆中 object 的对象实例，不代表这个对象实例不存在了。

强引用就是指在程序代码之中普遍存在的，类似“Object obj=new Object()”这类的引用，只要强引用还存在，垃圾收集器永远不会回收掉被引用的对象实例。

软引用是用来描述一些还有用但并非必需的对象。对于软引用关联着的对象，在系统将要发生内存溢出异常之前，将会把这些对象实例列进回收范围之中进行第二次回收。如果这次回收还没有足够的内存，才会抛出内存溢出异常。在 JDK 1.2 之后，提供了 **SoftReference** 类来实现软引用。

弱引用也是用来描述非必需对象的，但是它的强度比软引用更弱一些，被弱引用关联的对象实例只能生存到下一次垃圾收集发生之前。当垃圾收集器工作时，无论当前内存是否足够，都会回收掉只被弱引用关联的对象实例。在 JDK 1.2 之后，提供了 **WeakReference** 类来实现弱引用。

虚引用也称为幽灵引用或者幻影引用，它是最弱的一种引用关系。一个对象实例是否有虚引用的存在，完全不会对其生存时间构成影响，也无法通过虚引用来取得一个对象实例。为一个对象设置虚引用关联的唯一目的就是能在这个对象实例被收集器回收时收到一个系统通知。在 JDK 1.2 之后，提供了 **PhantomReference** 类来实现虚引用。

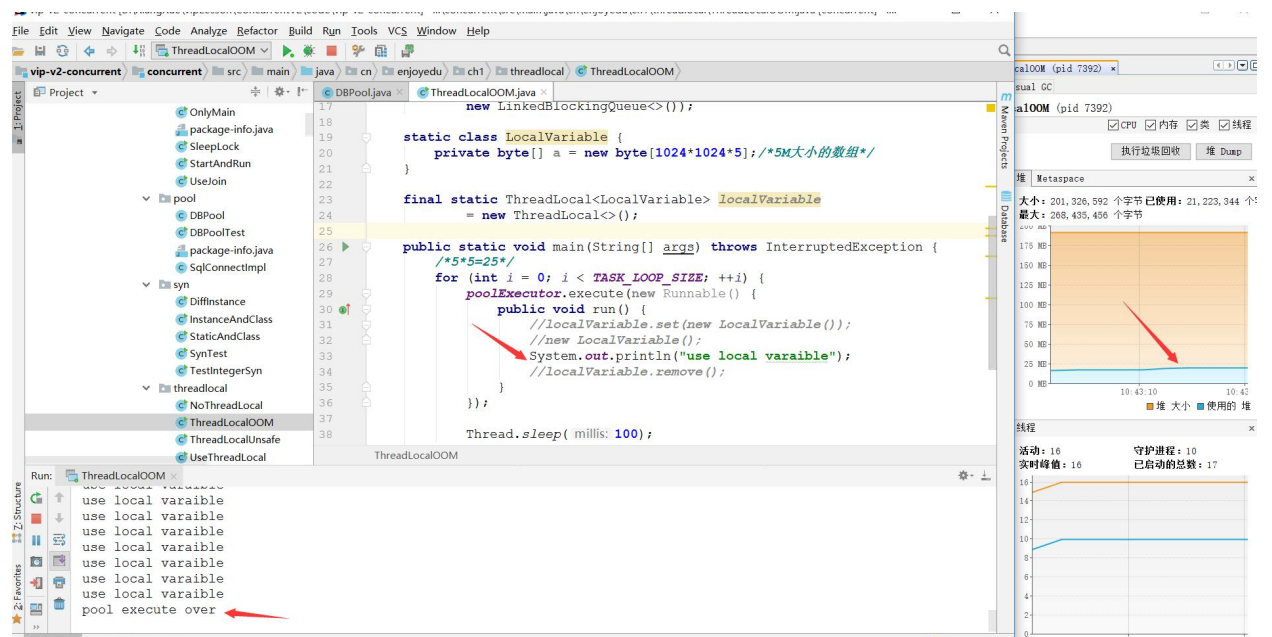
内存泄漏的现象

执行 cn.enjoyedu.ch1.threadlocal 下的 ThreadLocalOOM，并将堆内存大小设置为-Xmx256m，

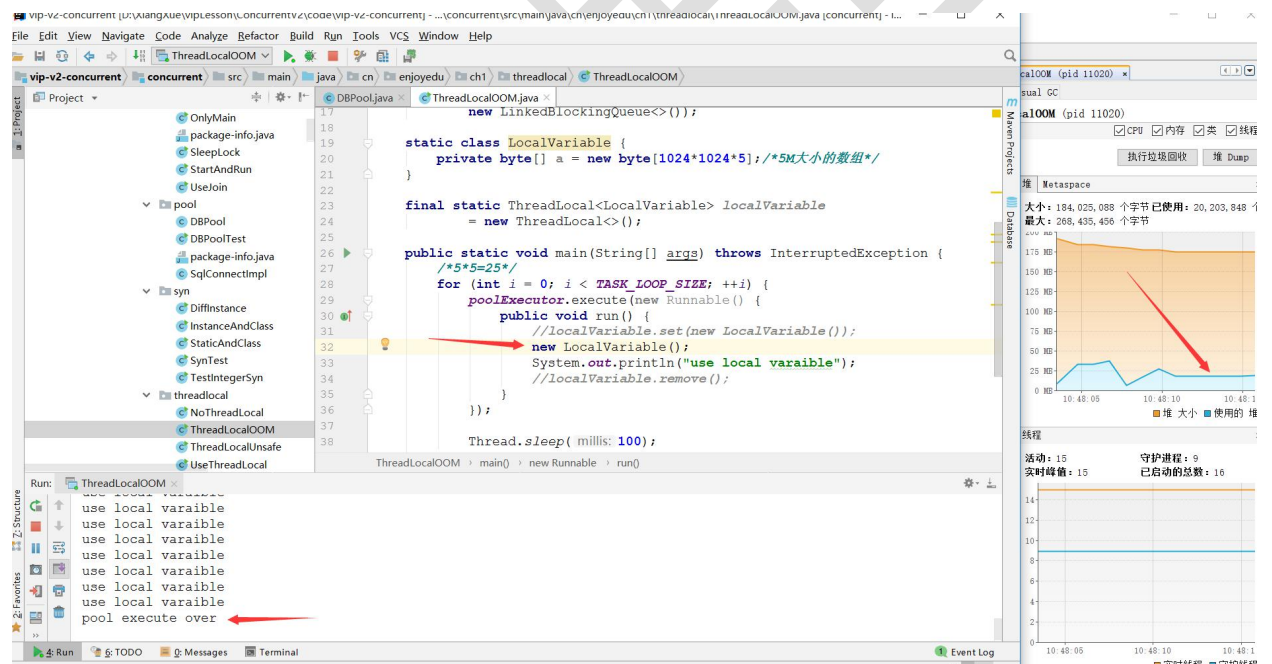
我们启用一个线程池，大小固定为 5 个线程

```
final static ThreadPoolExecutor poolExecutor
    = new ThreadPoolExecutor( corePoolSize: 5, maximumPoolSize: 5, keepAliveTime: 1,
        TimeUnit.MINUTES,
        new LinkedBlockingQueue<>());
```

场景 1，首先任务中不执行任何有意义的代码，当所有的任务提交执行完成后，可以看见，我们这个应用的内存占用基本上为 25M 左右



场景 2，然后我们只简单的在每个任务中 new 出一个数组，执行完成后我们可以看见，内存占用基本和场景 1 同

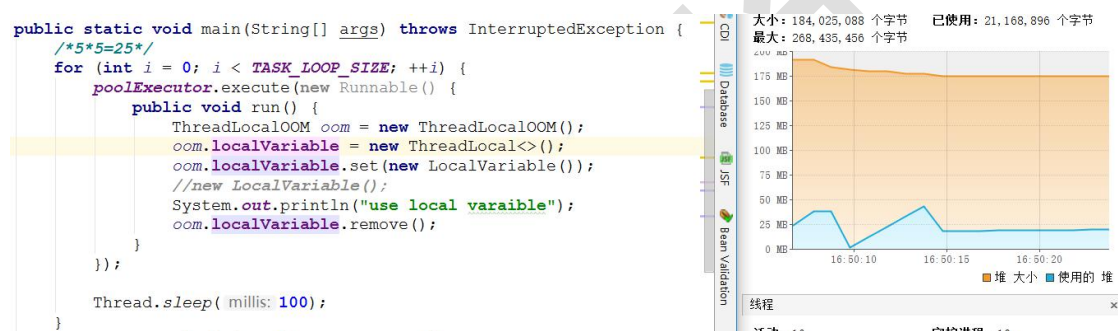


场景 3，当我们启用了 ThreadLocal 以后：



执行完成后我们可以看见，内存占用变为了 100 多 M

场景 4，于是，我们加入一行代码，再执行，看看内存情况：



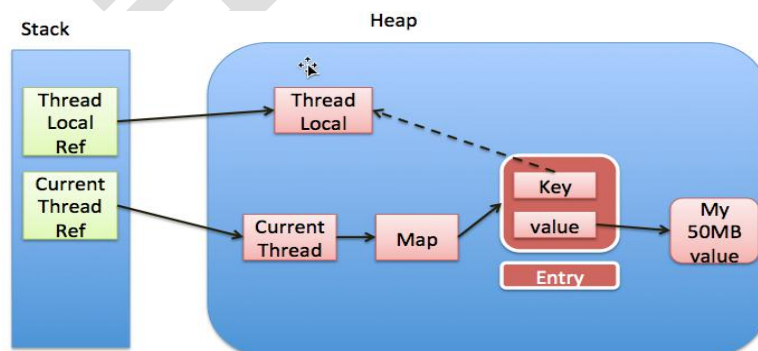
可以看见，内存占用基本和场景 1 同。

这就充分说明，场景 3，当我们启用了 ThreadLocal 以后确实发生了内存泄漏。

分析

根据我们前面对 ThreadLocal 的分析，我们可以知道每个 Thread 维护一个 ThreadLocalMap，这个映射表的 key 是 ThreadLocal 实例本身，value 是真正需要存储的 Object，也就是说 ThreadLocal 本身并不存储值，它只是作为一个 key 来让线程从 ThreadLocalMap 获取 value。仔细观察 ThreadLocalMap，这个 map 是使用 ThreadLocal 的弱引用作为 Key 的，弱引用的对象在 GC 时会被回收。

因此使用了 ThreadLocal 后，引用链如图所示



图中的虚线表示弱引用。

这样，当把 threadlocal 变量置为 null 以后，没有任何强引用指向 threadlocal 实例，所以 threadlocal 将会被 gc 回收。这样一来，ThreadLocalMap 中就会出现

key 为 null 的 Entry，就没有办法访问这些 key 为 null 的 Entry 的 value，如果当前线程再迟迟不结束的话，这些 key 为 null 的 Entry 的 value 就会一直存在一条强引用链：Thread Ref -> Thread -> ThreadLocalMap -> Entry -> value，而这块 value 永远不会被访问到了，所以存在着内存泄露。

只有当前 thread 结束以后，current thread 就不会存在栈中，强引用断开，Current Thread、Map value 将全部被 GC 回收。最好的做法是不在需要使用 ThreadLocal 变量后，都调用它的 remove()方法，清除数据。

所以回到我们前面的实验场景，场景 3 中，虽然线程池里面的任务执行完毕了，但是线程池里面的 5 个线程会一直存在直到 JVM 退出，我们 set 了线程的 localVariable 变量后没有调用 localVariable.remove()方法，导致线程池里面的 5 个线程的 threadLocals 变量里面的 new LocalVariable()实例没有被释放。

其实考察 ThreadLocal 的实现，我们可以看见，无论是 get()、set()在某些时候，调用了 expungeStaleEntry 方法来清除 Entry 中 Key 为 null 的 Value，但是这是不及时的，也不是每次都会执行的，所以一些情况下还是会发生内存泄露。只有 remove()方法中显式调用了 expungeStaleEntry 方法。

从表面上看内存泄漏的根源在于使用了弱引用，但是另一个问题也同样值得思考：为什么使用弱引用而不是强引用？

下面我们分两种情况讨论：

key 使用强引用：对 ThreadLocal 对象实例的引用被置为 null 了，但是 ThreadLocalMap 还持有这个 ThreadLocal 对象实例的强引用，如果没有手动删除，ThreadLocal 的对象实例不会被回收，导致 Entry 内存泄漏。

key 使用弱引用：对 ThreadLocal 对象实例的引用被置为 null 了，由于 ThreadLocalMap 持有 ThreadLocal 的弱引用，即使没有手动删除，ThreadLocal 的对象实例也会被回收。value 在下次 ThreadLocalMap 调用 set，get，remove 都有机会被回收。

比较两种情况，我们可以发现：由于 ThreadLocalMap 的生命周期跟 Thread 一样长，如果都没有手动删除对应 key，都会导致内存泄漏，但是使用弱引用可以多一层保障。

因此，ThreadLocal 内存泄漏的根源是：由于 ThreadLocalMap 的生命周期跟 Thread 一样长，如果没有手动删除对应 key 就会导致内存泄漏，而不是因为弱引用。

总结

JVM 利用设置 ThreadLocalMap 的 Key 为弱引用，来避免内存泄露。

JVM 利用调用 remove、get、set 方法的时候，回收弱引用。

当 ThreadLocal 存储很多 Key 为 null 的 Entry 的时候，而不再去调用 remove、get、set 方法，那么将导致内存泄漏。

使用线程池+ ThreadLocal 时要小心，因为这种情况下，线程是一直在不断的重复运行的，从而也就造成了 value 可能造成累积的情况。

错误使用 ThreadLocal 导致线程不安全

参见代码 `cn.enjoyedu.ch1.threadlocal.ThreadLocalUnsafe`
运行后的结果为

ThreadLocalUnsafe x
"C:\Program File
Thread-0=3
Thread-1=3
Thread-2=3
Thread-4=5
Thread-3=4

如果我们加入 SleepTools.ms(2);会看的更明显

The screenshot displays an IDE with a Java project. The left sidebar shows the 'Structure' view with 'ThreadLocalUnsafe' selected. The main editor shows the following code:

```

10 public class ThreadLocalUnsafe implements Runnable {
11
12     public static Number number = new Number(0);
13
14     public void run() {
15         // 每个线程计数加一
16         number.setNum(number.getNum()+1);
17         // 将其存储到ThreadLocal中
18         value.set(number);
19         SleepTools.ms( seconds: 2);
20         // 输出num值
21         System.out.println(Thread.currentThread().getName()+"="+value.get().getNum());
22     }
23
24     public static ThreadLocal<Number> value = new ThreadLocal<Number>() {
25     }
26 }

```

Below the code editor, the 'Run' tab shows the execution path: 'ThreadLocalUnsafe > run()'. The 'Run' output pane displays the command executed and the resulting output:

```

Run: ThreadLocalUnsafe x ThreadLocalUnsafe x ThreadLocalUnsafe x ThreadLocalUnsafe x ThreadLocalUnsafe x
"C:\Program Files\Java\jdk1.8.0_101\bin\java.exe" ...

Thread-1=5
Thread-3=5
Thread-2=5
Thread-4=5
Thread-0=5

```

The output shows five threads (Thread-0 to Thread-4) all having a value of 5, indicating that each thread maintained its own copy of the counter.

为什么每个线程都输出 5？难道他们没有独自保存自己的 **Number** 副本吗？为什么其他线程还是能够修改这个值？仔细考察 **ThreadLocal** 和 **Thead** 的代码，我们发现 **ThreadLocalMap** 中保存的其实是对象的一个引用，这样的话，当有其他线程对这个引用指向的对象实例做修改时，其实也同时影响了所有的线程持有的对象引用所指向的同一个对象实例。这也就是为什么上面的程序为什么会输出一样的结果：5 个线程中保存的是同一 **Number** 对象的引用，在线程睡眠的时候，其他线程将 **num** 变量进行了修改，而修改的对象 **Number** 的实例是同一份，因此它们最终输出的结果是相同的。

而上面的程序要正常的工作，应该的用法是让每个线程中的 `ThreadLocal` 都应该持有一个新的 `Number` 对象。

线程间的协作

线程之间相互配合，完成某项工作，比如：一个线程修改了一个对象的值，而另一个线程感知到了变化，然后进行相应的操作，整个过程开始于一个线程，而最终执行又是另一个线程。前者是生产者，后者就是消费者，这种模式隔离了“做什么”（**what**）和“怎么做”（**How**），简单的办法是让消费者线程不断地循环检查变量是否符合预期在 **while** 循环中设置不满足的条件，如果条件满足则退出 **while** 循环，从而完成消费者的工作。却存在如下问题：

- 1) 难以确保及时性。

2) 难以降低开销。如果降低睡眠的时间, 比如休眠 1 毫秒, 这样消费者能更加迅速地发现条件变化, 但是却可能消耗更多的处理器资源, 造成了无端的浪费。

等待/通知机制

是指一个线程 A 调用了对象 O 的 `wait()` 方法进入等待状态, 而另一个线程 B 调用了对象 O 的 `notify()` 或者 `notifyAll()` 方法, 线程 A 收到通知后从对象 O 的 `wait()` 方法返回, 进而执行后续操作。上述两个线程通过对象 O 来完成交互, 而对象上的 `wait()` 和 `notify/notifyAll()` 的关系就如同开关信号一样, 用来完成等待方和通知方之间的交互工作。

`notify()`:

通知一个在对象上等待的线程, 使其从 `wait` 方法返回, 而返回的前提是该线程获取到了对象的锁, 没有获得锁的线程重新进入 **WAITING** 状态。

`notifyAll()`:

通知所有等待在该对象上的线程

`wait()`

调用该方法的线程进入 **WAITING** 状态, 只有等待另外线程的通知或被中断才会返回。需要注意, 调用 `wait()` 方法后, 会释放对象的锁

`wait(long)`

超时等待一段时间, 这里的参数时间是毫秒, 也就是等待长达 n 毫秒, 如果没有通知就超时返回

`wait (long,int)`

对于超时时间更细粒度的控制, 可以达到纳秒

等待和通知的标准范式

等待方遵循如下原则。

- 1) 获取对象的锁。
- 2) 如果条件不满足, 那么调用对象的 `wait()` 方法, 被通知后仍要检查条件。
- 3) 条件满足则执行对应的逻辑。

```
synchronized(对象) {  
    while(条件不满足) {  
        对象.wait();  
    }  
    对应的处理逻辑  
}
```

通知方遵循如下原则。

- 1) 获得对象的锁。
- 2) 改变条件。
- 3) 通知所有等待在对象上的线程。

```
synchronized(对象) {  
    改变条件  
    对象.notifyAll();  
}
```

在调用 **wait()**、**notify()** 系列方法之前，线程必须要获得该对象的对象级别锁，即只能在同步方法或同步块中调用 **wait()** 方法、**notify()** 系列方法，进入 **wait()** 方法后，当前线程释放锁，在从 **wait()** 返回前，线程与其他线程竞争重新获得锁，执行 **notify()** 系列方法的线程退出调用了 **notifyAll()** 的 **synchronized** 代码块的时候后，他们就会去竞争。如果其中一个线程获得了该对象锁，它就会继续往下执行，在它退出 **synchronized** 代码块，释放锁后，其他的已经被唤醒的线程将会继续竞争获取该锁，一直进行下去，直到所有被唤醒的线程都执行完毕。

notify 和 notifyAll 应该用谁

尽可能用 **notifyAll()**，谨慎使用 **notify()**，因为 **notify()** 只会唤醒一个线程，我们无法确保被唤醒的这个线程一定就是我们需要唤醒的线程，具体表现参见代码：包 **cn.enjoyedu.ch1.wn** 下

等待超时模式实现一个连接池

调用场景：调用一个方法时等待一段时间（一般来说是给定一个时间段），如果该方法能够在给定的时间段之内得到结果，那么将结果立刻返回，反之，超时返回默认结果。

假设等待时间段是 **T**，那么可以推断出在当前时间 **now+T** 之后就会超时

等待持续时间：**REMAINING=T**。

• 超时时间：**FUTURE=now+T**。

// 对当前对象加锁

```
public synchronized Object get(long mills) throws InterruptedException {  
    long future = System.currentTimeMillis() + mills;  
    long remaining = mills;  
    // 当超时大于 0 并且 result 返回值不满足要求  
    while ((result == null) && remaining > 0) {  
        wait(remaining);  
        remaining = future - System.currentTimeMillis();  
    }  
    return result;  
}
```

具体实现参见：包下 **cn.enjoyedu.ch1.pool** 的代码

客户端获取连接的过程被设定为等待超时的模式，也就是在 **1000** 毫秒内如果无法获取到可用连接，将会返回给客户端一个 **null**。设定连接池的大小为 **10** 个，然后通过调节客户端的线程数来模拟无法获取连接的场景。

它通过构造函数初始化连接的最大上限，通过一个双向队列来维护连接，调用方需要先调用 `fetchConnection(long)` 方法来指定在多少毫秒内超时获取连接，当连接使用完成后，需要调用 `releaseConnection(Connection)` 方法将连接放回线程池

面试题

调用 `yield()`、`sleep()`、`wait()`、`notify()` 等方法对锁有何影响？

`yield()`、`sleep()` 被调用后，都不会释放当前线程所持有的锁。

调用 `wait()` 方法后，会释放当前线程持有的锁，而且当前被唤醒后，会重新去竞争锁，锁竞争到后才会执行 `wait` 方法后面的代码。

调用 `notify()` 系列方法后，对锁无影响，线程只有在 `syn` 同步代码执行完后才会自然而然的释放锁，所以 `notify()` 系列方法一般都是 `syn` 同步代码的最后一行。

2、线程的并发工具类

Fork-Join

java 下多线程的开发可以我们自己启用多线程，线程池，还可以使用 `forkjoin`，`forkjoin` 可以让我们不去了解诸如 `Thread`、`Runnable` 等相关的知识，只要遵循 `forkjoin` 的开发模式，就可以写出很好的多线程并发程序，

分而治之

同时 `forkjoin` 在处理某一类问题时非常的有用，哪一类问题？分而治之的问题。十大计算机经典算法：快速排序、堆排序、归并排序、二分查找、线性查找、

深度优先、广度优先、`Dijkstra`、动态规划、朴素贝叶斯分类，有几个属于分而治之？3 个，快速排序、归并排序、二分查找，还有大数据中 `M/R` 都是。

分治法的设计思想是：将一个难以直接解决的大问题，分割成一些规模较小的相同问题，以便各个击破，分而治之。

分治策略是：对于一个规模为 n 的问题，若该问题可以容易地解决（比如说规模 n 较小）则直接解决，否则将其分解为 k 个规模较小的子问题，**这些子问题互相独立且与原问题形式相同(子问题相互之间有联系就会变为动态规划算法)**，递归地解这些子问题，然后将各子问题的解合并得到原问题的解。这种算法设计策略叫做分治法。

归并排序

归并排序是建立在归并操作上的一种有效的排序算法。该算法是采用分治法的一个非常典型的应用。将已有序的子序列合并，得到完全有序的序列；即先使每个子序列有序，再使子序列段间有序。

若将两个有序表合并成一个有序表，称为 2-路归并，与之对应的还有多路归并。

对于给定的一组数据，利用递归与分治技术将数据序列划分成为越来越小的半子表，在对半子表排序后，再用递归方法将排好序的半子表合并成为越来越大的有序序列。

为了提升性能，有时我们在半子表的个数小于某个数（比如 15）的情况下，对半子表的排序采用其他排序算法，比如插入排序。

归并排序（降序）示例

原始数组 

先将数组划分为左右两个子表：

左子表 ，右子表 

然后继续左右两个子表拆分：

对最后的拆分的子表，两两进行排序

对有序的子表进行排序和比较合并

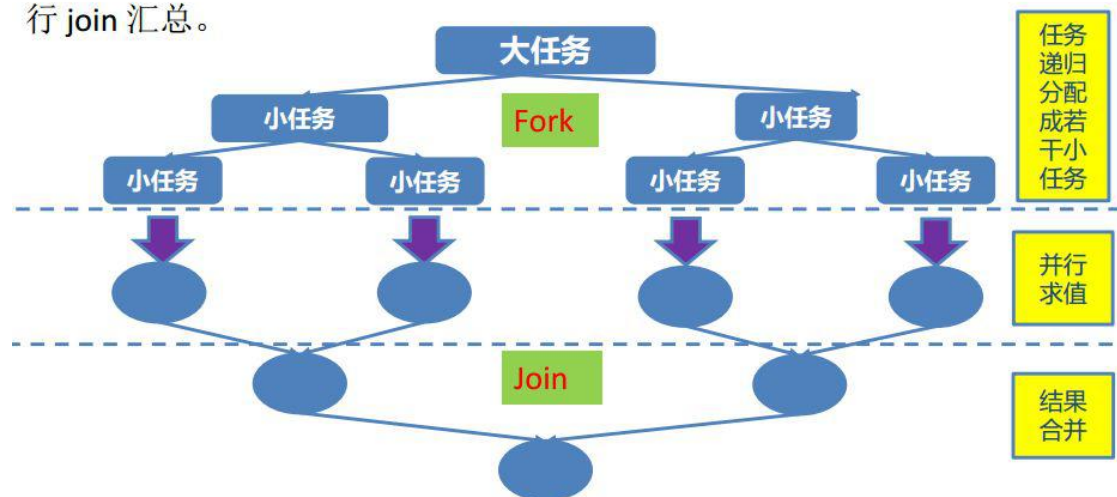
有序子表  
合并  

对合并后的子表继续比较合并


Fork-Join 原理

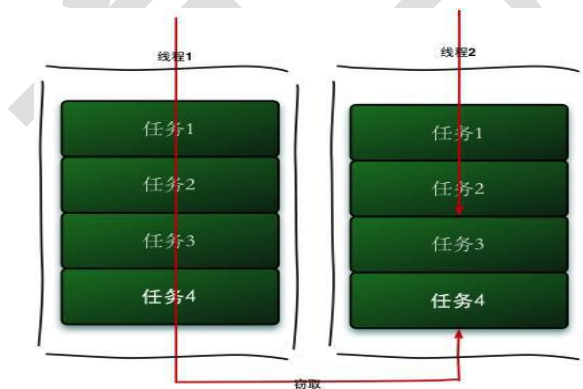
Fork/Join 框架：就是在必要的情况下，将一个大任务，进行拆分(fork)成若干个小任务（拆到不可再拆时），再将一个个的小任务运算的结果进行 join 汇总。



工作窃取

即当前线程的 Task 已经全被执行完毕，则自动取到其他线程的 Task 池中取出 Task 继续执行。

ForkJoinPool 中维护着多个线程（一般为 CPU 核数）在不断地执行 Task，每个线程除了执行自己职务内的 Task 之外，还会根据自己工作线程的闲置情况去获取其他繁忙的工作线程的 Task，如此一来就能减少线程阻塞或是闲置的时间，提高 CPU 利用率。



Fork/Join 实战

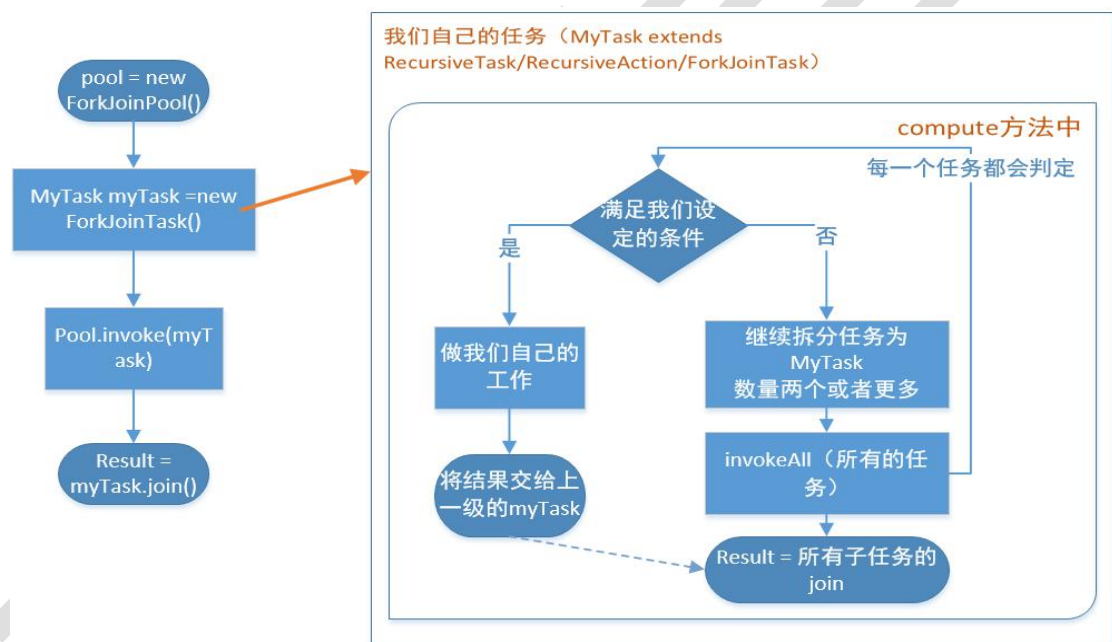
Fork/Join 使用的标准范式

我们要使用 ForkJoin 框架，必须首先创建一个 ForkJoin 任务。它提供在任务中执行 fork 和 join 的操作机制，通常我们不直接继承 ForkJoinTask 类，只需要直接继承其子类。

1. RecursiveAction，用于没有返回结果的任务
2. RecursiveTask，用于有返回值的任务

task 要通过 ForkJoinPool 来执行，使用 submit 或 invoke 提交，两者的区别是：invoke 是同步执行，调用之后需要等待任务完成，才能执行后面的代码；submit 是异步执行。

join()和 get 方法当任务完成的时候返回计算结果。



在我们自己实现的 compute 方法里，首先需要判断任务是否足够小，如果足够小就直接执行任务。如果不够小，就必须分割成两个子任务，每个子任务在调用 invokeAll 方法时，又会进入 compute 方法，看看当前子任务是否需要继续分割成孙任务，如果不需要继续分割，则执行当前子任务并返回结果。使用 join 方法会等待子任务执行完并得到其结果。

Fork/Join 的同步用法和异步用法

参见代码包 cn.enjoyedu.ch2.forkjoin 下

CountDownLatch

闭锁，`CountDownLatch` 这个类能够使一个线程等待其他线程完成各自的工作后再执行。例如，应用程序的主线程希望在负责启动框架服务的线程已经启动所有的框架服务之后再执行。

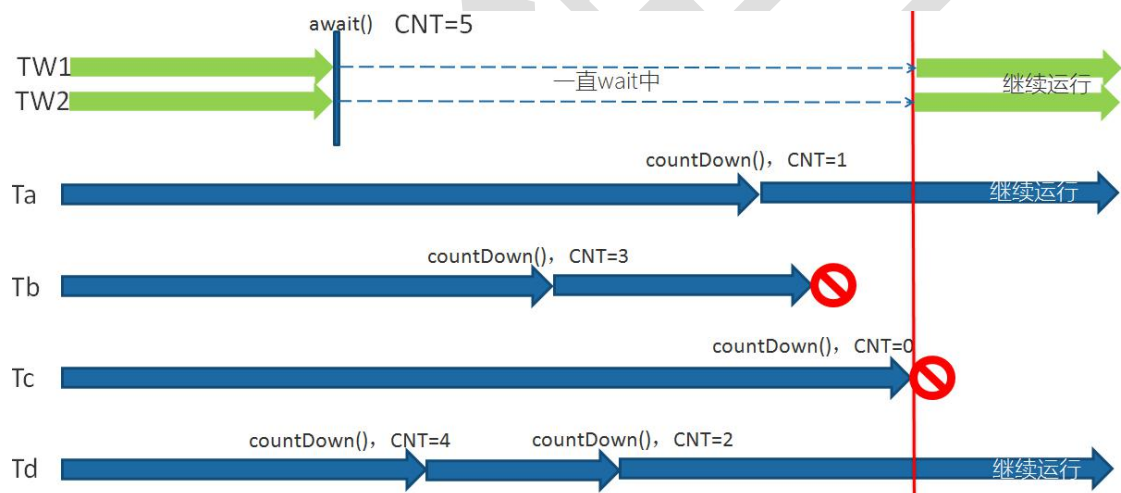
`CountDownLatch` 是通过一个计数器来实现的，计数器的初始值为初始任务的数量。每当完成了一个任务后，计数器的值就会减 1

（`CountDownLatch.countDown()`方法）。当计数器值到达 0 时，它表示所有的已经完成了任务，然后在闭锁上等待 `CountDownLatch.await()`方法的线程就可以恢复执行任务。

应用场景：

实现最大的并行性：有时我们想同时启动多个线程，实现最大程度的并行性。例如，我们想测试一个单例类。如果我们创建一个初始计数为 1 的 `CountDownLatch`，并让所有线程都在这个锁上等待，那么我们可以很轻松地完成测试。我们只需调用 一次 `countDown()`方法就可以让所有的等待线程同时恢复执行。

开始执行前等待 n 个线程完成各自任务：例如应用程序启动类要确保在处理用户请求前，所有 N 个外部系统已经启动和运行了，例如处理 excel 中多个表单。



参见代码包 `cn.enjoyedu.ch2.tools` 下

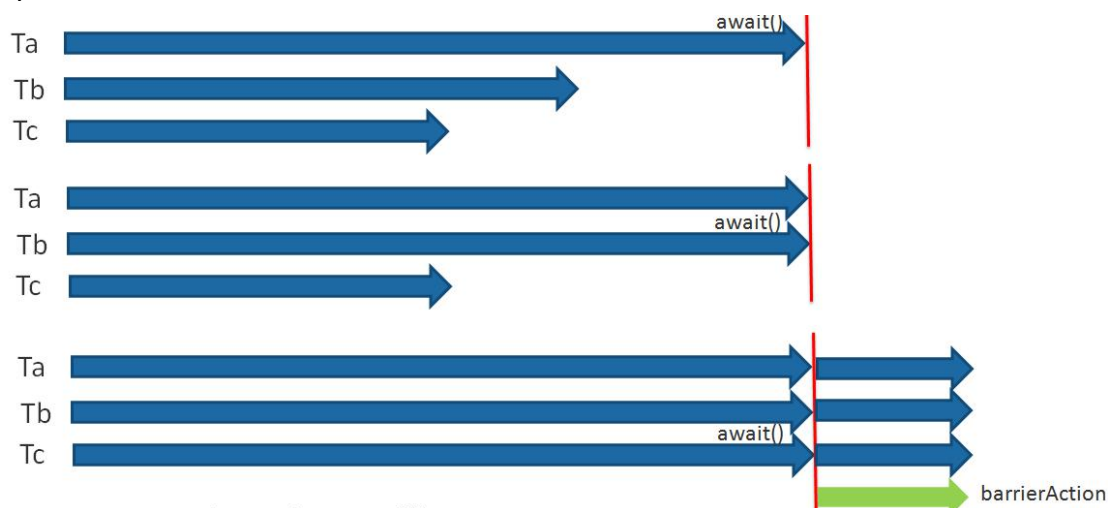
CyclicBarrier

`CyclicBarrier` 的字面意思是可循环使用（`Cyclic`）的屏障（`Barrier`）。它要做的事情是，让一组线程到达一个屏障（也可以叫同步点）时被阻塞，直到最后一个线程到达屏障时，屏障才会开门，所有被屏障拦截的线程才会继续运行。

`CyclicBarrier` 默认的构造方法是 `CyclicBarrier(int parties)`，其参数表示屏障拦截的线程数量，每个线程调用 `await` 方法告诉 `CyclicBarrier` 我已经到达了屏障，然后当前线程被阻塞。

`CyclicBarrier` 还提供一个更高级的构造函数 `CyclicBarrier(int parties, Runnable barrierAction)`，用于在线程到达屏障时，优先执行 `barrierAction`，方便处理更复杂的业务场景。

CyclicBarrier 可以用于多线程计算数据，最后合并计算结果的场景。



参见代码包 `cn.enjoyedu.ch2.tools` 下

CountDownLatch 和 CyclicBarrier 辨析

CountDownLatch 的计数器只能使用一次，而 CyclicBarrier 的计数器可以反复使用。

CountDownLatch.await 一般阻塞工作线程，所有的进行预备工作的线程执行 `countDown`，而 CyclicBarrier 通过工作线程调用 `await` 从而自行阻塞，直到所有工作线程达到指定屏障，再大家一起往下走。

在控制多个线程同时运行上，CountDownLatch 可以不限线程数量，而 CyclicBarrier 是固定线程数。

同时，CyclicBarrier 还可以提供一个 `barrierAction`，合并多线程计算结果。

Semaphore

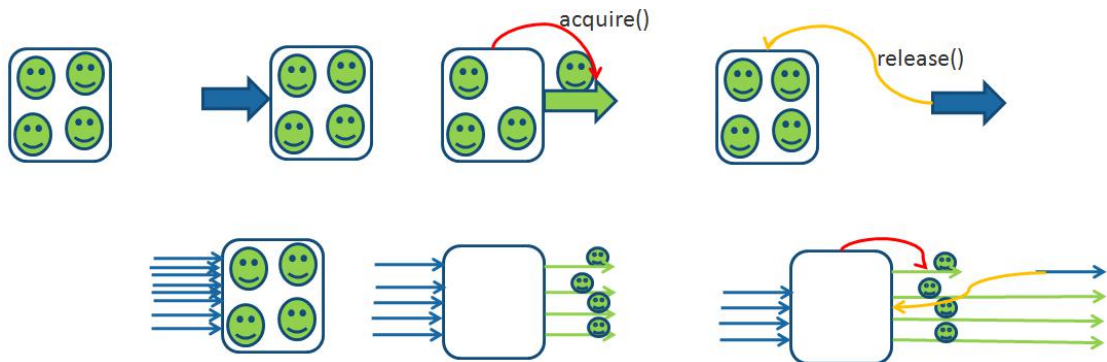
Semaphore（信号量）是用来控制同时访问特定资源的线程数量，它通过协调各个线程，以保证合理的使用公共资源。应用场景 Semaphore 可以用于做流量控制，特别是公用资源有限的应用场景，比如数据库连接。假如有一个需求，要读取几万个文件的数据，因为都是 IO 密集型任务，我们可以启动几十个线程并发地读取，但是如果读到内存后，还需要存储到数据库中，而数据库的连接数只有 10 个，这时我们必须控制只有 10 个线程同时获取数据库连接保存数据，否则会报错无法获取数据库连接。这个时候，就可以使用 Semaphore 来做流量控制。。Semaphore 的构造方法 `Semaphore(int permits)` 接受一个整型的数字，表示可用的许可证数量。Semaphore 的用法也很简单，首先线程使用 Semaphore 的 `acquire()` 方法获取一个许可证，使用完之后调用 `release()` 方法归还许可证。还可以用 `tryAcquire()` 方法尝试获取许可证。

Semaphore 还提供一些其他方法，具体如下。

- `int availablePermits()`: 返回此信号量中当前可用的许可证数。
- `int getQueueLength()`: 返回正在等待获取许可证的线程数。
- `boolean hasQueuedThreads()`: 是否有线程正在等待获取许可证。

•void reducePermits(int reduction): 减少 reduction 个许可证,是个 protected 方法。

•Collection getQueuedThreads(): 返回所有等待获取许可证的线程集合,是个 protected 方法。



用 Semaphore 实现数据库连接池

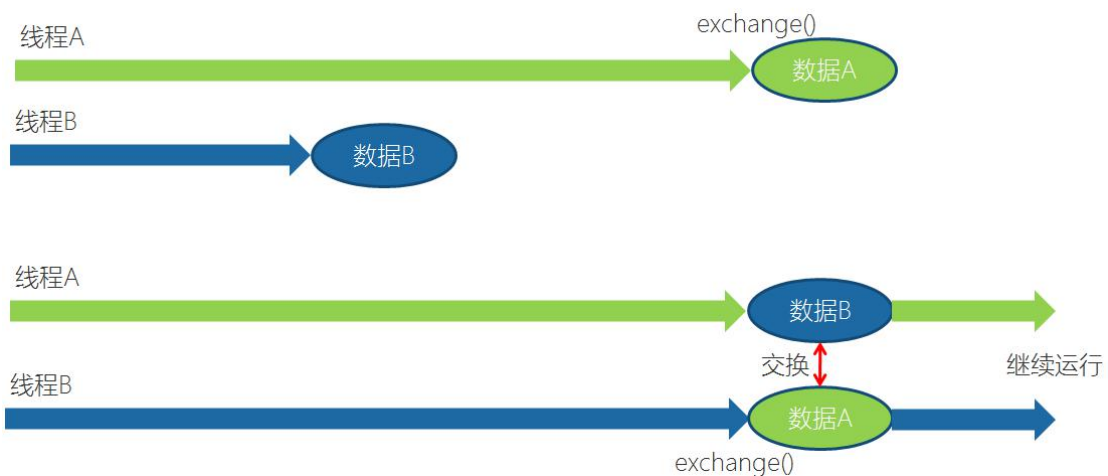
参见代码,包 cn.enjoyedu.ch2.tools.semaphore 下

Semaphore 注意事项

参见代码类 cn.enjoyedu.ch2.tools.semaphore. DBPoolNoUseless 下

Exchange

Exchanger (交换者) 是一个用于线程间协作的工具类。Exchanger 用于进行线程间的数据交换。它提供一个同步点,在这个同步点,两个线程可以交换彼此的数据。这两个线程通过 exchange 方法交换数据,如果第一个线程先执行 exchange()方法,它会一直等待第二个线程也执行 exchange 方法,当两个线程都到达同步点时,这两个线程就可以交换数据,将本线程生产出来的数据传递给对方。



参见代码包 cn.enjoyedu.ch2.tools 下

Callable、Future 和 FutureTask

Runnable 是一个接口，在它里面只声明了一个 run() 方法，由于 run() 方法返回值为 void 类型，所以在执行完任务之后无法返回任何结果。

Callable 位于 java.util.concurrent 包下，它也是一个接口，在它里面也只声明了一个方法，只不过这个方法叫做 call()，这是一个泛型接口，call() 函数返回的类型就是传递进来的 V 类型。

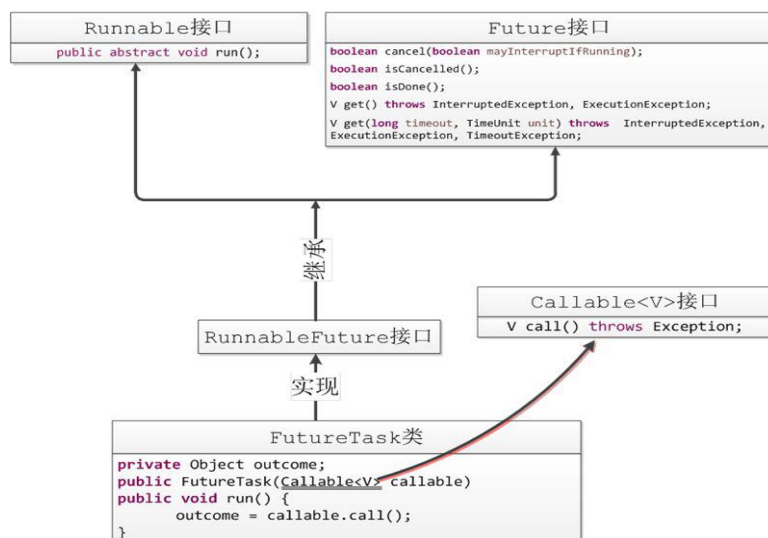
Future 就是对于具体的 Runnable 或者 Callable 任务的执行结果进行取消、查询是否完成、获取结果。必要时可以通过 get 方法获取执行结果，该方法会阻塞直到任务返回结果。

```
✓ I Future
  (m) cancel(boolean): boolean
  (m) isCancelled(): boolean
  (m) isDone(): boolean
  (m) get(): V
  (m) get(long, TimeUnit): V
```

因为 Future 只是一个接口，所以是无法直接用来创建对象使用的，因此就有了下面的 FutureTask。

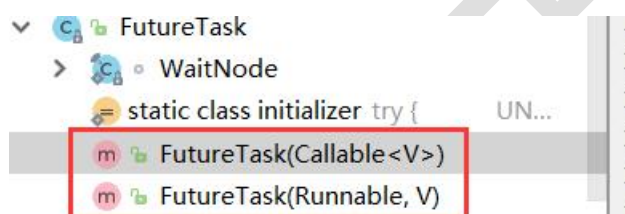
```
public class FutureTask<V> implements RunnableFuture<V> {
    /*
     * Sets this Future to the result of its computation
     * unless it has been cancelled.
     */
    void run();
}
```

FutureTask 类实现了 RunnableFuture 接口，RunnableFuture 继承了 Runnable 接口和 Future 接口，而 FutureTask 实现了 RunnableFuture 接口。所以它既可以作为 Runnable 被线程执行，又可以作为 Future 得到 Callable 的返回值。



因此我们通过一个线程运行 Callable，但是 Thread 不支持构造方法中传递 Callable 的实例，所以我们需要通过 FutureTask 把一个 Callable 包装成 Runnable，然后再通过这个 FutureTask 拿到 Callable 运行后的返回值。

要 new 一个 FutureTask 的实例，有两种方法



参见代码包 cn.enjoyedu.ch2.tools 下

3、原子操作 CAS

什么是原子操作？如何实现原子操作？

假定有两个操作 A 和 B，如果从执行 A 的线程来看，当另一个线程执行 B 时，要么将 B 全部执行完，要么完全不执行 B，那么 A 和 B 对彼此来说是原子的。

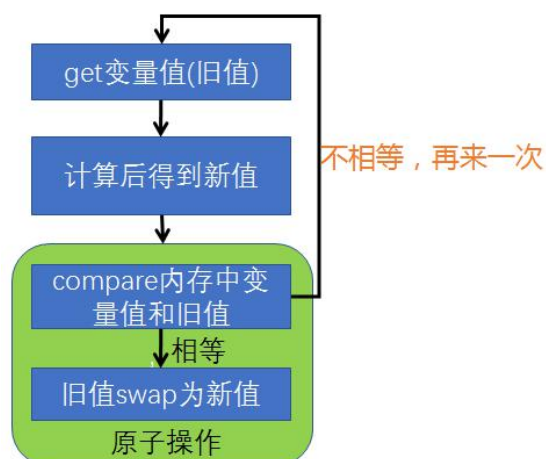
实现原子操作可以使用锁，锁机制，满足基本的需求是没有问题的了，但是有的时候我们的需求并非这么简单，我们需要更有效，更加灵活的机制，synchronized 关键字是基于阻塞的锁机制，也就是说当一个线程拥有锁的时候，访问同一资源的其它线程需要等待，直到该线程释放锁，

这里会有些问题：首先，如果被阻塞的线程优先级很高很重要怎么办？其次，如果获得锁的线程一直不释放锁怎么办？（这种情况是非常糟糕的）。还有一种情况，如果有大量的线程来竞争资源，那 CPU 将会花费大量的时间和资源来处理这些竞争，同时，还有可能出现一些例如死锁之类的情况，最后，其实锁机制是一种比较粗糙，粒度比较大的机制，相对于像计数器这样的需求有点儿过于笨重。

实现原子操作还可以使用当前的处理器基本都支持 `CAS()` 的指令，只不过每个厂家所实现的算法并不一样，每一个 `CAS` 操作过程都包含三个运算符：一个内存地址 `V`，一个期望的值 `A` 和一个新值 `B`，操作的时候如果这个地址上存放的值等于这个期望的值 `A`，则将地址上的值赋为新值 `B`，否则不做任何操作。

`CAS` 的基本思路就是，如果这个地址上的值和期望的值相等，则给予其赋予新值，否则不做任何事儿，但是要返回原值是多少。循环 `CAS` 就是在一个循环里不断的做 `cas` 操作，直到成功为止。

`CAS` 是怎么实现线程的安全呢？语言层面不做处理，我们将其交给硬件——`CPU` 和内存，利用 `CPU` 的多处理能力，实现硬件层面的阻塞，再加上 `volatile` 变量的特性即可实现基于原子操作的线程安全。



CAS 实现原子操作的三大问题

ABA 问题。

因为 `CAS` 需要在操作值的时候，检查值有没有发生变化，如果没有发生变化则更新，但是如果一个值原来是 `A`，变成了 `B`，又变成了 `A`，那么使用 `CAS` 进行检查时会发现它的值没有发生变化，但是实际上却变化了。

`ABA` 问题的解决思路就是使用版本号。在变量前面追加版本号，每次变量更新的时候把版本号加 1，那么 `A→B→A` 就会变成 `1A→2B→3A`。举个通俗点的例子，你倒了一杯水放桌子上，干了点别的事，然后同事把你水喝了又给你重新倒了一杯水，你回来看水还在，拿起来就喝，如果你不管水中间被人喝过，只关心水还在，这就是 `ABA` 问题。

如果你是一个讲卫生讲文明的小伙子，不但关心水在不在，还要在你离开的时候水被人动过没有，因为你是程序员，所以就想起了放了张纸在旁边，写上初始值 0，别人喝水前麻烦先做个累加才能喝水。

循环时间长开销大。

自旋 `CAS` 如果长时间不成功，会给 `CPU` 带来非常大的执行开销。

只能保证一个共享变量的原子操作。

当对一个共享变量执行操作时，我们可以使用循环 CAS 的方式来保证原子操作，但是对多个共享变量操作时，循环 CAS 就无法保证操作的原子性，这个时候就可以用锁。

还有一个取巧的办法，就是把多个共享变量合并成一个共享变量来操作。比如，有两个共享变量 $i=2$, $j=a$ ，合并一下 $ij=2a$ ，然后用 CAS 来操作 ij 。从 Java 1.5 开始，JDK 提供了 `AtomicReference` 类来保证引用对象之间的原子性，就可以把多个变量放在一个对象里来进行 CAS 操作。

Jdk 中相关原子操作类的使用

参见代码，包 `cn.enjoyedu.ch3` 下

AtomicInteger

- `int addAndGet (int delta)`：以原子方式将输入的数值与实例中的值（`AtomicInteger` 里的 `value`）相加，并返回结果。
- `boolean compareAndSet (int expect, int update)`：如果输入的数值等于预期值，则以原子方式将该值设置为输入的值。
- `int getAndIncrement()`：以原子方式将当前值加 1，注意，这里返回的是自增前的值。
- `int getAndSet (int newValue)`：以原子方式设置为 `newValue` 的值，并返回旧值。

AtomicIntegerArray

主要是提供原子的方式更新数组里的整型，其常用方法如下。

- `int addAndGet (int i, int delta)`：以原子方式将输入值与数组中索引 i 的元素相加。
- `boolean compareAndSet (int i, int expect, int update)`：如果当前值等于预期值，则以原子方式将数组位置 i 的元素设置成 `update` 值。

需要注意的是，数组 `value` 通过构造方法传递进去，然后 `AtomicIntegerArray` 会将当前数组复制一份，所以当 `AtomicIntegerArray` 对内部的数组元素进行修改时，不会影响传入的数组。

更新引用类型

原子更新基本类型的 `AtomicInteger`，只能更新一个变量，如果要原子更新多个变量，就需要使用这个原子更新引用类型提供的类。`Atomic` 包提供了以下 3 个类。

AtomicReference

原子更新引用类型。

AtomicStampedReference

利用版本戳的形式记录了每次改变以后的版本号，这样的话就不会存在 ABA 问题了。这就是 AtomicStampedReference 的解决方案。AtomicMarkableReference 跟 AtomicStampedReference 差不多，AtomicStampedReference 是使用 pair 的 int stamp 作为计数器使用，AtomicMarkableReference 的 pair 使用的是 boolean mark。还是那个水的例子，AtomicStampedReference 可能关心的是动过几次，AtomicMarkableReference 关心的是有没有被人动过，方法都比较简单。

AtomicMarkableReference :

原子更新带有标记位的引用类型。可以原子更新一个布尔类型的标记位和引用类型。构造方法是 AtomicMarkableReference (V initialRef, boolean initialMark)。

原子更新字段类

如果需原子地更新某个类里的某个字段时，就需要使用原子更新字段类，Atomic 包提供了以下 3 个类进行原子字段更新。

要想原子地更新字段类需要两步。第一步，因为原子更新字段类都是抽象类，每次使用的时候必须使用静态方法 newUpdater() 创建一个更新器，并且需要设置想要更新的类和属性。第二步，更新类的字段（属性）必须使用 public volatile 修饰符。

AtomicIntegerFieldUpdater :

原子更新整型的字段的更新器。

AtomicLongFieldUpdater :

原子更新长整型字段的更新器。

AtomicReferenceFieldUpdater :

原子更新引用类型里的字段。

4、显式锁和 AQS

显式锁

有了 synchronized 为什么还要 Lock?

Java 程序是靠 synchronized 关键字实现锁功能的，使用 synchronized 关键字将会隐式地获取锁，但是它将锁的获取和释放固化了，也就是先获取再释放。

特 性	描 述
尝试非阻塞地获取锁	当前线程尝试获取锁，如果这一时刻锁没有被其他线程获取到，则成功获取并持有锁
能被中断地获取锁	与 <code>synchronized</code> 不同，获取到锁的线程能够响应中断，当获取到锁的线程被中断时，中断异常将会被抛出，同时锁会被释放
超时获取锁	在指定的截止时间之前获取锁，如果截止时间到了仍旧无法获取锁，则返回

Lock 的标准用法

```
lock.lock();
try {
    count++;
} finally {
    lock.unlock();
}
```

在 `finally` 块中释放锁，目的是保证在获取到锁之后，最终能够被释放。

不要将获取锁的过程写在 `try` 块中，因为如果在获取锁（自定义锁的实现）时发生了异常，异常抛出的同时，也会导致锁无故释放。

Lock 的常用 API

方法名称	描 述
<code>void lock()</code>	获取锁，调用该方法当前线程将会获取锁，当锁获得后，从该方法返回
<code>void lockInterruptibly() throws InterruptedException</code>	可中断地获取锁，和 <code>lock()</code> 方法的不同之处在于该方法会响应中断，即在锁的获取中可以中断当前线程
<code>boolean tryLock()</code>	尝试非阻塞的获取锁，调用该方法后立刻返回，如果能够获取则返回 <code>true</code> ，否则返回 <code>false</code>
<code>boolean tryLock(long time, TimeUnit unit) throws InterruptedException</code>	超时的获取锁，当前线程在以下 3 种情况下会返回： ①当前线程在超时时间内获得了锁 ②当前线程在超时时间内被中断 ③超时时间结束，返回 <code>false</code>
<code>void unlock()</code>	释放锁

ReentrantLock

锁的可重入

简单地讲就是：“同一个线程对于已经获得到的锁，可以多次继续申请到该锁的使用权”。而 `synchronized` 关键字隐式的支持重进入，比如一个 `synchronized` 修饰的递归方法，在方法执行时，执行线程在获取了锁之后仍能连续多次地获得该锁。`ReentrantLock` 在调用 `lock()` 方法时，已经获取到锁的线程，能够再次调用 `lock()` 方法获取锁而不被阻塞。

公平和非公平锁

如果在时间上,先对锁进行获取的请求一定先被满足,那么这个锁是公平的,反之,是不公平的。公平的获取锁,也就是等待时间最长的线程最优先获取锁,也可以说锁获取是顺序的。 `ReentrantLock` 提供了一个构造函数,能够控制锁是否是公平的。事实上,公平的锁机制往往没有非公平的效率高。

在激烈竞争的情况下,非公平锁的性能高于公平锁的性能的一个原因是:在恢复一个被挂起的线程与该线程真正开始运行之间存在着严重的延迟。假设线程 A 持有一个锁,并且线程 B 请求这个锁。由于这个锁已被线程 A 持有,因此 B 将被挂起。当 A 释放锁时,B 将被唤醒,因此会再次尝试获取锁。与此同时,如果 C 也请求这个锁,那么 C 很可能在 B 被完全唤醒之前获得、使用以及释放这个锁。这样的情况是一种“双赢”的局面:B 获得锁的时刻并没有推迟,C 更早地获得了锁,并且吞吐量也获得了提高。

读写锁 `ReentrantReadWriteLock`

之前提到锁(如 `Mutex` 和 `ReentrantLock`)基本都是排他锁,这些锁在同一时刻只允许一个线程进行访问,而读写锁在同一时刻可以允许多个读线程访问,但是在写线程访问时,所有的读线程和其他写线程均被阻塞。读写锁维护了一对锁,一个读锁和一个写锁,通过分离读锁和写锁,使得并发性相比一般的排他锁有了很大提升。

除了保证写操作对读操作的可见性以及并发性的提升之外,读写锁能够简化读写交互场景的编程方式。假设在程序中定义一个共享的用作缓存数据结构,它大部分时间提供读服务(例如查询和搜索),而写操作占有的时间很少,但是写操作完成之后的更新需要对后续的读服务可见。

在没有读写锁支持的(`Java 5`之前)时候,如果需要完成上述工作就要使用 `Java` 的等待通知机制,就是当写操作开始时,所有晚于写操作的读操作均会进入等待状态,只有写操作完成并进行通知之后,所有等待的读操作才能继续执行(写操作之间依靠 `synchronized` 关键进行同步),这样做的目的是使读操作能读取到正确的数据,不会出现脏读。改用读写锁实现上述功能,只需要在读操作时获取读锁,写操作时获取写锁即可。当写锁被获取到时,后续(非当前写操作线程)的读写操作都会被阻塞,写锁释放之后,所有操作继续执行,编程方式相对于使用等待通知机制的实现方式而言,变得简单明了。

一般情况下,读写锁的性能都会比排它锁好,因为大多数场景读是多于写的。在读多于写的情况下,读写锁能够提供比排它锁更好的并发性和吞吐量

`ReentrantReadWriteLock` 其实实现的是 `ReadWriteLock` 接口

Condition 接口

任意一个 `Java` 对象,都拥有一组监视器方法(定义在 `java.lang.Object` 上),主要包括 `wait()`、`wait(long timeout)`、`notify()`以及 `notifyAll()`方法,这些方法与 `synchronized` 同步关键字配合,可以实现等待/通知模式。`Condition` 接口也提供了类似 `Object` 的监视器方法,与 `Lock` 配合可以实现等待/通知模式。

Condition 常用方法

方法名称	描 述
void await() throws InterruptedException	当前线程进入等待状态直到被通知（signal）或中断，当前线程将进入运行状态且从 await() 方法返回的情况，包括： 其他线程调用该 Condition 的 signal() 或 signalAll() 方法，而当前线程被选中唤醒 ☐ 其他线程（调用 interrupt() 方法）中断当前线程 ☐ 如果当前等待线程从 await() 方法返回，那么表明该线程已经获取了 Condition 对象所对应的锁
(续)	
方法名称	描 述
void awaitUninterruptibly()	当前线程进入等待状态直到被通知，从方法名称上可以看出该方法对中断不敏感
long awaitNanos(long nanosTimeout) throws InterruptedException	当前线程进入等待状态直到被通知、中断或者超时。返回值表示剩余的时间，如果在 nanosTimeout 纳秒之前被唤醒，那么返回值就是 (nanosTimeout- 实际耗时)。如果返回值是 0 或者负数，那么可以认定已经超时了
boolean awaitUntil(Date deadline) throws InterruptedException	当前线程进入等待状态直到被通知、中断或者到某个时间。如果没有到指定时间就被通知，方法返回 true，否则，表示到了指定时间，方法返回 false
void signal()	唤醒一个等待在 Condition 上的线程，该线程从等待方法返回前必须获得与 Condition 相关联的锁
void signalAll()	唤醒所有等待在 Condition 上的线程，能够从等待方法返回的线程必须获得与 Condition 相关联的锁

Condition 使用范式

```
Lock lock = new ReentrantLock();
Condition condition = lock.newCondition();
public void conditionWait() throws InterruptedException {
    lock.lock();
    try {
        condition.await();
    } finally {
        lock.unlock();
    }
}
public void conditionSignal() throws InterruptedException {
    lock.lock();
    try {
        condition.signal();
    } finally {
        lock.unlock();
    }
}
```

Condition 使用

参见 `cn.enjoyedu.ch4.condition.ExpressCond`

了解 LockSupport

LockSupport 定义了一组的公共静态方法，这些方法提供了最基本的线程阻塞和唤醒功能，而 LockSupport 也成为构建同步组件的基础工具。

LockSupport 定义了一组以 park 开头的方法用来阻塞当前线程，以及 unpark(Thread thread)方法来唤醒一个被阻塞的线程。LockSupport 增加了 park(Object blocker)、parkNanos(Object blocker,long nanos)和 parkUntil(Object blocker,long deadline)3 个方法，用于实现阻塞当前线程的功能，其中参数 blocker 是用来标识当前线程在等待的对象（以下称为阻塞对象），该对象主要用于问题排查和系统监控。

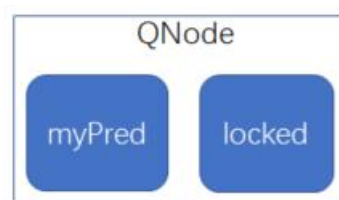
CLH 队列锁

CLH 队列锁即 Craig, Landin, and Hagersten (CLH) locks。

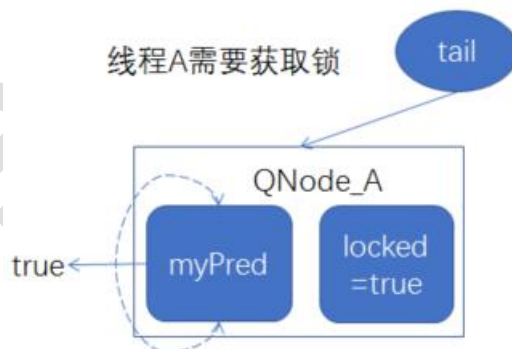
CLH 队列锁也是一种基于链表的可扩展、高性能、公平的自旋锁，申请线程仅仅在本地变量上自旋，它不断轮询前驱的状态，假设发现前驱释放了锁就结束自旋。

当一个线程需要获取锁时：

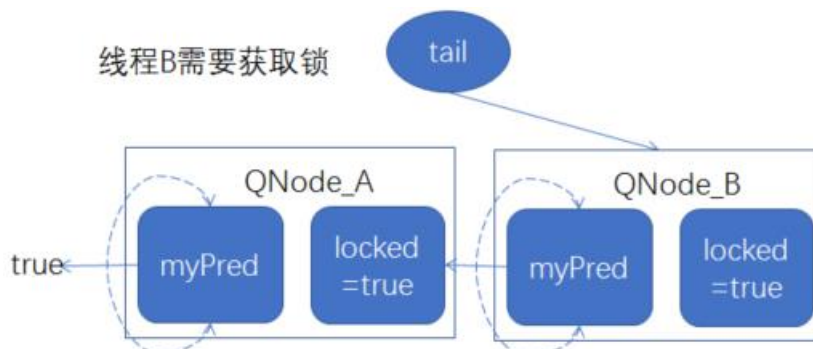
1. 创建一个的 QNode，将其中的 locked 设置为 true 表示需要获取锁，myPred 表示对其前驱结点的引用



2. 线程 A 对 tail 域调用 getAndSet 方法，使自己成为队列的尾部，同时获取一个指向其前驱结点的引用 myPred

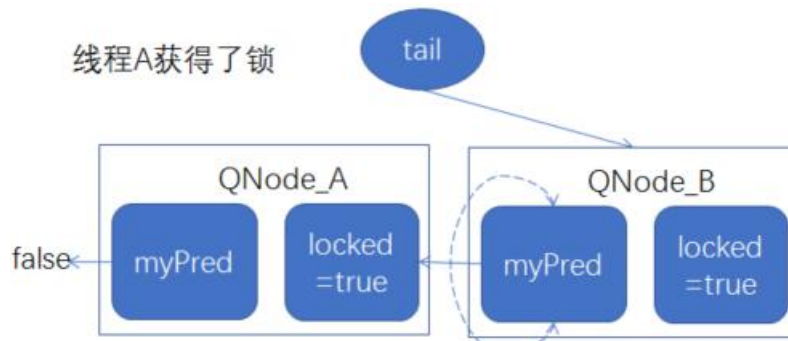


线程 B 需要获得锁，同样的流程再来一遍



3.线程就在前驱结点的 `locked` 字段上旋转，直到前驱结点释放锁(前驱节点的锁值 `locked == false`)

4.当一个线程需要释放锁时，将当前结点的 `locked` 域设置为 `false`，同时回收前驱结点



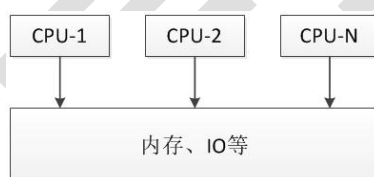
如上图所示，前驱结点释放锁，线程 A 的 `myPred` 所指向的前驱结点的 `locked` 字段变为 `false`，线程 A 就可以获取到锁。

CLH 队列锁的优点是空间复杂度低（如果有 n 个线程， L 个锁，每个线程每次只获取一个锁，那么需要的存储空间是 $O(L+n)$ ， n 个线程有 n 个 `myNode`， L 个锁有 L 个 `tail`）。CLH 队列锁常用在 SMP 体系结构下。

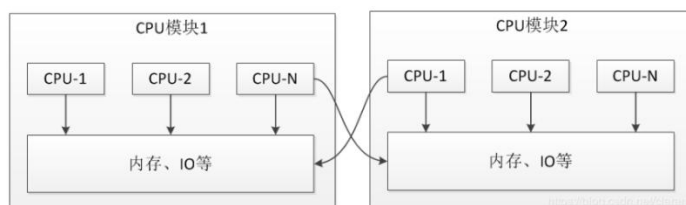
Java 中的 AQS 是 CLH 队列锁的一种变体实现。

扩展知识

SMP(Symmetric Multi-Processor)。即对称多处理器结构，指 `server` 中多个 CPU 对称工作，每一个 CPU 访问内存地址所需时间同样。其主要特征是共享，包括对 CPU，内存，I/O 等进行共享。SMP 的长处是可以保证内存一致性。缺点是这些共享的资源非常可能成为性能瓶颈。随着 CPU 数量的添加，每一个 CPU 都要访问同样的内存资源，可能导致内存访问冲突，可能会导致 CPU 资源的浪费。经常使用的 PC 机就属于这样的。



非一致存储访问，将 CPU 分为 CPU 模块，每个 CPU 模块由多个 CPU 组成，并且具有独立的本地内存、I/O 槽口等，模块之间可以通过互联模块相互访问，访问本地内存（本 CPU 模块的内存）的速度将远远高于访问远地内存（其他 CPU 模块的内存）的速度，这也是非一致存储访问的由来。NUMA 较好地解决 SMP 的扩展问题，当 CPU 数量增加时，因为访问远地内存的延时远远超过本地内存，系统性能无法线性增加。



CLH 唯一的缺点是在 NUMA 系统结构下性能很差，但是在 SMP 系统结构下该法还是非常有效的。解决 NUMA 系统结构的思路是 MCS 队列锁。

AbstractQueuedSynchronizer

学习 AQS 的必要性

队列同步器 AbstractQueuedSynchronizer（以下简称同步器或 AQS），是用来构建锁或者其他同步组件的基础框架，它使用了一个 int 成员变量表示同步状态，通过内置的 FIFO 队列来完成资源获取线程的排队工作。并发包的大师（Doug Lea）期望它能够成为实现大部分同步需求的基础。

AQS 使用方式和其中的设计模式

AQS 的主要使用方式是继承，子类通过继承 AQS 并实现它的抽象方法来管理同步状态，在 AQS 里由一个 int 型的 state 来代表这个状态，在抽象方法的实现过程中免不了要对同步状态进行更改，这时就需要使用同步器提供的 3 个方法（getState()、setState(int newState)和 compareAndSetState(int expect,int update)）来进行操作，因为它们能够保证状态的改变是安全的。

```
/**
 * The synchronization state.
 */
private volatile int state;
```

在实现上，子类推荐被定义为自定义同步组件的静态内部类，AQS 自身没有实现任何同步接口，它仅仅是定义了若干同步状态获取和释放的方法来供自定义同步组件使用，同步器既可以支持独占式地获取同步状态，也可以支持共享式地获取同步状态，这样就可以方便实现不同类型的同步组件（ReentrantLock、ReentrantReadWriteLock 和 CountdownLatch 等）。

同步器是实现锁（也可以是任意同步组件）的关键，在锁的实现中聚合同步器。可以这样理解二者之间的关系：

锁是面向使用者的，它定义了使用者与锁交互的接口（比如可以允许两个线程并行访问），隐藏了实现细节；

同步器面向的是锁的实现者，它简化了锁的实现方式，屏蔽了同步状态管理、线程的排队、等待与唤醒等底层操作。锁和同步器很好地隔离了使用者和实现者所需关注的领域。

实现者需要继承同步器并重写指定的方法，随后将同步器组合在自定义同步组件的实现中，并调用同步器提供的模板方法，而这些模板方法将会调用使用者重写的方法。

模板方法模式

同步器的设计基于模板方法模式。模板方法模式的意图是，定义一个操作中的算法的骨架，而将一些步骤的实现延迟到子类中。模板方法使得子类可以不改

变一个算法的结构即可重定义该算法的某些特定步骤。我们最常见的就是 Spring 框架里的各种 Template。

实际例子

我们开了个蛋糕店，蛋糕店不能只卖一种蛋糕呀，于是我们决定先卖奶油蛋糕，芝士蛋糕和慕斯蛋糕。三种蛋糕在制作方式上一样，都包括造型，烘焙和涂抹蛋糕上的东西。所以可以定义一个抽象蛋糕模型

```
public abstract class AbstractCake {  
    protected abstract void shape();  
    protected abstract void apply();  
    protected abstract void brake();  
}
```

```
/*模板方法*/  
public final void run() {  
    this.shape();  
    this.apply();  
    this.brake();  
}
```

然后就可以批量生产三种蛋糕

```
public class CheeseCake extends AbstractCake {  
    @Override  
    protected void shape() {  
        System.out.println("芝士蛋糕造型");  
    }  
  
    @Override  
    protected void apply() {  
        System.out.println("芝士蛋糕涂抹");  
    }  
  
    @Override  
    protected void brake() {  
        System.out.println("芝士蛋糕烘焙");  
    }  
}  
  
public class CreamCake extends AbstractCake {  
}  
  
public class MouseCake extends AbstractCake {  
}  
  
AbstractCake cake = new CheeseCake();  
//AbstractCake cake = new CreamCake();  
//AbstractCake cake = new MouseCake();  
cake.run();
```

这样一来，不但可以批量生产三种蛋糕，而且如果日后有扩展，只需要继承抽象蛋糕方法就可以了，十分方便，我们天天生意做得越来越赚钱。突然有一天，我们发现市面有一种最简单的小蛋糕销量很好，这种蛋糕就是简单烘烤成型就可以卖，并不需要涂抹什么食材，由于制作简单销售量大，这个品种也很赚钱，于是我们也想要生产这种蛋糕。但是我们发现了一个问题，抽象蛋糕是定义了抽象

的涂抹方法的，也就是说扩展的这种蛋糕是必须要实现涂抹方法，这就很鸡儿蛋疼了。怎么办？我们可以将原来的模板修改为带钩子的模板。

```
/*模板方法*/
public final void run() {
    this.shape();
    if(this.shouldApply()){
        this.apply();
    }
    this.brake();
}

protected boolean shouldApply(){
    return true;
}
```

做小蛋糕的时候通过 flag 来控制是否涂抹，其余已有的蛋糕制作不需要任何修改可以照常进行。

```
public class SmallCake extends AbstractCake {

    private boolean flag = false;
    public void setFlag(boolean shouldApply) {
        flag = shouldApply;
    }
    @Override
    protected boolean shouldApply() {
        return this.flag;
    }
    @Override
    protected void shape() {
        System.out.println("小蛋糕造型");
    }
}
```

AQS 中的方法

模板方法

实现自定义同步组件时，将会调用同步器提供的模板方法，

方法名称	描 述
<code>void acquire(int arg)</code>	独占式获取同步状态，如果当前线程获取同步状态成功，则由该方法返回，否则，将会进入同步队列等待，该方法将会调用重写的 <code>tryAcquire(int arg)</code> 方法
<code>void acquireInterruptibly(int arg)</code>	与 <code>acquire(int arg)</code> 相同，但是该方法响应中断，当前线程未获取到同步状态而进入同步队列中，如果当前线程被中断，则该方法会抛出 <code>InterruptedException</code> 并返回
<code>boolean tryAcquireNanos(int arg, long nanos)</code>	在 <code>acquireInterruptibly(int arg)</code> 基础上增加了超时限制，如果当前线程在超时时间内没有获取到同步状态，那么将会返回 <code>false</code> ，如果获取到了返回 <code>true</code>
<code>void acquireShared(int arg)</code>	共享式的获取同步状态，如果当前线程未获取到同步状态，将会进入同步队列等待，与独占式获取的主要区别是在同一时刻可以有多个线程获取到同步状态
<code>void acquireSharedInterruptibly(int arg)</code>	与 <code>acquireShared(int arg)</code> 相同，该方法响应中断
<code>boolean tryAcquireSharedNanos(int arg, long nanos)</code>	在 <code>acquireSharedInterruptibly(int arg)</code> 基础上增加了超时限制
<code>boolean release(int arg)</code>	独占式的释放同步状态，该方法会在释放同步状态之后，将同步队列中第一个节点包含的线程唤醒
<code>boolean releaseShared(int arg)</code>	共享式的释放同步状态
<code>Collection<Thread> getQueuedThreads()</code>	获取等待在同步队列上的线程集合

这些模板方法同步器提供的模板方法基本上分为 3 类：独占式获取与释放同步状态、共享式获取与释放、同步状态和查询同步队列中的等待线程情况。

可重写的方法

方法名称	描 述
<code>protected boolean tryAcquire(int arg)</code>	独占式获取同步状态，实现该方法需要查询当前状态并判断同步状态是否符合预期，然后再进行 CAS 设置同步状态
<code>protected boolean tryRelease(int arg)</code>	独占式释放同步状态，等待获取同步状态的线程将有机会获取同步状态
<code>protected int tryAcquireShared(int arg)</code>	共享式获取同步状态，返回大于等于 0 的值，表示获取成功，反之，获取失败
<code>protected boolean tryReleaseShared(int arg)</code>	共享式释放同步状态
<code>protected boolean isHeldExclusively()</code>	当前同步器是否在独占模式下被线程占用，一般该方法表示是否被当前线程所独占

访问或修改同步状态的方法

重写同步器指定的方法时，需要使用同步器提供的如下 3 个方法来访问或修改同步状态。

- `getState()`：获取当前同步状态。
- `setState(int newState)`：设置当前同步状态。
- `compareAndSetState(int expect,int update)`：使用 CAS 设置当前状态，该方法能够保证状态设置的原子性。

实现一个自己的独占锁

参见代码 `cn.enjoyedu.ch4.aqs.SelfLock`

深入源码

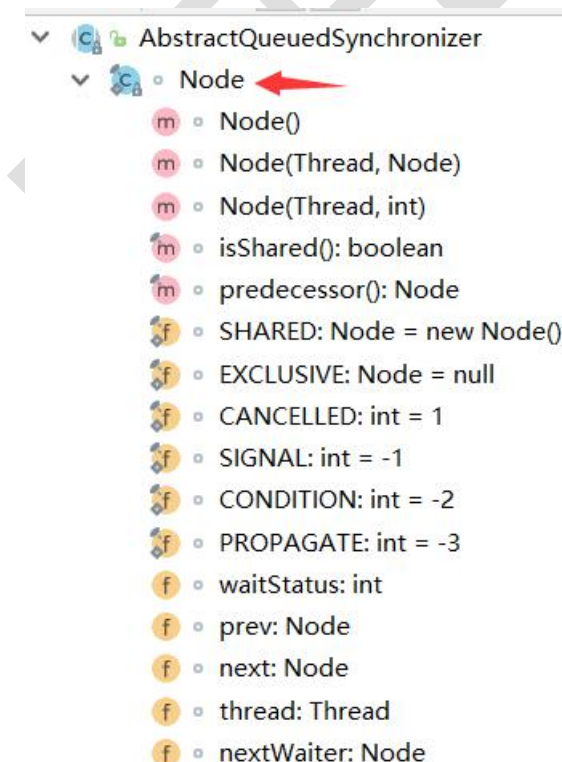
AQS 中的数据结构-节点和同步队列

节点 Node

既然说 Java 中的 AQS 是 CLH 队列锁的一种变体实现，毫无疑问，作为队列来说，必然要有一个节点的数据结构来保存我们前面所说的各种域，比如前驱节点，节点的状态等，这个数据结构就是 AQS 中的内部类 **Node**。作为这个数据结构应该关心些什么信息？

- 1、线程信息，肯定要知道我是哪个线程；
- 2、队列中线程状态，既然知道是哪一个线程，肯定还要知道线程当前处在什么状态，是已经取消了“获锁”请求，还是在“”等待中”，或者说“即将得到锁”
- 3、前驱和后继线程，因为是一个等待队列，那么也就需要知道当前线程前面的是哪个线程，当前线程后面的是哪个线程（因为当前线程释放锁以后，理当立马通知后继线程去获取锁）。

所以这个 **Node** 类是这么设计的：



其中包括了：

线程的 2 种等待模式：

SHARED：表示线程以共享的模式等待锁（如 ReadLock）

EXCLUSIVE：表示线程以互斥的模式等待锁（如 ReentrantLock），互斥就是一把锁只能由一个线程持有，不能同时存在多个线程使用同一个锁

线程在队列中的状态枚举：

CANCELLED：值为 1，表示线程的获锁请求已经“取消”

SIGNAL：值为-1，表示该线程一切都准备好了,就等待锁空闲出来给我

CONDITION：值为-2，表示线程等待某一个条件（Condition）被满足

PROPAGATE：值为-3，当线程处在“SHARED”模式时，该字段才会被使用上

初始化 Node 对象时，默认为 0

成员变量：

waitStatus：该 int 变量表示线程在队列中的状态，其值就是上述提到的 CANCELLED、SIGNAL、CONDITION、PROPAGATE

prev：该变量类型为 Node 对象，表示该节点的前一个 Node 节点（前驱）

next：该变量类型为 Node 对象，表示该节点的后一个 Node 节点（后继）

thread：该变量类型为 Thread 对象，表示该节点代表的线程

nextWaiter：该变量类型为 Node 对象，表示等待 condition 条件的 Node 节点

当前线程获取同步状态失败时，同步器会将当前线程以及等待状态等信息构造成为一个节点（Node）并将其加入同步队列，同时会阻塞当前线程，当同步状态释放时，会把首节点中的线程唤醒，使其再次尝试获取同步状态。同步队列中的节点（Node）用来保存获取同步状态失败的线程引用、等待状态以及前驱和后继节点。

head 和 tail

AQS 还拥有首节点（head）和尾节点（tail）两个引用，一个指向队列头节点，而另一个指向队列尾节点。

注意：因为首节点 head 是不保存线程信息的节点，仅仅是因为数据结构设计上的需要，在数据结构上，这种做法往往叫做“空头节点链表”。对应的就有“非空头结点链表”

节点在同步队列中的增加和移出

节点加入到同步队列

当一个线程成功地获取了同步状态（或者锁），其他线程将无法获取到同步状态，也就是获取同步状态失败，AQS 会将这个线程以及等待状态等信息构造成为一个节点（Node）并将其加入同步队列的尾部。而这个加入队列的过程必须

要保证线程安全，因此同步器提供了一个基于 CAS 的设置尾节点的方法：`compareAndSetTail(Node expect, Node update)`，它需要传递当前线程“认为”的尾节点和当前节点，只有设置成功后，当前节点才正式与之前的尾节点建立关联。

首节点的变化

首节点是获取同步状态成功的节点，首节点的线程在释放同步状态时，将会唤醒后继节点，而后继节点将会在获取同步状态成功时将自己设置为首节点。设置首节点是通过获取同步状态成功的线程来完成的，由于只有一个线程能够成功获取到同步状态，因此设置头节点的方法并不需要使用 CAS 来保证，它只需要将首节点设置成为原首节点的后继节点并断开原首节点的 `next` 引用即可。

独占式同步状态获取与释放

获取

通过调用同步器的 `acquire(int arg)` 方法可以获取同步状态，主要完成了同步状态获取、节点构造、加入同步队列以及在同步队列中自旋等待的相关工作，其主要逻辑是：

```
public final void acquire(int arg) {  
    if (!tryAcquire(arg) &&  
        acquireQueued(addWaiter(Node.EXCLUSIVE), arg))  
        selfInterrupt();  
}
```

首先调用自定义同步器实现的 `tryAcquire(int arg)` 方法，该方法需要保证线程安全的获取同步状态。

如果同步状态获取失败（`tryAcquire` 返回 `false`），则构造同步节点（独占式 `Node.EXCLUSIVE`，同一时刻只能有一个线程成功获取同步状态）并通过 `addWaiter(Node node)` 方法将该节点加入到同步队列的尾部，

最后调用 `acquireQueued(Node node, int arg)` 方法，使得该节点以“死循环”的方式获取同步状态。如果获取不到则阻塞节点中的线程，而被阻塞线程的唤醒主要依靠前驱节点的出队或阻塞线程被中断来实现。

`addWaiter(Node node)` 方法中

```

private Node addWaiter(Node mode) {
    Node node = new Node(Thread.currentThread(), mode);
    // Try the fast path of enq; backup to full enq on failure
    Node pred = tail;
    if (pred != null) {
        node.prev = pred;
        if (compareAndSetTail(pred, node)) {
            pred.next = node;
            return node;
        }
    }
    enq(node);
    return node;
}

```

将当前线程包装成 Node 后，队列不为空的情况下，先尝试把当前节点加入队列并成为尾节点，如果不成功或者队列为空进入 enq(final Node node)方法。

```

private Node enq(final Node node) {
    for (;;) {
        Node t = tail;
        if (t == null) { // Must initialize
            if (compareAndSetHead(new Node()))
                tail = head;
        } else {
            node.prev = t;
            if (compareAndSetTail(t, node)) {
                t.next = node;
                return t;
            }
        }
    }
}

```

在 enq(final Node node)方法中，同步器通过“死循环”来保证节点的正确添加，这个死循环中，做了两件事，第一件，如果队列为空，初始化队列，new 出一个空节点，并让首节点（head）和尾节点（tail）两个引用都指向这个空节点；第二件事，把当前节点加入队列。

在“死循环”中只有通过 CAS 将节点设置成为尾节点之后，当前线程才能从该方法返回，否则，当前线程不断地尝试设置。

节点进入同步队列之后，观察 acquireQueued(Node node,int arg)方法

```

final boolean acquireQueued(final Node node, int arg) {
    boolean failed = true;
    try {
        boolean interrupted = false;
        for (;;) {
            final Node p = node.predecessor();
            if (p == head && tryAcquire(arg)) {
                setHead(node);
                p.next = null; // help GC
                failed = false;
                return interrupted;
            }
            if (shouldParkAfterFailedAcquire(p, node) &&
                parkAndCheckInterrupt())
                interrupted = true;
        }
    } finally {
        if (failed)
            cancelAcquire(node);
    }
}

```

获取到了锁

线程进入阻塞状态，等待被唤醒

其实就是一个自旋的过程，每个节点（或者说每个线程）都在自省地观察，当条件满足，获取到了同步状态，就可以从这个自旋过程中退出，否则依旧留在这个自旋过程中（并会阻塞节点的线程）。

在 `acquireQueued(final Node node, int arg)` 方法中，当前线程在“死循环”中尝试获取同步状态，而只有前驱节点是头节点才能够尝试获取同步状态，这是为什么？原因有两个。

第一，头节点是成功获取到同步状态的节点，而头节点的线程释放了同步状态之后，将会唤醒其后继节点，后继节点的线程被唤醒后需要检查自己的前驱节点是否是头节点。

第二，维护同步队列的 FIFO 原则。

当前线程获取到同步状态后，让首节点（head）这个引用指向自己所在节点。当同步状态获取成功后，当前线程就从 `acquire` 方法返回了。如果同步器实现的是锁，那就代表当前线程获得了锁。

释放

当前线程获取同步状态并执行了相应逻辑之后，就需要释放同步状态，使得后续节点能够继续获取同步状态。通过调用同步器的 `release(int arg)` 方法可以释放同步状态，该方法在释放了同步状态之后，会唤醒其后继节点（进而使后继节点重新尝试获取同步状态）。


```

public final boolean release(int arg) {
    if (tryRelease(arg)) {
        Node h = head;
        if (h != null && h.waitStatus != 0)
            unparkSuccessor(h);
        return true;
    }
    return false;
}

```

该方法执行时，会唤醒首节点（head）所指向节点的后继节点线程，`unparkSuccessor(Node node)`方法使用 `LockSupport` 来唤醒处于等待状态的线程。而在 `unparkSuccessor` 中，

```

Node s = node.next;
if (s == null || s.waitStatus > 0) {
    s = null;
    for (Node t = tail; t != null && t != node; t = t.prev)
        if (t.waitStatus <= 0)
            s = t;
}
if (s != null)
    LockSupport.unpark(s.thread);

```

这段代码的意思，一般情况下，被唤醒的是 `head` 指向节点的后继节点线程，如果这个后继节点处于被 `cancel` 状态，（我推测开发者的思路这样的：后继节点处于被 `cancel` 状态，意味着当锁竞争激烈时，队列的第一个节点等了很久（一直被还未加入队列的节点抢走锁），包括后续的节点 `cancel` 的几率都比较大，所以）先从尾开始遍历，找到最前面且没有被 `cancel` 的节点。

总结

在获取同步状态时，同步器维护一个同步队列，获取状态失败的线程都会被加入到队列中并在队列中进行自旋；移出队列（或停止自旋）的条件是前驱节点为头节点且成功获取了同步状态。在释放同步状态时，同步器调用 `tryRelease(int arg)`方法释放同步状态，然后唤醒 `head` 指向节点的后继节点。

共享式同步状态获取与释放

共享式获取与独占式获取最主要的区别在于同一时刻能否有多个线程同时获取到同步状态。以读写为例，如果一个程序在进行读操作，那么这一时刻写操作均被阻塞，而读操作能够同时进行。写操作要求对资源的独占式访问，而读操作可以是共享式访问。

在 `acquireShared(int arg)`方法中，同步器调用 `tryAcquireShared(int arg)`方法尝试获取同步状态，`tryAcquireShared(int arg)`方法返回值为 `int` 类型，当返回值大于等于 0 时，表示能够获取到同步状态。因此，在共享式获取的自旋过程中，成功获取到同步状态并退出自旋的条件就是 `tryAcquireShared(int arg)`方法返回值大于等于 0。可以看到，在 `doAcquireShared(int arg)`方法的自旋过程中，如果当前节

点的前驱为头节点时，尝试获取同步状态，如果返回值大于等于 0，表示该次获取同步状态成功并从自旋过程中退出。

该方法在释放同步状态之后，将会唤醒后续处于等待状态的节点。对于能够支持多个线程同时访问的并发组件（比如 Semaphore），它和独占式主要区别在于 `tryReleaseShared(int arg)` 方法必须确保同步状态（或者资源数）线程安全释放，一般是通过循环和 CAS 来保证的，因为释放同步状态的操作会同时来自多个线程。

共享式的同步工具类

设计一个同步工具：该工具在同一时刻，只允许至多 3 个线程同时访问，超过 3 个线程的访问将被阻塞。

首先，确定访问模式。TrinityLock 能够在同一时刻支持多个线程的访问，这显然是共享式访问，因此，需要使用同步器提供的 `acquireShared(int args)` 方法和 `Shared` 相关的方法，这就要求 TwinsLock 必须重写 `tryAcquireShared(int args)` 方法和 `tryReleaseShared(int args)` 方法，这样才能保证同步器的共享式同步状态的获取与释放方法得以执行。

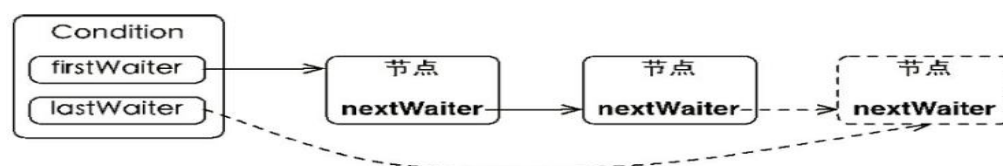
其次，定义资源数。TrinityLock 在同一时刻允许至多三个线程的同时访问，表明同步资源数为 3，这样可以设置初始状态 `status` 为 3，当一个线程进行获取，`status` 减 1，该线程释放，则 `status` 加 1，状态的合法范围为 0、1 和 2,3，其中 0 表示当前已经有 3 个线程获取了同步资源，此时再有其他线程对同步状态进行获取，该线程只能被阻塞。在同步状态变更时，需要使用 `compareAndSet(int expect,int update)` 方法做原子性保障。

最后，组合自定义同步器。前面的章节提到，自定义同步组件通过组合自定义同步器来完成同步功能，一般情况下自定义同步器会被定义为自定义同步组件的内部类。

了解 Condition 的实现

Condition 的数据结构

等待队列是一个 FIFO 的队列，在队列中的每个节点都包含了一个线程引用，该线程就是在 Condition 对象上等待的线程，如果一个线程调用了 `Condition.await()` 方法，那么该线程将会释放锁、构造成节点加入等待队列并进入等待状态。事实上，节点的定义复用了同步器中节点的定义，也就是说，同步队列和等待队列中节点类型都是同步器的静态内部类。



一个 Condition 包含一个等待队列，Condition 拥有首节点（`firstWaiter`）和尾节点（`lastWaiter`）。当前线程调用 `Condition.await()` 方法，将会以当前线程构造节点，并将节点从尾部加入等待队列。Condition 拥有首尾节点的引用，而新增

调用 `Condition` 的 `signal()` 方法，将会唤醒在等待队列中等待时间最长的节点（首节点），在唤醒节点之前，会将节点移到同步队列中。

调用该方法的前置条件是当前线程必须获取了锁，可以看到 `signal()` 方法进行了 `isHeldExclusively()` 检查，也就是当前线程必须是获取了锁的线程。接着获取等待队列的首节点，将其移动到同步队列并使用 `LockSupport` 唤醒节点中的线程。

通过调用同步器的 `enq(Node node)` 方法，等待队列中的头节点线程安全地移动到同步队列。当节点移动到同步队列后，当前线程再使用 `LockSupport` 唤醒该节点的线程。

被唤醒后的线程，将从 `await()` 方法中的 `while` 循环中退出（`isOnSyncQueue(Node node)` 方法返回 `true`，节点已经在同步队列中），进而调用同步器的 `acquireQueued()` 方法加入到获取同步状态的竞争中。

成功获取同步状态（或者说锁）之后，被唤醒的线程将从先前调用的 `await()` 方法返回，此时该线程已经成功地获取了锁。

`Condition` 的 `signalAll()` 方法，相当于对等待队列中的每个节点均执行一次 `signal()` 方法，效果就是将等待队列中所有节点全部移动到同步队列中，并唤醒每个节点的线程。

回头看 Lock 的实现

ReentrantLock 的实现

锁的可重入

重进入是指任意线程在获取到锁之后能够再次获取该锁而不会被锁所阻塞，该特性的实现需要解决以下两个问题。

1) 线程再次获取锁。锁需要去识别获取锁的线程是否为当前占据锁的线程，如果是，则再次成功获取。

2) 锁的最终释放。线程重复 n 次获取了锁，随后在第 n 次释放该锁后，其他线程能够获取到该锁。锁的最终释放要求锁对于获取进行计数自增，计数表示当前锁被重复获取的次数，而锁被释放时，计数自减，当计数等于 0 时表示锁已经成功释放。

`nonfairTryAcquire` 方法增加了再次获取同步状态的处理逻辑：通过判断当前线程是否为获取锁的线程来决定获取操作是否成功，如果是获取锁的线程再次请求，则将同步状态值进行增加并返回 `true`，表示获取同步状态成功。同步状态表示锁被一个线程重复获取的次数。

如果该锁被获取了 n 次，那么前 $(n-1)$ 次 `tryRelease(int releases)` 方法必须返回 `false`，而只有同步状态完全释放了，才能返回 `true`。可以看到，该方法将同步状态是否为 0 作为最终释放的条件，当同步状态为 0 时，将占有线程设置为 `null`，并返回 `true`，表示释放成功。

公平和非公平锁

`ReentrantLock` 的构造函数中，默认的非参构造函数将会把 `Sync` 对象创建为 `NonfairSync` 对象，这是一个“非公平锁”；而另一个构造函数 `ReentrantLock(boolean fair)` 传入参数为 `true` 时将会把 `Sync` 对象创建为“公平锁” `FairSync`。

`nonfairTryAcquire(int acquires)` 方法，对于非公平锁，只要 CAS 设置同步状态成功，则表示当前线程获取了锁，而公平锁则不同。`tryAcquire` 方法，该方法与 `nonfairTryAcquire(int acquires)` 比较，唯一不同的位置为判断条件多了 `hasQueuedPredecessors()` 方法，即加入了同步队列中当前节点是否有前驱节点的判断，如果该方法返回 `true`，则表示有线程比当前线程更早地请求获取锁，因此需要等待前驱线程获取并释放锁之后才能继续获取锁。

改造我们的独占锁为可重入

参见代码 `cn.enjoyedu.ch4.aqs.ReenterSelfLock`

ReentrantReadWriteLock 的实现

读写状态的设计

读写锁同样依赖自定义同步器来实现同步功能，而读写状态就是其同步器的同步状态。

回想 `ReentrantLock` 中自定义同步器的实现，同步状态表示锁被一个线程重复获取的次数，而读写锁的自定义同步器需要在同步状态（一个整型变量）上维护多个读线程和一个写线程的状态，使得该状态的设计成为读写锁实现的关键。

如果在一个整型变量上维护多种状态，就一定需要“按位切割使用”这个变量，读写锁将变量切分成了两个部分，高 16 位表示读，低 16 位表示写，读写锁是如何迅速确定读和写各自的状态呢？

答案是通过位运算。假设当前同步状态值为 `S`，写状态等于 `S&0x0000FFFF`（将高 16 位全部抹去），读状态等于 `S>>>16`（无符号补 0 右移 16 位）。当写状态增加 1 时，等于 `S+1`，当读状态增加 1 时，等于 `S+(1<<16)`，也就是 `S+0x00010000`。根据状态的划分能得出一个推论：`S` 不等于 0 时，当写状态（`S&0x0000FFFF`）等于 0 时，则读状态（`S>>>16`）大于 0，即读锁已被获取。

写锁的获取与释放

写锁是一个支持重进入的排它锁。如果当前线程已经获取了写锁，则增加写状态。如果当前线程在获取写锁时，读锁已经被获取（读状态不为 0）或者该线程不是已经获取写锁的线程，则当前线程进入等待状态。

该方法除了重入条件（当前线程为获取了写锁的线程）之外，增加了一个读锁是否存在的判断。如果存在读锁，则写锁不能被获取，原因在于：读写锁要确保写锁的操作对读锁可见，如果允许读锁在已被获取的情况下对写锁的获取，那

么正在运行的其他读线程就无法感知到当前写线程的操作。因此，只有等待其他读线程都释放了读锁，写锁才能被当前线程获取，而写锁一旦被获取，则其他读写线程的后续访问均被阻塞。

写锁的释放与 `ReentrantLock` 的释放过程基本类似，每次释放均减少写状态，当写状态为 0 时表示写锁已被释放，从而等待的读写线程能够继续访问读写锁，同时前次写线程的修改对后续读写线程可见。

读锁的获取与释放

读锁是一个支持重进入的共享锁，它能够被多个线程同时获取，在没有其他写线程访问（或者写状态为 0）时，读锁总会被成功地获取，而所做的也只是（线程安全的）增加读状态。如果当前线程已经获取了读锁，则增加读状态。

如果当前线程在获取读锁时，写锁已被其他线程获取，则进入等待状态。读状态是所有线程获取读锁次数的总和，而每个线程各自获取读锁的次数只能选择保存在 `ThreadLocal` 中，由线程自身维护。在 `tryAcquireShared(int unused)` 方法中，如果其他线程已经获取了写锁，则当前线程获取读锁失败，进入等待状态。如果当前线程获取了写锁或者写锁未被获取，则当前线程（线程安全，依靠 CAS 保证）增加读状态，成功获取读锁。读锁的每次释放（线程安全的，可能有多个读线程同时释放读锁）均减少读状态。

锁的升降级

锁降级指的是写锁降级成为读锁。如果当前线程拥有写锁，然后将其释放，最后再获取读锁，这种分段完成的过程不能称之为锁降级。

锁降级是指把持住（当前拥有的）写锁，再获取到读锁，随后释放（先前拥有的）写锁的过程。

`ReentrantReadWriteLock` 不支持锁升级（把持读锁、获取写锁，最后释放读锁的过程）。目的是保证数据可见性，如果读锁已被多个线程获取，其中任意线程成功获取了写锁并更新了数据，则其更新对其他获取到读锁的线程是不可见的。

5、并发容器

预备知识

hash

就是把任意长度的输入（又叫做预映射，`pre-image`），通过散列算法，转换成固定长度的输出，该输出就是散列值。这种转换是一种压缩映射，也就是，散列值的空间通常远小于输入的空间，不同的输入可能会散列成相同的输出，所以不可能从散列值来确定唯一的输入值。简单的说就是一种将任意长度的消息压缩到某一固定长度的消息摘要的函数。常用 HASH 函数：直接取余法、乘法取整法、平方取中法。

处理冲突方法:

1. [开放寻址法](#);
2. 再[散列法](#);
3. 链地址法(拉链法)

常用 hash 算法的介绍: (1)MD4 (2)[MD5](#) 它对输入仍以 512 位分组, 其输出是 4 个 32 位字的级联 (3)[SHA-1](#) 及其他。

位运算

二进制

从小学一年级开始, 我们就开始学加减乘除, 特别是学加法的时候, 刚开始算超过十的时候, 如果不熟练, 我们会掰着手指头数, 有时候恨不得把脚趾头也加上。正是因为我们一般人有十个手指头, 所以, 从我们老老。。。祖先开始就用十进制。那计算机怎么办? 计算机又没十个手指头。自然界要找出有十种状态的东西很难, 但是只有两种状态的东西或者现象还是很容易的, 所以计算机中要用二进制。

逢十进一, 意味着至少两点, 1, 每个位上的数字不能超过 10, 最大只能到 9, 第二, 如果某个位上的数字因为运算超过了 10, 怎么办? 往高位进位。那么二进制也是一样的, 不能说你二进制比较二, 规则就不一样了, 基本法还是要遵守的。所以二进制里同样要遵循: 1, 每个位上的数字不能超过 2, 最大只能到 1, 第二, 如果某个位上的数字因为运算超过了 2, 怎么办? 往高位进位。

十进制: 逢十进一

$$208 = 2 \times 100 + 0 \times 10 + 8 \times 1 \\ = 2 \times 10^2 + 0 \times 10^1 + 8 \times 10^0$$

高位		低位
2	1	0
2	0	8
$2 \times 10 \times 10$ (10的2次方)	0×10 (10的1次方)	8×1 (10的0次方)

二进制: 逢二进一

$$208 = 1 \times 2^7 + 1 \times 2^6 + 0 \times 2^5 + 1 \times 2^4 + 0 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 0 \times 2^0 \\ = 1 \times 128 + 1 \times 64 + 0 \times 32 + 1 \times 16 + 0 \times 8 + 0 \times 4 + 0 \times 2 + 0 \times 0$$

高位	7	6	5	4	3	2	1	0	低位
	1	1	0	1	0	0	0	0	
	1×2 的7次方	1×2 的6次方	0×2 的5次方	1×2 的4次方	0×2 的3次方	0×2 的2次方	0×2 的1次方	0×2 的0次方	

常用位运算

位与 & ($1 \& 1 = 1$ $0 \& 0 = 0$ $1 \& 0 = 0$)

位或 | ($1 \mid 1 = 1$ $0 \mid 0 = 0$ $1 \mid 0 = 1$)

位非 ~ ($\sim 1 = 0$ $\sim 0 = 1$)

位异或 ^ ($1 \wedge 1 = 0$ $1 \wedge 0 = 1$ $0 \wedge 0 = 0$)

有符号右移 >> (若正数, 高位补 0, 负数, 高位补 1)

有符号左移 <<

无符号右移 >>> (不论正负, 高位均补 0)

有趣的取模性质：取模 $a \% (2^n)$ 等价于 $a \& (2^n - 1)$ ，所以在 map 里的数组个数一定是 2 的乘方数，计算 key 值在哪个元素中的时候，就用位运算来快速定位。

具体位运算的运行结果参见代码 `cn.enjoyedu.ch5.bitwise.IntToBinary`

位运算运用场景

Java 中的类修饰符、成员变量修饰符、方法修饰符，比如 Class 类中

Java 容器中的 HashMap 和 ConcurrentHashMap 的实现

权限控制或者商品属性

简单可逆加密，比如异或运算($1 \wedge 1 = 0$; $0 \wedge 1 = 1$)

实战：权限控制

参见代码 `cn.enjoyedu.ch5.bitwise.Permission`

使用位运算的优劣势：

节省很多代码量、效率高、属性变动影响小、不直观

为什么要使用 ConcurrentHashMap

1.7 中 HashMap 死循环分析

在多线程环境下，使用 HashMap 进行 put 操作会引起死循环，导致 CPU 利用率接近 100%，HashMap 在并发执行 put 操作时会引起死循环，是因为多线程会导致 HashMap 的 Entry 链表

形成环形数据结构，一旦形成环形数据结构，Entry 的 next 节点永远不为空，就会产生死循环获取 Entry。那么这个死循环是如何生成的呢？我们来仔细分析一下。

HashMap 扩容流程

原理

引发死循环，是在 HashMap 的扩容操作中。

正常的扩容操作是这个流程。HashMap 的扩容在 put 操作中会触发扩容，主要是三个方法：

```

void addEntry(int hash, K key, V value, int bucketIndex) {
    if ((size >= threshold) && (null != table[bucketIndex])) {
        resize(newCapacity: 2 * table.length);
        hash = (null != key) ? hash(key) : 0;
        bucketIndex = indexFor(hash, table.length);
    }
}

void resize(int newCapacity) {
    Entry[] oldTable = table;
    int oldCapacity = oldTable.length;
    if (oldCapacity == MAXIMUM_CAPACITY) {
        threshold = Integer.MAX_VALUE;
        return;
    }

    Entry[] newTable = new Entry[newCapacity];
    transfer(newTable, initHashSeedAsNeeded(newCapacity));
    table = newTable;
}

void transfer(Entry[] newTable, boolean rehash) {
    int newCapacity = newTable.length;
    for (Entry<K,V> e : table) {
        while(null != e) {
            Entry<K,V> next = e.next;
            if (rehash) {
                e.hash = null == e.key ? 0 : hash(e.key);
            }
            int i = indexFor(e.hash, newCapacity);
            e.next = newTable[i];
            newTable[i] = e;
            e = next;
        }
    }
}

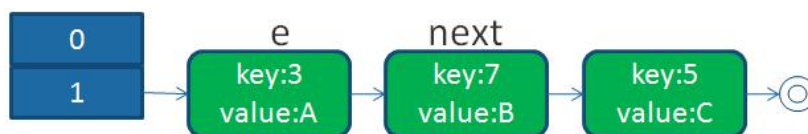
```

综合来说，HashMap 一次扩容的过程：

- 1、取当前 table 的 2 倍作为新 table 的大小
- 2、根据算出的新 table 的大小 new 出一个新的 Entry 数组来，名为 newTable
- 3、轮询原 table 的每一个位置，将每个位置上连接的 Entry，算出在新 table 上的位置，并以链表形式连接
- 4、原 table 上的所有 Entry 全部轮询完毕之后，意味着原 table 上面的所有 Entry 已经移到了新的 table 上，HashMap 中的 table 指向 newTable

实例

现在 hashmap 中有三个元素，Hash 表的 size=2，所以 key = 3, 7, 5，在 mod 2 以后都冲突在 table[1]这里了。



按照方法中的代码

```

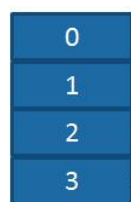
for (Entry<K,V> e : table) {
    while(null != e) {
        Entry<K,V> next = e.next;
        if (rehash) {
            e.hash = null == e.key ? 0 : hash(e.key);
        }
        int i = indexFor(e.hash, newCapacity);
        e.next = newTable[i];
        newTable[i] = e;
        e = next;
    }
}

```

对 table[1]中的链表来说, 进入 while 循环, 此时 e=key(3), 那么 next=key(7), 经过计算重新定位 e=key(3)在新表中的位置, 并把 e=key(3)挂在 newTable[3]的位置

The screenshot shows the `transfer` method in `HashMap`. Annotations in red explain the process:

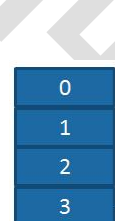
- `int i = indexFor(e.hash, newCapacity);` is annotated with `i=3`.
- `e.next = newTable[i];` is annotated with "按照头插法, key(3)的next指向newTable[3]此时的元素, 当然现在为null".
- `e = next;` is annotated with "现在将e指向next, 也就是key(7)".
- A red arrow points to `newTable[i]` with the text "将key(3)挂在newTable[3]的位置".



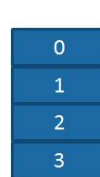
第1次



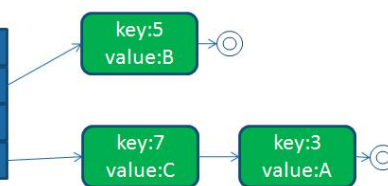
这样循环下去, 将 table[1]中的链表循环完成后, 于是 HashMap 就完成了扩容



第2次



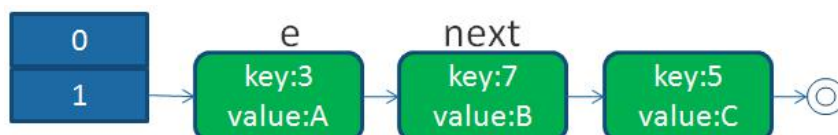
第3次



并发下的扩容

上面都是单线程下的扩容, 当多线程进行扩容时, 会是什么样子呢?

初始的 HashM 还是:



我们现在假设有两个线程并发操作，都进入了扩容操作，

线程1

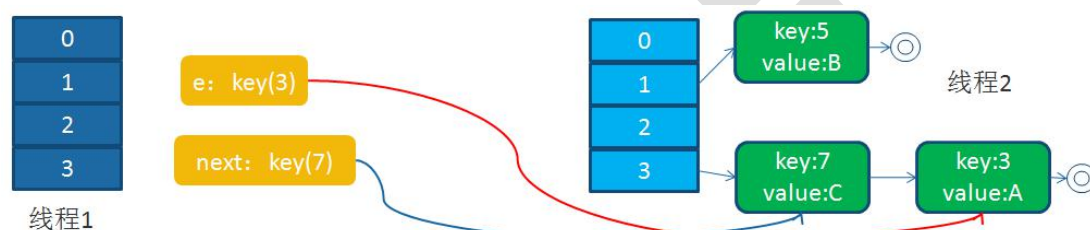
线程2

我们以颜色进行区分两个线程。

回顾我们的扩容代码，我们假设，线程1执行到 `Entry<K,V> next = e.next;` 时被操作系统调度挂起了，而线程2执行完成了扩容操作

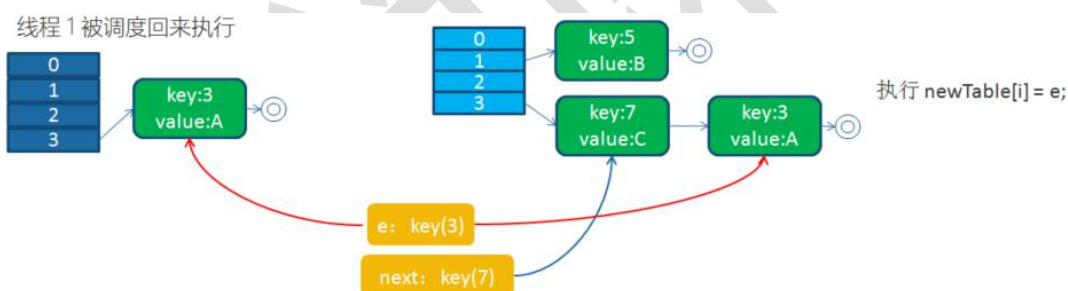
```
void transfer(Entry[] newTable, boolean rehash) {  
    int newCapacity = newTable.length;  
    for (Entry<K,V> e : table) {  
        while (null != e) {  
            Entry<K,V> next = e.next; ← 线程1执行到这里被挂起  
            if (rehash) {  
                e.hash = null == e.key ? 0 : hash(e.key);  
            }  
            newTable[e.key] = e;  
            e = next;  
        }  
    }  
}
```

于是，在线程1,2看来，就应该是这个样子



接下来，线程1被调度回来执行：

1)

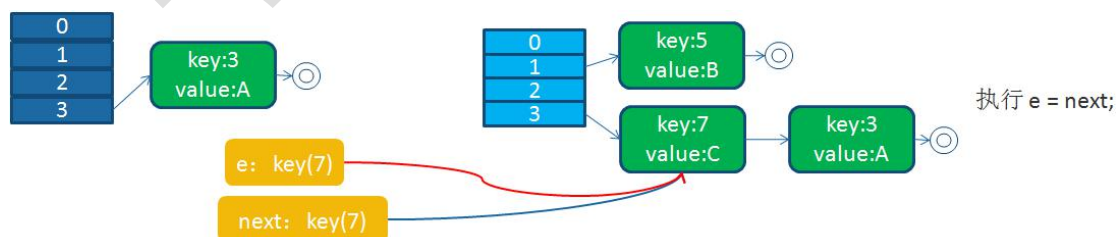


注意：

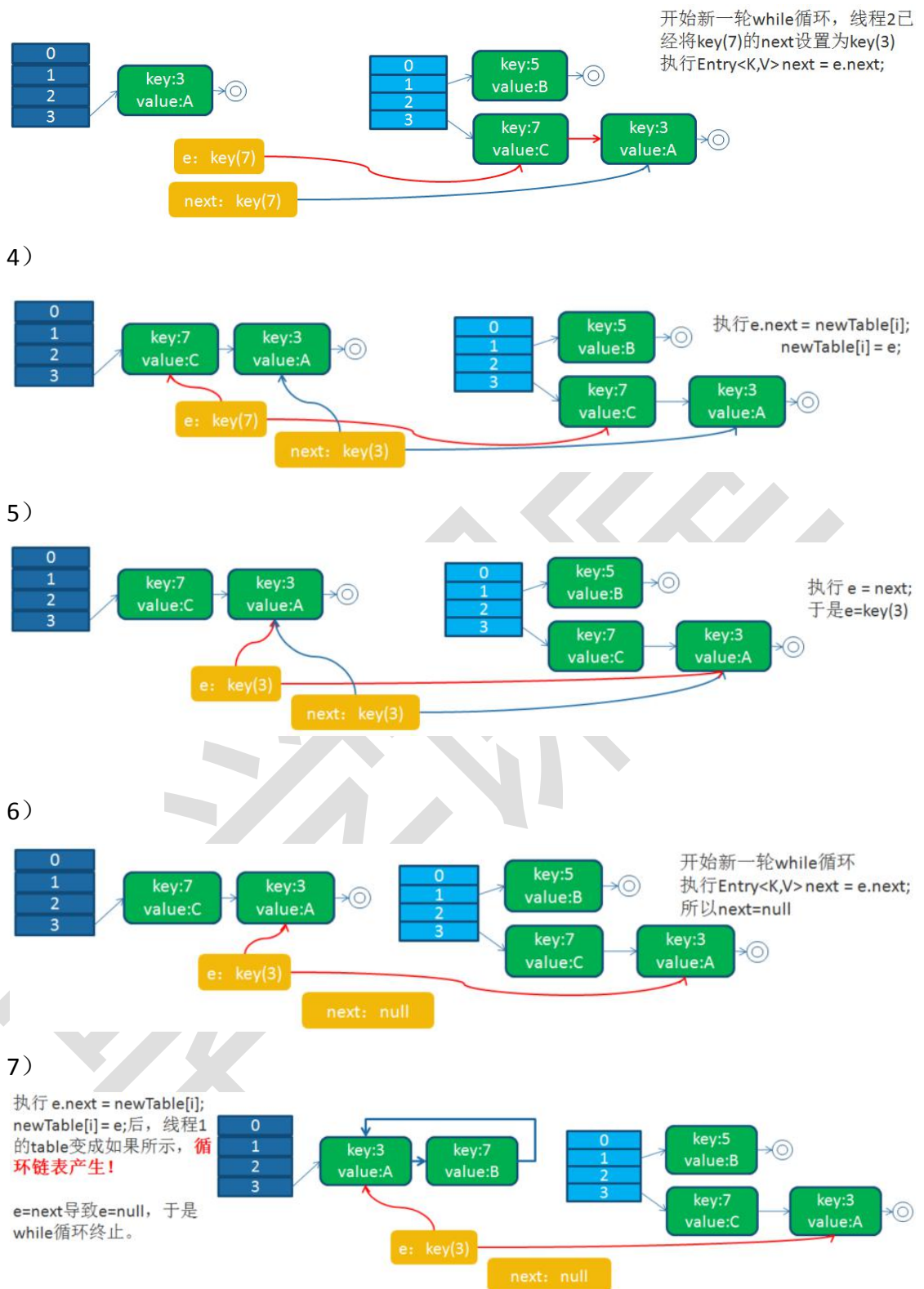
key:5 value:B key:7 value:C key:3 value:A

在内存中仅有一份实例,HashMap的table中拥有的只是这些元素的一份引用

2)



3)



循环列表产生后，一旦线程 1 调用 get (11,15 之类的元素) 时，就会进入一个死循环的情况，将 CPU 的消耗到 100%。

总结

HashMap 之所以在并发下的扩容造成死循环，是因为，多个线程并发进行时，因为一个线程先期完成了扩容，将原 Map 的链表重新散列到自己的表中，并且链表变成了倒序，后一个线程再扩容时，又进行自己的散列，再次将倒序链表变为正序链表。于是形成了一个环形链表，当 get 表中不存在的元素时，造成死循环。

ConcurrentHashMap

使用

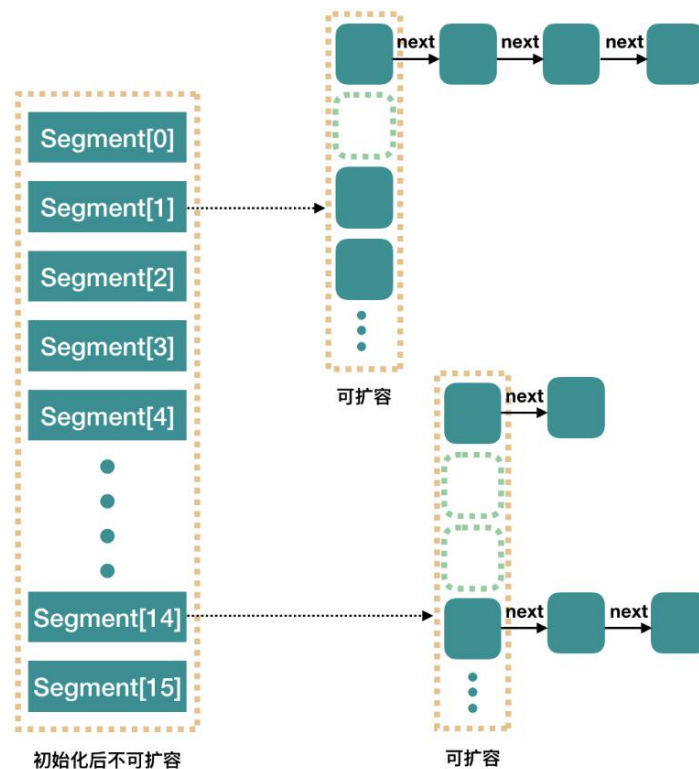
除了 Map 系列应该有的线程安全的 get, put 等方法外，ConcurrentHashMap 还提供了一个在并发下比较有用的方法 putIfAbsent，如果传入 key 对应的 value 已经存在，就返回存在的 value，不进行替换。如果不存在，就添加 key 和 value，返回 null。在代码层面它的作用类似于：

```
synchronized(map){
    if (map.get(key) == null){
        return map.put(key, value);
    } else{
        return map.get(key);
    }
}
```

它让上述代码的整个操作是线程安全的。

ConcurrentHashMap 实现分析

在 1.7 下的实现



```
static final class Segment<K,V> extends ReentrantLock
```

ConcurrentHashMap 是由 Segment 数组结构和 HashEntry 数组结构组成。Segment 是一种可重入锁（ReentrantLock），在 ConcurrentHashMap 里扮演锁的角色；HashEntry 则用于存储键值对数据。一个 ConcurrentHashMap 里包含一个 Segment 数组。Segment 的结构和 HashMap 类似，是一种数组和链表结构。一个 Segment 里包含一个 HashEntry 数组，每个 HashEntry 是一个链表结构的元素，每个 Segment 守护着一个 HashEntry 数组里的元素，当对 HashEntry 数组的数据进行修改时，必须首先获得与它对应的 Segment 锁。

构造方法和初始化

```
public ConcurrentHashMap(int initialCapacity,  
                        float loadFactor, int concurrencyLevel) {
```

ConcurrentHashMap 初始化方法是通过 initialCapacity、loadFactor 和 concurrencyLevel(参数 concurrencyLevel 是用户估计的并发级别，就是你觉得最多有多少线程共同修改这个 map，根据这个来确定 Segment 数组的大小 concurrencyLevel 默认是 DEFAULT_CONCURRENCY_LEVEL = 16;)等几个参数来初始化 segment 数组、段偏移量 segmentShift、段掩码 segmentMask 和每个 segment 里的 HashEntry 数组来实现的。

并发级别可以理解为程序运行时能够同时更新 `ConcurrentHashMap` 且不产生锁竞争的最大线程数，实际上就是 `ConcurrentHashMap` 中的分段锁个数，即 `Segment[]` 的数组长度。`ConcurrentHashMap` 默认的并发度为 16，但用户也可以在构造函数中设置并发度。当用户设置并发度时，`ConcurrentHashMap` 会使用大于等于该值的最小 2 幂指数作为实际并发度（假如用户设置并发度为 17，实际并发度则为 32）。

如果并发度设置的过小，会带来严重的锁竞争问题；如果并发度设置的过大，原本位于同一个 `Segment` 内的访问会扩散到不同的 `Segment` 中，CPU cache 命中率会下降，从而引起程序性能下降。（文档的说法是根据你并发的线程数量决定，太多会导致性能降低）

`segments` 数组的长度 `ssize` 是通过 `concurrencyLevel` 计算得出的。为了能通过按位与的散列算法来定位 `segments` 数组的索引，必须保证 `segments` 数组的长度是 2 的 N 次方（power-of-two size），所以必须计算出一个大于或等于 `concurrencyLevel` 的最小的 2 的 N 次方值来作为 `segments` 数组的长度。假如 `concurrencyLevel` 等于 14、15 或 16，`ssize` 都会等于 16，即容器里锁的个数也是 16。

以下知识了解即可，无需深究

初始化 `segmentShift` 和 `segmentMask`

这两个全局变量需要在定位 `segment` 时的散列算法里使用，`sshift` 等于 `ssize` 从 1 向左移位的次数，在默认情况下 `concurrencyLevel` 等于 16，1 需要向左移位移动 4 次，所以 `sshift` 等于 4。`segmentShift` 用于定位参与散列运算的位数，`segmentShift` 等于 32 减 `sshift`，所以等于 28，这里之所以用 32 是因为 `ConcurrentHashMap` 里的 `hash()` 方法输出的最大数是 32 位的。`segmentMask` 是散列运算的掩码，等于 `ssize` 减 1，即 15，掩码的二进制各个位的值都是 1。因为 `ssize` 的最大长度是 65536，所以 `segmentShift` 最大值是 16，`segmentMask` 最大值是 65535，对应的二进制是 16 位，每个位都是 1。

```
int cap = MIN_SEGMENT_TABLE_CAPACITY;
while (cap < c)
    cap <<= 1;
// create segments and segments[0]
Segment<K,V> s0 =
    new Segment<K,V>(loadFactor, (int)(cap * loadFactor),
        (HashEntry<K,V>[]) new HashEntry[cap]);
Segment<K,V>[] ss = (Segment<K,V>[]) new Segment[ssize];
UNSAFE.putOrderedObject(ss, SBASE, s0); // ordered write of segmen
this.segments = ss;
```

输入参数 `initialCapacity` 是 `ConcurrentHashMap` 的初始化容量，`loadfactor` 是每个 `segment` 的负载因子，在构造方法里需要通过这两个参数来初始化数组中的每个 `segment`。上面代码中的变量 `cap` 就是 `segment` 里 `HashEntry` 数组的长度，它等于 `initialCapacity` 除以 `ssize` 的倍数 `c`，如果 `c` 大于 1，就会取大于等于 `c` 的 2 的 N 次方值，所以 `segment` 里 `HashEntry` 数组的长度不是 1，就是 2 的 N 次方。

在默认情况下，`ssize = 16`，`initialCapacity = 16`，`loadFactor = 0.75f`，那么 `cap = 1`，`threshold = (int) cap * loadFactor = 0`。

既然 ConcurrentHashMap 使用分段锁 Segment 来保护不同段的数据，那么在插入和获取元素的时候，必须先通过散列算法定位到 Segment。

ConcurrentHashMap 会首先使用 Wang/Jenkins hash 的变种算法对元素的 hashCode 进行一次再散列。

ConcurrentHashMap 完全允许多个读操作并发进行，读操作并不需要加锁。ConcurrentHashMap 实现技术是保证 HashEntry 几乎是不可变的以及 volatile 关键字。

```
static final class HashEntry<K,V> {
    final int hash;
    final K key;
    volatile V value;
    volatile HashEntry<K,V> next;
```

get 操作

```
public V get(Object key) {
    Segment<K,V> s; // manually integrate access methods to reduce overhead
    HashEntry<K,V>[] tab;
    int h = hash(key); // 准备定位的hash值
    long u = (((h >>> segmentShift) & segmentMask) << SSHIFT) + SBASE;
    if ((s = (Segment<K,V>) UNSAFE.getObjectVolatile(segments, u)) != null &&
        (tab = s.table) != null) {
        for (HashEntry<K,V> e = (HashEntry<K,V>) UNSAFE.getObjectVolatile(
            tab, l: ((long) ((tab.length - 1) & h)) << TSHIFT) + TBASE);
            e != null; e = e.next) { // 遍历table下指定的HashEntry链表
                K k;
                if ((k = e.key) == key || (e.hash == h && key.equals(k)))
                    return e.value;
            }
        }
    }
    return null;
}
```

get 操作先经过一次再散列，然后使用这个散列值通过散列运算定位到 Segment(使用了散列值的高位部分)，再通过散列算法定位到 table(使用了散列值的全部)。整个 get 过程，没有加锁，而是通过 volatile 保证 get 总是可以拿到最新值。

```
/**
 * The per-segment table. Elements are accessed via
 * entryAt/setEntryAt providing volatile semantics.
 */
transient volatile HashEntry<K,V>[] table;
```

put 操作

```
@SuppressWarnings("unchecked")
public V put(K key, V value) {
    Segment<K,V> s;
    if (value == null) // 定位所需hash值
        throw new NullPointerException();
    int hash = hash(key);
    int j = (hash >>> segmentShift) & segmentMask; // 初始化segment[j], 因为整个map初始化时, 只初始化了segment[0]
    if ((s = (Segment<K,V>) UNSAFE.getObjectVolatile(segments, l: (j << SSHIFT) + SBASE)) == null) // nonvolatile; recheck
        s = ensureSegment(j); // in ensureSegment
    return s.put(key, hash, value, onlyIfAbsent: false);
}
```

ConcurrentHashMap 初始化的时候会初始化第一个槽 segment[0], 对于其他槽, 在插入第一个值的时候再进行初始化。

ensureSegment 方法考虑了并发情况, 多个线程同时进入初始化同一个槽 segment[k], 但只要有一个成功就可以了。

具体实现是

```
private Segment<K,V> ensureSegment(int k) {
    final Segment<K,V>[] ss = this.segments;
    long u = (k << SSHIFT) + SBASE; // raw offset
    Segment<K,V> seg;
    if ((seg = (Segment<K,V>) UNSAFE.getObjectVolatile(ss, u)) == null) {
        Segment<K,V> proto = ss[0]; // use segment 0 as prototype
        int cap = proto.table.length;
        float lf = proto.loadFactor;
        int threshold = (int)(cap * lf); // 多次检查
        HashEntry<K,V>[] tab = (HashEntry<K,V>[]) new HashEntry[cap];
        if ((seg = (Segment<K,V>) UNSAFE.getObjectVolatile(ss, u))
            == null) { // recheck
            Segment<K,V> s = new Segment<K,V>(lf, threshold, tab);
            while ((seg = (Segment<K,V>) UNSAFE.getObjectVolatile(ss, u))
                == null) {
                if (UNSAFE.compareAndSwapObject(ss, u, 0L, null, seg = s))
                    break;
            }
        }
    }
    return seg;
}
```

循环CAS操作, 保证多线程下只有一个线程可以成功

```
final V put(K key, int hash, V value, boolean onlyIfAbsent) {
    HashEntry<K,V> node = tryLock() ? null :
        scanAndLockForPut(key, hash, value);
    V oldValue;
```

put 方法会通过 tryLock() 方法尝试获得锁, 获得了锁, node 为 null 进入 try 语句块, 没有获得锁, 调用 scanAndLockForPut 方法自旋等待获得锁。

```
while (!tryLock()) {
    HashEntry<K,V> f; // to recheck first below
    if (retries < 0) {
        if (e == null) {
            if (node == null) // speculatively create node
                node = new HashEntry<K,V>(hash, key, value,
                    retries = 0);
        }
        else if (key.equals(e.key))
            retries = 0;
        else
            e = e.next;
    }
    else if (++retries > MAX_SCAN_RETRIES) {
        lock();
        break;
    }
    else if ((retries & 1) == 0 &&
        (f = entryForHash(seg: this, hash)) != first) {
        e = first = f; // re-traverse if entry changed
        retries = -1;
    }
}
return node;
```

scanAndLockForPut 方法里在尝试获得锁的过程中会对对应 hashCode 的链表进行遍历，如果遍历完毕仍然找不到与 key 相同的 HashEntry 节点，则为后续的 put 操作提前创建一个 HashEntry。当 tryLock 一定次数后仍无法获得锁，则通过 lock 申请锁。

```
for (HashEntry<K,V> e = first;;) {
    if (e != null) {
        K k;
        if ((k = e.key) == key ||
            (e.hash == hash && key.equals(k))) {
            oldValue = e.value;
            if (!onlyIfAbsent) {
                e.value = value;
                ++modCount;
            }
            break;
        }
        e = e.next;
    }
}
```

在获得锁之后，Segment 对链表进行遍历，如果某个 HashEntry 节点具有相同的 key，则更新该 HashEntry 的 value 值，

```
else {
    if (node != null)
        node.setNext(first);
    else
        node = new HashEntry<K,V>(hash, key, value, first);
    int c = count + 1;
    if (c > threshold && tab.length < MAXIMUM_CAPACITY)
        rehash(node);
    else
        setEntryAt(tab, index, node);
    ++modCount;
    count = c;
    oldValue = null;
    break;
}
```

否则新建一个 HashEntry 节点，采用头插法，将它设置为链表的新 head 节点并将原头节点设为新 head 的下一个节点。新建过程中如果节点总数（含新建的 HashEntry）超过 threshold，则调用 rehash() 方法对 Segment 进行扩容，最后将新建 HashEntry 写入到数组中。

rehash 操作

扩容是新创建了数组，然后进行迁移数据，最后再将 newTable 设置给属性 table。

为了避免让所有的节点都进行复制操作：由于扩容是基于 2 的幂指来操作，假设扩容前某 HashEntry 对应到 Segment 中数组的 index 为 i，数组的容量为 capacity，那么扩容后该 HashEntry 对应到新数组中的 index 只可能为 i 或者 i+capacity，因此很多 HashEntry 节点在扩容前后 index 可以保持不变。

假设原来 table 长度为 4，那么元素在 table 中的分布是这样的

Hash值	15	23	34	56	77
在table中下标	$3 = 15\%4$	$3 = 23\%4$	$2 = 34\%4$	$0 = 56\%4$	$1 = 77\%4$

扩容后 table 长度变为 8，那么元素在 table 中的分布变成：

Hash值	56		34			77		15,23
下标	0	1	2	3	4	5	6	7

可以看见 hash 值为 34 和 56 的下标保持不变，而 15,23,77 的下标都是在原来下标的基础上+4 即可，可以快速定位和减少重排次数。

该方法没有考虑并发，因为执行该方法之前已经获取了锁。

remove 操作

与 put 方法类似，都是在操作前需要拿到锁，以保证操作的线程安全性。

ConcurrentHashMap 的弱一致性

然后对链表遍历判断是否存在 key 相同的节点以及获得该节点的 value。但由于遍历过程中其他线程可能对链表结构做了调整，因此 get 和 containsKey 返回的可能是过时的数据，这一点是 ConcurrentHashMap 在弱一致性上的体现。如果要求强一致性，那么必须使用 Collections.synchronizedMap() 方法。

size、containsValue

这些方法都是基于整个 ConcurrentHashMap 来进行操作的，他们的原理也基本类似：首先不加锁循环执行以下操作：循环所有的 Segment，获得对应的值以及所有 Segment 的 modcount 之和。在 put、remove 和 clean 方法里操作元素前都会将变量 modCount 进行变动，如果连续两次所有 Segment 的 modcount 和相等，则过程中没有发生其他线程修改 ConcurrentHashMap 的情况，返回获得的值。

当循环次数超过预定义的值时，这时需要对所有的 Segment 依次进行加锁，获取返回值后再依次解锁。所以一般来说，应该避免在多线程环境下使用 size 和 containsValue 方法。

在 1.8 下的实现

改进

改进一：取消 segments 字段，直接采用 transient volatile HashEntry<K,V>[] table 保存数据，采用 table 数组元素作为锁，从而实现了缩小锁的粒度，进一步减少并发冲突的概率，并大量使用了 CAS + synchronized 来保证并发安全性。

改进二：将原先 table 数组+单向链表的数据结构，变更为 table 数组+单向链表+红黑树的结构。对于 hash 表来说，最核心的能力在于将 key hash 之后能均匀的分布在数组中。如果 hash 之后散列的很均匀，那么 table 数组中的每个队列长度主要为 0 或者 1。但实际情况并非总是如此理想，虽然 ConcurrentHashMap 类默认的加载因子为 0.75，但是在数据量过大或者运气不佳

的情况下，还是会存在一些队列长度过长的情况，如果还是采用单向列表方式，那么查询某个节点的时间复杂度为 $O(n)$ ；因此，对于个数超过 8(默认值)的列表，jdk1.8 中采用了红黑树的结构，那么查询的时间复杂度可以降低到 $O(\log N)$ ，可以改进性能。

使用 Node (1.7 为 Entry) 作为链表的数据结点，仍然包含 key, value, hash 和 next 四个属性。红黑树的情况使用的是 TreeNode (extends Node)。

根据数组元素中，第一个结点数据类型是 Node 还是 TreeNode 可以判断该位置下是链表还是红黑树。

用于判断是否需要将链表转换为红黑树的阈值

```
static final int TREEIFY_THRESHOLD = 8;
```

用于判断是否需要将红黑树转换为链表的阈值

```
static final int UNTREEIFY_THRESHOLD = 6;
```

核心数据结构和属性

Node

Node 是最核心的内部类，它包装了 key-value 键值对。

```
static class Node<K,V> implements Map.Entry<K,V> {
    final int hash;
    final K key;
    volatile V val;
    volatile Node<K,V> next;
```

定义基本和 1.7 中的 HashEntry 相同。而这个 map 本身所持有的也是一个 Node 型的数组

```
transient volatile Node<K,V>[] table;
```

增加了一个 find 方法来用以辅助 map.get() 方法。其实就是遍历链表，子类中会覆盖这个方法。

```
Node<K,V> find(int h, Object k) {
    Node<K,V> e = this;
    if (k != null) {
        do {
```

在 map 中还定义了 Segment 这个类，不过只是为了向前兼容而已，不做过多考虑。

TreeNode

树节点类，另外一个核心的数据结构。当链表长度过长的时候，会转换为 TreeNode。


```
static final class TreeNode<K,V> extends Node<K,V> {
    TreeNode<K,V> parent; // red-black tree links
    TreeNode<K,V> left;
    TreeNode<K,V> right;
    TreeNode<K,V> prev; // needed to unlink next u
    boolean red;
}
```

与 1.8 中 HashMap 不同点:

- 1、它并不是直接转换为红黑树，而是把这些结点放在 TreeBin 对象中，由 TreeBin 完成对红黑树的包装。
- 2、TreeNode 在 ConcurrentHashMap 扩展自 Node 类，而并非 HashMap 中的扩展自 LinkedHashMap.Entry<K,V>类，也就是说 TreeNode 带有 next 指针。

TreeBin

负责 TreeNode 节点。它代替了 TreeNode 的根节点，也就是说在实际的 ConcurrentHashMap “数组” 中，存放的是 TreeBin 对象，而不是 TreeNode 对象。另外这个类还带有了读写锁机制。

```
static final class TreeBin<K,V> extends Node<K,V> {
    TreeNode<K,V> root;
    volatile TreeNode<K,V> first;
    volatile Thread waiter;
    volatile int lockState;
    // values for lockState
    static final int WRITER = 1; // set while holding write lock
    static final int WAITER = 2; // set when waiting for write lock
    static final int READER = 4; // increment value for setting read lock
}
```

特殊的 ForwardingNode

一个特殊的 Node 结点，hash 值为 -1，其中存储 nextTable 的引用。有 table 发生扩容的时候，ForwardingNode 发挥作用，作为一个占位符放在 table 中表示当前结点为 null 或者已经被移动。

sizeCtl

用来控制 table 的初始化和扩容操作。

负数代表正在进行初始化或扩容操作

- -1 代表正在初始化
- -N 表示有 N-1 个线程正在进行扩容操作

0 为默认值，代表当时的 table 还没有被初始化

正数表示初始化大小或 Map 中的元素达到这个数量时，需要进行扩容了。

核心方法

```
/*利用硬件级别的原子操作，获得在i位置上的Node节点
 * Unsafe.getObjectVolatile可以直接获取指定内存的数据，
 * 保证了每次拿到数据都是最新的*/
static final <K,V> Node<K,V> tabAt(Node<K,V>[] tab, int i) {
    return (Node<K,V>) U.getObjectVolatile(tab, (long)i << ASHIFT) + AB
}

/*利用CAS操作设置i位置上的Node节点*/
static final <K,V> boolean casTabAt(Node<K,V>[] tab, int i,
    Node<K,V> c, Node<K,V> v) {
    return U.compareAndSwapObject(tab, (long)i << ASHIFT) + ABASE, c,
}

/*利用硬件级别的原子操作，设置在i位置上的Node节点
 * Unsafe.putObjectVolatile可以直接设定指定内存的数据，
 * 保证了其他线程访问这个节点时一定可以看到最新的数据*/
static final <K,V> void setTabAt(Node<K,V>[] tab, int i, Node<K,V> v) {
    U.putObjectVolatile(tab, (long)i << ASHIFT) + ABASE, v);
}
```

构造方法

```
public ConcurrentHashMap(int initialCapacity,
    float loadFactor, int concurrencyLevel) {
    if (!(loadFactor > 0.0f) || initialCapacity < 0 || concurrencyLevel <= 0)
        throw new IllegalArgumentException();
    if (initialCapacity < concurrencyLevel) // Use at least as many bins
        initialCapacity = concurrencyLevel; // as estimated threads
    long size = (long)(1.0 + (long)initialCapacity / loadFactor);
    int cap = (size >= (long)MAXIMUM_CAPACITY) ?
        MAXIMUM_CAPACITY : tableSizeFor((int)size);
    this.sizeCtl = cap;
}
```

可以发现，在 new 出一个 map 的实例时，并不会创建其中的数组等等相关的部件，只是进行简单的属性设置而已，同样的，table 的大小也被规定为必须是 2 的乘方数。

真正的初始化在放在了是在向 ConcurrentHashMap 中插入元素的时候发生的。如调用 put、computeIfAbsent、compute、merge 等方法的时候，调用时机是检查 table==null。

get 操作

get 方法比较简单，给定一个 key 来确定 value 的时候，必须满足两个条件 key 相同 hash 值相同，对于节点可能在链表或树上的情况，需要分别去查找。

```

public V get(Object key) {
    Node<K,V>[] tab; Node<K,V> e, p; int n, eh; K ek;
    int h = spread(key.hashCode()); /*计算hash值*/
    /*根据hash值确定节点位置*/
    if ((tab = table) != null && (n = tab.length) > 0 &&
        (e = tabAt(tab, i: (n - 1) & h)) != null) {
        /*Node数组中的节点就是要找的节点*/
        if ((eh = e.hash) == h) {
            if ((ek = e.key) == key || (ek != null && key.equals(ek)))
                return e.val;
        }
        /*eh<0 说明这个节点在树上 调用树的find方法寻找*/
        else if (eh < 0)
            return (p = e.find(h, key)) != null ? p.val : null;
        /*到这一步说明是个链表, 遍历链表找到对应的值并返回*/
        while ((e = e.next) != null) {
            if (e.hash == h &&
                ((ek = e.key) == key || (ek != null && key.equals(ek))))
                return e.val;
        }
    }
    return null;
}

```

put 操作

```

/** Implementation for put and putIfAbsent */
final V putVal(K key, V value, boolean onlyIfAbsent) {
    if (key == null || value == null) throw new NullPointerException();
    int hash = spread(key.hashCode()); /*计算hash值*/
    int binCount = 0;
    /*死循环 何时插入成功 何时跳出*/
    for (Node<K,V>[] tab = table;;) {
        Node<K,V> f; int n, i, fh;
        if (tab == null || (n = tab.length) == 0)
            tab = initTable(); /*如果table为空的话, 初始化table*/
        else if ((f = tabAt(tab, i = (n - 1) & hash)) == null) {
            /*Node数组中的元素, 这个位置没有值, 使用CAS操作放进去*/
            if (casTabAt(tab, i, c: null,
                new Node<K,V>(hash, key, value, next: null)))
                break; // no lock when adding to empty
        }
        else if ((fh = f.hash) == MOVED)
            /*正在进行扩容, 当前线程帮忙扩容*/
            tab = helpTransfer(tab, f);
        else {
            V oldVal = null;
            /*锁Node数组中的元素, 这个位置是Hash冲突组成链表的头结点
            * 或者是红黑树的根节点*/
            synchronized (f) {
                if (tabAt(tab, i) == f) {
                    /*fh>0 说明这个节点是一个链表的节点 不是树的节点*/
                    if (fh >= 0) {
                        binCount = 1;

```



```

synchronized (t) {
    if (tabAt(tab, i) == f) {
        /*fh>0 说明这个节点是一个链表的节点 不是树的节点*/
        if (fh >= 0) {
            binCount = 1;
            for (Node<K,V> e = f;; ++binCount) {
                K ek;
                /*put操作和putIfAbsent操作业务实现*/
                if (e.hash == hash &&
                    ((ek = e.key) == key ||
                     (ek != null && key.equals(ek)))) {
                    oldVal = e.val;
                    if (!onlyIfAbsent)
                        e.val = value;
                    break;
                }
                Node<K,V> pred = e;
                /*如果遍历到了最后一个结点, 使用尾插法, 把它插入在链表尾部*/
                if ((e = e.next) == null) {
                    pred.next = new Node<K,V>(hash, key,
                                                value, next: null);
                    break;
                }
            }
            /*按照树的方式插入值*/
        } else if (f instanceof TreeBin) {
            Node<K,V> p;
            /*按照树的方式插入值*/
            else if (f instanceof TreeBin) {
                Node<K,V> p;
                binCount = 2;
                if ((p = ((TreeBin<K,V>) f).putTreeVal(hash, key,
                                                         value)) != null) {
                    oldVal = p.val;
                    if (!onlyIfAbsent)
                        p.val = value;
                }
            }
        }
        if (binCount != 0) {
            /*达到临界值8 就需要把链表转换为树结构*/
            if (binCount >= TREEIFY_THRESHOLD)
                treeifyBin(tab, i);
            if (oldVal != null)
                return oldVal;
            break;
        }
    }
    /*Map的元素数量+1, 并检查是否需要扩容*/
    addCount(x: 1L, binCount);
    return null;
}

```

总结来说, put 方法就是, 沿用 HashMap 的 put 方法的思想, 根据 hash 值计算这个新插入的点在 table 中的位置 i, 如果 i 位置是空的, 直接放进去, 否则进行判断, 如果 i 位置是树节点, 按照树的方式插入新的节点, 否则把 i 插入到链表的末尾。

整体流程上，就是首先定义不允许 key 或 value 为 null 的情况放入 对于每一个放入的值，首先利用 spread 方法对 key 的 hashCode 进行一次 hash 计算，由此来确定这个值在 table 中的位置。

如果这个位置是空的，那么直接放入，而且不需要加锁操作。

如果这个位置存在结点，说明发生了 hash 碰撞，首先判断这个结点的类型。如果是链表结点，则得到的结点就是 hash 值相同的节点组成的链表的头节点。需要依次向后遍历确定这个新加入的值所在位置。如果遇到 hash 值与 key 值都与新加入节点是一致的情况，则只需要更新 value 值即可。否则依次向后遍历，直到链表尾插入这个结点。如果加入这个节点以后链表长度大于 8，就把这个链表转换成红黑树。如果这个结点的类型已经是树节点的话，直接调用树节点的插入方法进行插入新的值。

初始化

前面说过，构造方法中并没有真正初始化，真正的初始化在放在了是在向 ConcurrentHashMap 中插入元素的时候发生的。具体实现的方法就是 initTable

```
private final Node<K,V>[] initTable() {
    Node<K,V>[] tab; int sc;
    while ((tab = table) == null || tab.length == 0) {
        /*小于0表示有其他线程正在进行初始化操作，把当前线程CPU时间让出来。
        因为对于table的初始化工作，只能有一个线程在进行。*/
        if ((sc = sizeCtl) < 0)
            Thread.yield(); // lost initialization race; just spin
        /*利用CAS操作把sizeCtl的值置为-1 表示本线程正在进行初始化*/
        else if (U.compareAndSwapInt(0: this, SIZECTL, sc, i1: -1)) {
            try {
                if ((tab = table) == null || tab.length == 0) {
                    int n = (sc > 0) ? sc : DEFAULT_CAPACITY;
                    /unchecked/
                    Node<K,V>[] nt = (Node<K,V>[])new Node<?,?>[n];
                    table = tab = nt;
                    /*n右移2位本质上就是n变为n原值的1/4，所以
                    * sc=0.75*n */
                    sc = n - (n >>> 2);
                }
            } finally {
                /*将设置成扩容的阈值*/
                sizeCtl = sc;
            }
            break;
        }
    }
    return tab;
}
```

transfer

当 ConcurrentHashMap 容量不足的时候，需要对 table 进行扩容。这个方法的基本思想跟 HashMap 是很像的，但是由于它是支持并发扩容的，所以要复杂的多。我们不深入源码去讲述，只讲述其大概原理。

为何要并发扩容？因为在扩容的时候，总是会涉及到从一个“数组”到另一个“数组”拷贝的操作，如果这个操作能够并发进行，就能利用并发处理去减少扩容带来的时间影响。

整个扩容操作分为两个部分：

第一部分是构建一个 `nextTable`，它的容量是原来的 2 倍。

```
if (nextTab == null) { // initiating
    try {
        @SuppressWarnings("unchecked")
        Node<K,V>[] nt = (Node<K,V>[])new Node<?,?>[n << 1];
        nextTab = nt;
    }
}
```

第二个部分就是将原来 `table` 中的元素复制到 `nextTable` 中，这里允许多线程进行操作。

整个扩容流程就是遍历和复制：

为 `null` 或者已经处理过的节点，会被设置为 `forwardNode` 节点，当线程准备扩容时，发现节点是 `forwardNode` 节点，跳过这个节点，继续寻找未处理的节点，找到了，对节点上锁，

如果这个位置是 `Node` 节点 (`fh >= 0`)，说明它是一个链表，就构造一个反序链表，把他们分别放在 `nextTable` 的 `i` 和 `i+n` 的位置上

如果这个位置是 `TreeBin` 节点 (`fh < 0`)，也做一个反序处理，并且判断是否需要红黑树转链表，把处理的结果分别放在 `nextTable` 的 `i` 和 `i+n` 的位置上

遍历过所有的节点以后就完成了复制工作，这时让 `nextTable` 作为新的 `table`，并且更新 `sizeCtl` 为新容量的 0.75 倍，完成扩容。

并发扩容其实就是将数据迁移任务拆分成多个小迁移任务，在实现上使用了一个变量 `stride` 作为步长控制，每个线程每次负责迁移其中的一部分。

remove

移除方法的基本流程和 `put` 方法很类似，只不过操作由插入数据变为移除数据而已，而且如果存在红黑树的情况下，会检查是否需要将红黑树转为链表的步骤。不再重复讲述。

treeifyBin

用于将过长的链表转换为 `TreeBin` 对象。但是他并不是直接转换，而是进行一次容量判断，如果容量没有达到转换的要求，直接进行扩容操作并返回；如果满足条件才将链表的结构转换为 `TreeBin`，这与 `HashMap` 不同的是，它并没有把 `TreeNode` 直接放入红黑树，而是利用了 `TreeBin` 这个小容器来封装所有的 `TreeNode`。

size

在 `JDK1.8` 版本中，对于 `size` 的计算，在扩容和 `addCount()` 方法就已经有处理了，可以注意一下 `Put` 函数，里面就有 `addCount()` 函数，早就计算好的，然后你 `size` 的时候直接给你。`JDK1.7` 是在调用 `size()` 方法才去计算，其实在并发集合中去计算 `size` 是没有多大的意义的，因为 `size` 是实时在变的。

在具体实现上，计算大小的核心方法都是 `sumCount()`

```
final long sumCount() {  
    CounterCell[] as = counterCells; CounterCell a;  
    long sum = baseCount;  
    if (as != null) {  
        for (int i = 0; i < as.length; ++i) {  
            if ((a = as[i]) != null)  
                sum += a.value;  
        }  
    }  
    return sum;  
}
```

可以看见，统计数量时使用了 `baseCount`、和 `CounterCell` 类型的变量 `counterCells`。其实 `baseCount` 就是记录容器数量的，而 `counterCells` 则是记录 CAS 更新 `baseCounter` 值时，由于高并发而导致失败的值。这两个变量的变化在 `addCount()` 方法中有体现，大致的流程就是：

- 1、对 `baseCount` 做 CAS 自增操作。
- 2、如果并发导致 `baseCount` CAS 失败了，则使用 `counterCells`。
- 3、如果 `counterCells` CAS 失败了，在 `fullAddCount` 方法中，会继续死循环操作，直到成功。

HashTable

`HashTable` 容器使用 `synchronized` 来保证线程安全，但在线程竞争激烈的情况下 `HashTable` 的效率非常低下。因为当一个线程访问 `HashTable` 的同步方法，其他线程也访问 `HashTable` 的同步方法时，会进入阻塞或轮询状态。如线程 1 使用 `put` 进行元素添加，线程 2 不但不能使用 `put` 方法添加元素，也不能使用 `get` 方法来获取元素，所以竞争越激烈效率越低。

并发下的 Map 常见面试题汇总

Q: `HashMap` 和 `HashTable` 有什么区别？

- A: ①、`HashMap` 是线程不安全的，`HashTable` 是线程安全的；
- ②、由于线程安全，所以 `HashTable` 的效率比不上 `HashMap`；
- ③、`HashMap` 最多只允许一条记录的键为 `null`，允许多条记录的值为 `null`，而 `HashTable` 不允许；
- ④、`HashMap` 默认初始化数组的大小为 16，`HashTable` 为 11，前者扩容时，扩大两倍，后者扩大两倍+1；
- ⑤、`HashMap` 需要重新计算 `hash` 值，而 `HashTable` 直接使用对象的 `hashCode`

Q: Java 中的另一个线程安全的与 `HashMap` 极其类似的类是什么？同样是线程安全，它与 `HashTable` 在线程同步上有什么不同？

A: `ConcurrentHashMap` 类（是 Java 并发包 `java.util.concurrent` 中提供的一个线程安全且高效的 `HashMap` 实现）。

HashTable 是使用 synchronize 关键字加锁的原理（就是对对象加锁）；

而针对 ConcurrentHashMap，在 JDK 1.7 中采用分段锁的方式；JDK 1.8 中直接采用了 CAS（无锁算法）+ synchronized，也采用分段锁的方式并大大缩小了锁的粒度。

HashMap & ConcurrentHashMap 的区别？

A：除了加锁，原理上无太大区别。

另外，HashMap 的键值对允许有 null，但是 ConcurrentHashMap 都不允许。
在数据结构上，红黑树相关的节点类

Q：为什么 ConcurrentHashMap 比 HashTable 效率要高？

A：HashTable 使用一把锁（锁住整个链表结构）处理并发问题，多个线程竞争一把锁，容易阻塞；

ConcurrentHashMap

JDK 1.7 中使用分段锁（ReentrantLock + Segment + HashEntry），相当于把一个 HashMap 分成多个段，每段分配一把锁，这样支持多线程访问。锁粒度：基于 Segment，包含多个 HashEntry。

JDK 1.8 中使用 CAS + synchronized + Node + 红黑树。锁粒度：Node（首结点）（实现 Map.Entry<K,V>）。锁粒度降低了。

Q：针对 ConcurrentHashMap 锁机制具体分析（JDK 1.7 VS JDK 1.8）？

JDK 1.7 中，采用分段锁的机制，实现并发的更新操作，底层采用数组+链表的存储结构，包括两个核心静态内部类 Segment 和 HashEntry。

①、Segment 继承 ReentrantLock（重入锁）用来充当锁的角色，每个 Segment 对象守护每个散列映射表的若干个桶；

②、HashEntry 用来封装映射表的键-值对；

③、每个桶是由若干个 HashEntry 对象链接起来的链表。

JDK 1.8 中，采用 Node + CAS + Synchronized 来保证并发安全。取消类 Segment，直接用 table 数组存储键值对；当 HashEntry 对象组成的链表长度超过 TREEIFY_THRESHOLD 时，链表转换为红黑树，提升性能。底层变更为数组 + 链表 + 红黑树。

Q：ConcurrentHashMap 在 JDK 1.8 中，为什么要使用内置锁 synchronized 来代替重入锁 ReentrantLock？

A：

1、JVM 开发团队在 1.8 中对 synchronized 做了大量性能上的优化，而且基于 JVM 的 synchronized 优化空间更大，更加自然。

2、在大量的数据操作下，对于 JVM 的内存压力，基于 API 的 ReentrantLock 会开销更多的内存。

Q: ConcurrentHashMap 简单介绍？

A:

①、重要的常量：

`private transient volatile int sizeCtl;`

当为负数时，-1 表示正在初始化，-N 表示 N - 1 个线程正在进行扩容；

当为 0 时，表示 table 还没有初始化；

当为其他正数时，表示初始化或者下一次进行扩容的大小。

②、数据结构：

Node 是存储结构的基本单元，继承 HashMap 中的 Entry，用于存储数据；

TreeNode 继承 Node，但是数据结构换成了二叉树结构，是红黑树的存储结构，用于红黑树中存储数据；

TreeBin 是封装 TreeNode 的容器，提供转换红黑树的一些条件和锁的控制。

③、存储对象时（put() 方法）：

1.如果没有初始化，就调用 initTable() 方法来进行初始化；

2.如果没有 hash 冲突就直接 CAS 无锁插入；

3.如果需要扩容，就先进行扩容；

4.如果存在 hash 冲突，就加锁来保证线程安全，两种情况：一种是链表形式就直接遍历到尾端插入，一种是红黑树就按照红黑树结构插入；

5.如果该链表的数量大于阈值 8，就要先转换成红黑树的结构，break 再一次进入循环

6.如果添加成功就调用 addCount() 方法统计 size，并且检查是否需要扩容。

④、扩容方法 transfer()：默认容量为 16，扩容时，容量变为原来的两倍。

helpTransfer()：调用多个工作线程一起帮助进行扩容，这样的效率就会更高。

⑤、获取对象时（get()方法）：

1.计算 hash 值，定位到该 table 索引位置，如果是首结点符合就返回；

2.如果遇到扩容时，会调用标记正在扩容结点 ForwardingNode.find()方法，查找该结点，匹配就返回；

3.以上都不符合的话，就往下遍历结点，匹配就返回，否则最后就返回 null。

Q: ConcurrentHashMap 的并发度是什么？

A: 1.7 中程序运行时能够同时更新 ConcurrentHashMap 且不产生锁竞争的最大线程数。默认为 16，且可以在构造函数中设置。当用户设置并发度时，ConcurrentHashMap 会使用大于等于该值的最小 2 幂指数作为实际并发度（假如用户设置并发度为 17，实际并发度则为 32）。

1.8 中并发度则无太大的实际意义了，主要用处就是当设置的初始容量小于并发度，将初始容量提升至并发度大小。

ConcurrentSkipList 系列

ConcurrentSkipListMap 有序 Map

ConcurrentSkipListSet 有序 Set

TreeMap 和 TreeSet 使用红黑树按照 key 的顺序（自然顺序、自定义顺序）来使得键值对有序存储，但是只能在单线程下安全使用；多线程下想要使键值对按照 key 的顺序来存储，则需要使用 ConcurrentSkipListMap 和 ConcurrentSkipListSet，分别用以代替 TreeMap 和 TreeSet，存入的数据按 key 排序。在实现上，ConcurrentSkipListSet 本质上就是 ConcurrentSkipListMap。

```
private final ConcurrentNavigableMap<E, Object> m;

/**
 * Constructs a new, empty set that orders its elements according to
 * their {@link Comparable Comparable} natural ordering.
 */
public ConcurrentSkipListSet() {
    m = new ConcurrentSkipListMap<E, Object>();
}
```

了解什么是 SkipList ?

二分查找和 AVL 树查找

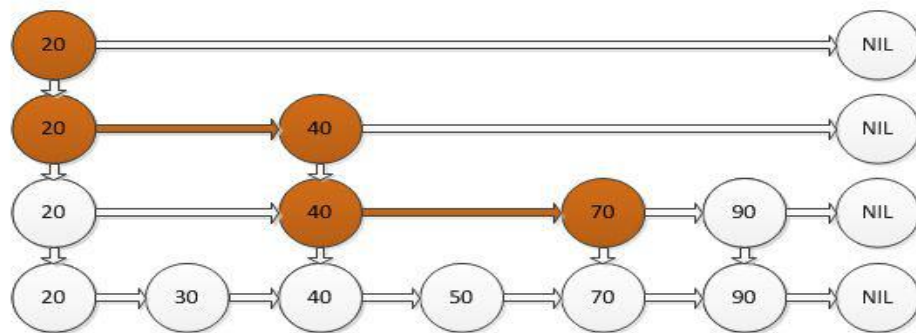
二分查找要求元素可以随机访问，所以决定了需要把元素存储在连续内存。这样查找确实很快，但是插入和删除元素的时候，为了保证元素的有序性，就需要大量的移动元素了。

如果需要的是一个能够进行二分查找，又能快速添加和删除元素的数据结构，首先就是二叉查找树，二叉查找树在最坏情况下可能变成一个链表。

于是，就出现了平衡二叉树，根据平衡算法的不同有 AVL 树，B-Tree，B+Tree，红黑树等，但是 AVL 树实现起来比较复杂，平衡操作较难理解，这时候就可以用 SkipList 跳跃表结构。

什么是跳表

传统意义的单链表是一个线性结构，向有序的链表中插入一个节点需要 $O(n)$ 的时间，查找操作需要 $O(n)$ 的时间。



如果我们使用上图所示的跳跃表，就可以减少查找所需时间为 $O(n/2)$ ，因为我们可以先通过每个节点的最上面的指针先进行查找，这样子就能跳过一半的节点。

比如我们想查找 50，首先和 20 比较，大于 20 之后，在和 40 进行比较，然后在和 70 进行比较，发现 70 大于 50，说明查找的点在 40 和 50 之间，从这个过程中，我们可以看出，查找的时候跳过了 30。

跳跃表其实也是一种通过“空间来换取时间”的一个算法，令链表的每个结点不仅记录 next 结点位置，还可以按照 level 层级分别记录后继第 level 个结点。此法使用的就是“先大步查找确定范围，再逐渐缩小逼近”的思想进行的查找。跳跃表在算法效率上很接近红黑树。

跳跃表又被称为概率，或者说是随机化的数据结构，目前开源软件 Redis 和 lucence 都有用到它。

都是线程安全的 Map 实现，ConcurrentHashMap 的性能和存储空间要优于 ConcurrentSkipListMap，但是 ConcurrentSkipListMap 有一个功能：它会按照键的顺序进行排序。

ConcurrentLinkedQueue

无界非阻塞队列，它是一个基于链表的无界线程安全队列。该队列的元素遵循先进先出的原则。头是最先加入的，尾是最近加入的。插入元素是追加到尾上。提取一个元素是从头提取。

大家可以看成是 LinkedList 的并发版本，常用方法：

```
concurrentLinkedQueue.add("c");
```

```
concurrentLinkedQueue.offer("d"); // 将指定元素插入到此队列的尾部。
```

```
concurrentLinkedQueue.peek(); // 检索并不移除此队列的头，如果此队列为空，则返回 null。
```

```
concurrentLinkedQueue.poll(); // 检索并移除此队列的头，如果此队列为空，则返回 null。
```

写时复制容器

什么是写时复制容器

`CopyOnWriteArrayList` 和 `CopyOnWriteArraySet`

`CopyOnWrite` 容器即写时复制的容器。通俗的理解是当我们往一个容器添加元素的时候，不直接往当前容器添加，而是先将当前容器进行 `Copy`，复制出一个新的容器，然后新的容器里添加元素，添加完元素之后，再将原容器的引用指向新的容器。

这样做的好处是我们可以对 `CopyOnWrite` 容器进行并发的读，而不需要加锁，因为当前容器不会添加任何元素。所以 `CopyOnWrite` 容器也是一种读写分离的思想，读和写不同的容器。如果读的时候有多个线程正在向 `CopyOnWriteArrayList` 添加数据，读还是会读到旧的数据，因为写的时候不会锁住旧的 `CopyOnWriteArrayList`。

`CopyOnWrite` 并发容器用于对于绝大部分访问都是读，且只是偶尔写的并发场景。比如白名单，黑名单，商品类目的访问和更新场景，假如我们有一个搜索网站，用户在这个网站的搜索框中，输入关键字搜索内容，但是某些关键字不允许被搜索。这些不能被搜索的关键字会被放在一个黑名单当中，黑名单每天晚上更新一次。当用户搜索时，会检查当前关键字是否在黑名单当中，如果在，则提示不能搜索。

使用 `CopyOnWriteMap` 需要注意两件事情：

1. 减少扩容开销。根据实际需要，初始化 `CopyOnWriteMap` 的大小，避免写时 `CopyOnWriteMap` 扩容的开销。
2. 使用批量添加。因为每次添加，容器每次都会进行复制，所以减少添加次数，可以减少容器的复制次数。

写时复制容器的问题

性能问题

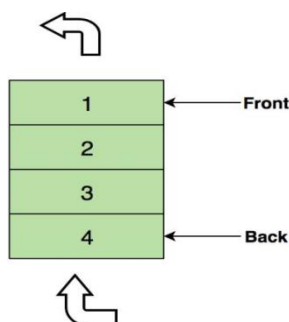
每次修改都创建一个新数组，然后复制所有内容，如果数组比较大，修改操作又比较频繁，可以想象，性能是很低的，而且内存开销会很大。

数据一致性问题。

`CopyOnWrite` 容器只能保证数据的最终一致性，不能保证数据的实时一致性。所以如果你希望写入的数据，马上能读到，不要使用 `CopyOnWrite` 容器。

阻塞队列 BlockingQueue

队列



队列是一种特殊的线性表，特殊之处在于它只允许在表的前端（front）进行删除操作，而在表的后端（rear）进行插入操作，和栈一样，队列是一种操作受限制的线性表。进行插入操作的端称为队尾，进行删除操作的端称为队头。

在队列中插入一个队列元素称为入队，从队列中删除一个队列元素称为出队。因为队列只允许在一端插入，在另一端删除，所以只有最早进入队列的元素才能最先从队列中删除，故队列又称为先进先出（FIFO—first in first out）线性表。

什么是阻塞队列

1) 支持阻塞的插入方法：意思是当队列满时，队列会阻塞插入元素的线程，直到队列不满。

2) 支持阻塞的移除方法：意思是在队列为空时，获取元素的线程会等待队列变为非空。

在并发编程中使用**生产者和消费者模式**能够解决绝大多数并发问题。该模式通过平衡生产线程和消费线程的工作能力来提高程序整体处理数据的速度。

在线程世界里，生产者就是生产数据的线程，消费者就是消费数据的线程。在多线程开发中，如果生产者处理速度很快，而消费者处理速度很慢，那么生产者就必须等待消费者处理完，才能继续生产数据。同样的道理，如果消费者的处理能力大于生产者，那么消费者就必须等待生产者。

为了解决这种生产消费能力不均衡的问题，便有了生产者和消费者模式。生产者和消费者模式是通过一个容器来解决生产者和消费者的强耦合问题。生产者和消费者彼此之间不直接通信，而是通过阻塞队列来进行通信，所以生产者生产完数据之后不用等待消费者处理，直接扔给阻塞队列，消费者不找生产者要数据，而是直接从阻塞队列里取，阻塞队列就相当于一个缓冲区，平衡了生产者和消费者的处理能力。

阻塞队列常用于生产者和消费者的场景，生产者是向队列里添加元素的线程，消费者是从队列里取元素的线程。阻塞队列就是生产者用来存放元素、消费者用来获取元素的容器。

方法 / 处理方式	抛出异常	返回特殊值	一直阻塞	超时退出
插入方法	add (e)	offer (e)	put (e)	offer (e, time, unit)
移除方法	remove()	poll()	take()	poll (time, unit)
检查方法	element()	peek()	不可用	不可用

- 抛出异常：当队列满时，如果再往队列里插入元素，会抛出 `IllegalStateException ("Queuefull")` 异常。当队列空时，从队列里获取元素会抛出 `NoSuchElementException` 异常。

- 返回特殊值：当往队列插入元素时，会返回元素是否插入成功，成功返回 `true`。如果是移除方法，则是从队列里取出一个元素，如果没有则返回 `null`。

- 一直阻塞：当阻塞队列满时，如果生产者线程往队列里 `put` 元素，队列会一直阻塞生产者线程，直到队列可用或者响应中断退出。当队列空时，如果消费者线程从队列里 `take` 元素，队列会阻塞住消费者线程，直到队列不为空。

- 超时退出：当阻塞队列满时，如果生产者线程往队列里插入元素，队列会阻塞生产者线程一段时间，如果超过了指定的时间，生产者线程就会退出。

常用阻塞队列

- `ArrayBlockingQueue`：一个由数组结构组成的有界阻塞队列。
- `LinkedBlockingQueue`：一个由链表结构组成的有界阻塞队列。
- `PriorityBlockingQueue`：一个支持优先级排序的无界阻塞队列。
- `DelayQueue`：一个使用优先级队列实现的无界阻塞队列。
- `SynchronousQueue`：一个不存储元素的阻塞队列。
- `LinkedTransferQueue`：一个由链表结构组成的无界阻塞队列。
- `LinkedBlockingDeque`：一个由链表结构组成的双向阻塞队列。

以上的阻塞队列都实现了 `BlockingQueue` 接口，也都是线程安全的。

有界无界？

有限队列就是长度有限，满了以后生产者会阻塞，无界队列就是里面能放无数的东西而不会因为队列长度限制被阻塞，当然空间限制来源于系统资源的限制，如果处理不及时，导致队列越来越大越来越大，超出一定的限制致使内存超限，操作系统或者 JVM 帮你解决烦恼，直接把你 `OOM kill` 省事了。

无界也会阻塞，为何？因为阻塞不仅仅体现在生产者放入元素时会阻塞，消费者拿取元素时，如果没有元素，同样也会阻塞。

`ArrayBlockingQueue`

是一个用数组实现的有界阻塞队列。此队列按照先进先出（FIFO）的原则对元素进行排序。默认情况下不保证线程公平的访问队列，所谓公平访问队列是指阻塞的线程，可以按照阻塞的先后顺序访问队列，即先阻塞线程先访问队列。非公平性是对先等待的线程是非公平的，当队列可用时，阻塞的线程都可以争夺访问队列的资格，有可能先阻塞的线程最后才访问队列。初始化时有参数可以设置

LinkedBlockingQueue

是一个用链表实现的有界阻塞队列。此队列的默认和最大长度为 `Integer.MAX_VALUE`。此队列按照先进先出的原则对元素进行排序。

Array 实现和 Linked 实现的区别

1. 队列中锁的实现不同

`ArrayBlockingQueue` 实现的队列中的锁是没有分离的，即生产和消费用的是同一个锁；

`LinkedBlockingQueue` 实现的队列中的锁是分离的，即生产用的是 `putLock`，消费是 `takeLock`

2. 在生产或消费时操作不同

`ArrayBlockingQueue` 实现的队列中在生产和消费的时候，是直接将枚举对象插入或移除的；

`LinkedBlockingQueue` 实现的队列中在生产和消费的时候，需要把枚举对象转换为 `Node<E>` 进行插入或移除，会影响性能

3. 队列大小初始化方式不同

`ArrayBlockingQueue` 实现的队列中必须指定队列的大小；

`LinkedBlockingQueue` 实现的队列中可以不指定队列的大小，但是默认是 `Integer.MAX_VALUE`

PriorityBlockingQueue

`PriorityBlockingQueue` 是一个支持优先级的无界阻塞队列。默认情况下元素采取自然顺序升序排列。也可以自定义类实现 `compareTo()` 方法来指定元素排序规则，或者初始化 `PriorityBlockingQueue` 时，指定构造参数 `Comparator` 来对元素进行排序。需要注意的是不能保证同优先级元素的顺序。

DelayQueue

是一个支持延时获取元素的无界阻塞队列。队列使用 `PriorityQueue` 来实现。队列中的元素必须实现 `Delayed` 接口，在创建元素时可以指定多久才能从队列中获取当前元素。只有在延迟期满时才能从队列中提取元素。

`DelayQueue` 非常有用，可以将 `DelayQueue` 运用在以下应用场景。

缓存系统的设计：可以用 `DelayQueue` 保存缓存元素的有效期，使用一个线程循环查询 `DelayQueue`，一旦能从 `DelayQueue` 中获取元素时，表示缓存有效期到了。**还有订单到期，限时支付等等**

SynchronousQueue

是一个不存储元素的阻塞队列。每一个 `put` 操作必须等待一个 `take` 操作，否则不能继续添加元素。`SynchronousQueue` 可以看成是一个传球手，负责把生产者线程处理的数据直接传递给消费者线程。队列本身并不存储任何元素，非常适合传递性场景。`SynchronousQueue` 的吞吐量高于 `LinkedBlockingQueue` 和 `ArrayBlockingQueue`。

LinkedTransferQueue

多了 `tryTransfer` 和 `transfer` 方法，

(1) transfer 方法

如果当前有消费者正在等待接收元素（消费者使用 `take()` 方法或带时间限制的 `poll()` 方法时），`transfer` 方法可以把生产者传入的元素立刻 `transfer`（传输）给消费者。如果没有消费者在等待接收元素，`transfer` 方法会将元素存放在队列的 `tail` 节点，并等到该元素被消费者消费了才返回。

(2) tryTransfer 方法

`tryTransfer` 方法是用来试探生产者传入的元素是否能直接传给消费者。如果没有消费者等待接收元素，则返回 `false`。和 `transfer` 方法的区别是 `tryTransfer` 方法无论消费者是否接收，方法立即返回，而 `transfer` 方法是必须等到消费者消费了才返回。

LinkedBlockingDeque

`LinkedBlockingDeque` 是一个由链表结构组成的双向阻塞队列。所谓双向队列指的是可以从队列的两端插入和移出元素。双向队列因为多了一个操作队列的入口，在多线程同时入队时，也就减少了一半的竞争。

多了 `addFirst`、`addLast`、`offerFirst`、`offerLast`、`peekFirst` 和 `peekLast` 等方法，以 `First` 单词结尾的方法，表示插入、获取（`peek`）或移除双端队列的第一个元素。以 `Last` 单词结尾的方法，表示插入、获取或移除双端队列的最后一个元素。另外，插入方法 `add` 等同于 `addLast`，移除方法 `remove` 等效于 `removeFirst`。但是 `take` 方法却等同于 `takeFirst`，不知道是不是 JDK 的 bug，使用时还是用带有 `First` 和 `Last` 后缀的方法更清楚。在初始化 `LinkedBlockingDeque` 时可以设置容量防止其过度膨胀。另外，双向阻塞队列可以运用在“工作窃取”模式中。

了解阻塞队列的实现原理

使用了等待通知模式实现。所谓通知模式，就是当生产者往满的队列里添加元素时会阻塞住生产者，当消费者消费了一个队列中的元素后，会通知生产者当前队列可用。通过查看 JDK 源码发现 `ArrayBlockingQueue` 使用了 `Condition` 来实现。其余队列的实现，大家可以自行查看，队列的实现代码总体来说，并不复杂。

6、线程池

为什么要用线程池？

Java 中的线程池是运用场景最多的并发框架，几乎所有需要异步或并发执行任务的程序都可以使用线程池。在开发过程中，合理地使用线程池能够带来 3 个好处。

第一：降低资源消耗。通过重复利用已创建的线程降低线程创建和销毁造成的消耗。

第二：提高响应速度。当任务到达时，任务可以不需要等到线程创建就能立即执行。假设一个服务器完成一项任务所需时间为：T1 创建线程时间，T2 在线程中执行任务的时间，T3 销毁线程时间。如果：T1 + T3 远大于 T2，则可以采用线程池，以提高服务器性能。线程池技术正是关注如何缩短或调整 T1,T3 时间的技术，从而提高服务器程序性能的。它把 T1，T3 分别安排在服务器程序的启动和结束的时间段或者一些空闲的时间段，这样在服务器程序处理客户请求时，不会有 T1，T3 的开销了。

第三：提高线程的可管理性。线程是稀缺资源，如果无限制地创建，不仅会消耗系统资源，还会降低系统的稳定性，使用线程池可以进行统一分配、调优和监控。

假设一个服务器一天要处理 50000 个请求，并且每个请求需要一个单独的线程完成。在线程池中，线程数一般是固定的，所以产生线程总数不会超过线程池中线程的数目，而如果服务器不利用线程池来处理这些请求则线程总数为 50000。一般线程池大小是远小于 50000。所以利用线程池的服务器程序不会为了创建 50000 而在处理请求时浪费时间，从而提高效率。

ThreadPoolExecutor 的类关系

Executor 是一个接口，它是 Executor 框架的基础，它将任务的提交与任务的执行分离开来。

ExecutorService 接口继承了 Executor，在其上做了一些 shutdown()、submit() 的扩展，可以说是真正的线程池接口；

AbstractExecutorService 抽象类实现了 ExecutorService 接口中的大部分方法；

ThreadPoolExecutor 是线程池的核心实现类，用来执行被提交的任务。

ScheduledExecutorService 接口继承了 ExecutorService 接口，提供了带"周期执行"功能 ExecutorService；

ScheduledThreadPoolExecutor 是一个实现类，可以在给定的延迟后运行命令，或者定期执行命令。ScheduledThreadPoolExecutor 比 Timer 更灵活，功能更强大。

线程池的创建各个参数含义

```
public ThreadPoolExecutor(int corePoolSize,int maximumPoolSize,long
keepAliveTime,TimeUnit unit,BlockingQueue<Runnable> workQueue,ThreadFactory
threadFactory,RejectedExecutionHandler handler)
```

corePoolSize

线程池中的核心线程数，当提交一个任务时，线程池创建一个新线程执行任务，直到当前线程数等于 corePoolSize；

如果当前线程数为 corePoolSize，继续提交的任务被保存到阻塞队列中，等待被执行；

如果执行了线程池的 prestartAllCoreThreads() 方法，线程池会提前创建并启动所有核心线程。

maximumPoolSize

线程池中允许的最大线程数。如果当前阻塞队列满了，且继续提交任务，则创建新的线程执行任务，前提是当前线程数小于 `maximumPoolSize`

keepAliveTime

线程空闲时的存活时间，即当线程没有任务执行时，继续存活的时间。默认情况下，该参数只在线程数大于 `corePoolSize` 时才有用

`TimeUnit`

`keepAliveTime` 的时间单位

`workQueue`

`workQueue` 必须是 `BlockingQueue` 阻塞队列。当线程池中的线程数超过它的 `corePoolSize` 的时候，线程会进入阻塞队列进行阻塞等待。通过 `workQueue`，线程池实现了阻塞功能

workQueue

用于保存等待执行的任务的阻塞队列，一般来说，我们应该尽量使用有界队列，因为使用无界队列作为工作队列会对线程池带来如下影响。

- 1) 当线程池中的线程数达到 `corePoolSize` 后，新任务将在无界队列中等待，因此线程池中的线程数不会超过 `corePoolSize`。
- 2) 由于 1，使用无界队列时 `maximumPoolSize` 将是一个无效参数。
- 3) 由于 1 和 2，使用无界队列时 `keepAliveTime` 将是一个无效参数。
- 4) 更重要的，使用无界 queue 可能会耗尽系统资源，有界队列则有助于防止资源耗尽，同时即使使用有界队列，也要尽量控制队列的大小在一个合适的范围。

所以我们一般会使用，`ArrayBlockingQueue`、`LinkedBlockingQueue`、`SynchronousQueue`、`PriorityBlockingQueue`。

threadFactory

创建线程的工厂，通过自定义的线程工厂可以给每个新建的线程设置一个具有识别度的线程名，当然还可以更加自由的对线程做更多的设置，比如设置所有的线程为守护线程。

参见代码 `cn.enjoyedu.ch6.ThreadPoolAdv`

`Executors` 静态工厂里默认的 `threadFactory`，线程的命名规则是“pool-数字-thread-数字”。

RejectedExecutionHandler

线程池的饱和策略，当阻塞队列满了，且没有空闲的工作线程，如果继续提交任务，必须采取一种策略处理该任务，线程池提供了 4 种策略：

-
- (1) **AbortPolicy**: 直接抛出异常，默认策略；
 - (2) **CallerRunsPolicy**: 用调用者所在的线程来执行任务；
 - (3) **DiscardOldestPolicy**: 丢弃阻塞队列中靠最前的任务，并执行当前任务；
 - (4) **DiscardPolicy**: 直接丢弃任务；

当然也可以根据应用场景实现 **RejectedExecutionHandler** 接口，自定义饱和策略，如记录日志或持久化存储不能处理的任务。

扩展线程池

能扩展线程池的功能吗？比如在任务执行的前后做一点我们自己的业务工作？实际上，JDK 的线程池已经为我们预留的接口，在线程池核心方法中，有 2 个方法是空的，就是给我们预留的。还有一个线程池退出时会调用的方法。参见代码 `cn.enjoyedu.ch6.ThreadPoolExt`

可以看到，每个任务执行前后都会调用 `beforeExecute` 和 `afterExecute` 方法。相当于执行了一个切面。而在调用 `shutdown` 方法后则会调用 `terminated` 方法。

线程池的工作机制

- 1) 如果当前运行的线程少于 `corePoolSize`，则创建新线程来执行任务（注意，执行这一步骤需要获取全局锁）。
- 2) 如果运行的线程等于或多于 `corePoolSize`，则将任务加入 `BlockingQueue`。
- 3) 如果无法将任务加入 `BlockingQueue`（队列已满），则创建新的线程来处理任务。
- 4) 如果创建新线程将使当前运行的线程超出 `maximumPoolSize`，任务将被拒绝，并调用 `RejectedExecutionHandler.rejectedExecution()` 方法。

提交任务

`execute()` 方法用于提交不需要返回值的任务，所以无法判断任务是否被线程池执行成功。

`submit()` 方法用于提交需要返回值的任务。线程池会返回一个 `future` 类型的对象，通过这个 `future` 对象可以判断任务是否执行成功，并且可以通过 `future` 的 `get()` 方法来获取返回值，`get()` 方法会阻塞当前线程直到任务完成，而使用 `get(long timeout, TimeUnit unit)` 方法则会阻塞当前线程一段时间后立即返回，这时候有可能任务没有执行完。

关闭线程池

可以通过调用线程池的 `shutdown` 或 `shutdownNow` 方法来关闭线程池。它们的原理是遍历线程池中的工作线程，然后逐个调用线程的 `interrupt` 方法来中断线程，所以无法响应中断的任务可能永远无法终止。但是它们存在一定的区别，`shutdownNow` 首先将线程池的状态设置成 `STOP`，然后尝试停止所有的正在执行或暂停任务的线程，并返回等待执行任务的列表，而 `shutdown` 只是将线程池的状态设置成 `SHUTDOWN` 状态，然后中断所有没有正在执行任务的线程。

只要调用了这两个关闭方法中的任意一个，`isShutdown` 方法就会返回 `true`。当所有的任务都已关闭后，才表示线程池关闭成功，这时调用 `isTerminated` 方法会返回 `true`。至于应该调用哪一种方法来关闭线程池，应该由提交到线程池的任务特性决定，通常调用 `shutdown` 方法来关闭线程池，如果任务不一定要执行完，则可以调用 `shutdownNow` 方法。

合理地配置线程池

要想合理地配置线程池，就必须首先分析任务特性

要想合理地配置线程池，就必须首先分析任务特性，可以从以下几个角度来分析。

- 任务的性质：CPU 密集型任务、IO 密集型任务和混合型任务。
- 任务的优先级：高、中和低。
- 任务的执行时间：长、中和短。
- 任务的依赖性：是否依赖其他系统资源，如数据库连接。

性质不同的任务可以用不同规模的线程池分开处理。

CPU 密集型任务应配置尽可能小的线程，如配置 `Ncpu+1` 个线程的线程池。由于 IO 密集型任务线程并不是一直在执行任务，则应配置尽可能多的线程，如 `2*Ncpu`。

混合型的任务，如果可以拆分，将其拆分成一个 CPU 密集型任务和一个 IO 密集型任务，只要这两个任务执行的时间相差不是太大，那么分解后执行的吞吐量将高于串行执行的吞吐量。如果这两个任务执行时间相差太大，则没必要进行分解。可以通过 `Runtime.getRuntime().availableProcessors()` 方法获得当前设备的 CPU 个数。

对于 IO 型的任务的最佳线程数，有个公式可以计算

$$N_{threads} = N_{CPU} * U_{CPU} * (1 + W/C)$$

其中：

- `NCPU` 是处理器的核的数目
- `UCPU` 是期望的 CPU 利用率（该值应该介于 0 和 1 之间）
- `W/C` 是等待时间与计算时间的比率

等待时间与计算时间我们在 Linux 下使用相关的 `vmstat` 命令或者 `top` 命令查看。

优先级不同的任务可以使用优先级队列 `PriorityBlockingQueue` 来处理。它可以让优先级高的任务先执行。

执行时间不同的任务可以交给不同规模的线程池来处理，或者可以使用优先级队列，让执行时间短的任务先执行。

依赖数据库连接池的任务，因为线程提交 SQL 后需要等待数据库返回结果，等待的时间越长，则 CPU 空闲时间就越长，那么线程数应该设置得越大，这样才能更好地利用 CPU。

建议使用有界队列。有界队列能增加系统的稳定性和预警能力，可以根据需要设大一点儿，比如几千。

假设，我们现在有一个 Web 系统，里面使用了线程池来处理业务，在某些情况下，系统里后台任务线程池的队列和线程池全满了，不断抛出抛弃任务的异常，通过排查发现是数据库出现了问题，导致执行 SQL 变得非常缓慢，因为后台任务线程池里的任务全是需要向数据库查询和插入数据的，所以导致线程池里的工作线程全部阻塞，任务积压在线程池里。

如果当时我们设置成无界队列，那么线程池的队列就会越来越多，有可能会撑满内存，导致整个系统不可用，而不只是后台任务出现问题。

预定义线程池

FixedThreadPool 详解

创建使用固定线程数的 `FixedThreadPool` 的 API。适用于为了满足资源管理的需求，而需要限制当前线程数量的应用场景，它适用于负载比较重的服务器。`FixedThreadPool` 的 `corePoolSize` 和 `maximumPoolSize` 都被设置为创建 `FixedThreadPool` 时指定的参数 `nThreads`。

当线程池中的线程数大于 `corePoolSize` 时，`keepAliveTime` 为多余的空闲线程等待新任务的

最长时间，超过这个时间后多余的线程将被终止。这里把 `keepAliveTime` 设置为 `0L`，意味着多余的空闲线程会被立即终止。

`FixedThreadPool` 使用有界队列 `LinkedBlockingQueue` 作为线程池的工作队列（队列的容量为 `Integer.MAX_VALUE`）。

SingleThreadExecutor

创建使用单个线程的 `SingleThread-Executor` 的 API，于需要保证顺序地执行各个任务；并且在任意时间点，不会有多个线程是活动的应用场景。

`corePoolSize` 和 `maximumPoolSize` 被设置为 1。其他参数与 `FixedThreadPool` 相同。`SingleThreadExecutor` 使用有界队列 `LinkedBlockingQueue` 作为线程池的工作队列（队列的容量为 `Integer.MAX_VALUE`）。

CachedThreadPool

创建一个会根据需要创建新线程的 `CachedThreadPool` 的 API。大小无界的线程池，适用于执行很多的短期异步任务的小程序，或者是负载较轻的服务器。

`corePoolSize` 被设置为 0，即 `corePool` 为空；`maximumPoolSize` 被设置为 `Integer.MAX_VALUE`。这里把 `keepAliveTime` 设置为 `60L`，意味着 `CachedThreadPool` 中的空闲线程等待新任务的最长时间为 60 秒，空闲线程超过 60 秒后将会被终止。

`FixedThreadPool` 和 `SingleThreadExecutor` 使用有界队列 `LinkedBlockingQueue` 作为线程池的工作队列。`CachedThreadPool` 使用没有容量的 `SynchronousQueue` 作为线程池的工作队列，但 `CachedThreadPool` 的 `maximumPool` 是无界的。这意

味着，如果主线程提交任务的速度高于 `maximumPool` 中线程处理任务的速度时，`CachedThreadPool` 会不断创建新线程。极端情况下，`CachedThreadPool` 会因为创建过多线程而耗尽 CPU 和内存资源。

WorkStealingPool

利用所有运行的处理器数目来创建一个工作窃取的线程池，使用 `forkjoin` 实现

ScheduledThreadPoolExecutor

使用工厂类 `Executors` 来创建。`Executors` 可以创建 2 种类型的 `ScheduledThreadPoolExecutor`，如下。

- `ScheduledThreadPoolExecutor`。包含若干个线程的 `ScheduledThreadPoolExecutor`。
- `SingleThreadScheduledExecutor`。只包含一个线程的 `ScheduledThreadPoolExecutor`。

`ScheduledThreadPoolExecutor` 适用于需要多个后台线程执行周期任务，同时为了满足资源管理的需求而需要限制后台线程的数量的应用场景。

`SingleThreadScheduledExecutor` 适用于需要单个后台线程执行周期任务，同时需要保证顺序地执行各个任务的应用场景。

提交定时任务

```
public ScheduledFuture<?> schedule(Runnable command, long delay, TimeUnit unit)
```

//向定时任务线程池提交一个延时 `Runnable` 任务（仅执行一次）

```
public <V> ScheduledFuture<V> schedule(Callable<V> callable, long delay,
TimeUnit unit);
```

//向定时任务线程池提交一个延时的 `Callable` 任务（仅执行一次）

```
public ScheduledFuture<?> scheduleAtFixedRate(Runnable command, long
initialDelay, long period, TimeUnit unit)
```

//向定时任务线程池提交一个固定时间间隔执行的任务

```
public ScheduledFuture<?> scheduleWithFixedDelay(Runnable command, long
initialDelay, long delay, TimeUnit unit);
```

//向定时任务线程池提交一个固定延时间隔执行的任务

固定时间间隔的任务不论每次任务花费多少时间，下次任务开始执行时间从理论上讲是确定的，当然执行任务的时间不能超过执行周期。

固定延时间隔的任务是指每次执行完任务以后都延时一个固定的时间。由于操作系统调度以及每次任务执行的语句可能不同，所以每次任务执行所花费的时间是不确定的，也就导致了每次任务的执行周期存在一定的波动。

定时任务超时问题

`scheduleAtFixedRate` 中，若任务处理时长超出设置的定时频率时长，本次任务执行完才开始下次任务，下次任务已经处于超时状态，会马上开始执行。

若任务处理时长小于定时频率时长，任务执行完后，定时器等待，下次任务会在定时器等待频率时长后执行。

如下例子：

设置定时任务每 60s 执行一次，那么从理论上应该第一次任务在第 0s 开始，第二次任务在第 60s 开始，第三次任务在 120s 开始，但实际运行时第一次任务时长 80s，第二次任务时长 30s，第三次任务时长 50s，则实际运行结果为：

第一次任务第 0s 开始,第 80s 结束；

第二次任务第 80s 开始,第 110s 结束(上次任务已超时,本次不会再等待 60s,会马上开始)；

第三次任务第 120s 开始,第 170s 结束.

第四次任务第 180s 开始.....

具体代码，参见 `cn.enjoyedu.ch6.schd. ScheduleWorkerTime`

CompletionService

`CompletionService` 实际上可以看做是 `Executor` 和 `BlockingQueue` 的结合体。`CompletionService` 在接收到要执行的任务时，通过类似 `BlockingQueue` 的 `put` 和 `take` 获得任务执行的结果。

`CompletionService` 的一个实现是 `ExecutorCompletionService`，`ExecutorCompletionService` 把具体的计算任务交给 `Executor` 完成。

在实现上，`ExecutorCompletionService` 在构造函数中会创建一个 `BlockingQueue`（使用的基于链表的 `LinkedBlockingQueue`），该 `BlockingQueue` 的作用是保存 `Executor` 执行的结果。

当提交一个任务到 `ExecutorCompletionService` 时，首先将任务包装成 `QueueingFuture`，它是 `FutureTask` 的一个子类，然后改写 `FutureTask` 的 `done` 方法，之后把 `Executor` 执行的计算结果放入 `BlockingQueue` 中。

与 `ExecutorService` 最主要的区别在于 `submit` 的 `task` 不一定是按照加入时的顺序完成的。`CompletionService` 对 `ExecutorService` 进行了包装，内部维护一个保存 `Future` 对象的 `BlockingQueue`。只有当这个 `Future` 对象状态是结束的时候，才会加入到这个 `Queue` 中，`take()` 方法其实就是 `Producer-Consumer` 中的 `Consumer`。它会从 `Queue` 中取出 `Future` 对象，如果 `Queue` 是空的，就会阻塞在那里，直到有完成的 `Future` 对象加入到 `Queue` 中。所以，先完成的必定先被取出。这样就减少了不必要的等待时间。

参见代码 `cn.enjoyedu.ch6.comps. CompletionCase`，我们可以得出结论：

使用方法一，自己创建一个集合来保存 `Future` 存根并循环调用其返回结果的时候，主线程并不能保证首先获得的是最先完成任务的线程返回值。它只是按加入线程池的顺序返回。因为 `take` 方法是阻塞方法，后面的任务完成了，前面

的任务却没有完成，主程序就那样等待在那儿，只到前面的完成了，它才知道原来后面的也完成了。

使用方法二，使用 `CompletionService` 来维护处理线程不返回结果时，主线程总是能够拿到最先完成的任务的返回值，而不管它们加入线程池的顺序。

7、并发安全

什么是线程安全性

在《Java 并发编程实战》中，定义如下：

当多个线程访问某个类时，不管运行时环境采用何种调度方式或者这些线程将如何交替执行，并且在调用代码中不需要任何额外的同步或者协同，这个类都能表现出正确的行为，那么就称这个类是线程安全的。

线程封闭

实现好的并发是一件困难的事情，所以很多时候我们都想躲避并发。避免并发最简单的方法就是线程封闭。什么是线程封闭呢？

就是把对象封装到一个线程里，只有这一个线程能看到此对象。那么这个对象就算不是线程安全的也不会出现任何安全问题。实现线程封闭有哪些方法呢？

ad-hoc 线程封闭

这是完全靠实现者控制的线程封闭，他的线程封闭完全靠实现者实现。

Ad-hoc 线程封闭非常脆弱，应该尽量避免使用。

栈封闭

栈封闭是我们编程当中遇到的最多的线程封闭。什么是栈封闭呢？简单的说就是局部变量。多个线程访问一个方法，此方法中的局部变量都会被拷贝一份到线程栈中。所以局部变量是不被多个线程所共享的，也就不会出现并发问题。所以能用局部变量就别用全局的变量，全局变量容易引起并发问题。

无状态的类

没有任何成员变量的类，就叫无状态的类，这种类一定是线程安全的。参见代码：`cn.enjoyedu.ch7.safeclass.StatelessClass`。

如果这个类的方法参数中使用了对象，也是线程安全的吗？比如：


```

public class StatelessClass {
    public int service(int a, int b) {
        return a + b;
    }

    public void serviceUser(UserVo user) {
        //do sth
    }
}

```

当然也是，为何？因为多线程下的使用，固然 `user` 这个对象的实例会不正常，但是对于 `StatelessClass` 这个类的对象实例来说，它并不持有 `UserVo` 的对象实例，它自己并不会有问题，有问题的是 `UserVo` 这个类，而非 `StatelessClass` 本身。

让类不可变

让状态不可变，两种方式：

- 1、加 `final` 关键字，对于一个类，所有的成员变量应该是私有的，同样的只要有可能，所有的成员变量应该加上 `final` 关键字，但是加上 `final`，要注意如果成员变量又是一个对象时，这个对象所对应的类也要是不可变，才能保证整个类是不可变的。参见代码 `cn.enjoyedu.ch7.safeClass.ImmutableClass`

- 2、根本就不提供任何可供修改成员变量的地方，同时成员变量也不作为方法的返回值。参见代码 `cn.enjoyedu.ch7.safeClass.ImmutableClassToo`

但是要注意，一旦类的成员变量中有对象，上述的 `final` 关键字保证不可变并不能保证类的安全性，为何？因为在多线程下，虽然对象的引用不可变，但是对象在堆上的实例是有可能被多个线程同时修改的，没有正确处理的情况下，对象实例在堆中的数据是不可预知的。这就牵涉到了如何安全的发布对象这个问题。

```

public class ImmutableClass {
    private final int a;
    private final String b;
    private final UserVo user; //加上他就不安全了

    public UserVo getUser() {
        return user;
    }
}

```

volatile

并不能保证类的线程安全性，只能保证类的可见性，最适合一个线程写，多个线程读的情景。

加锁和 CAS

我们最常使用的保证线程安全的手段，使用 `synchronized` 关键字，使用显式锁，使用各种原子变量，修改数据时使用 `CAS` 机制等等。

安全的发布

类中持有的成员变量，如果是基本类型，发布出去，并没有关系，因为发布出去的其实是这个变量的一个副本，参见代码 `cn.enjoyedu.ch7.safeClass`。

`SafePublish`。

但是如果类中持有的成员变量是对象的引用，如果这个成员对象不是线程安全的，通过 `get` 等方法发布出去，会造成这个成员对象本身持有的数据在多线程下不正确的修改，从而造成整个类线程不安全的问题。参见代码 `cn.enjoyedu.ch7.safeClass`。 `UnsafePublish`，可以看见，

```
private List<Integer> list = new ArrayList<>(initialCapacity: 3);

public UnsafePublish() {
    list.add(1);
    list.add(2);
    list.add(3);
}

public List getList() {
    return list;
}

public static void main(String[] args) {
    UnsafePublish unsafePublish = new UnsafePublish();
    List<Integer> list = unsafePublish.getList();
    System.out.println(list);
    list.add(4);
    System.out.println(list);
    System.out.println(unsafePublish.getList());
}
```

这个 `list` 发布出去后，是可以被外部线程之间修改，那么在多个线程同时修改的情况下不安全问题肯定存在的，怎么修正这个问题呢？我们在发布这对象出去的时候，就应该用线程安全的方式包装这个对象。参见代码

`cn.enjoyedu.ch7.safeClass`。 `SafePublishToo`。我们将 `list` 用

`Collections.synchronizedList` 进行包装以后，无论多少线程使用这个 `list`，就都是线程安全的了。

```
private List<Integer> list
    = Collections.synchronizedList(new ArrayList<>(initialCapacity: 3));
```

对于我们自己使用或者声明的类，JDK 自然没有提供这种包装类的办法，但是我们可以仿造这种模式或者委托给线程安全的类，当然，对这种通过 `get` 等方法发布出去的对象，最根本的解决办法还是应该在实现上就考虑到线程安全问题，参见代码包 `cn.enjoyedu.ch7.safeClass`。 `safePublish` 下的代码

ThreadLocal

`ThreadLocal` 是实现线程封闭的最好方法。`ThreadLocal` 内部维护了一个 `Map`，`Map` 的 `key` 是每个线程的名称，而 `Map` 的值就是我们要封闭的对象。每个线程中的对象都对应着 `Map` 中一个值，也就是 `ThreadLocal` 利用 `Map` 实现了对象的线程封闭。

Servlet 辨析

不是线程安全的类，为什么我们平时没感觉到：

1、在需求上，很少有共享的需求，2、接收到了请求，返回应答的时候，一般都是由一个线程来负责的。

但是只要 Servlet 中有成员变量，一旦有多线程下的写，就很容易产生线程安全问题。

死锁

概念

是指两个或两个以上的进程在执行过程中，由于竞争资源或者由于彼此通信而造成的一种阻塞的现象，若无外力作用，它们都将无法推进下去。此时称系统处于死锁状态或系统产生了死锁。

举个例子：A 和 B 去按摩洗脚，都想在洗脚的时候，同时顺便做个头部按摩，13 技师擅长足底按摩，14 擅长头部按摩。

这个时候 A 先抢到 14，B 先抢到 13，两个人都想同时洗脚和头部按摩，于是就互不相让，扬言我死也不让你，这样的话，A 抢到 14，想要 13，B 抢到 13，想要 14，在这个想同时洗脚和头部按摩的事情上 A 和 B 就产生了死锁。怎么解决这个问题呢？

第一种，假如这个时候，来了个 15，刚好也是擅长头部按摩的，A 又没有两个脑袋，自然就归了 B，于是 B 就美滋滋的洗脚和做头部按摩，剩下 A 在旁边气鼓鼓的，这个时候死锁这种情况就被打破了，不存在了。

第二种，C 出场了，用武力强迫 A 和 B，必须先做洗脚，再头部按摩，这种情况下，A 和 B 谁先抢到 13，谁就可以进行下去，另外一个没抢到的，就等着，这种情况下，也不会产生死锁。

所以总结一下：

死锁是必然发生在多操作者 ($M \geq 2$ 个) 情况下，争夺多个资源 ($N \geq 2$ 个，且 $N \leq M$) 才会发生这种情况。很明显，单线程自然不会有死锁，只有 B 一个去，不要 2 个，打十个都没问题；单资源呢？只有 13，A 和 B 也只会产生激烈竞争，打得不可开交，谁抢到就是谁的，但不会产生死锁。同时，死锁还有一个重要的要求，争夺资源的顺序不对，如果争夺资源的顺序是一样的，也不会产生死锁。

学术化的定义

死锁的发生必须具备以下四个必要条件。

1) 互斥条件：指进程对所分配到的资源进行排它性使用，即在一段时间内某资源只由一个进程占用。如果此时还有其它进程请求资源，则请求者只能等待，直至占有资源的进程用毕释放。

2) 请求和保持条件：指进程已经保持至少一个资源，但又提出了新的资源请求，而该资源已被其它进程占有，此时请求进程阻塞，但又对自己已获得的其它资源保持不放。

3) 不剥夺条件：指进程已获得的资源，在未使用完之前，不能被剥夺，只能在使用完时由自己释放。

4) 环路等待条件：指在发生死锁时，必然存在一个进程——资源的环形链，即进程集合 $\{P_0, P_1, P_2, \dots, P_n\}$ 中的 P_0 正在等待一个 P_1 占用的资源； P_1 正在等待 P_2 占用的资源，……， P_n 正在等待已被 P_0 占用的资源。

理解了死锁的原因，尤其是产生死锁的四个必要条件，就可以最大可能地避免、预防和解除死锁。只要打破四个必要条件之一就能有效预防死锁的发生：打破互斥条件：改造独占性资源为虚拟资源，大部分资源已无法改造。打破不可抢占条件：当一进程占有一独占性资源后又申请一独占性资源而无法获得，则退出原占有的资源。打破占有且申请条件：采用资源预先分配策略，即进程运行前申请全部资源，满足则运行，不然就等待，这样就不会占有且申请。打破循环等待条件：实现资源有序分配策略，对所有设备实现分类编号，所有进程只能采用按序号递增的形式申请资源。

避免死锁常见的算法有有序资源分配法、银行家算法。

现象、危害和解决

在我们 IT 世界有没有存在死锁的情况，有：数据库里多事务而且同时要操作多个表的情况下。所以数据库设计的时候就考虑到了检测死锁和从死锁中恢复的机制。比如 oracle 提供了检测和处理死锁的语句，而 mysql 也提供了“循环依赖检测的机制”

```
Mysql:
频繁报错: Deadlock found when trying to get to lock; try restarting transaction.
1: 查看当前的事务
SELECT * FROM INFORMATION_SCHEMA.INNODB_TRX;
2: 查看当前锁定的事务
SELECT * FROM INFORMATION_SCHEMA.INNODB_LOCKS;
3: 查看当前等锁的事务
SELECT * FROM INFORMATION_SCHEMA.INNODB_LOCK_WAITS;

杀死进程id (就是上面命令的trx_mysql_thread_id列)
kill 线程ID
```

Oracle

1) 用dba用户执行以下语句

```
select username,lockwait,status,machine,program from v$session where sid  
in (select session id from v$locked object)
```

如果有输出的结果, 则说明有死锁, 且能看到死锁的机器是哪一台。字段说明:

Username: 死锁语句所用的数据库用户;

Lockwait: 死锁的状态, 如果有内容表示被死锁。

Status: 状态, active表示被死锁

Machine: 死锁语句所在的机器。

Program: 产生死锁的语句主要来自哪个应用程序。

2) 用dba用户执行以下语句, 可以查看到被死锁的语句

```
select sql_text from v$sql where hash_value in  
(select sql_hash_value from v$session where sid in  
(select session_id from v$locked_object))
```

3) 查找死锁的进程:

```
sqlplus "/as sysdba" (sys/change_on_install)
```

```
SELECT s.username,l.OBJECT_ID,l.SESSION_ID,s.SERIAL#,l.ORACLE_USERNAME,l.OS_USER_NAME,l.PROCESS  
FROM V$LOCKED_OBJECT l,V$SESSION S  
WHERE l.SESSION_ID=S.SID;
```

4) kill掉这个死锁的进程:

```
alter system kill session 'sid,serial#'; (其中sid=l.session_id)
```

在 Java 世界里存在着多线程争夺多个资源, 不可避免的存在着死锁。那么我们在编写代码的时候什么情况下会发生呢?

现象

简单顺序死锁

参见代码 cn.enjoyedu.ch7. NormalDeadLock

动态顺序死锁

顾名思义也是和获取锁的顺序有关, 但是比较隐蔽, 不像简单顺序死锁, 往往从代码一眼就看出获取锁的顺序不对。

参见代码 cn.enjoyedu.ch7.tranfer.service. UserAccount

危害

1、线程不工作了, 但是整个程序还是活着的 2、没有任何的异常信息可以供我们检查。3、一旦程序发生了发生了死锁, 是没有任何的办法恢复的, 只能重启程序, 对生产平台的程序来说, 这是个很严重的问题。

实际工作中的死锁

时间不定, 不是每次必现; 一旦出现没有任何异常信息, 只知道这个应用的所有业务越来越慢, 最后停止服务, 无法确定是哪个具体业务导致的问题; 测试部门也无法复现, 并发量不够。

解决

定位

要解决死锁, 当然要先找到死锁, 怎么找?

通过 **jps** 查询应用的 id, 再通过 **jstack id** 查看应用的锁的持有情况


```

"SubTestThread" #11 prio=5 os_prio=0 tid=0x00000001c559800 nid=0xb3020 waiting for monitor entry [0x000000001d1ae000]
java.lang.Thread.State: BLOCKED (on object monitor)
    at com.xiangxue.ch7.NormalDeadLock.SecondToFisrt(NormalDeadLock.java:34)
    - waiting to lock <0x00000000780ab6378> (a java.lang.Object)
    - locked <0x00000000780ab6388> (a java.lang.Object)
    at com.xiangxue.ch7.NormalDeadLock.access$0(NormalDeadLock.java:28)
    at com.xiangxue.ch7.NormalDeadLock$TestThread.run(NormalDeadLock.java:51)

```

```

"TestDeadLock" #1 prio=5 os_prio=0 tid=0x000000002f20800 nid=0xb33b0 waiting for monitor entry [0x000000002e8f000]
java.lang.Thread.State: BLOCKED (on object monitor)
    at com.xiangxue.ch7.NormalDeadLock.fisrtToSecond(NormalDeadLock.java:21)
    - waiting to lock <0x00000000780ab6378> (a java.lang.Object)
    - locked <0x00000000780ab6378> (a java.lang.Object)
    at com.xiangxue.ch7.NormalDeadLock.main(NormalDeadLock.java:63)

```

Found one Java-level deadlock:

```

=====
"SubTestThread":
    waiting to lock monitor 0x00000000301d4c8 (object 0x00000000780ab6378, a java.lang.Object),
    which is held by "TestDeadLock"
"TestDeadLock":
    waiting to lock monitor 0x00000000301bec8 (object 0x00000000780ab6388, a java.lang.Object),
    which is held by "SubTestThread"

```

Java stack information for the threads listed above:

```

=====
"SubTestThread":
    at com.xiangxue.ch7.NormalDeadLock.SecondToFisrt(NormalDeadLock.java:34)
    - waiting to lock <0x00000000780ab6378> (a java.lang.Object)
    - locked <0x00000000780ab6388> (a java.lang.Object)
    at com.xiangxue.ch7.NormalDeadLock.access$0(NormalDeadLock.java:28)
    at com.xiangxue.ch7.NormalDeadLock$TestThread.run(NormalDeadLock.java:51)
"TestDeadLock":
    at com.xiangxue.ch7.NormalDeadLock.fisrtToSecond(NormalDeadLock.java:21)
    - waiting to lock <0x00000000780ab6378> (a java.lang.Object)
    - locked <0x00000000780ab6378> (a java.lang.Object)
    at com.xiangxue.ch7.NormalDeadLock.main(NormalDeadLock.java:63)

```

Found 1 deadlock.

修正

关键是保证拿锁的顺序一致

两种解决方式

- 1、内部通过顺序比较，确定拿锁的顺序；
- 2、采用尝试拿锁的机制。

参见代码 cn.enjoyedu.ch7.transfer.service.SafeOperate 和 SafeOperateToo

其他安全问题

活锁

两个线程在尝试拿锁的机制中，发生多个线程之间互相谦让，不断发生同一个线程总是拿到同一把锁，在尝试拿另一把锁时因为拿不到，而将本来已经持有的锁释放的过程。

解决办法：每个线程休眠随机数，错开拿锁的时间。

线程饥饿

低优先级的线程，总是拿不到执行时间

并发下的性能

使用并发的目标是为了提高性能，引入多线程后，其实会引入额外的开销，如线程之间的协调、增加的上下文切换，线程的创建和销毁，线程的调度等等。过度的使用和不恰当的使用，会导致多线程程序甚至比单线程还要低。

衡量应用的程序的性能：服务时间，延迟时间，吞吐量，可伸缩性等等，其中服务时间，延迟时间（多快），吞吐量（处理能力的指标，完成工作的多少）。多快和多少，完全独立，甚至是相互矛盾的。

对服务器应用来说：多少（可伸缩性，吞吐量）这个方面比多快更受重视。

我们做应用的时候：

- 1、先保证程序正确，确实达不到要求的时候，再提高速度。（黄金原则）
- 2、一定要以测试为基准。

线程引入的开销

上下文切换

如果主线程是唯一的线程,那么它基本上不会被调度出去。另一方面,如果可运行的线程数大于 CPU 的数量,那么操作系统最终会将某个正在运行的线程调度出来,从而使其他线程能够使用 CPU。这将导致一次上下文切换,在这个过程中将保存当前运行线程的执行上下文,并将新调度进来的线程的执行上下文设置为当前上下文。上下文切换有点像我们同时阅读几本书,在来回切换书本的同时我们需要记住每本书当前读到的页码。

切换上下文需要一定的开销,而在线程调度过程中需要访问由操作系统和 JVM 共享的数据结构。应用程序、操作系统以及 JVM 都使用一组相同的 CPU。在 JVM 和操作系统的代码中消耗越多的 CPU 时钟周期,应用程序的可用 CPU 时钟周期就越少。但上下文切换的开销并不只是包含 JVM 和操作系统的开销。当一个新的线程被切换进来时,它所需要的数据可能不在当前处理器的本地缓存中,因此上下文切换将导致一些缓存缺失,因而线程在首次调度运行时会更加缓慢。

当线程由于等待某个发生竞争的锁而被阻塞时,JVM 通常会将这个线程挂起,并允许它被交换出去。如果线程频繁地发生阻塞,那么它们将无法使用完整的调度时间片。在程序中发生越多的阻塞(包括阻塞 IO,等待获取发生竞争的锁,或者在条件变量上等待),与 CPU 密集型的程序就会发生越多的上下文切换,从而增加调度开销,并因此而降低吞吐量。

上下文切换是计算密集型操作。也就是说,它需要相当可观的处理器时间。所以,上下文切换对系统来说意味着消耗大量的 CPU 时间,事实上,可能是操作系统中时间消耗最大的操作。上下文切换的实际开销会随着平台的不同而变化,然而按照经验来看:在大多数通用的处理器中,上下文切换的开销相当于 50~10000 个时钟周期,也就是几微秒。

UNIX 系统的 `vmstat` 命令能报告上下文切换次数以及在内核中执行时间所占比例等信息。如果内核占用率较高(超过 10%),那么通常表示调度活动发生得很频繁,这很可能是由 IO 或竞争锁导致的阻塞引起的。

内存同步

同步操作的性能开销包括多个方面。在 `synchronized` 和 `volatile` 提供的可见性保证中可能会使用一些特殊指令,即内存栅栏(`Memory Barrier`)。

内存栅栏可以刷新缓存,使缓存无效刷新硬件的写缓冲,以及停止执行管道。

内存栅栏可能同样会对性能带来间接的影响,因为它们将抑制一些编译器优化操作。在内存栅栏中,大多数操作都是不能被重排序的。

阻塞

引起阻塞的原因: 包括阻塞 IO,等待获取发生竞争的锁,或者在条件变量上等待等等。

阻塞会导致线程挂起【挂起: 挂起进程在操作系统中可以定义为暂时被淘汰出内存的进程,机器的资源是有限的,在资源不足的情况下,操作系统对在内存中的程序进行合理的安排,其中有的进程被暂时调离出内存,当条件允许的时候,会被操作系统再次调回内存,重新进入等待被执行的状态即就绪态,系统在超过一定的时间没有任何动作】。

很明显这个操作至少包括两次额外的上下文切换,还有相关的操作系统级的操作等等。

如何减少锁的竞争

减少锁的粒度

使用锁的时候,锁所保护的對象是多个,当这些多个对象其实是独立变化的时候,不如用多个锁来一一保护这些对象。但是如果有同时要持有多个锁的业务方法,要注意避免发生死锁

缩小锁的范围

对锁的持有实现快进快出,尽量缩短持有锁的时间。将一些与锁无关的代码移出锁的范围,特别是一些耗时,可能阻塞的操作

避免多余的锁

两次加锁之间的语句非常简单,导致加锁的时间比执行这些语句还长,这个时候应该进行锁粗化—扩大锁的范围。

锁分段

`ConcurrentHashMap` 就是典型的锁分段。

替换独占锁

在业务允许的情况下：

- 1、使用读写锁，
- 2、用自旋 CAS
- 3、使用系统的并发容器

线程安全的单例模式

参见代码包 cn.enjoyedu.ch7.dcl 下

双重检查锁定

```
5  * 懒汉式-双重检查
6  */
7  public class SingleDcl {
8      private int a;
9      private User user;
10     private static SingleDcl singleDcl;
11     private SingleDcl() {
12     }
13
14     public static SingleDcl getInstance() {
15         if (singleDcl == null) {
16             synchronized (SingleDcl.class) {
17                 if (singleDcl == null) {
18                     singleDcl = new SingleDcl();
19                 }
20             }
21         }
22         return singleDcl;
23     }
24 }
25 singleDcl.getUser.getId() ---> NullPointerException
```

对象的域不一定赋值完成了

对象的引用有了

解决办法，加 **volatile** 关键字

解决之道

懒汉式

类初始化模式，也叫延迟占位模式。在单例类的内部由一个私有静态内部类来持有这个单例类的实例。

延迟占位模式还可以用在多线程下实例域的延迟赋值。

饿汉式

在声明的时候就 **new** 这个类的实例，因为在 JVM 中，对类的加载和类初始化，由虚拟机保证线程安全。

或者使用枚举

8、实战项目

并发任务执行框架

架构师是什么？

在一个软件项目开发过程中，将客户的需求转换为规范的开发计划及文本，并制定这个项目的总体架构，指导整个开发团队完成这个计划的那个人，就是架构师。一般是一个项目里的最资深的专业技术人员，可以说架构师首先一定是个 Java 高级开发人员。

主要职责

主要是架构设计、软件开发，具体来说包括

1、确认需求

在项目开发过程中，架构师是在需求规格说明书完成后介入的，需求规格说明书必须得到架构师的认可。架构师需要和分析人员反复交流，以保证自己完整并准确地理解用户需求。

2、系统分解

依据用户需求，整个系统是否需要分层，如何进行分层，架构师将系统整体分解为更小的子系统和组件，从而形成不同的逻辑层或服务。随后，架构师会确定各层的接口，层与层相互之间的关系。架构师不仅要对整个系统分层，进行“纵向”分解，还要对同一逻辑层分块，进行“横向”分解。

软件架构师的功力基本体现于此，这是一项相对复杂的工作。

3、技术选型

架构师通过对系统的一系列的分解，最终形成了软件的整体架构。技术选择主要取决于软件架构，就是不断找到系统的瓶颈和弱点，采用分而治之、缓存、异步、集群等手段逐渐化解，并平衡处理系统各项要求（性能、安全、可用性、伸缩性、扩展性…）的过程。由此形成了架构。

什么样的架构才是好的架构？

答案：是适用于当前业务和团队成员，并保留适当前瞻性(最多半年的业务增长)的就是好架构。

Web Server 运行在 Windows 上还是 Linux 上？数据库采用 MSSql、Oracle 还是 Mysql？需要不需要采用 MVC 或者 Spring 等轻量级的框架？前端采用富客户端还是瘦客户端方式？类似的工作，都需要在这个阶段提出，并进行评估。

架构师对产品和技术的选型仅仅限于评估，没有决定权，最终的决定权归项目经理。架构师提出的技术方案为项目经理提供了重要的参考信息，项目经理会从项目预算、人力资源、时间进度等实际情况进行权衡，最终进行确认。

4、制定技术规格说明

架构师在项目开发过程中,是技术权威。他需要协调所有的开发人员,与开发人员一直保持沟通,始终保证开发者依照它的架构意图去实现各项功能。

5、核心、关键或者难点任务的开发

6、开发管理

通常还需要承担一些管理职能:规划产品路线、估算人力资源和时间资源、安排人员职责分工,确定计划里程碑点、指导工程师工作、过程风险评估与控制等。这些管理事务需要对产品技术架构、功能模块划分、技术风险都熟悉的架构师参与或直接负责。

7、沟通协调

项目目组内外各种角色沟通协调,可以说架构师相当多的时间用在和人打交道上。处理好人的关系对架构和项目的成功至关重要。

架构师的方方面面

作用

负责系统架构设计,同时也要负责架构的实施落地、演化发展、推广重构。

充当救火队员的角色,系统出现故障或者“灵异现象”,会请他们出马解决。

架构师对某一领域有较深刻的认识,有时候甚至是坚定的技术信仰,乐于同他人分享自己的知识,希望能够推广自己的技术主张。

效果

不管项目有多么艰难复杂,只要有优秀的架构师,大家就会坚信,项目一定能顺利完成。优秀的架构师带给项目组的,不只是技术和方法,更重要的是必胜的信念。这种信念是架构师自己积累起来的气场和影响力。

架构师通常会开发项目中最具技术难度和挑战性的模块,从而为整个项目的顺利进行铺平道路。这些模块包括基础框架、公共组件、通用服务等平台类产品。在大型互联网应用中,基础服务承担着海量的数据存储和核心业务处理服务,有许多挑战性的工作。所以我们的实战就是实现一个基础框架和对一个项目进行性能优化。

实现一个基础框架

需求的产生和分析

公司里有两个项目组,考试组有批量的离线文档要生成,题库组则经常有批量的题目进行排重和根据条件批量修改题目的内容。

架构组通过对实际的上线产品进行用户调查,发现这些功能在实际使用时,用户都反应速度很慢,而且提交任务后,不知道任务的进行情况,做没做?做到哪一步了?有哪些成功?哪些失败了?都一概不知道。

架构组和实际的开发人员沟通,他们都说,因为前端提交任务到 Web 后台以后,是一次要处理多个文档和题目,所以速度快不起来。提示用多线程进行改进,实际的开发人员表示多线程没有用过,不知道如何使用,也担心用不好。综

合以上情况，架构组决定在公司的基础构件库中提供一个并发任务执行框架，以解决上述用户和业务开发人员的痛点：

- 1、对批量型任务提供统一的开发接口
- 2、在使用上尽可能的对业务开发人员友好
- 3、要求可以查询批量任务的执行进度

需要做什么

要实现这么一个批量任务并发执行的框架，我们来分析一下我们要做些什么？

- 1、批量任务，为提高性能，

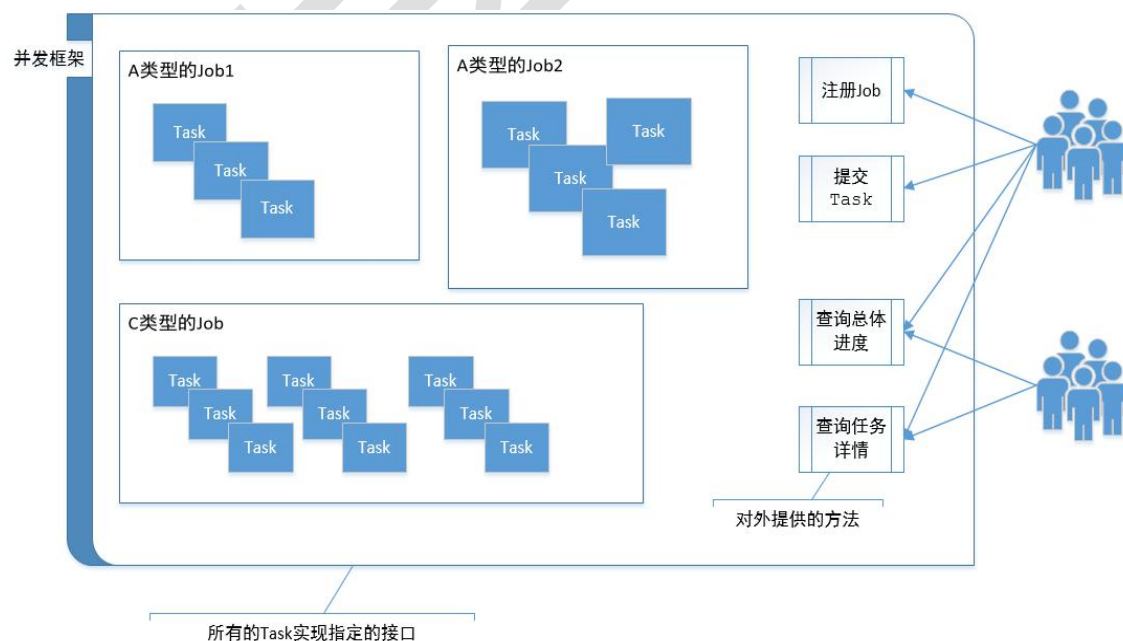
必然的我们要使用 java 里的多线程，为了在使用上尽可能的对业务开发人员友好和简单，需要屏蔽一些底层 java 并发编程中的细节，让他们不需要去了解并发容器，阻塞队列，异步任务，线程安全等方面的知识，只要专心于自己的业务处理即可。

- 4、每个批量任务拥有自己的上下文环境，

因为一个项目组里同时要处理的批量任务可能有多个，比如考试组，可能就会有不同的学校的批量的离线文档生成，而题库组则会不同的学科都会有老师同时进行工作，因此需要一个并发安全的容器保存每个任务的属性信息，

- 3、自动清除已完成和过期任务

因为要提供进度查询，系统需要在内存中维护每个任务的进度信息以供查询，但是这种查询又是有时时间限制的，一个任务完成一段时间后，就不再提供进度查询了，则就需要我们自动清除已完成和过期任务，用定时轮询吗？



具体实现

可查询进度的并发任务执行框架

1、用户业务方法的结果？

一个方法执行的结果有几种可能？三种，成功：按预想的流程出了结果；失败：按预想的流程没出结果；异常：没按预想的流程抛出了预料之外的错误。因此我们定义了一个枚举，表示这三种情况，

```
public enum TaskResultType {  
    Success, /*方法执行完成，业务结果也正确*/  
    Failure, /*方法执行完成，业务结果错误*/  
    Exception /*方法执行抛出了异常*/  
}
```

对于方法的业务执行结果，返回值有很多种可能，基本类型，系统定义的对象类型，用户自定义的对象类型都是存在的，我们需要用泛型来说表示这个结果。同时方法执行失败了，我们还需要告诉用户或者业务开发人员，失败的原因，我们再定义了一个任务的结果类。

```
public class TaskResult<R> {  
  
    private final TaskResultType resultType; /*方法执行结果*/  
  
    private final R returnValue; /*方法执行后的结果数据*/  
  
    private final String reason; /*如果方法失败，这里可以填充原因*/  
}
```

2、如何执行用户的业务方法？

我们是个框架，用户的业务各种各样，都要放到我们框架里执行，怎么办？当然是定义个接口，我们的框架就只执行这个方法，而使用我们框架的业务方都应该来实现这个接口，当然因为用户业务的数据多样性，意味着我们这个方法的参数也应该用泛型。

```
/**  
 *类说明：要求框架使用者实现的任务接口，因为任务的性质在调用时才知道，  
 *所以传入的参数和方法的返回值均使用泛型  
 */  
public interface ITaskProcessor<T, R> {  
    TaskResult<R> taskExecute(T data);  
}
```

3、用户如何提交他的工作和查询任务进度？

用户在前端提交了工作（JOB）到后台，我们需要提供一种封装机制，让业务开发人员可以将任务的相关信息提交给这个封装机制，用户的需要查询进度的时候，也从这个封装机制中取得，同时我们的封装机制内部也要负责清除已完成任务。

在这个封装机制里我们定义了一个类 JobInfo，抽象了对用户工作的封装，一个工作可以包含多个子任务（TASK），这个 JobInfo 中就包括了这个工作的相关信息，比如工作名，用以区分框架中唯一的工作，也可以避免重复提交，也方

便查询时快速定位工作，除了工作名以外，工作中任务的列表，工作中任务的处理器都在其中定义。

```
public class JobInfo<R> {  
  
    private final String jobName; /*工作名，用以区分框架中唯一的工作*/  
  
    private final int jobLength; /*工作中任务的列表*/  
  
    private final ITaskProcessor<?, ?> taskProcessor; /*处理工作中任务的处理器*/  
  
    private AtomicInteger successCount; /*任务的成功次数*/  
    private AtomicInteger taskProcessCount; /*工作中任务目前已经处理的次数*/  
  
    /*存放每个任务的处理结果，供查询用*/  
    private LinkedBlockingDeque<TaskResult<R>> taskDetailQueues;  
  
    private final long expireTime; /*保留的工作的结果信息供查询的时长*/  
}
```

同时 JobInfo 还有相当多的关于这个工作的方法，比如查询工作进度，查询每个任务的处理结果，记录每个任务的处理结果等等

```
JobInfo  
  JobInfo(String, int, ITaskProcessor<?, ?>, long)  
  addTaskResult(TaskResult<R>, CheckJobProc  
  getFailCount(): int  
  getJobLength(): int  
  getSuccCount(): int  
  getTaskDetail(): List<TaskResult<R>>  
  getTaskProcessCount(): int  
  getTaskProcessor(): ITaskProcessor<?, ?>  
  getTotalProcess(): String
```

负责清除已完成任务，我们则交给 CheckJobProcessor 类来完成，定时轮询的机制不够优雅，因此我们选用了 DelayQueue 来实现这个功能

```
/**  
 *类说明：任务完成后，在一定的时间供查询结果，  
 * 之后为释放资源节约内存，需要定期处理过期的任务  
 */  
public class CheckJobProcessor {  
    /*存放任务的队列*/  
    private static DelayQueue<ItemVo<String>> queue  
        = new DelayQueue<~>();  
}
```

并且在其中定义了清除已完成任务的 Runnable 和相关的工作线程。

```
/*处理队列中到期任务*/  
private static class FetchJob implements Runnable{  
  
    static {  
        Thread thread = new Thread(new FetchJob());  
        thread.setDaemon(true);  
        thread.start();  
        System.out.println("开启任务过期检查守护线程.....");  
    }  
}
```

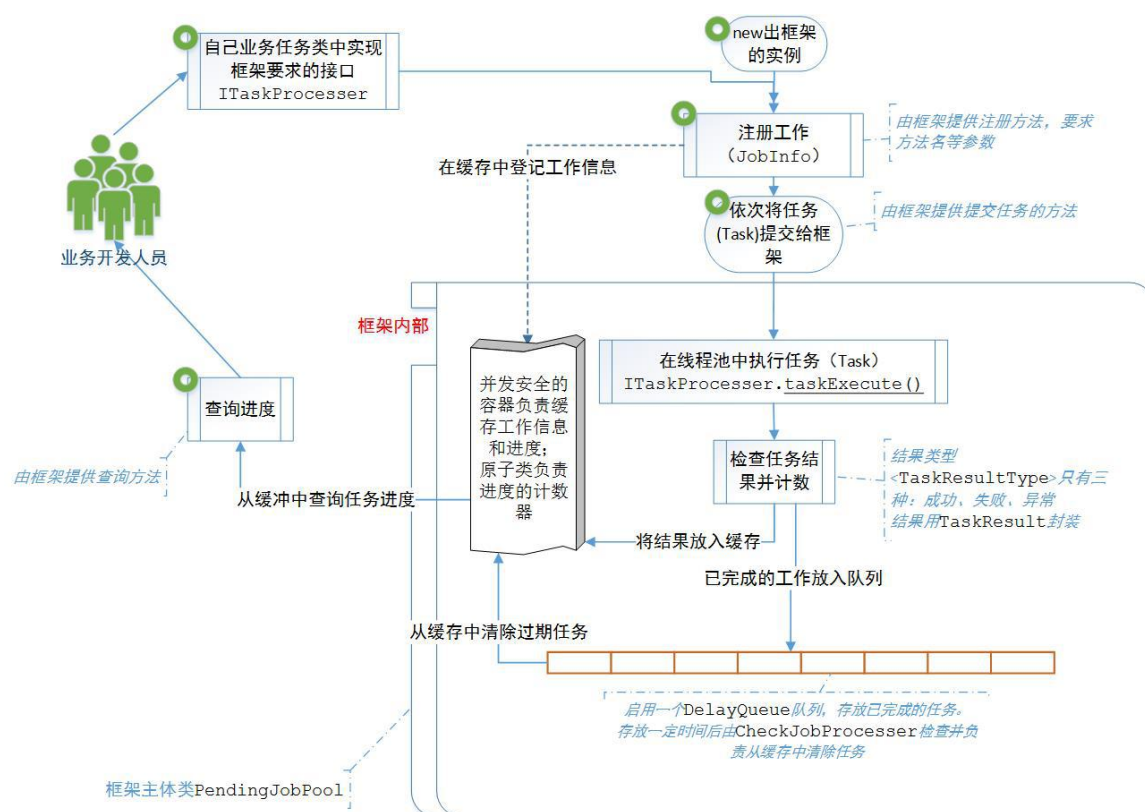

4、框架的主体类

主体类则是 PendingJobPool，这也是业务开发人员主要使用的类。这个类主要负责调度，例如工作（JOB）和任务（TASK）的提交，任务（TASK）的保存，任务（TASK）的并发执行，工作进度的查询接口和任务执行情况的查询等等。

```
/**
 *类说明：框架的主体类，也是调用者主要使用的类
 */
public class PendingJobPool {
    /*框架运行时的线程数，与机器的CPU数相同*/
    private static final int THREAD_COUNTS
        = Runtime.getRuntime().availableProcessors();
    /*队列，线程池使用，用以存放待处理的任务*/
    private static BlockingQueue<Runnable> taskQueue
        = new ArrayBlockingQueue<>( capacity: 5000);
    /*线程池，固定大小，有界队列*/
    private static ExecutorService taskExecutor
        = new ThreadPoolExecutor(THREAD_COUNTS, THREAD_COUNTS,
            keepAliveTime: 60, TimeUnit.SECONDS, taskQueue);
    /*工作信息的存放容器*/
    private static ConcurrentHashMap<String, JobInfo<?>> jobInfoMap
        = new ConcurrentHashMap<>();
}
```

- ▼ PendingJobPool
 - > JobPoolHolder
 - > PendingTask
 - Ⓜ PendingJobPool()
 - Ⓜ getInstance(): PendingJobPool
 - Ⓜ getJob(String): JobInfo<R>
 - Ⓜ getMap(): Map<String, JobInfo<?>>
 - Ⓜ getTaskDetail(String): List<TaskResult<R>>
 - Ⓜ getTaskProgress(String): String
 - Ⓜ putTask(String, T): void
 - Ⓜ registerJob(String, int, ITaskProcessor<?, ?>, long)

流程图



测试

参见代码包 `cn.enjoyedu.ch8a.demo` 下，以及和 `spring` 的集成，在模块 `TaskFramework` 中。

当然和 `spring` 集成的话，某些单例化的部分就不需要了。

性能优化实战

具体优化过程，参见代码包 `cn.enjoyedu.ch8b` 下和视频

项目背景和问题

这个项目来自为电信教育系统设计开发的一套自适应的考试学习系统，面向的用户主要是职业学院的老师和学生以及短时间脱产学习的在职人员。什么叫自适应呢？就是当时提出一种教育理念，对学员的学习要求经常考试进行检查，学员的成绩出来以后，老师会要求系统根据每个学员的考卷上错误的题目从容量为 10 万左右的题库中抽取题目，为每个学员生成一套各自个性化的考后复习和练习的离线练习册。所以，每次考完试，特别是比较大型的考试后，要求生成的离线文档数量是比较多的，一个考试 2000 多人，就要求生成 2000 多份文档。当时我们在做这个项目的时候，因为时间紧，人员少，很快做出第一版就上线运营了。

当然，大家可以想到，问题是很多的，但是反应最大，用户最不满意的就是这个离线文档生成的功能，用户最不满意的点：离线文档生成的速度非常慢，慢

到什么程度呢？一份离线文档的生成平均时长在 50~55 秒左右，遇到成绩不好的学生，文档内容多的，生成甚至需要 3 分钟，大家可以算一下，2000 人，平均 55 秒，全部生成完，需要 $2000 \times 55 = 110000$ 秒，大约是 30 个小时。

为什么如此之慢？这跟离线文档的生成机制密切相关，对于每一个题目要从保存题库的数据库中找到需要的题目，单个题目的表现形式如图，数据库中存储则采用类 html 形式保存，对于每个题目而言，解析题目文本，找到需要下载的图片，每道题目都含有大量或大型的图片需要下载，等到文档中所有题目图片下载到本地完成后，整个文档才能继续进行处理。

分析和改进

第一版的实现，服务器在接收到老师的请求后，就会把批量生成请求分解为一个一个单独的任务，然后串行的完成。



第一版实现

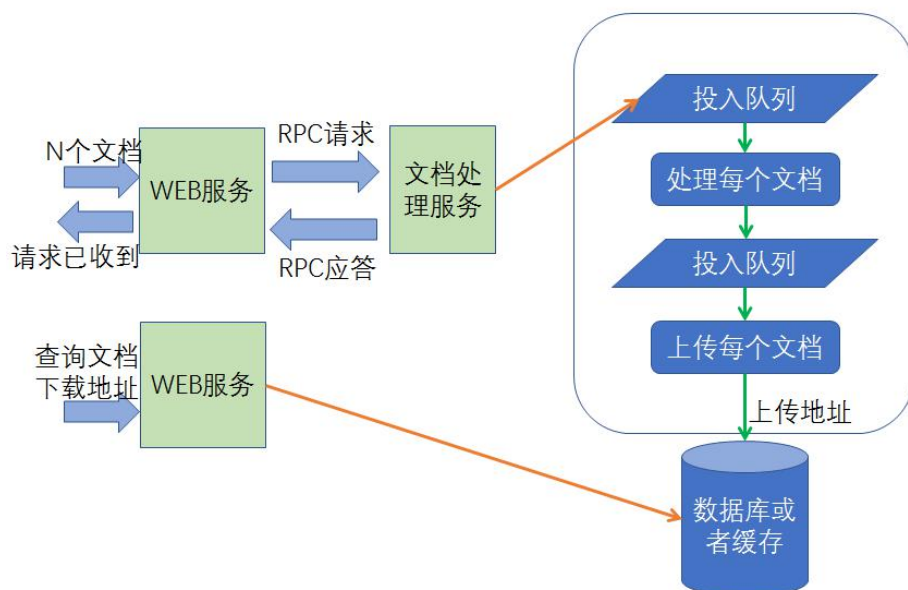
于是在第二版的实现上，首先我们做了个服务拆分，将生成离线文档的功能拆了出来成为了单独的服务，对外提供 RPC 接口，在 WEB 服务器接收到了老师们提出的批量生成离线文档的要求以后，将请求拆分后再一一调用离线文档生成 RPC 服务，这个 RPC 服务在实现的时候有一个缓冲的机制，会将收到的请求进行缓存，然后迅速返回一个结果给调用者，告诉调用者已经收到了请求，这样 WEB 服务器也可以很快的对用户的请求进行应答。

所以我们有了第一次改进，参见 `cn.enjoyedu.ch8b.RpcServiceWebV1`

我们看这个离线文档，每份文档的生成独立性是很高的，天生就适用于多线程并发进行。所以在 RPC 服务实现的时候，使用了生产者消费者模式，RPC 接口的实现收到了一个调用方的请求时，会把请求打包放入一个容器，然后会有多个线程进行消费处理，也就是生成每个具体文档。

当文档生成后，再使用一次生产者消费者模式，投入另一个阻塞队列，由另外的一组线程负责进行上传。当上传成功完成后，由上传线程返回文档的下载地址，表示当前文档已经成功完成。

文档具体的下载地址则由 WEB 服务器单独去数据库或者缓存中去查询。



对于每个离线文档生成本身，我们来看看它的业务，

- 1、从容量为 10 万左右的题库中为每个学生抽取适合他的题目，
- 2、每道题目都含有大量的图片需要下载到本地，和文字部分一起渲染。

但是我们仔细考察整个系统的业务就会发现，我们是在一次考试后为学员生成自适应的练习册，换句话说，不管考试考察的内容如何，学生的成绩如何，每次考试的知识点是有限的，而从这些知识点中可以抽取的相关联的题目数也总是有限的，不同的学生之间所需要的题目会有很大的重复性。

举个例子我们为甲学生因为他考卷上的错误部分抽取了 80 个题目，有很大的概率其他学生跟甲学生错误的地方会有重复，相对应的题目也会有重复。对于这部分题目，我们是完全没有必要重复处理的，包括从数据库中重新获取题目、解析和下载图片。这也是我们可供优化的一大突破点。

其次，一篇练习册是由很多的题目组成的，每个题目相互之间是独立的，我们也可以完全并行的、异步的处理每个题目。

具体怎么做？要避免重复工作肯定是使用缓存机制，对已处理过的题目进行缓存。我们看看怎么使用缓存机制进行优化。这个业务，毋庸置疑，map 肯定是最适合的，因为我们要根据题目的 id 来找题目的详情，用哪个 map？我们现在是在多线程下使用，考虑的是并发安全的 `concurrentHashMap`。

当我们的服务接收到处理一个题目的请求，首先会在缓存中 `get` 一次，没有找到，可以认为这是个新题目，准备向数据库请求题目数据并进行题目的解析，图片的下载。

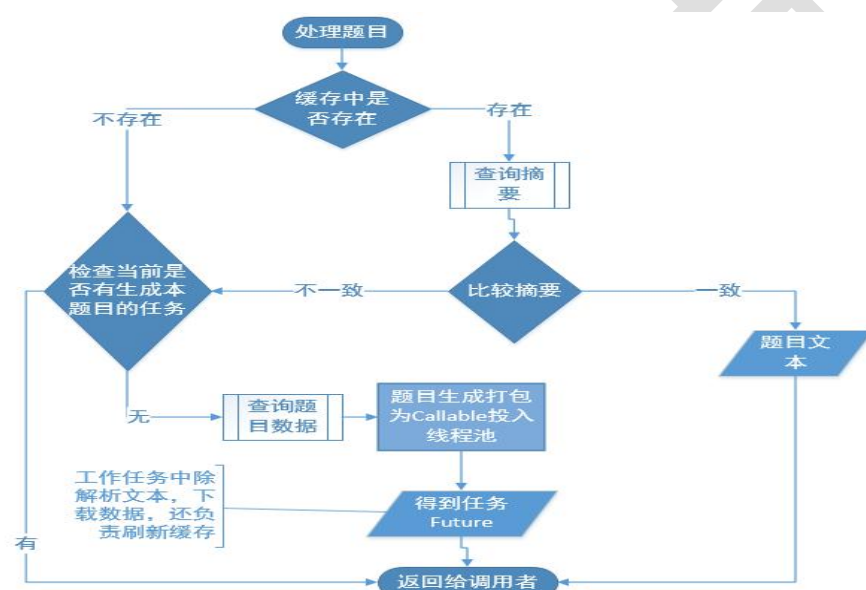
这里有一个并发安全的点需要注意，因为是多线程的应用，会发生多个线程在处理多个文档时有同时进行处理相同题目的情况，这种情况下不做控制，一是会造成数据冲突和混乱，比如同时读写同一个磁盘文件，二是会造成计算资源的浪费，同时为了防止文档的生成阻塞在当前题目上，因此每个新题目的处理过程会包装成一个 `Callable` 投入一个线程池中 而把处理结果作为一个 `Future` 返回，等到线程在实际生成文档时再从 `Future` 中 `get` 出结果进行处理。因此在每个新题目实际处理前，还会检查当前是否有这个题目的处理任务正在进行。

如果题目在缓存中被找到，并不是直接引用就可以了，因为题库中的题目因为种种关系存在被修改的可能，比如存在错误，比如可能内容被替换，这个时候缓存中数据其实是失效过期的，所以需要先行检查一次。如何检查？

我们前面说过题库中的题目平均长度在 800 个字节左右，直接 equals 来检查题目正文是否变动过，明显效率比较低，所以我们这里又做了一番处理，什么处理？对题目正文事先做了一次 SHA 的摘要并保存在数据库，并且要求题库开发小组在处理题目数据入库的时候进行 SHA 摘要。

在本机缓存中同样保存了这个摘要信息，在比较题目是否变动过时，首先检查摘要是否一致，摘要一致说明题目不需要更新，摘要不一致时，才需要更新题目文本，将这个题目视为新题目，进入新题目的处理流程，这样的话就减少了数据的传输量，也降低了数据库的压力。

题目处理的流程就变为：



所以我们有第二次改进，

- 1、在题目实体类 QuestionInDBVo 中增加一个

//题目内容的sha串

```
private final String sha;
```

- 2、增加一个题目保存在缓存中的实体类 QuestionInCacheVo

```
public class QuestionInCacheVo {
    private final String questionDetail;
    private final String questionSha;
```

- 3、增加一个并发处理时返回的题目结果实体类 TaskResultVo

```
public class TaskResultVo {
    private final String questionDetail; //题目处理后的文本
    private final Future<QuestionInCacheVo> questionFuture; //处理题目的任务
```


按照我们前面的描述，我们可以得知，题目要么已经处理完成，要么正在处理，所以在获取题目结果时，先从 `questionDetail` 获取一次，获取为 `null`，则从 `questionFuture` 获取。那么这个类的构造方法需要单独处理一下。

```
public TaskResultVo(String questionDetail) {
    this.questionDetail = questionDetail;
    this.questionFuture = null;
}

public TaskResultVo(Future<QuestionInCacheVo> questionFuture) {
    this.questionDetail = null;
    this.questionFuture = questionFuture;
}
```

4、在处理文档的服务的类 `ProduceDocService` 中增加一个处理文档的新方法 `makeDocAsyn`

```
public static String makeDocAsyn(SrcDocVo pendingDocVo)
    throws ExecutionException, InterruptedException {
```

在这个方法中，会调用一个并发处理题目的方法。

5、增加一个优化题目处理的类 `ParallelQstService`，其中提供了并发处理题目的方法，还包括了

```
public class ParallelQstService {

    //已处理过题目的缓存
    private static ConcurrentHashMap<Integer, QuestionInCacheVo> questionCache
        = new ConcurrentHashMap<>();
    //正在处理的题目的缓存，防止多个线程同时生成一个题目的内容
    private static ConcurrentHashMap<Integer, Future<QuestionInCacheVo>>
        processingQuestionCache = new ConcurrentHashMap<>();
    //处理题目的线程池
    private static ExecutorService makeQuestionExecutor =
        Executors.newCachedThreadPool();
```

主程序参见 `cn.enjoyedu.ch8b.RpcServiceWebV2`

继续改进

数据结构的改进

但是我们仔细分析就会发现，作为一个长期运行的服务，如果我们使用 `concurrentHashMap`，意味着随着时间的推进，缓存对内存的占用会不断的增长。最极端的情况，十万个题目全部被加载到内存，这种情况下会占据多少内存呢？我们做了统计，题库中题目的平均长度在 800 个字节左右，十万个题目大约会使用 75M 左右的空间。

看起来还好，但是有几点，第一，我们除了题目本身还会有其他的一些附属信息需要缓存，比如题目图片在本地磁盘的存储位置等等，那就说，实际缓存的数据内容会远远超过 800 个字节，

第二，`map` 类型的内存使用效率是比较低的，以 `hashmap` 为例，内存利用率一般只有 20%到 40%左右，而 `concurrentHashMap` 只会更低，有时候只有

hashmap 的十分之一到 4 分之一,这也就是说十万个题目放在 `concurrentHashMap` 中会实际占据几百兆的内存空间,是很容易造成内存溢出的,也就是大家常见的 OOM。

考虑到这种情况,我们需要一种数据结构有 `map` 的方便但同时可以限制内存的占用大小或者可以根据需要按照某种策略刷新缓存。最后,在实际的工作中,我们选择了 `ConcurrentLinkedHashMap`,这是由 Google 开源一个线程安全的 `hashmap`,它本身是对 `ConcurrentHashMap` 的封装,可以限定最大容量,并实现一个基于 LRU 也就是最近最少使用算法策略的进行更新的缓存。很完美的契合了我们的要求,对于已经缓冲的题目,越少使用的就可以认为这个题目离当前考试考察的章节越远,被再次选中的概率就越小,在容量已满,需要腾出空间给新缓冲的题目时,越少使用就会优先被清除。

线程数的设置

原来我们设置的线程数按照我们通用的 IO 密集型任务,两个线程池设置的都是机器的 CPU 核心数*2,但是这个就是最佳的吗?不一定,通过反复试验我们发现,处理文档的线程池线程数设置为 CPU 核心数*4,继续提高线程数并不能带来性能上的提升。而因为我们改进后处理文档的时间和上传文档的时间基本在 1: 4 到 1: 3 的样子,所以处理文档的线程池线程数设置为 CPU 核心数*4*3。

这时我们有了第三次改进,参见 `cn.enjoyedu.ch8b.RpcServiceWebV3`

缓存的改进

在这里我们除了本地内存缓存还使用了本地文件存储,启用了二级缓存机制。为什么要使用本地文件存储?因为考虑到服务器会升级、会宕机,已经在内存中缓存的数据会丢失,为了避免这一点,我们将相关的数据在本地进行了一个持久化的操作,保存在了本地磁盘。

改进后的效果

- 1、原单 WEB 串行处理,3 个文档耗时

```
System.out.println("题库开始初始化.....");
SL_QuestionBank.initBank();
System.out.println("题库初始化完成。");
List<SrcDocVo> docList = CreatePendingDocs.makePendingDoc(count 3);
long startTotal = System.currentTimeMillis();
for (SrcDocVo doc: docList) {
    System.out.println("["+doc.getDocName()+"]共有题目: "
        +doc.getQuestionList().size()+"个");
    System.out.println("开始处理文档: "+doc.getDocName()+".....");
    long start = System.currentTimeMillis();
    String localName = ProduceDocService.makeDoc(doc);
    System.out.println("文档"+localName+"生成耗时: "+(System.currentTimeMillis()
        - start));
    String remoteUrl = ProduceDocService.uploadDoc(localName);
    System.out.println("文档"+localName+"上传耗时: "+(System.currentTimeMillis()
        - start));
}
```

文档complete_1557907758105_pending_0.pdf生成耗时: 56044ms
已上传至 [http://www.xxxx.com/file/upload/complete_1557907758105_pending_0.pdf] 耗时: 9105ms
[pending_1]共有题目: 71个
开始处理文档: pending_1.....
开始处理文档: pending_1
文档complete_1557907804256_pending_1.pdf生成耗时: 37046ms
已上传至 [http://www.xxxx.com/file/upload/complete_1557907804256_pending_1.pdf] 耗时: 9213ms
[pending_2]共有题目: 64个
开始处理文档: pending_2.....
开始处理文档: pending_2
文档complete_1557907846618_pending_2.pdf生成耗时: 33149ms
已上传至 [http://www.xxxx.com/file/upload/complete_1557907846618_pending_2.pdf] 耗时: 9397ms
共耗时: 153954ms

平均一个文档耗时 51 秒。

2、服务化，文档生成并行化后，60 个文档耗时

```
//执行文档生成的线程池
private static ExecutorService docMakeService
    = Executors.newFixedThreadPool( nThreads: Consts.THREAD_COUNT*2);
//执行文档上传的线程池
private static ExecutorService docUploadService
    = Executors.newFixedThreadPool( nThreads: Consts.THREAD_COUNT*2);
//执行文档生成的服务
private static CompletionService<String> docCompletionService
    = new ExecutorCompletionService<>(docMakeService);
//执行文档上传的服务
private static CompletionService<String> docUploadCompletionService
    = new ExecutorCompletionService<>(docUploadService);

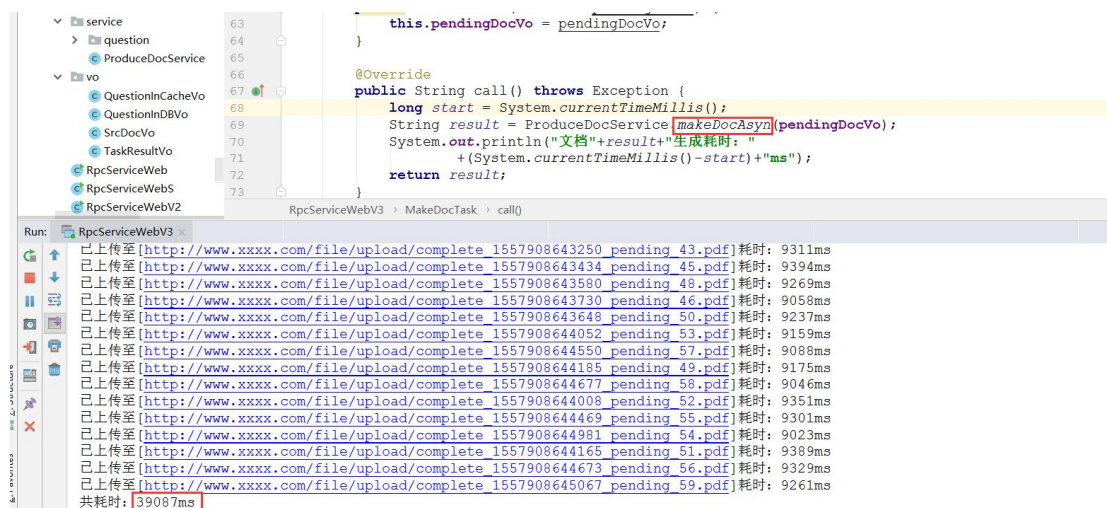
public static ExecutorService getDocMakeService() {
    return docMakeService;
}

public static void main(String[] args)
    throws ExecutionException, InterruptedException {
    int docCount = 60;
}
```

已上传至 [http://www.xxxx.com/file/upload/complete_1557908157960_pending_51.pdf] 耗时: 9108ms
文档complete_1557908167588_pending_54.pdf生成耗时: 51738ms
已上传至 [http://www.xxxx.com/file/upload/complete_1557908159246_pending_53.pdf] 耗时: 9377ms
已上传至 [http://www.xxxx.com/file/upload/complete_1557908165900_pending_52.pdf] 耗时: 9027ms
已上传至 [http://www.xxxx.com/file/upload/complete_1557908166593_pending_55.pdf] 耗时: 9135ms
已上传至 [http://www.xxxx.com/file/upload/complete_1557908167588_pending_54.pdf] 耗时: 9170ms
文档complete_1557908180320_pending_56.pdf生成耗时: 58093ms
文档complete_1557908183040_pending_59.pdf生成耗时: 58128ms
已上传至 [http://www.xxxx.com/file/upload/complete_1557908180320_pending_56.pdf] 耗时: 9327ms
已上传至 [http://www.xxxx.com/file/upload/complete_1557908183040_pending_59.pdf] 耗时: 9166ms
共耗时: 214216ms

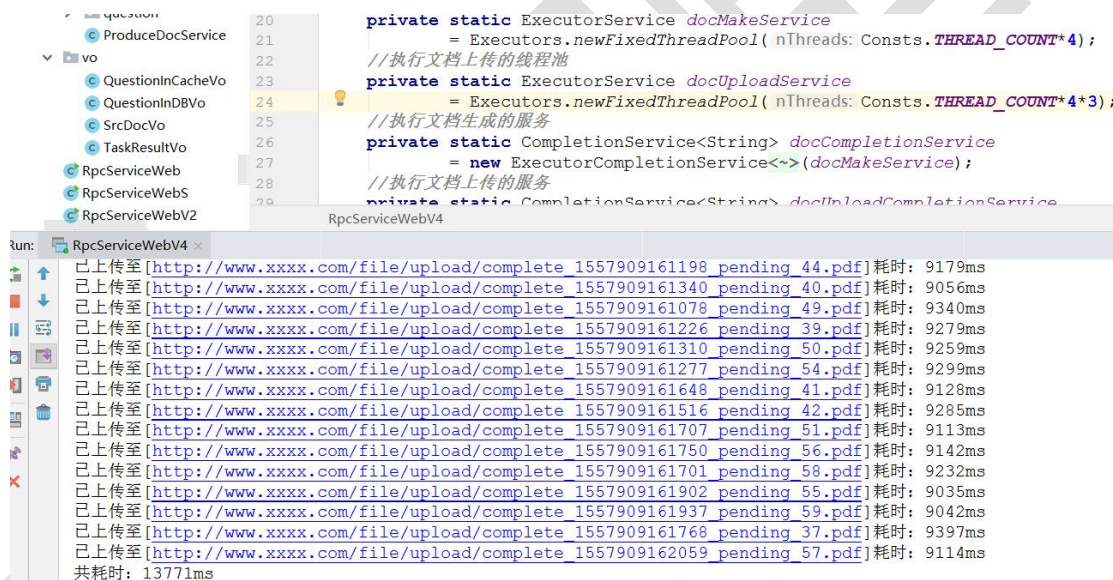
平均一个文档耗时 3.5 秒，已经比单 WEB 串行版的实现有了数量级上的提高。

3、引入缓存避免重复工作、题目处理并行和异步化后，60 个文档耗时



平均一个文档耗时 0.65 秒，再次有了数量级上的提高。

4、调整线程数后，60 个文档耗时



平均一个文档耗时 0.23 秒，再次提升了 3 倍的速度,而相对我们第一版的性能而言，平均一个文档处理性能提升了 $51/0.23=221$ 倍。

这就是善用并发编程后威力！

用户体验的改进

还可以和我们前面实战的并发任务执行框架中的思想相结合，在前端显示处理进度，给用户带来更好的使用体验。

启示

这次项目的优化给我们带来了什么样的启示呢？

性能优化一定要建立在对业务的深入分析上，比如我们在性能优化的切入点，在缓存数据结构的选择就建立在对业务的深入理解上；

性能优化要善于利用语言的高并发特性，

性能优化多多利用缓存，异步任务等机制，正是因为我们使用这些特性和机制，才让我们的应用在性能上有个质的飞跃；

5、引入各种机制的同时要注意避免带来新的不安全因素和瓶颈，比如说缓存数据过期的问题，并发时的线程安全问题，都是需要我们去克服和解决的。

9、JMM 和底层实现原理

JMM 基础-计算机原理

Java 内存模型即 Java Memory Model，简称 JMM。JMM 定义了 Java 虚拟机 (JVM) 在计算机内存 (RAM) 中的工作方式。JVM 是整个计算机虚拟模型，所以 JMM 是隶属于 JVM 的。Java1.5 版本对其进行了重构，现在的 Java 仍沿用了 Java1.5 的版本。Jmm 遇到的问题与现代计算机中遇到的问题是差不多的。

物理计算机中的并发问题，物理机遇到的并发问题与虚拟机中的情况有不少相似之处，物理机对并发的处理方案对于虚拟机的实现也有相当大的参考意义。

根据《Jeff Dean 在 Google 全体工程大会的报告》我们可以看到

操作	响应时间
打开一个站点	几秒
数据库查询一条记录（有索引）	十几毫秒
1.6G的CPU执行一条指令	0.6纳秒
从机械磁盘顺序读取1M数据	2-10毫秒
从SSD磁盘顺序读取1M数据	0.3毫秒
从内存连续读取1M数据	250微秒
CPU读取一次内存	100纳秒
1G网卡，网络传输2Kb数据	20微秒

1秒=1000毫秒 1毫秒=1000微秒 1微秒=1000纳秒

计算机在做一些我们平时的基本操作时，需要的响应时间是不一样的。

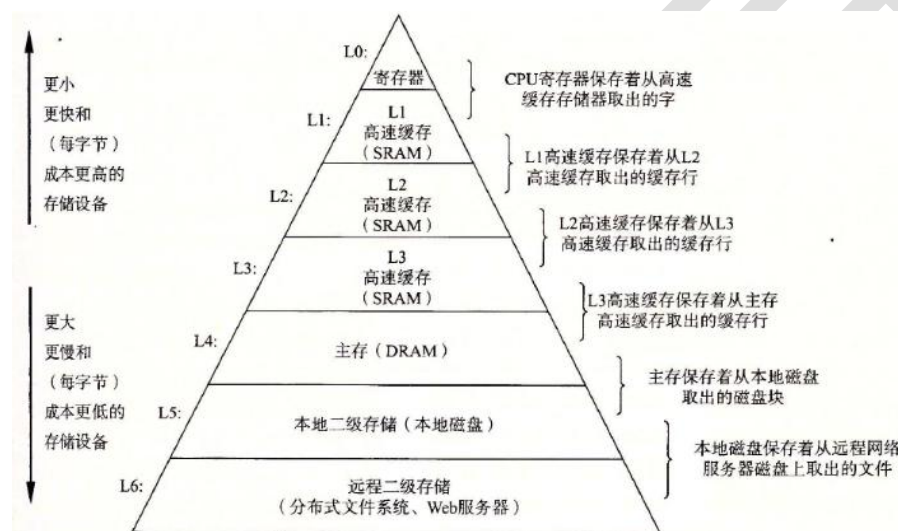
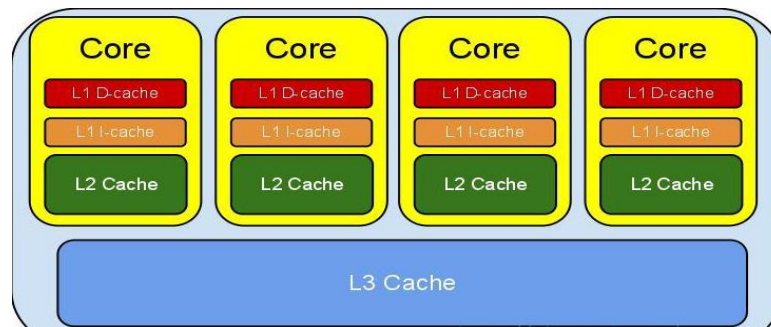
（以下案例仅做说明，并不代表真实情况。）

如果从内存中读取 1M 的 int 型数据由 CPU 进行累加，耗时要多久？

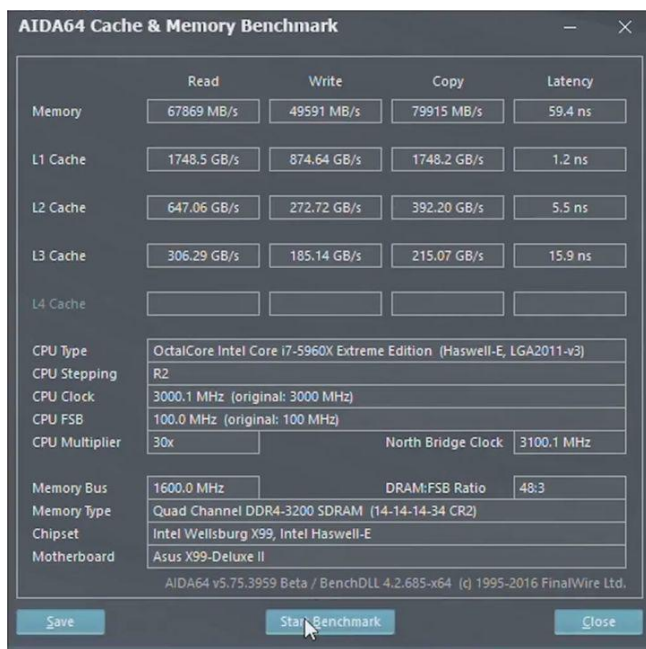
做个简单的计算，1M 的数据，Java 里 int 型为 32 位，4 个字节，共有 $1024 * 1024 / 4 = 262144$ 个整数，则 CPU 计算耗时： $262144 * 0.6 = 157286$ 纳秒，而我们知道从内存读取 1M 数据需要 250000 纳秒，两者虽然有差距（当然这个差距并不小，十万纳秒的时间足够 CPU 执行将近二十万条指令了），但是还在一个数量级上。但是，没有任何缓存机制的情况下，意味着每个数都需要从内存中读取，这样加上 CPU 读取一次内存需要 100 纳秒，262144 个整数从内存读取到 CPU 加上计算时间一共需要 $262144 * 100 + 250000 = 26464400$ 纳秒，这就存在着数量级上的差异了。

而且现实情况中绝大多数的运算任务都不可能只靠处理器“计算”就能完成，处理器至少要与内存交互，如读取运算数据、存储运算结果等，这个 I/O 操作是基本上是无法消除的（无法仅靠寄存器来完成所有运算任务）。早期计算机中

cpu 和内存的速度是差不多的，但在现代计算机中，cpu 的指令速度远超内存的存取速度,由于计算机的存储设备与处理器的运算速度有几个数量级的差距，所以现代计算机系统都不得不加入一层读写速度尽可能接近处理器运算速度的高速缓存（Cache）来作为内存与处理器之间的缓冲：将运算需要使用到的数据复制到缓存中，让运算能快速进行，当运算结束后再从缓存同步回内存之中，这样处理器就无须等待缓慢的内存读写了。



在计算机系统中，寄存器划是 L0 级缓存，接着依次是 L1，L2，L3（接下来是内存，本地磁盘，远程存储）。越往上的缓存存储空间越小，速度越快，成本也更高；越往下的存储空间越大，速度更慢，成本也更低。从上至下，每一层都可以看做是更下一层的缓存，即：L0 寄存器是 L1 一级缓存的缓存，L1 是 L2 的缓存，依次类推；每一层的数据都是来至它的下一层，所以每一层的数据是下一层的数据的子集。



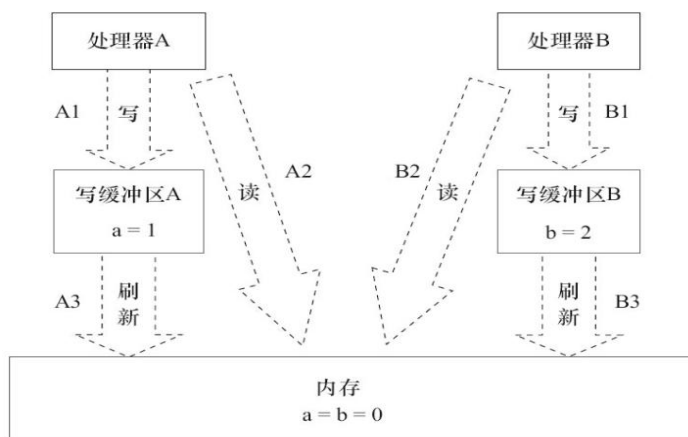
在现代 CPU 上，一般来说 L0， L1， L2， L3 都集成在 CPU 内部，而 L1 还分为一级数据缓存（Data Cache， D-Cache， L1d）和一级指令缓存（Instruction Cache， I-Cache， L1i）， 分别用于存放数据和执行数据的指令解码。每个核心拥有独立的运算处理单元、控制器、寄存器、L1、L2 缓存， 然后一个 CPU 的多个核心共享最后一层 CPU 缓存 L3

物理内存模型带来的问题

基于高速缓存的存储交互很好地解决了处理器与内存的速度矛盾，但是也为计算机系统带来更高的复杂度，因为它引入了一个新的问题：缓存一致性（Cache Coherence）。在多处理器系统中，每个处理器都有自己的高速缓存，而它们又共享同一主内存（MainMemory）。当多个处理器的运算任务都涉及同一块主内存区域时，将可能导致各自的缓存数据不一致。

现代的处理器的使用写缓冲区临时保存向内存写入的数据。写缓冲区可以保证指令流水线持续运行，它可以避免由于处理器停顿下来等待向内存写入数据而产生的延迟。同时，通过以批处理的方式刷新写缓冲区，以及合并写缓冲区中对同一内存地址的多次写，减少对内存总线的占用。虽然写缓冲区有这么多好处，但每个处理器上的写缓冲区，仅仅对它所在的处理器可见。这个特性会对内存操作的执行顺序产生重要的影响：处理器对内存的读/写操作的执行顺序，不一定与内存实际发生的读/写操作顺序一致。

示例项	处理器	Processor A	Processor B
	代码	a = 1; //A1 x = b; //A2	b = 2; //B1 y = a; //B2
运行结果	初始状态: a = b = 0 处理器允许执行后得到结果: x = y = 0		



处理器 A 和处理器 B 按程序的顺序并行执行内存访问，最终可能得到 $x=y=0$ 的结果。

处理器 A 和处理器 B 可以同时把共享变量写入自己的写缓冲区（步骤 A1，B1），然后从内存中读取另一个共享变量（步骤 A2，B2），最后才把自己写缓冲区中保存的脏数据刷新到内存中（步骤 A3，B3）。当以这种时序执行时，程序就可以得到 $x=y=0$ 的结果。

从内存操作实际发生的顺序来看，直到处理器 A 执行 A3 来刷新自己的写缓冲区，写操作 A1 才算真正执行了。虽然处理器 A 执行内存操作的顺序为：A1→A2，但内存操作实际发生的顺序却是 A2→A1。

如果真的发生这种情况，那同步回到主内存时以谁的缓存数据为准呢？为了解决一致性的问题，需要各个处理器访问缓存时都遵循一些协议，在读写时要根据协议来进行操作，这类协议有 MSI、MESI（Illinois Protocol）、MOSI、Synapse、Firefly 及 Dragon Protocol 等。

伪共享

前面我们已经知道，CPU 中有好几级高速缓存。但是 CPU 缓存系统中是以缓存行（cache line）为单位存储的。目前主流的 CPU Cache 的 Cache Line 大小都是 64Bytes。Cache Line 可以简单的理解为 CPU Cache 中的最小缓存单位，今天的 CPU 不再是按字节访问内存，而是以 64 字节为单位的块(chunk)拿取，称为一个缓存行(cache line)。当你读一个特定的内存地址，整个缓存行将从主存换入缓存。

一个缓存行可以存储多个变量（存满当前缓存行的字节数）；而 CPU 对缓存的修改又是以缓存行为最小单位的，在多线程情况下，如果需要修改“共享同一个缓存行的变量”，就会无意中影响彼此的性能，这就是伪共享(False Sharing)。

为了避免伪共享，我们可以使用数据填充的方式来避免，即单个数据填满一个 CacheLine。这本质是一种空间换时间的做法。但是这种方式在 Java7 以后可能失效。

Java8 中已经提供了官方的解决方案，Java8 中新增了一个注解 `@sun.misc.Contended`。

比如 JDK 的 `ConcurrentHashMap` 中就有使用

```
@sun.misc.Contended static final class CounterCell {
    volatile long value;
    CounterCell(long x) { value = x; }
}
```

加上这个注解的类会自动补齐缓存行，需要注意的是此注解默认是无效的，需要在 jvm 启动时设置-XX:-RestrictContended 才会生效。

测试代码，参见 cn.enjoyedu.ch9. FalseSharing

一个类中，只有一个 long 类型的变量：

```
public final static class VolatileLong {
    public volatile long value = 0L;
}
```

定义一个 VolatileLong 类型的数组，然后让多个线程同时并发访问这个数组，这时可以想到，在多个线程同时处理数据时，数组中的多个 VolatileLong 对象可能存在同一个缓存行中。

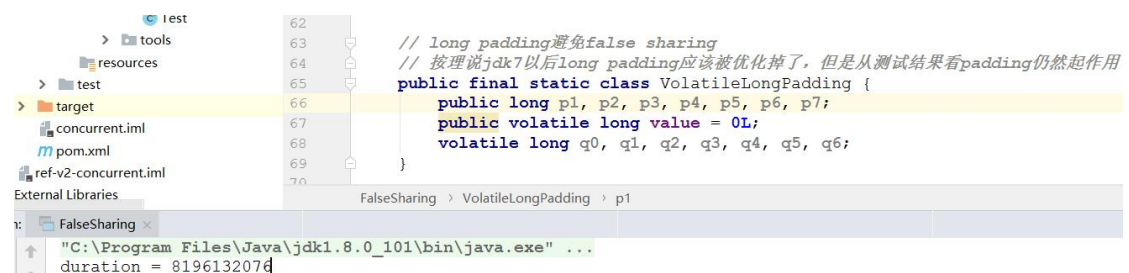
```
private static VolatileLongAnno[] longs = new VolatileLongAnno[NUM_THREADS];
static{
    /*将数组初始化*/
    for (int i = 0; i < longs.length; i++){
        longs[i] = new VolatileLongAnno();
    }
}
```

运行后，可以得到运行时间



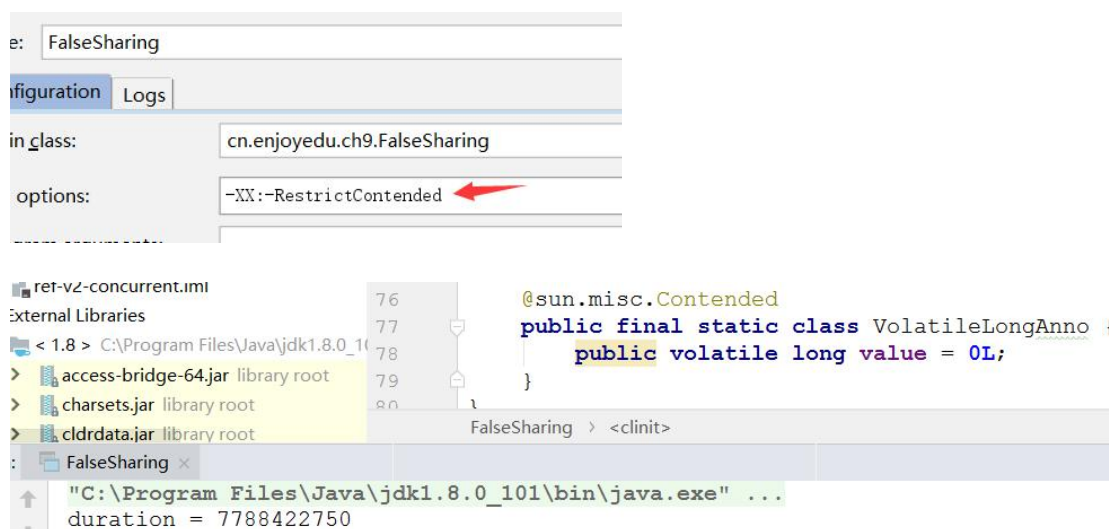
花费了 39 秒多。

我们改用进行了缓存行填充的变量



花费了 8.1 秒，如果任意注释上下填充行的任何一行，时间表现不稳定，从 8 秒到 20 秒都有，但是还是比不填充要快。具体原因目前未知。

再次改用注解标识的变量，同时加入参数-XX:-RestrictContended

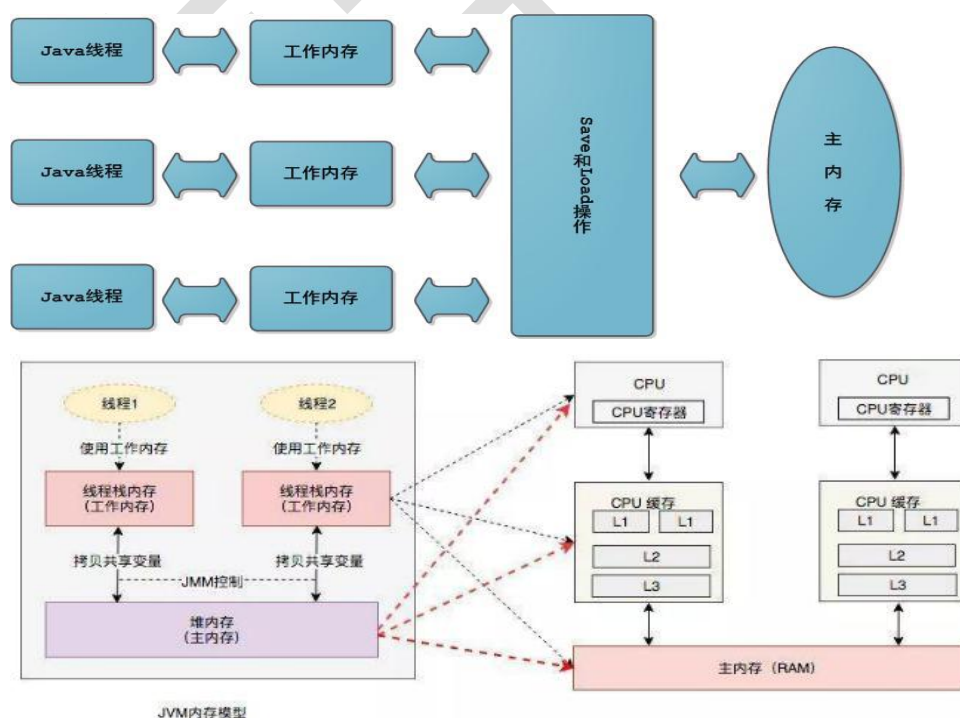


花费了 7.7 秒。

由上述的实验结果表明，伪共享确实会影响应用的性能。

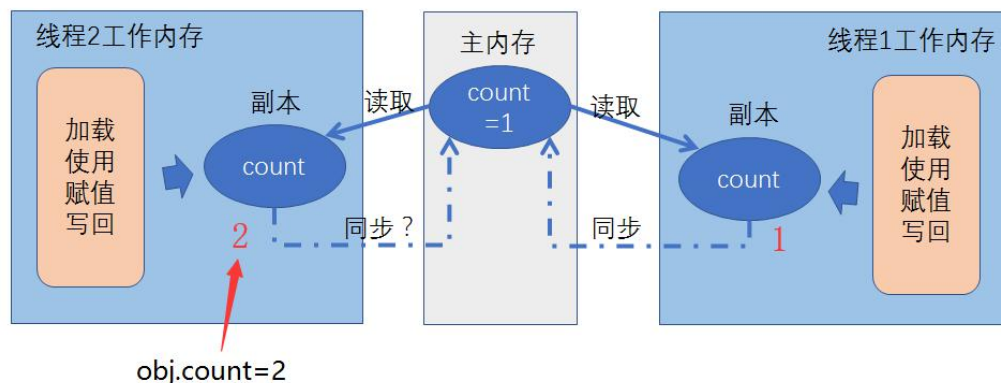
Java 内存模型 (JMM)

从抽象的角度来看，JMM 定义了线程和主内存之间的抽象关系：线程之间的共享变量存储在主内存（Main Memory）中，每个线程都有一个私有的本地内存（Local Memory），本地内存中存储了该线程以读/写共享变量的副本。本地内存是 JMM 的一个抽象概念，并不真实存在。它涵盖了缓存、写缓冲区、寄存器以及其他的硬件和编译器优化。



Java 内存模型带来的问题

可见性问题



可见性问题

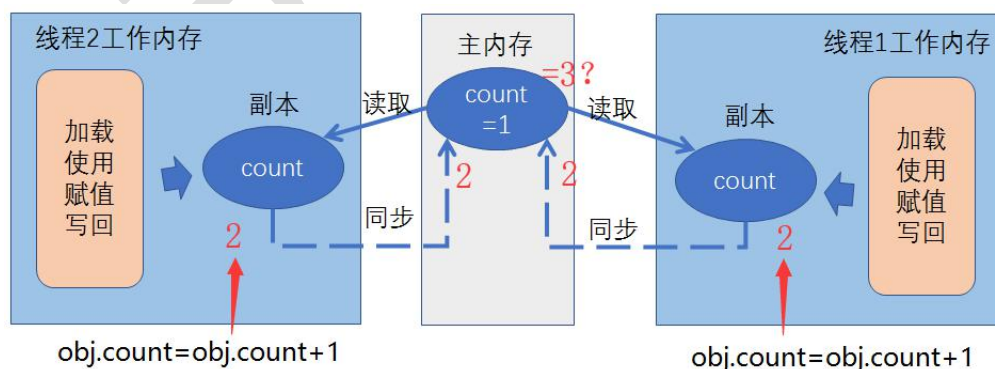
左边 CPU 中运行的线程从主存中拷贝共享对象 `obj` 到它的 CPU 缓存，把对象 `obj` 的 `count` 变量改为 2。但这个变更对运行在右边 CPU 中的线程不可见，因为这个更改还没有 flush 到主存中。

在多线程的环境下，如果某个线程首次读取共享变量，则首先到主内存中获取该变量，然后存入工作内存中，以后只需要在工作内存中读取该变量即可。同样如果对该变量执行了修改的操作，则先将新值写入工作内存中，然后再刷新至主内存中。但是什么时候最新的值会被刷新至主内存中是不太确定，一般来说会很快，但具体时间不知。

要解决共享对象可见性这个问题，我们可以使用 `volatile` 关键字或者是加锁。

竞争问题

线程 A 和线程 B 共享一个对象 `obj`。假设线程 A 从主存读取 `Obj.count` 变量到自己的 CPU 缓存，同时，线程 B 也读取了 `Obj.count` 变量到它的 CPU 缓存，并且这两个线程都对 `Obj.count` 做了加 1 操作。此时，`Obj.count` 加 1 操作被执行了两次，不过都在不同的 CPU 缓存中。



竞争问题

如果这两个加 1 操作是串行执行的，那么 `Obj.count` 变量便会在原始值上加 2，最终主存中的 `Obj.count` 的值会是 3。然而图中两个加 1 操作是并行的，不管是线程 A 还是线程 B 先 flush 计算结果到主存，最终主存中的 `Obj.count` 只会增加 1 次变成 2，尽管一共有两次加 1 操作。要解决上面的问题我们可以使用 `java synchronized` 代码块

重排序

重排序类型

除了共享内存和工作内存带来的问题，还存在重排序的问题：在执行程序时，为了提高性能，编译器和处理器常常会对指令做重排序。重排序分 3 种类型。

1) 编译器优化的重排序。编译器在不改变单线程程序语义的前提下，可以重新安排语句的执行顺序。

2) 指令级并行的重排序。现代处理器采用了指令级并行技术（Instruction-Level Parallelism, ILP）来将多条指令重叠执行。如果不存在数据依赖性，处理器可以改变语句对应机器指令的执行顺序。

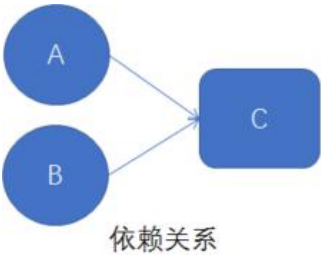
3) 内存系统的重排序。由于处理器使用缓存和读/写缓冲区，这使得加载和存储操作看上去可能是在乱序执行。

数据依赖性

数据依赖性：如果两个操作访问同一个变量，且这两个操作中有一个为写操作，此时这两个操作之间就存在数据依赖性。数据依赖分为下列 3 种类型，上面 3 种情况，只要重排序两个操作的执行顺序，程序的执行结果就会被改变。

名称	代码示例	说明
写后读	<code>a = 1;</code> <code>b = a;</code>	写一个变量之后，再读这个位置。
写后写	<code>a = 1;</code> <code>a = 2;</code>	写一个变量之后，再写这个变量。
读后写	<code>a = b;</code> <code>b = 1;</code>	读一个变量之后，再写这个变量。

例如：



很明显，A 和 C 存在数据依赖，B 和 C 也存在数据依赖，而 A 和 B 之间不存在数据依赖，如果重排序了 A 和 C 或者 B 和 C 的执行顺序，程序的执行结果就会被改变。

很明显，不管如何重排序，都必须保证代码在单线程下的运行正确，连单线程下都无法正确，更不用讨论多线程并发的情况，所以就提出了一个 **as-if-serial** 的概念。

as-if-serial

as-if-serial 语义的意思是：不管怎么重排序（编译器和处理器为了提高并行度），（单线程）程序的执行结果不能被改变。编译器、runtime 和处理器都必须遵守 **as-if-serial** 语义。

为了遵守 **as-if-serial** 语义，编译器和处理器不会对存在数据依赖关系的操作做重排序，因为这种重排序会改变执行结果。（强调一下，这里所说的数据依赖性仅针对单个处理器中执行的指令序列和单个线程中执行的操作，不同处理器之间和不同线程之间的数据依赖性不被编译器和处理器考虑。）但是，如果操作之间不存在数据依赖关系，这些操作依然可能被编译器和处理器重排序。

A 和 C 之间存在数据依赖关系，同时 B 和 C 之间也存在数据依赖关系。因此在最终执行的指令序列中，C 不能被重排序到 A 和 B 的前面（C 排到 A 和 B 的前面，程序的结果将会被改变）。但 A 和 B 之间没有数据依赖关系，编译器和处理器可以重排序 A 和 B 之间的执行顺序。



as-if-serial 语义把单线程程序保护了起来，遵守 **as-if-serial** 语义的编译器、runtime 和处理器可以让我们感觉到：单线程程序看起来是按程序的顺序来执行的。**as-if-serial** 语义使单线程程序员无需担心重排序会干扰他们，也无需担心内存可见性问题。

控制依赖性

```
int a = 0;
boolean flag = false;

public void init() {
    a = 1; // 1
    flag = true; // 2
    //.....
}

public void use() {
    if (flag) { // 3
        int i = a * a; // 4
    }
    //.....
}
```

上述代码中，flag 变量是个标记，用来标识变量 a 是否已被写入，在 use 方法中变量 i 的赋值依赖 if (flag) 的判断，这里就叫控制依赖，如果发生了重排序，结果就不对了。

考察代码，我们可以看见，

操作 1 和操作 2 没有数据依赖关系，编译器和处理器可以对这两个操作重排序；同样，操作 3 和操作 4 没有数据依赖关系，编译器和处理器也可以对这两个操作重排序。操作 3 和操作 4 则存在所谓**控制依赖关系**。

在程序中，当代码中存在控制依赖性时，会影响指令序列执行的并行度。为此，编译器和处理器会采用猜测（Speculation）执行来克服控制相关性对并行度的影响。以处理器的猜测执行为例，执行线程 B 的处理器可以提前读取并计算 $a*a$ ，然后把计算结果临时保存到一个名为重排序缓冲（Reorder Buffer, ROB）的硬件缓存中。当操作 3 的条件判断为真时，就把该计算结果写入变量 i 中。猜测执行实质上对操作 3 和 4 做了重排序，问题在于这时候，a 的值还没被线程 A 赋值。

在单线程程序中，对存在控制依赖的操作重排序，不会改变执行结果（这也是 as-if-serial 语义允许对存在控制依赖的操作做重排序的原因）。

但是对多线程来说就完全不同了：这里假设有两个线程 A 和 B，A 首先执行 init () 方法，随后 B 线程接着执行 use () 方法。线程 B 在执行操作 4 时，能否看到线程 A 在操作 1 对共享变量 a 的写入呢？答案是：不一定能看到。

让我们先来看看，当操作 1 和操作 2 重排序，操作 3 和操作 4 重排序时，可能会产生什么效果？操作 1 和操作 2 做了重排序。程序执行时，线程 A 首先写标记变量 flag，随后线程 B 读这个变量。由于条件判断为真，线程 B 将读取变量 a。此时，变量 a 还没有被线程 A 写入，这时就会发生错误！

所以在多线程程序中，对存在控制依赖的操作重排序，可能会改变程序的执行结果。

内存屏障

Java 编译器在生成指令序列的适当位置会插入内存屏障指令来禁止特定类型的处理器重排序，从而让程序按我们预想的流程去执行。

- 1、保证特定操作的执行顺序。
- 2、影响某些数据（或则是某条指令的执行结果）的内存可见性。

编译器和 CPU 能够重排序指令，保证最终相同的结果，尝试优化性能。插入一条 Memory Barrier 会告诉编译器和 CPU：不管什么指令都不能和这条 Memory Barrier 指令重排序。

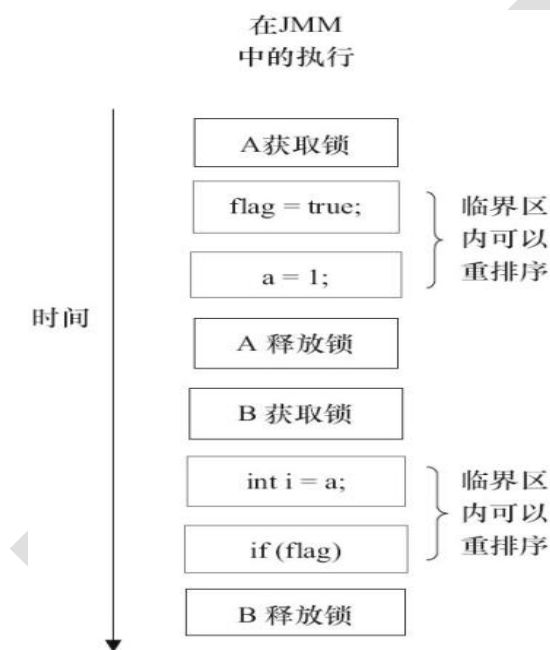
Memory Barrier 所做的另外一件事是强制刷出各种 CPU cache，如一个 Write-Barrier（写入屏障）将刷出所有在 Barrier 之前写入 cache 的数据，因此，任何 CPU 上的线程都能读取到这些数据的最新版本。

JMM 把内存屏障指令分为 4 类

屏障类型	指令示例	说明
LoadLoad Barriers	Load1; LoadLoad ; Load2	确保Load1数据的装载，之前于Load2及所有后续装载指令的装载。
StoreStore Barriers	Store1; StoreStore ; Store2	确保Store1数据对其他处理器可见（刷新到内存），之前于Store2及所有后续存储指令的存储。
LoadStore Barriers	Load1; LoadStore ; Store2	确保Load1数据装载，之前于Store2及所有后续的存储指令刷新到内存。
StoreLoad Barriers	Store1; StoreLoad ; Load2	确保Store1数据对其他处理器变得可见（指刷新到内存），之前于Load2及所有后续装载指令的装载。StoreLoad Barriers会使该屏障之前的所有内存访问指令（存储和装载指令）完成之后，才执行该屏障之后的内存访问指令。

StoreLoad Barriers 是一个“全能型”的屏障，它同时具有其他 3 个屏障的效果。现代的多处理器大多支持该屏障（其他类型的屏障不一定被所有处理器支持）。

临界区



JMM 会在退出临界区和进入临界区这两个关键时间点做一些特别处理，使得多线程在这两个时间点按某种顺序执行。

临界区内的代码则可以重排序（但 JMM 不允许临界区内的代码“逸出”到临界区之外，那样会破坏监视器的语义）。虽然线程 A 在临界区内做了重排序，但由于监视器互斥执行的特性，这里的线程 B 根本无法“观察”到线程 A 在临界区内的重排序。这种重排序既提高了执行效率，又没有改变程序的执行结果。

回想一下，为啥线程安全的单例模式中一般的双重检查不能保证真正的线程安全？

happens-before

在 Java 规范提案中为让大家理解内存可见性的这个概念，提出了 happens-before 的概念来阐述操作之间的内存可见性。对应 Java 程序员来说，理解 happens-before 是理解 JMM 的关键。

JMM 这么做的原因是：程序员对于这两个操作是否真的被重排序并不关心，程序员关心的是程序执行时的语义不能被改变（即执行结果不能被改变）。因此，happens-before 关系本质上和 as-if-serial 语义是一回事。as-if-serial 语义保证单线程内程序的执行结果不被改变，happens-before 关系保证**正确同步**的多线程程序的执行结果不被改变。

定义

用 happens-before 的概念来阐述操作之间的内存可见性。在 JMM 中，如果一个操作执行的结果需要对另一个操作可见，那么这两个操作之间必须要存在 happens-before 关系。

两个操作之间具有 happens-before 关系，并不意味着前一个操作必须要在后一个操作之前执行！happens-before 仅仅要求前一个操作（执行的结果）对后一个操作可见，且前一个操作按顺序排在第二个操作之前（the first is visible to and ordered before the second）

加深理解

上面的定义看起来很矛盾，其实它是站在不同的角度说的。

1) 站在 Java 程序员的角度来说：JMM 保证，如果一个操作 happens-before 另一个操作，那么第一个操作的执行结果将对第二个操作可见，而且第一个操作的执行顺序排在第二个操作之前。

2) 站在编译器和处理器的角度来说：JMM 允许，两个操作之间存在 happens-before 关系，**不要求 Java 平台的具体实现必须要按照 happens-before 关系指定的顺序来执行**。如果重排序之后的执行结果，与按 happens-before 关系来执行的结果一致，那么这种重排序是允许的。

回顾我们前面存在数据依赖性的代码：

```
double x = 0.03 //A
double y = 0.01 //B
double z = x*x*y //C
```

站在我们 Java 程序员的角度：

- 1、A happens-before B
- 2、B happens-before C
- 3、A happens-before C

但是仔细考察，2、3 是必需的，而 1 并不是必需的，因此 JMM 对这三个 happens-before 关系的处理就分为两类：

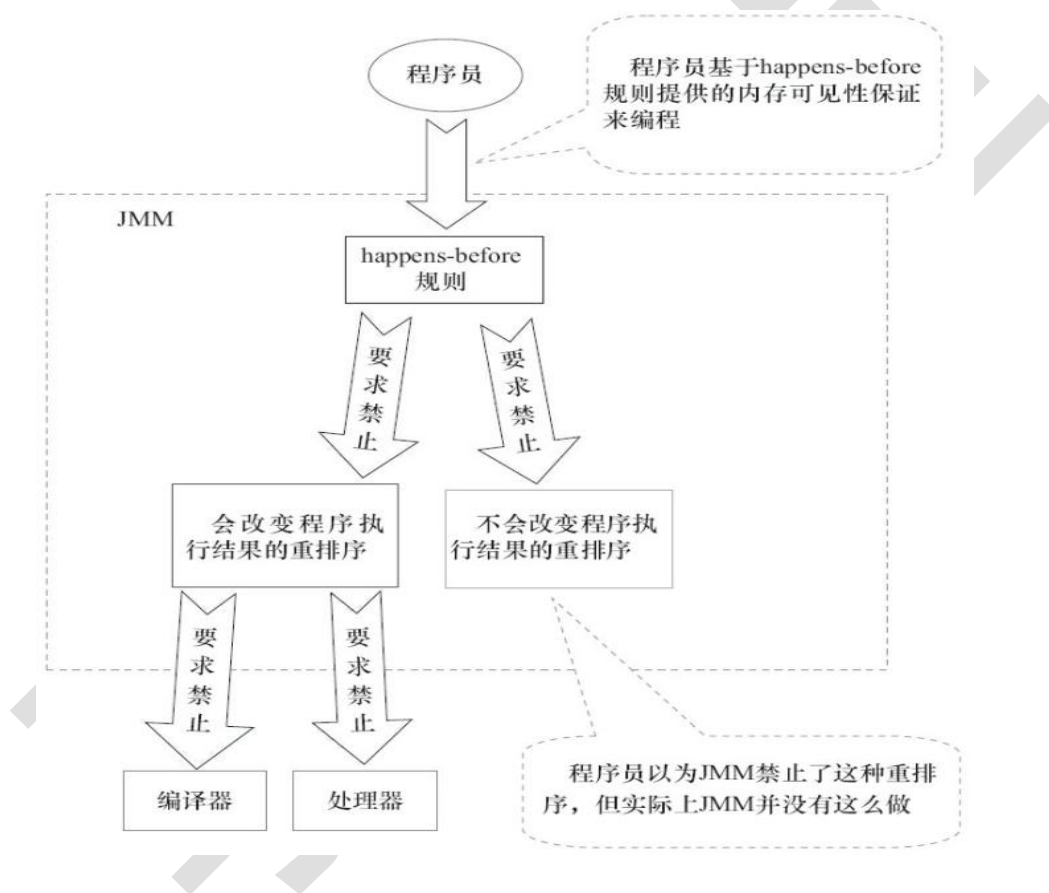
1. 会改变程序执行结果的重排序
2. 不会改变程序执行结果的重排序

JMM 对这两种不同性质的重排序，采用了不同的策略，如下：

1. 对于会改变程序执行结果的重排序，JMM 要求编译器和处理器必须禁止这种重排序；

2. 对于不会改变程序执行结果的重排序，JMM 对编译器和处理器不做要求。

于是，站在我们程序员的角度，看起来这个三个操作满足了 happens-before 关系，而站在编译器和处理器的角度，进行了重排序，而排序后的执行结果，也是满足 happens-before 关系的。



Happens-Before 规则

JMM 为我们提供了以下的 Happens-Before 规则：

1) 程序顺序规则：一个线程中的每个操作，happens-before 于该线程中的任意后续操作。

2) 监视器锁规则：对一个锁的解锁，happens-before 于随后对这个锁的加锁。

3) volatile 变量规则：对一个 volatile 域的写，happens-before 于任意后续对这个 volatile 域的读。

4) 传递性: 如果 A happens-before B, 且 B happens-before C, 那么 A happens-before C。

5) start()规则: 如果线程 A 执行操作 ThreadB.start() (启动线程 B), 那么 A 线程的 ThreadB.start()操作 happens-before 于线程 B 中的任意操作。

6) join()规则: 如果线程 A 执行操作 ThreadB.join()并成功返回, 那么线程 B 中的任意操作 happens-before 于线程 A 从 ThreadB.join()操作成功返回。

7) 线程中断规则:对线程 interrupt 方法的调用 happens-before 于被中断线程的代码检测到中断事件的发生。

volatile 详解

volatile 特性

可以把对 volatile 变量的单个读/写, 看成是使用同一个锁对这些单个读/写操作做了同步

```
public class Vola {
    volatile long i = 0L; // 使用volatile声明64位的long型变量

    public long getI() {
        return i; // 单个volatile变量的读
    }

    public void setI(long i) {
        this.i = i; // 单个volatile变量的写
    }

    public void inc() {
        i++; // 复合(多个)volatile变量的读/写
    }
}
```

可以看成

```
public class VolaLikeSyn {
    long i = 0L; // 普通的long型变量

    public synchronized long getI() { // 对单个的普通变量的写用同一个锁同步
        return i;
    }

    public synchronized void setI(long i) { // 对单个的普通变量的读用同一个锁同步
        this.i = i; // 单个volatile变量的写
    }

    public void inc() { // 普通方法调用
        long temp = getI(); // 调用已同步的读方法
        temp = temp + 1L; // 普通写操作
        setI(temp); // 调用已同步的写方法
    }
}
```

所以 volatile 变量自身具有下列特性:

可见性。对一个 `volatile` 变量的读，总是能看到（任意线程）对这个 `volatile` 变量最后的写入。

原子性：对任意单个 `volatile` 变量的读/写具有原子性，但类似于 `volatile++` 这种复合操作不具有原子性。

`volatile` 的内存语义

内存语义：可以简单理解为 `volatile`, `synchronize`, `atomic`, `lock` 之类的在 JVM 中的内存方面实现原则。

`volatile` 写的内存语义如下：

当写一个 `volatile` 变量时，JMM 会把该线程对应的本地内存中的共享变量值刷新到主内存。

`volatile` 读的内存语义如下：

当读一个 `volatile` 变量时，JMM 会把该线程对应的本地内存置为无效。线程接下来将从主内存中读取共享变量。

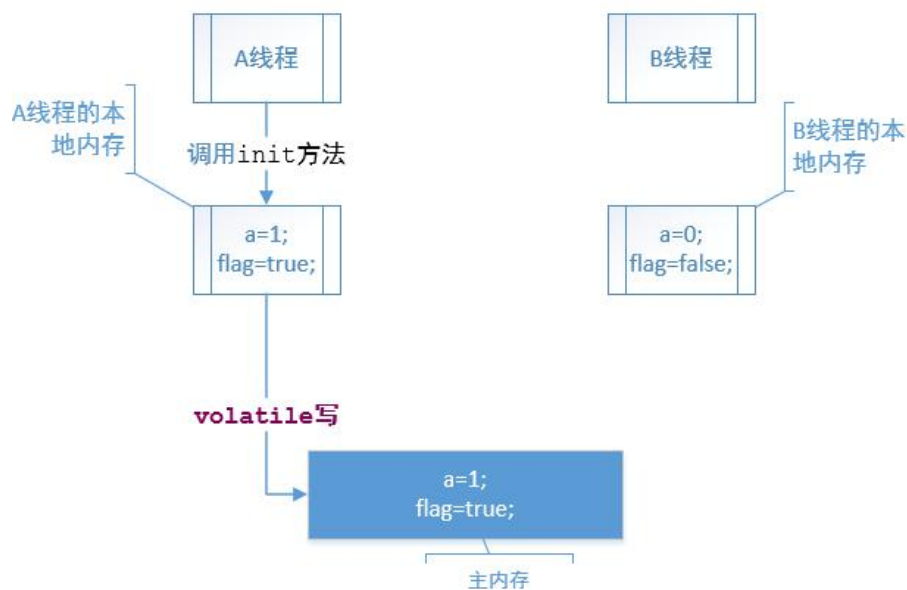
所以对于代码

```
public class ControlDep {
    int a = 0;
    volatile boolean flag = false;

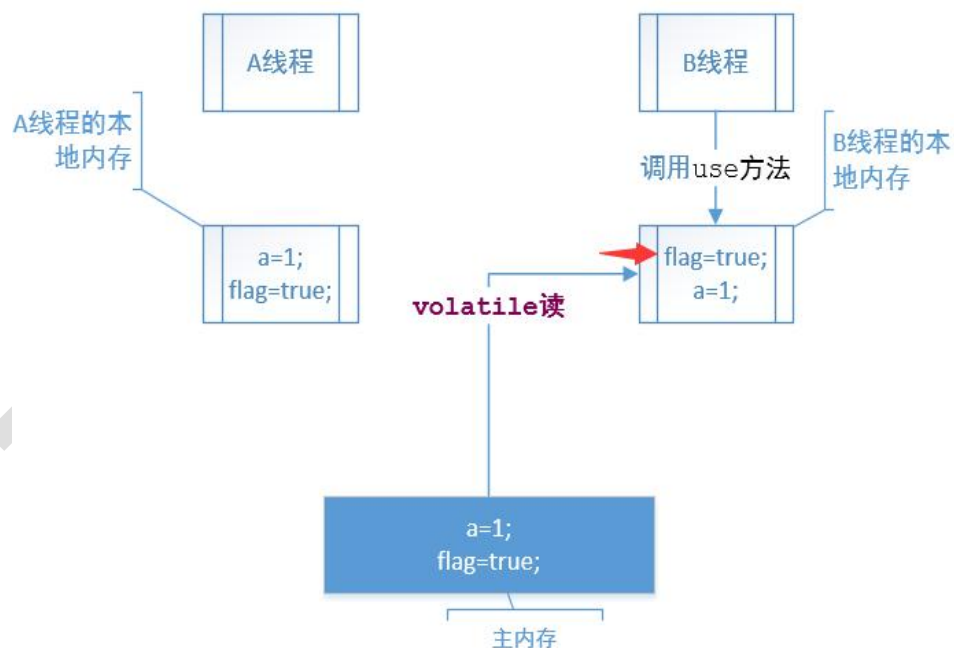
    public void init() {
        a = 1; // 1
        flag = true; // 2
        //.....
    }

    public void use() {
        if (flag) { // 3
            int i = a * a; // 4
        }
        //.....
    }
}
```

如果我们将 `flag` 变量以 `volatile` 关键字修饰，那么实际上：线程 A 在写 `flag` 变量后，本地内存 A 中被线程 A 更新过的两个共享变量的值都被刷新到主内存中。

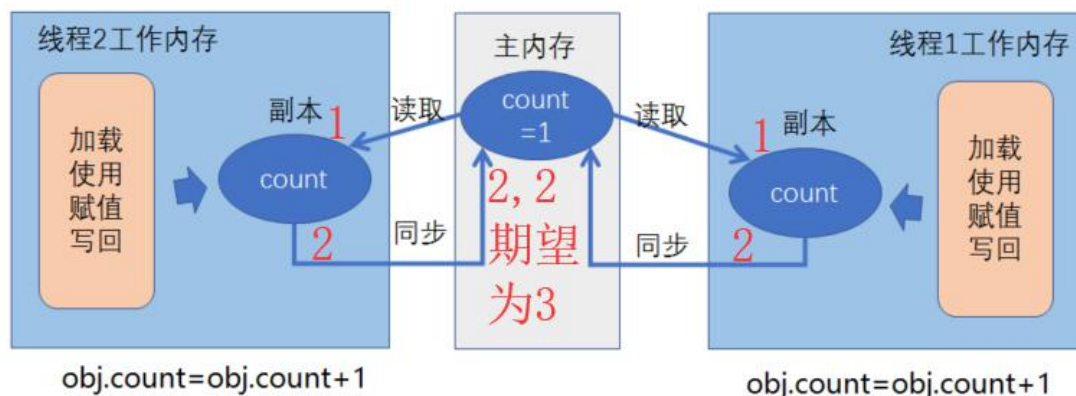


在读 `flag` 变量后，本地内存 B 包含的值已经被置为无效。此时，线程 B 必须从主内存中读取共享变量。线程 B 的读取操作将导致本地内存 B 与主内存中的共享变量的值变成一致。



如果我们将 **volatile 写** 和 **volatile 读** 两个步骤综合起来看的话，在读线程 B 读一个 **volatile** 变量后，写线程 A 在写这个 **volatile** 变量之前所有可见的共享变量的值都将立即变得对读线程 B 可见。

为何 volatile 不是线程安全的



保证可见性，不能保证原子性

volatile 内存语义的实现

volatile 重排序规则表

第一个操作	第二个操作		
	普通读/写	volatile读	volatile写
普通读/写			不允许
volatile读	不允许	不允许	不允许
volatile写		不允许	不允许

总结起来就是：

当第二个操作是 volatile 写时，不管第一个操作是什么，都不能重排序。这个规则确保 volatile 写之前的操作不会被编译器重排序到 volatile 写之后。

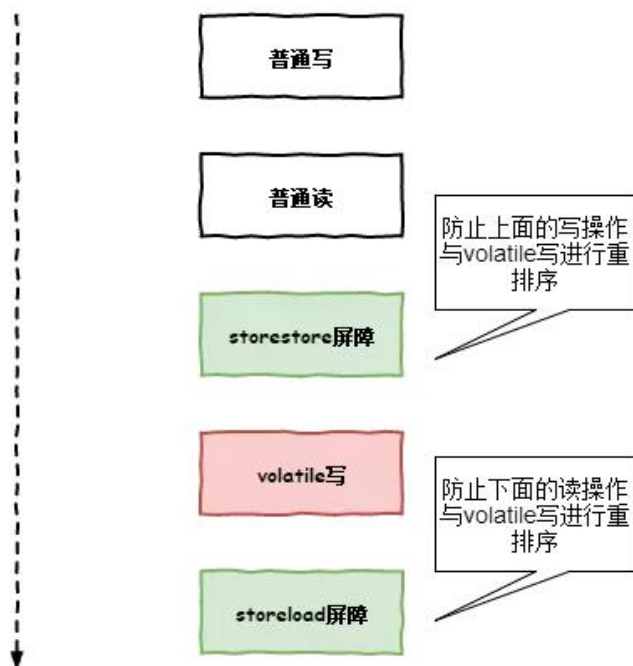
当第一个操作是 volatile 读时，不管第二个操作是什么，都不能重排序。这个规则确保 volatile 读之后的操作不会被编译器重排序到 volatile 读之前。

当第一个操作是 volatile 写，第二个操作是 volatile 读时，不能重排序。

volatile 的内存屏障

在 Java 中对于 volatile 修饰的变量，编译器在生成字节码时，会在指令序列中插入内存屏障来禁止特定类型的处理器重排序问题。

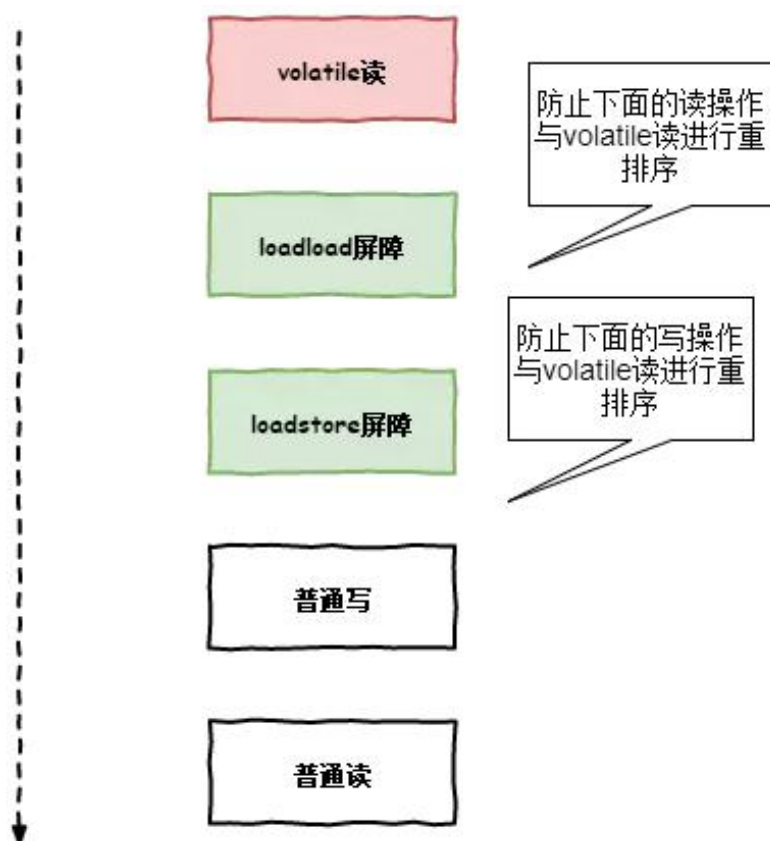
volatile 写



storestore 屏障：对于这样的语句 `store1; storestore; store2`，在 `store2` 及后续写入操作执行前，保证 `store1` 的写入操作对其它处理器可见。(也就是说如果出现 `storestore` 屏障，那么 `store1` 指令一定会在 `store2` 之前执行，CPU 不会 `store1` 与 `store2` 进行重排序)

storeload 屏障：对于这样的语句 `store1; storeload; load2`，在 `load2` 及后续所有读取操作执行前，保证 `store1` 的写入对所有处理器可见。(也就是说如果出现 `storeload` 屏障，那么 `store1` 指令一定会在 `load2` 之前执行,CPU 不会对 `store1` 与 `load2` 进行重排序)

volatile 读



在每个 volatile 读操作的后面插入一个 LoadLoad 屏障。在每个 volatile 读操作的后面插入一个 loadstore 屏障。

loadload 屏障：对于这样的语句 `load1; loadload; load2`，在 `load2` 及后续读取操作要读取的数据被访问前，保证 `load1` 要读取的数据被读取完毕。（也就是说，如果出现 `loadload` 屏障，那么 `load1` 指令一定会在 `load2` 之前执行，CPU 不会对 `load1` 与 `load2` 进行重排序）

loadstore 屏障：对于这样的语句 `load1; loadstore; store2`，在 `store2` 及后续写入操作被刷出前，保证 `load1` 要读取的数据被读取完毕。（也就是说，如果出现 `loadstore` 屏障，那么 `load1` 指令一定会在 `store2` 之前执行，CPU 不会对 `load1` 与 `store2` 进行重排序）

volatile 的实现原理

通过对 OpenJDK 中的 `unsafe.cpp` 源码的分析，会发现被 `volatile` 关键字修饰的变量会存在一个“lock:”的前缀。

Lock 前缀，Lock 不是一种内存屏障，但是它能完成类似内存屏障的功能。**Lock** 会对 CPU 总线 and 高速缓存加锁，可以理解为 CPU 指令级的一种锁。

同时该指令会将当前处理器缓存行的数据直接写会到系统内存中，且这个写回内存的操作会使在其他 CPU 里缓存了该地址的数据无效。

- 6、在具体的执行上，它先对总线和缓存加锁，然后执行后面的指令，最后释放锁后会把高速缓存中的脏数据全部刷新回主内存。在 Lock 锁住总线的时候，其他 CPU 的读写请求都会被阻塞，直到锁释放。

final 的内存语义

在构造线程的类时，我们有种方式就是让类中所有的成员变量都不可变，利用的就是 final 关键字，那么这个 final 为何可以做到呢？重排序这种优化动作对构造方法，一样也是存在的。这就说明，一个成员变量加了 final 关键字后，JMM 一定是做了相关处理的。

final 的两个重排序规则

对应 final 域，编译器和处理器需要遵守两个重排序规则。我们以代码 `cn.enjoyedu.ch9.semantics.FinalMemory` 来说明

```
public class FinalMemory {  
    int i;           // 普通变量  
    final int j;     // final变量  
    static FinalMemory obj;  
  
    public FinalMemory() { // 构造函数  
        i = 1;           // 写普通域  
        j = 2;           // 写final域  
    }  
  
    public static void writer() { // 写线程A执行  
        obj = new FinalMemory();  
    }  
  
    public static void reader() { // 读线程B执行  
        FinalMemory object = obj; // 读对象引用  
        int a = object.i;         // 读普通域  
        int b = object.j;         // 读final域  
    }  
}
```

我们假设一个线程 A 执行 `writer` 方法，随后另一个线程 B 执行 `reader` 方法。

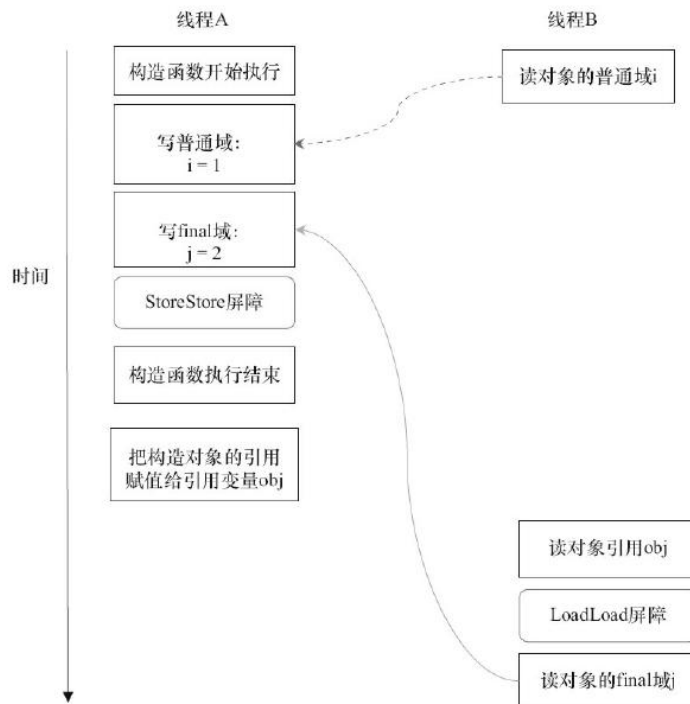
1、在构造函数内对一个 final 域的写入，与随后把这个被构造对象的引用赋值给一个引用变量，这两个操作之间不能重排序。

看 `write()` 方法，只包含一行代码 `obj = new FinalMemory();`。这一行代码包含两个步骤：

构造一个 `FinalMemory` 类型的对象。

把这个对象的引用赋值给引用变量 `obj`。

假设线程 B 读对象引用（`FinalMemory object = obj`）与读对象的成员域之间（`int a = object.i; int b = object.j`）没有重排序，下面的图是一种可能的执行时序：



从上面可能的时序图中我们可以看到，写普通域被编译器重排序到了构造函数之外，读线程 B 错误的读取了普通变量 i 初始化之前的值。而写 final 域的操作，被写 final 域的重排序规则“限制”到了构造函数之内，读线程 B 正确读取了 final 变量初始化之后的值。

总结：写 final 域的重排序规则可以确保在对象引用为任意线程可见之前，对象的 final 域已经被正常的初始化了，而普通域不具有这样的保证。

2、初次读一个包含 final 域的对象引用，与随后初次读这个 final 域，这两个操作之间不能重排序

在一个线程中，初次读对象引用与初次读该对象包含的 final 域，JMM 禁止处理器重排序这两个操作。编译器会在读 final 域操作的前面插入一个 LoadLoad 屏障。

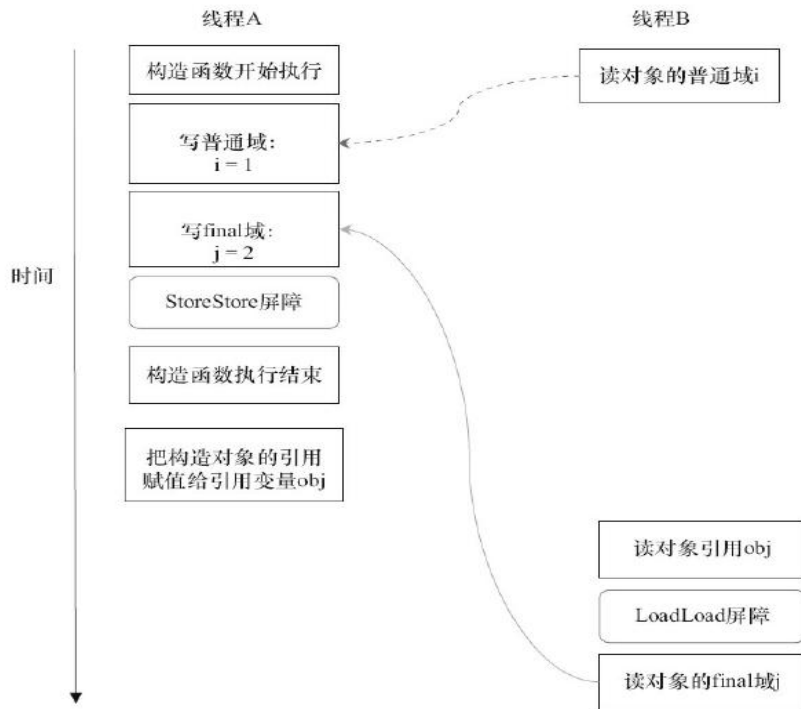
reader()方法包含 3 个步骤：

初次读引用变量 obj

初次读引用变量 obj 指向对象的普通域 i

初次读引用变量 obj 指向对象的 final 域 j

我们假设写线程 A 没有发生任何重排序，则下图是一种可能的时序：



读对象的普通域的操作被处理器重排序到读对象引用之前。读普通域时，该域还没有被线程 A 写入，所以上面的是一个错误的读取操作。但是读 final 域的重排序规则把读对象 final 域的操作“限定”在读对象引用之后，该 final 域已经被 A 线程初始化了，是一个正确的读取操作。

总结：读 final 域的重排序规则可以确保在读一个对象的 final 域之前，一定会先读包含这个 final 域的对象引用。

final 域为引用类型

我们以代码 `cn.enjoyedu.ch9.semantics.FinalRefMemory` 来说明

```
public class FinalRefMemory {
    final int[] intArray; // final 是引用类型
    static FinalRefMemory obj;

    public FinalRefMemory() { // 构造函数
        intArray = new int[1]; // 1
        intArray[0] = 1; // 2
    }

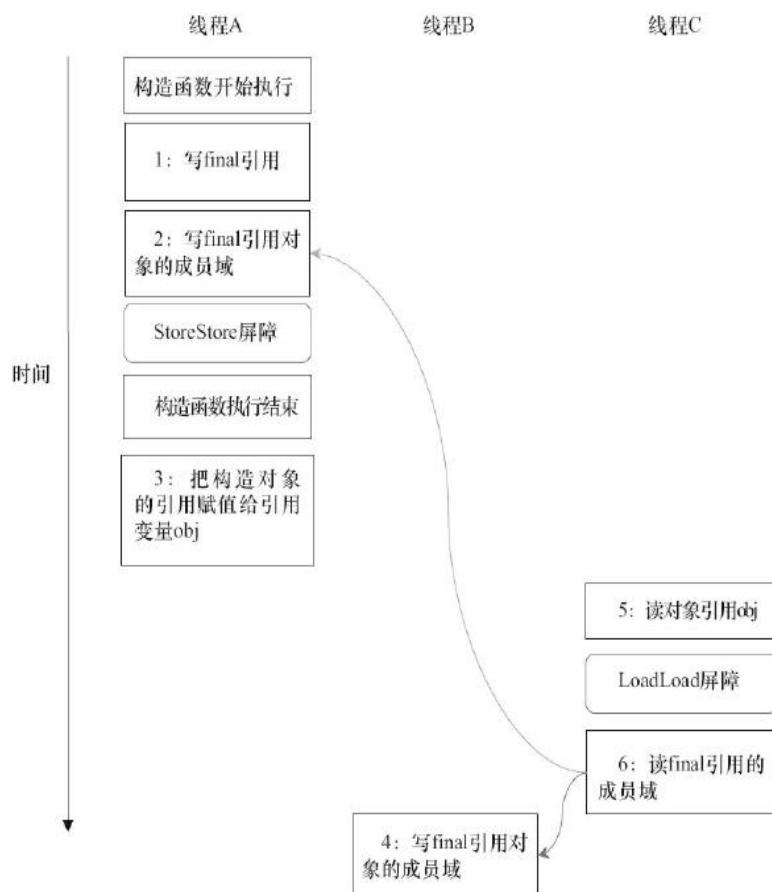
    public static void writerOne() { // 写线程A执行
        obj = new FinalRefMemory(); // 3
    }

    public static void writeTwo() { // 写线程B执行
        obj.intArray[0] = 2; // 4
    }

    public static void reader() { // 读线程C执行
        if (obj != null) { // 5
            int temp1 = obj.intArray[0]; // 6
        }
    }
}
```

在上面的代码中，`final` 域是一个引用类型，它引用了一个 `int` 类型的数组，对于引用类型，写 `final` 域的重排序规则对编译器和处理器增加了一下的约束：在构造函数内对一个 `final` 引用的对象的成员域的写入，与随后在构造函数外把这个被构造对象的引用赋值给一个引用变量，这两个操作之间不能重排序。

我们假设线程 A 先执行 `write0` 操作，执行完后线程 B 执行 `write1` 操作，执行完后线程 C 执行 `reader` 操作，下图是一种可能的执行时序：



1 是对 `final` 域的写入，2 是对这个 `final` 域引用的对象的成员域的写入，3 是把被构造的对象的引用赋值给某个引用变量。这里除了前面提到的 1 不能和 3 重排序外，2 和 3 也不能重排序。

JMM 可以确保读线程 C 至少能看到写线程 A 在构造函数中对 `final` 引用对象的成员域的写入。即 C 至少能看到数组下标 0 的值为 1。而写线程 B 对数组元素的写入，读线程 C 可能看得到，也可能看不到。JMM 不保证线程 B 的写入对读线程 C 可见，因为写线程 B 和读线程 C 之间存在数据竞争，此时的执行结果不可预知。

如果想要确保读线程 C 看到写线程 B 对数组元素的写入，写线程 B 和读线程 C 之间需要使用同步（`lock` 或 `volatile`）来确保内存可见性。

final 引用不能从构造函数内逃逸

写 `final` 域的重排序规则可以确保：在引用变量为任意线程可见之前，该引用变量指向的对象的 `final` 域已经在构造函数中被正确初始化过了。其实，要得

到这个效果，还需要一个保证：在构造函数内部，不能让这个被构造对象的引用为其他线程所见，也就是对象引用不能在构造函数中逃逸。

我们以 `cn.enjoyedu.ch9.semantics.FinalEscape` 为例来说明

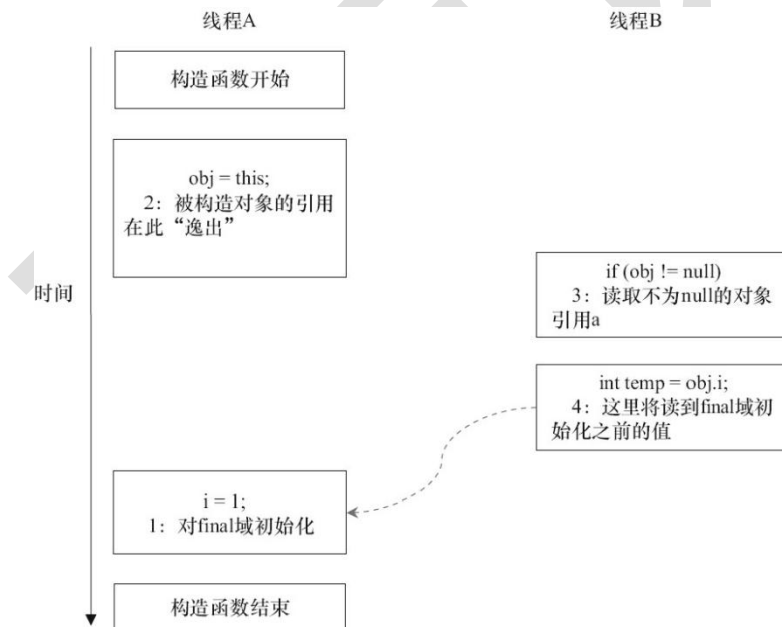
```
public class FinalEscape {
    final int i;
    static FinalEscape obj;

    public FinalEscape() {
        i = 10;           //写final域
        obj = this;       //this引用溢出
    }

    public static void writer() {
        new FinalEscape();
    }

    public static void reader() {
        if (obj != null) { //3
            int temp = obj.i; //4
        }
    }
}
```

假设一个线程 A 执行 `writer()` 方法，另一个线程 B 执行 `reader()` 方法。这里的操作 2 使得对象还未完成构造前就为线程 B 可见。即使这里的操作 2 是构造函数的最后一步，且在程序中操作 2 排在操作 1 后面，执行 `read()` 方法的线程仍然可能无法看到 `final` 域被初始化后的值，因为这里的操作 1 和操作 2 之间可能被重排序。



因此在构造函数返回前，被构造对象的引用不能为其他线程所见，因为此时的 `final` 域可能还没有被初始化。

final 语义的实现

会要求编译器在 final 域的写之后，构造函数 return 之前插入一个 StoreStore 屏障。

读 final 域的重排序规则要求编译器在读 final 域的操作前面插入一个 LoadLoad 屏障

锁的内存语义

当线程释放锁时，JMM 会把该线程对应的本地内存中的共享变量刷新到主内存中。

当线程获取锁时，JMM 会把该线程对应的本地内存置为无效。从而使得被监视器保护的临界区代码必须从主内存中读取共享变量。



如果我们回顾第一章的 VolatileCase，我们知道，为了让子线程可以及时看到 ready 变量的修改，我们需要将 ready 变量以 volatile 来修饰。

```
public class VolatileCase {
    private volatile static boolean ready;
    private static int number;

    //
    private static class PrintThread extends Thread{
        @Override
        public void run() {
            System.out.println("PrintThread is running.....");
            while (!ready); //无限循环
            System.out.println("number = "+number);
        }
    }

    public static void main(String[] args) {
        new PrintThread().start();
        SleepTools.second( seconds: 1);
        number = 51;
        ready = true;
    }
}
```

但是，当我们将程序做如下改造

```

public class SynMemory {
    private static boolean ready; ← 普通变量
    private static int number;

    private static class PrintThread extends Thread{
        @Override
        public void run() {
            while(!ready){
                System.out.println("number = "+number);
            }
            System.out.println("number = "+number); 打印 number
        }
    }

    public static void main(String[] args) {
        new PrintThread().start();
        SleepTools.second( seconds: 1);
        number = 51;
        ready = true;
        SleepTools.second( seconds: 5);
        System.out.println("main is ended!");
    }
}

```

我们可以看见子线程同样可以中止,为何? 我们观察 `System.out.println` 的实现,

```

public void println(String x) {
    synchronized (this) {
        print(x);
        newLine();
    }
}

```

结合前面锁的内存语义,我们可以知道,当进入 **synchronized** 语句块时,子线程会被强制从主内存中读取共享变量,其中就包括了 `ready` 变量,所以子线程同样中止了。

synchronized 的实现原理

Synchronized 在 JVM 里的实现都是基于进入和退出 **Monitor** 对象来实现方法同步和代码块同步,虽然具体实现细节不一样,但是都可以通过成对的 **MonitorEnter** 和 **MonitorExit** 指令来实现。

对同步块, **MonitorEnter** 指令插入在同步代码块的开始位置,当代码执行到该指令时,将会尝试获取该对象 **Monitor** 的所有权,即尝试获得该对象的锁,而 **monitorExit** 指令则插入在方法结束处和异常处,JVM 保证每个 **MonitorEnter** 必须有对应的 **MonitorExit**。

对同步方法,从同步方法反编译的结果来看,方法的同步并没有通过指令 **monitorenter** 和 **monitorexit** 来实现,相对于普通方法,其常量池中多了 **ACC_SYNCHRONIZED** 标示符。

JVM 就是根据该标示符来实现方法的同步的:当方法被调用时,调用指令将会检查方法的 **ACC_SYNCHRONIZED** 访问标志是否被设置,如果设置了,执行线程将先获取 **monitor**,获取成功之后才能执行方法体,方法执行完后再释放 **monitor**。在方法执行期间,其他任何线程都无法再获得同一个 **monitor** 对象。

synchronized 使用的锁是存放在 Java 对象头里面,

长 度	内 容	说 明
32/64bit	Mark Word	存储对象的 hashCode 或锁信息等
32/64bit	Class Metadata Address	存储到对象类型数据的指针
32/32bit	Array length	数组的长度（如果当前对象是数组）

具体位置是对象头里面的 MarkWord，MarkWord 里默认数据是存储对象的 hashCode 等信息，

锁状态	25bit	4bit	1bit 是否是偏向锁	2bit 锁标志位
无锁状态	对象的 hashCode	对象分代年龄	0	01

但是会随着对象的运行改变而发生变化，不同的锁状态对应着不同的记录存储方式

锁状态	25 bit		4bit	1bit	2bit
	23bit	2bit		是否是偏向锁	锁标志位
轻量级锁	指向栈中锁记录的指针				00
重量级锁	指向互斥量（重量级锁）的指针				10
GC标记	空				11
偏向锁	线程ID	Epoch	对象分代年龄	1	01

了解各种锁

自旋锁

原理

自旋锁原理非常简单，如果持有锁的线程能在很短时间内释放锁资源，那么那些等待竞争锁的线程就不需要做内核态和用户态之间的切换进入阻塞挂起状态，它们只需要等一等（自旋），等持有锁的线程释放锁后即可立即获取锁，这样就避免用户线程和内核的切换的消耗。

但是线程自旋是需要消耗 CPU 的，说白了就是让 CPU 在做无用功，线程不能一直占用 CPU 自旋做无用功，所以需要设定一个自旋等待的最大时间。

如果持有锁的线程执行的时间超过自旋等待的最大时间扔没有释放锁，就会导致其它争用锁的线程在最大等待时间内还是获取不到锁，这时争用线程会停止自旋进入阻塞状态。

自旋锁的优缺点

自旋锁尽可能的减少线程的阻塞，这对于锁的竞争不激烈，且占用锁时间非常短的代码块来说性能能大幅度的提升，因为自旋的消耗会小于线程阻塞挂起操作的消耗！

但是如果锁的竞争激烈，或者持有锁的线程需要长时间占用锁执行同步块，这时候就不适合使用自旋锁了，因为自旋锁在获取锁前一直都是占用 `cpu` 做无用功，占着 `xx` 不 `xx`，线程自旋的消耗大于线程阻塞挂起操作的消耗，其它需要 `cup` 的线程又不能获取到 `cpu`，造成 `cpu` 的浪费。

自旋锁时间阈值

自旋锁的目的是为了占着 `CPU` 的资源不释放，等到获取到锁立即进行处理。但是如何去选择自旋的执行时间呢？如果自旋执行时间太长，会有大量的线程处于自旋状态占用 `CPU` 资源，进而会影响整体系统的性能。因此自旋次数很重要

`JVM` 对于自旋次数的选择，`jdk1.5` 默认为 10 次，在 1.6 引入了适应性自旋锁，适应性自旋锁意味着自旋的时间不再是固定的了，而是由前一次在同一个锁上的自旋时间以及锁的拥有者的状态来决定，基本认为一个线程上下文切换的时间是最佳的一个时间。

`JDK1.6` 中 `-XX:+UseSpinning` 开启自旋锁；`JDK1.7` 后，去掉此参数，由 `jvm` 控制；

锁的状态

一共有四种状态，**无锁状态**，**偏向锁状态**，**轻量级锁状态**和**重量级锁状态**，它会随着竞争情况逐渐升级。锁可以升级但不能降级，目的是为了提高获得锁和释放锁的效率。

偏向锁

引入背景：大多数情况下锁不仅不存在多线程竞争，而且总是由同一线程多次获得，为了让线程获得锁的代价更低而引入了偏向锁，减少不必要的 `CAS` 操作。

偏向锁，顾名思义，它会偏向于第一个访问锁的线程，如果在运行过程中，同步锁只有一个线程访问，不存在多线程争用的情况，则线程是不需要触发同步的，减少加锁 / 解锁的一些 `CAS` 操作（比如等待队列的一些 `CAS` 操作），这种情况下，就会给线程加一个偏向锁。如果在运行过程中，遇到了其他线程抢占锁，则持有偏向锁的线程会被挂起，`JVM` 会消除它身上的偏向锁，将锁恢复到标准的轻量级锁。它通过消除资源无竞争情况下的同步原语，进一步提高了程序的运行性能。

偏向锁获取过程：

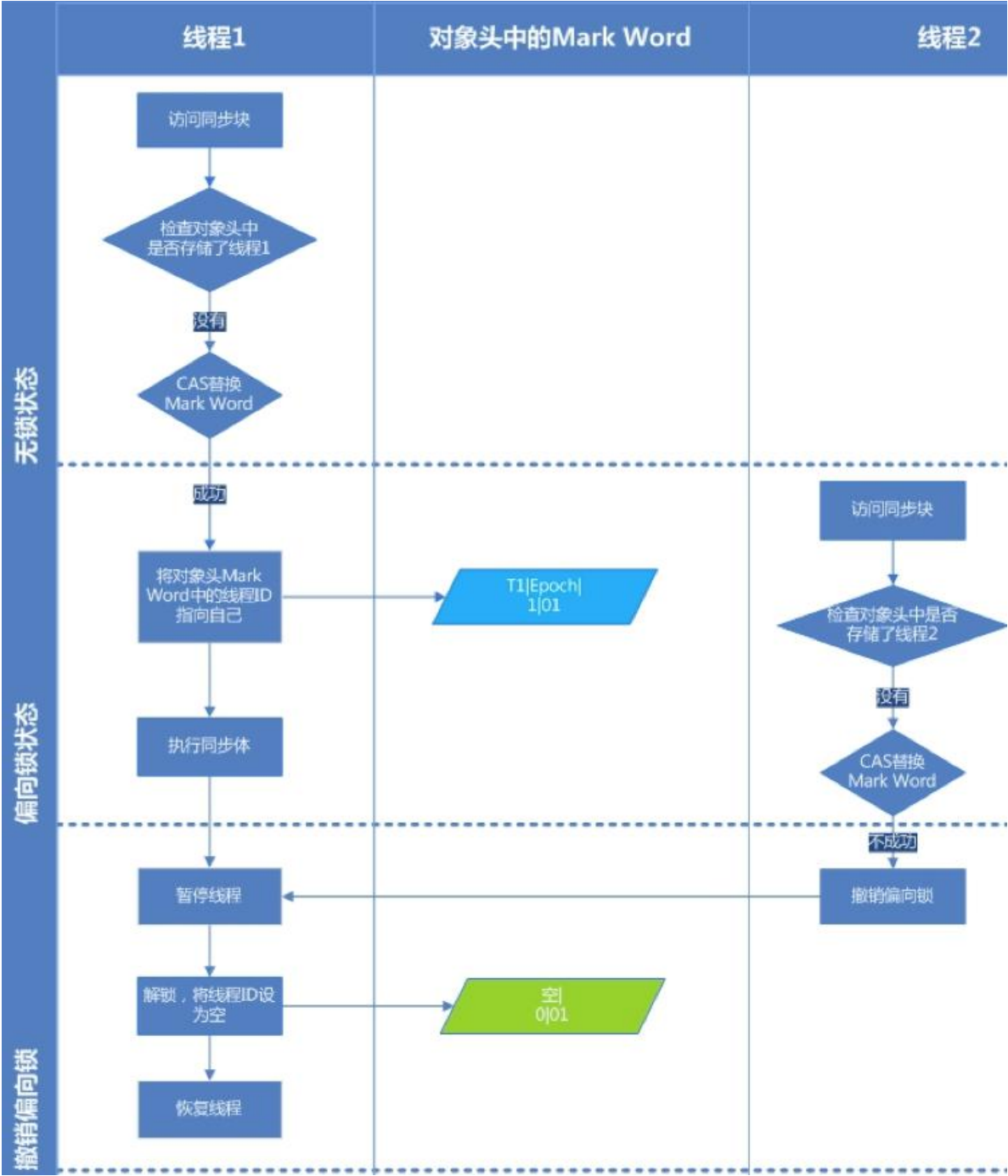
步骤 1、访问 `Mark Word` 中偏向锁的标识是否设置成 1，锁标志位是否为 01，确认为可偏向状态。

步骤 2、如果为可偏向状态，则测试线程 ID 是否指向当前线程，如果是，进入步骤 5，否则进入步骤 3。

步骤 3、 如果线程 ID 并未指向当前线程，则通过 CAS 操作竞争锁。如果竞争成功，则将 Mark Word 中线程 ID 设置为当前线程 ID，然后执行 5；如果竞争失败，执行 4。

步骤 4、 如果 CAS 获取偏向锁失败，则表示有竞争。当到达全局安全点（safepoint）时获得偏向锁的线程被挂起，偏向锁升级为轻量级锁，然后被阻塞在安全点的线程继续往下执行同步代码。（撤销偏向锁的时候会导致 stop the word）

步骤 5、 执行同步代码。



偏向锁的释放：

偏向锁的撤销在上述第四步骤中有提到。偏向锁只有遇到其他线程尝试竞争偏向锁时，持有偏向锁的线程才会释放偏向锁，线程不会主动去释放偏向锁。偏向锁的撤销，需要等待全局安全点（在这个时间点上没有字节码正在执行），它会首先暂停拥有偏向锁的线程，判断锁对象是否处于被锁定状态，撤销偏向锁后恢复到未锁定（标志位为“01”）或轻量级锁（标志位为“00”）的状态。

偏向锁的适用场景

始终只有一个线程在执行同步块，在它没有执行完释放锁之前，没有其它线程去执行同步块，在锁无竞争的情况下使用，一旦有了竞争就升级为轻量级锁，升级为轻量级锁的时候需要撤销偏向锁，撤销偏向锁的时候会导致 **stop the world** 操作；

在有锁的竞争时，偏向锁会多做很多额外操作，尤其是撤销偏向锁的时候会导致进入安全点，安全点会导致 **stw**，导致性能下降，这种情况下应当禁用。

jvm 开启/关闭偏向锁

开启偏向锁：-XX:+UseBiasedLocking -XX:BiasedLockingStartupDelay=0

关闭偏向锁：-XX:-UseBiasedLocking

轻量级锁

轻量级锁是由偏向锁升级来的，偏向锁运行在一个线程进入同步块的情况下，当第二个线程加入锁争用的时候，偏向锁就会升级为轻量级锁；

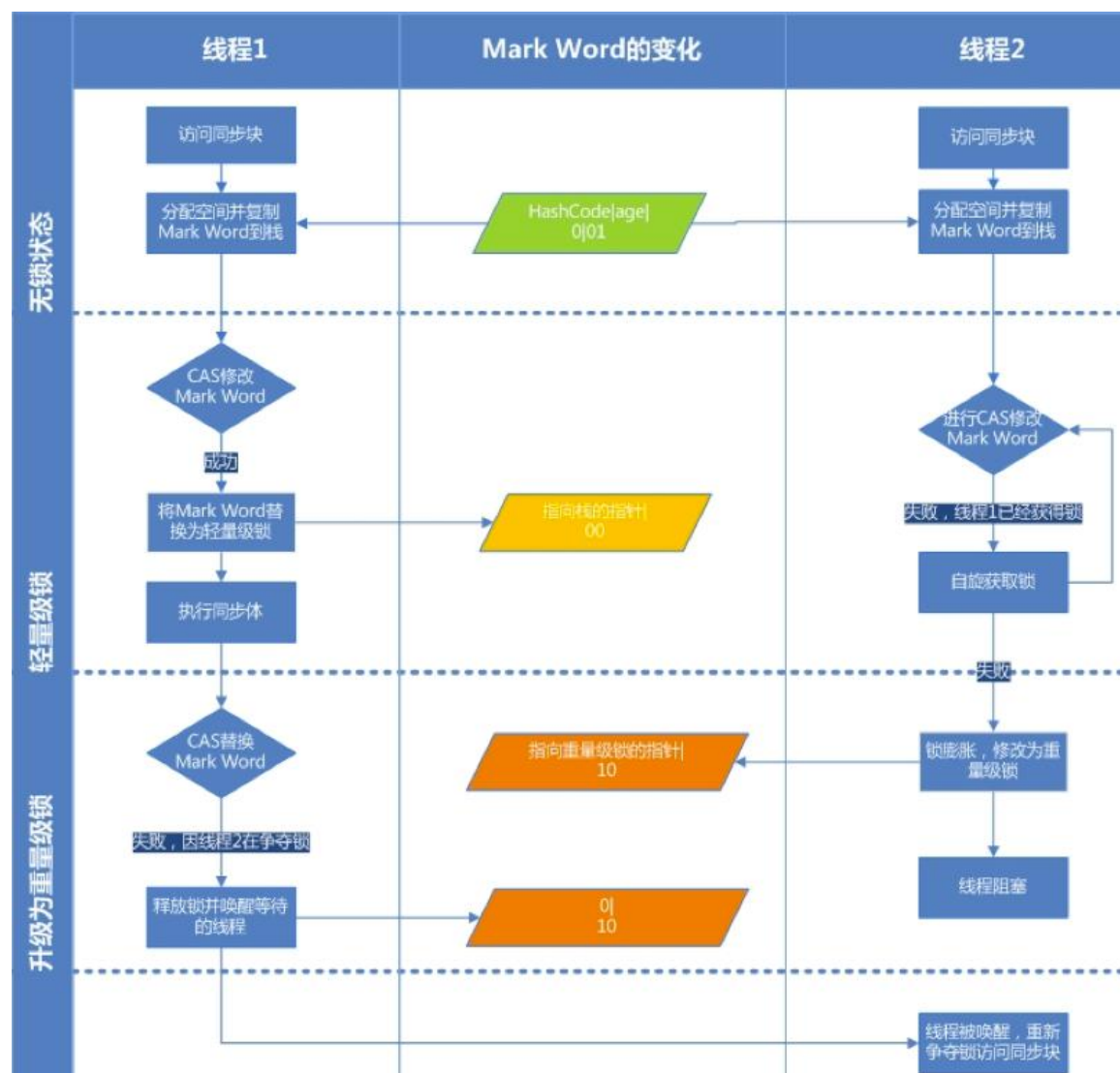
轻量级锁的加锁过程：

在代码进入同步块的时候，如果同步对象锁状态为无锁状态且不允许进行偏向（锁标志位为“01”状态，是否为偏向锁为“0”），虚拟机首先将在当前线程的栈帧中建立一个名为锁记录（Lock Record）的空间，用于存储锁对象目前的 Mark Word 的拷贝，官方称之为 Displaced Mark Word。

拷贝成功后，虚拟机将使用 CAS 操作尝试将对象的 Mark Word 更新为指向 Lock Record 的指针，并将 Lock record 里的 owner 指针指向 object mark word。如果更新成功，则执行步骤 4，否则执行步骤 5。

如果这个更新动作成功了，那么这个线程就拥有了该对象的锁，并且对象 Mark Word 的锁标志位设置为“00”，即表示此对象处于轻量级锁定状态

如果这个更新操作失败了，虚拟机首先会检查对象的 Mark Word 是否指向当前线程的栈帧，如果是就说明当前线程已经拥有了这个对象的锁，那就可以直接进入同步块继续执行。否则说明多个线程竞争锁，当竞争线程尝试占用轻量级锁失败多次之后，轻量级锁就会膨胀为重量级锁，重量级线程指针指向竞争线程，竞争线程也会阻塞，等待轻量级线程释放锁后唤醒他。锁标志的状态值变为“10”，Mark Word 中存储的就是指向重量级锁（互斥量）的指针，后面等待锁的线程也要进入阻塞状态。



不同锁的比较

锁	优点	缺点	适用场景
偏向锁	加锁和解锁不需要额外的消耗，和执行非同步方法比仅存在纳秒级的差距。	如果线程间存在锁竞争，会带来额外的锁撤销的消耗。	适用于只有一个线程访问同步块场景。
轻量级锁	竞争的线程不会阻塞，提高了程序的响应速度。	如果始终得不到锁竞争的线程使用自旋会消耗CPU。	追求响应时间。 同步块执行速度非常快。
重量级锁	线程竞争不使用自旋，不会消耗CPU。	线程阻塞，响应时间缓慢。	追求吞吐量。 同步块执行速度较长。

JDK 对锁的更多优化措施

逃逸分析

如果证明一个对象不会逃逸方法外或者线程外，则可针对此变量进行优化：

同步消除 `synchronization Elimination`，如果一个对象不会逃逸出线程，则对此变量的同步措施可消除。

锁消除和粗化

锁消除：虚拟机的运行时编译器在运行时如果检测到一些要求同步的代码上不可能发生共享数据竞争，则会去掉这些锁。

锁粗化：将临近的代码块用同一个锁合并起来。

消除无意义的锁获取和释放，可以提高程序运行性能。

10、Java8 新增的并发

原子操作 CAS

LongAdder

JDK1.8 时，`java.util.concurrent.atomic` 包中提供了一个新的原子类：`LongAdder`。根据 Oracle 官方文档的介绍，`LongAdder` 在高并发的场景下会比它的前辈——`AtomicLong` 具有更好的性能，代价是消耗更多的内存空间。

`AtomicLong` 是利用了底层的 CAS 操作来提供并发性的，调用了 `Unsafe` 类的 `getAndAddLong` 方法，该方法是个 `native` 方法，它的逻辑是采用自旋的方式不断更新目标值，直到更新成功。

在并发量较低的环境下，线程冲突的概率比较小，自旋的次数不会很多。但是，高并发环境下，N 个线程同时进行自旋操作，会出现大量失败并不断自旋的情况，此时 `AtomicLong` 的自旋会成为瓶颈。

这就是 `LongAdder` 引入的初衷——解决高并发环境下 `AtomicLong` 的自旋瓶颈问题。

`AtomicLong` 中有个内部变量 `value` 保存着实际的 long 值，所有的操作都是针对该变量进行。也就是说，高并发环境下，`value` 变量其实是一个热点，也就是 N 个线程竞争一个热点。

```
private volatile long value;
```

`LongAdder` 的基本思路就是分散热点，将 `value` 值分散到一个数组中，不同线程会命中到数组的不同槽中，各个线程只对自己槽中的那个值进行 CAS 操作，

这样热点就被分散了，冲突的概率就小很多。如果要获取真正的 long 值，只要将各个槽中的变量值累加返回。

这种做法和 ConcurrentHashMap 中的“分段锁”其实就是类似的思路。

LongAdder 提供的 API 和 AtomicLong 比较接近，两者都能以原子的方式对 long 型变量进行增减。

但是 AtomicLong 提供的功能其实更丰富，尤其是 addAndGet、decrementAndGet、compareAndSet 这些方法。

addAndGet、decrementAndGet 除了单纯的做自增自减外，还可以立即获取增减后的值，而 LongAdder 则需要做同步控制才能精确获取增减后的值。如果业务需求需要精确的控制计数，做计数比较，AtomicLong 也更合适。

另外，从空间方面考虑，LongAdder 其实是一种“空间换时间”的思想，从这一点来讲 AtomicLong 更适合。

总之，低并发、一般的业务场景下 AtomicLong 是足够了。如果并发量很多，存在大量写多读少的情况，那 LongAdder 可能更合适。适合的才是最好的，如果真出现了需要考虑到到底用 AtomicLong 好还是 LongAdder 的业务场景，那么这样的讨论是没有意义的，因为这种情况下要么进行性能测试，以准确评估在当前业务场景下两者的性能，要么换个思路寻求其它解决方案。

对于 LongAdder 来说，内部有一个 base 变量，一个 Cell[] 数组。

base 变量：非竞态条件下，直接累加到该变量上。

Cell[] 数组：竞态条件下，累加到各个线程自己的槽 Cell[i] 中。

```
transient volatile Cell[] cells;  
transient volatile long base;
```

所以，最终结果的计算应该是

```
public long sum() {  
    Cell[] as = cells; Cell a;  
    long sum = base;  
    if (as != null) {  
        for (int i = 0; i < as.length; ++i) {  
            if ((a = as[i]) != null)  
                sum += a.value;  
        }  
    }  
    return sum;  
}
```

$$value = base + \sum_{i=0}^n Cell[i]$$

在实际运用的时候，只有从未出现过并发冲突的时候，base 基数才会使用到，一旦出现了并发冲突，之后所有的操作都只针对 Cell[] 数组中的单元 Cell。


```

public void add(long x) {
    Cell[] as; long b, v; int m; Cell a;
    if ((as = cells) != null || !casBase(b = base, val: b + x)) {
        boolean uncontended = true;
        if (as == null || (m = as.length - 1) < 0 ||
            (a = as[getProbe() & m]) == null ||
            !(uncontended = a.cas(v = a.value, val: v + x)))
            longAccumulate(x, fn: null, uncontended);
    }
}

```

而 LongAdder 最终结果的求和,并没有使用全局锁,返回值不是绝对准确的,因为调用这个方法时还有其他线程可能正在进行计数累加,所以只能得到某个时刻的近似值,这也就是 LongAdder 并不能完全替代 LongAtomic 的原因之一。

而且从测试情况来看,线程数越多,并发操作数越大,LongAdder 的优势越大,线程数较小时,AtomicLong 的性能还超过了 LongAdder。

其他新增

除了新引入 LongAdder 外,还有引入了它的三个兄弟类: LongAccumulator、DoubleAdder、DoubleAccumulator。

LongAccumulator 是 LongAdder 的增强版。LongAdder 只能针对数值的进行加减运算,而 LongAccumulator 提供了自定义的函数操作。

通过 LongBinaryOperator,可以自定义对入参的任意操作,并返回结果(LongBinaryOperator 接收 2 个 long 作为参数,并返回 1 个 long)。

LongAccumulator 内部原理和 LongAdder 几乎完全一样。

DoubleAdder 和 DoubleAccumulator 用于操作 double 原始类型。

StampedLock

StampedLock 是 Java8 引入的一种新的锁机制,简单的理解,可以认为它是读写锁的一个改进版本,读写锁虽然分离了读和写的功能,使得读与读之间可以完全并发,但是读和写之间依然是冲突的,读锁会完全阻塞写锁,它使用的依然是悲观的锁策略。如果有大量的读线程,他也有可能引起写线程的饥饿。

而 StampedLock 则提供了一种乐观的读策略,这种乐观策略的锁非常类似于无锁的操作,使得乐观锁完全不会阻塞写线程。

它的思想是读写锁中读不仅不阻塞读,同时也不应该阻塞写。

读不阻塞写的实现思路:

在读的时候如果发生了写,则应当重读而不是在读的时候直接阻塞写!即读写之间不会阻塞对方,但是写和写之间还是阻塞的!

StampedLock 的内部实现是基于 CLH 的。

参考代码,参见 cn.enjoyedu.cha. StampedLockDemo

CompletableFuture

Future 的不足

Future 是 Java 5 添加的类，用来描述一个异步计算的结果。你可以使用 `isDone` 方法检查计算是否完成，或者使用 `get` 阻塞住调用线程，直到计算完成返回结果，你也可以使用 `cancel` 方法停止任务的执行。

虽然 Future 以及相关使用方法提供了异步执行任务的能力，但是对于结果的获取却是很不方便，只能通过阻塞或者轮询的方式得到任务的结果。阻塞的方式显然和我们的异步编程的初衷相违背，轮询的方式又会耗费无谓的 CPU 资源，而且也不能及时地得到计算结果，为什么不能用观察者设计模式当计算结果完成及时通知监听者呢？。

Java 的一些框架，比如 Netty，自己扩展了 Java 的 Future 接口，提供了 `addListener` 等多个扩展方法，Google guava 也提供了通用的扩展 `Future:ListenableFuture`、`SettableFuture` 以及辅助类 `Futures` 等，方便异步编程。

同时 Future 接口很难直接表述多个 Future 结果之间的依赖性。实际开发中，我们经常需要达成以下目的：

将两个异步计算合并为一个——这两个异步计算之间相互独立，同时第二个又依赖于第一个的结果。

等待 Future 集合中的所有任务都完成。

仅等待 Future 集合中最快结束的任务完成（有可能因为它们试图通过不同的方式计算同一个值），并返回它的结果。

应对 Future 的完成事件（即当 Future 的完成事件发生时收到通知，并能使用 Future 计算的结果进行下一步的操作，不只是简单地阻塞等待操作的结果）

CompletableFuture

JDK1.8 才新加入的一个实现类 `CompletableFuture`，实现了 `Future<T>`，`CompletionStage<T>` 两个接口。实现了 Future 接口，意味着可以像以前一样通过阻塞或者轮询的方式获得结果。

创建

除了直接 new 出一个 `CompletableFuture` 的实例，还可以通过工厂方法创建 `CompletableFuture` 的实例

工厂方法：

```
  runAsync(Runnable): CompletableFuture<Void>  
  runAsync(Runnable, Executor): CompletableFuture<Void>  
  
  supplyAsync(Supplier<U>): CompletableFuture<U>  
  supplyAsync(Supplier<U>, Executor): CompletableFuture<U>
```

Async 表示异步,而 supplyAsync 与 runAsync 不同在与前者异步返回一个结果,后者是 void.第二个函数第二个参数表示是用我们自己创建的线程池,否则采用默认的 ForkJoinPool.commonPool()作为它的线程池。

获得结果的方法

```
public T get()
public T get(long timeout, TimeUnit unit)
public T getNow(T valIfAbsent)
public T join()
```

getNow 有点特殊,如果结果已经计算完则返回结果或者抛出异常,否则返回给定的 valIfAbsent 值。

join 返回计算的结果或者抛出一个 unchecked 异常(CompletionException),它和 get 对抛出的异常的处理有些细微的区别。

参见 cn.enjoyedu.cha.cfdemo 下 CFDemo 和 JoinAndGet

辅助方法

```
public static CompletableFuture<Void> allOf(CompletableFuture<?>... cfs)
public static CompletableFuture<Object> anyOf(CompletableFuture<?>... cfs)
allOf 方法是当所有的 CompletableFuture 都执行完后执行计算。
```

anyOf 方法是当任意一个 CompletableFuture 执行完后就会执行计算,计算的结果相同。

参见 cn.enjoyedu.cha.cfdemo 下 AllOfAnyOf

CompletionStage 是一个接口,从命名上看得知是一个完成的阶段,它代表了一个特定的计算的阶段,可以同步或者异步的被完成。你可以把它看成一个计算流水线上的一个单元,并最终会产生一个最终结果,这意味着几个 CompletionStage 可以串联起来,一个完成的阶段可以触发下一阶段的执行,接着触发下一次,再接着触发下一次,.....。

总结 CompletableFuture 几个关键点:

1、计算可以由 Future ,Consumer 或者 Runnable 接口中的 apply, accept 或者 run 等方法表示。

2、计算的执行主要有以下

a. 默认执行

b. 使用默认的 CompletionStage 的异步执行提供者异步执行。这些方法名使用 someActionAsync 这种格式表示。

c. 使用 Executor 提供者异步执行。这些方法同样也是 someActionAsync 这种格式,但是会增加一个 Executor 参数。

CompletableFuture 里大约有五十种方法,但是可以进行归类,

变换类 thenApply:

```

m thenApply(Function<? super T, ? extends U>): CompletableFuture<U>
m thenApplyAsync(Function<? super T, ? extends U>): CompletableFuture<U>
m thenApplyAsync(Function<? super T, ? extends U>, Executor): CompletableFuture<U>

```

关键入参是函数式接口 `Function`。它的入参是上一个阶段计算后的结果，返回值是经过转化后结果。

消费类 `thenAccept`:

```

m thenAccept(Consumer<? super T>): CompletableFuture<U>
m thenAcceptAsync(Consumer<? super T>): CompletableFuture<U>
m thenAcceptAsync(Consumer<? super T>, Executor): CompletableFuture<U>

```

关键入参是函数式接口 `Consumer`。它的入参是上一个阶段计算后的结果，没有返回值。

执行操作类 `thenRun`:

```

m thenRun(Runnable): CompletableFuture<Void>
m thenRunAsync(Runnable): CompletableFuture<Void>
m thenRunAsync(Runnable, Executor): CompletableFuture<Void>

```

对上一步的计算结果不关心，执行下一个操作，入参是一个 `Runnable` 的实例，表示上一步完成后执行的操作。

结合转化类:

```

m thenCombine(CompletionStage<? extends U>, BiFunction<? super T, ? super U, ? extends V>): CompletableFuture<V>
m thenCombineAsync(CompletionStage<? extends U>, BiFunction<? super T, ? super U, ? extends V>): CompletableFuture<V>
m thenCombineAsync(CompletionStage<? extends U>, BiFunction<? super T, ? super U, ? extends V>, Executor): CompletableFuture<V>

```

需要上一步的处理返回值，并且 `other` 代表的 `CompletionStage` 有返回值之后，利用这两个返回值，进行转换后返回指定类型的值。

两个 `CompletionStage` 是并行执行的，它们之间并没有先后依赖顺序，`other` 并不会等待先前的 `CompletableFuture` 执行完毕后再执行。

结合转化类

```

m thenCompose(Function<? super T, ? extends CompletionStage<U>>): CompletableFuture<U>
m thenComposeAsync(Function<? super T, ? extends CompletionStage<U>>): CompletableFuture<U>
m thenComposeAsync(Function<? super T, ? extends CompletionStage<U>>, Executor): CompletableFuture<U>

```

对于 `Compose` 可以连接两个 `CompletableFuture`，其内部处理逻辑是当第一个 `CompletableFuture` 处理没有完成时会合并成一个 `CompletableFuture`，如果处理完成，第二个 `future` 会紧接上一个 `CompletableFuture` 进行处理。

第一个 `CompletableFuture` 的处理结果是第二个 `future` 需要的输入参数。

结合消费类:

```

m thenAcceptBoth(CompletionStage<? extends U>, BiConsumer<? super T, ? super U>): CompletableFuture<Void>
m thenAcceptBothAsync(CompletionStage<? extends U>, BiConsumer<? super T, ? super U>): CompletableFuture<Void>
m thenAcceptBothAsync(CompletionStage<? extends U>, BiConsumer<? super T, ? super U>, Executor): CompletableFuture<Void>

```

需要上一步的处理返回值，并且 `other` 代表的 `CompletionStage` 有返回值之后，利用这两个返回值，进行消费

运行后执行类：

- runAfterBoth(CompletionStage<?>, Runnable): CompletableFuture<Void> ↑CompletionStage
- runAfterBothAsync(CompletionStage<?>, Runnable): CompletableFuture<Void> ↑CompletionStage
- runAfterBothAsync(CompletionStage<?>, Runnable, Executor): CompletableFuture<Void> ↑CompletionStage

不关心这两个 CompletionStage 的结果，只关心这两个 CompletionStage 都执行完毕，之后再进行操作（Runnable）。

取最快转换类：

- applyToEither(CompletionStage<? extends T>, Function<? super T, U>): CompletableFuture<U> ↑CompletionStage
- applyToEitherAsync(CompletionStage<? extends T>, Function<? super T, U>): CompletableFuture<U> ↑CompletionStage
- applyToEitherAsync(CompletionStage<? extends T>, Function<? super T, U>, Executor): CompletableFuture<U> ↑CompletionStage

两个 CompletionStage，谁计算的快，我就用那个 CompletionStage 的结果进行下一步的转化操作。现实开发场景中，总会碰到有两种渠道完成同一个事情，所以就可以调用这个方法，找一个最快的结果进行处理。

取最快消费类：

- acceptEither(CompletionStage<? extends T>, Consumer<? super T>): CompletableFuture<Void> ↑CompletionStage
- acceptEitherAsync(CompletionStage<? extends T>, Consumer<? super T>): CompletableFuture<Void> ↑CompletionStage
- acceptEitherAsync(CompletionStage<? extends T>, Consumer<? super T>, Executor): CompletableFuture<Void> ↑CompletionStage

两个 CompletionStage，谁计算的快，我就用那个 CompletionStage 的结果进行下一步的消费操作。

取最快运行后执行类：

- runAfterEither(CompletionStage<?>, Runnable): CompletableFuture<Void> ↑CompletionStage
- runAfterEitherAsync(CompletionStage<?>, Runnable): CompletableFuture<Void> ↑CompletionStage
- runAfterEitherAsync(CompletionStage<?>, Runnable, Executor): CompletableFuture<Void> ↑CompletionStage

两个 CompletionStage，任何一个完成了都会执行下一步的操作（Runnable）。

异常补偿类：

```
public CompletableFuture<T> exceptionally(  
    Function<Throwable, ? extends T> fn) {  
    return uniExceptionallyStage(fn);  
}
```

当运行时出现了异常，可以通过 exceptionally 进行补偿。

运行后记录结果类：

- whenComplete(BiConsumer<? super T, ? super Throwable>): CompletableFuture<T> ↑CompletionStage
- whenCompleteAsync(BiConsumer<? super T, ? super Throwable>): CompletableFuture<T> ↑CompletionStage
- whenCompleteAsync(BiConsumer<? super T, ? super Throwable>, Executor): CompletableFuture<T> ↑CompletionStage

action 执行完毕后它的结果返回原始的 CompletableFuture 的计算结果或者返回异常。所以不会对结果产生任何的作用。

运行后处理结果类：


```
m 🐞 handle(BiFunction<? super T, Throwable, ? extends U>): CompletableFuture<U> ↑CompletionStag
m 🐞 handleAsync(BiFunction<? super T, Throwable, ? extends U>): CompletableFuture<U> ↑Completi
m 🐞 handleAsync(BiFunction<? super T, Throwable, ? extends U>, Executor): CompletableFuture<U> ↑C
```

运行完成时，对结果的处理。这里的完成时有两种情况，一种是正常执行，返回值。另外一种是在遇到异常抛出造成程序的中断。

补充：Lambda 速成

本补充章节仅为没接触过 Lambda 的同学快速入门和速查，更具体的 Lambda 的知识请自行查阅相关书籍和博客。相关代码放在 [cn.enjoyedu.cha.lambda](https://github.com/cn-enjoyedu/cha-lambda) 下

现在有一个实体类，我们会对这个实体类进行操作。

```
/**
 * 类说明：实体类
 */
public class Circle {
    private int radius;
    private String color;
    private int x;
    private int y;
```

第一步

我们想从一批 Circle 中挑选出半径为 2 的圆，于是我们写了一个方法

```
/*1、挑选出半径为2的圆*/
public static List<Circle> getCircles(List<Circle> circles) {
    List<Circle> circleList = new ArrayList<>();
    for(Circle circle : circles) {
        if(circle.getRadius() == 2) {
            circleList.add(circle);
        }
    }
    return circleList;
}
```

这样，无疑很不优雅，如果我们想挑选半径为 3 的圆，难道还要再写一个方法？于是我们考虑将选择条件进行参数化，比如根据颜色挑选出圆或者根据半径挑选出圆

```
/*2.1、选择条件参数化，根据颜色挑选出圆*/
public static List<Circle> getCircleByColor(List<Circle> circles, String color) {
    List<Circle> circleList = new ArrayList<>();
    for(Circle circle : circles) {
        if(color.equals(circle.getColor())) {
            circleList.add(circle);
        }
    }
    return circleList;
}
```

/*2.2、选择条件参数化，根据半径挑选出圆*/

```
public static List<Circle> getCircleByRadius(List<Circle> circles,int radius){
    List<Circle> circleList = new ArrayList<>();
    for(Circle circle :circles){
        if(radius==circle.getRadius()){
            circleList.add(circle);
        }
    }
    return circleList;
}
```

但是，这种实现，还是有问题的，1、选择条件变化了，那么相应的方法也要变，比如我们想挑选半径大于3的圆，怎么办？如果我要根据多个条件选择，怎么办？难道把所有的条件都传入吗？于是，我们考虑定义一个挑选圆的接口，程序进化到了第二步

第二步

进行行为参数化，定义一个接口

/*定义挑选圆的行为接口*/

```
static interface ChoiceCircle{
    boolean getCircle(Circle circle);
}
```

在进行圆的挑选的方法里，我们把这个接口作为参数进行传递

/*行为参数化，根据条件挑选出圆*/

```
public static List<Circle> getCircleByChoice(List<Circle> circles,ChoiceCircle choice){
    List<Circle> circleList = new ArrayList<>();
    for(Circle circle :circles){
        if(choice.getCircle(circle)){
            circleList.add(circle);
        }
    }
    return circleList;
}
```

然后，我们只要按业务需求实现接口，并传入实现类的实例即可

/*挑选圆的行为实现之二，选出半径为2的圆*/

```
static class CircleByRadiusTwo implements ChoiceCircle{

    @Override
    public boolean getCircle(Circle circle) {
        return circle.getRadius()==2;
    }

}

public static void service(){
    List<Circle> src = new ArrayList<>();/*待处理的圆的集合*/
    List<Circle> result = getCircleByChoice(src, new CircleByRed());
}
```

这种方式可以提高灵活性，但是业务上每增加一个挑选行为，我们就需要显式声明一个接口 ChoiceCircle 的实现类，于是我们可以考虑使用内部匿名类，进入第三步。

第三步

在实际使用时，我们不再声明一个接口 ChoiceCircle 的实现类

```

public static void service() {
    List<Circle> src = new ArrayList<>(); /*待处理的圆的集合*/
    List<Circle> radiusTwos = getCircleByChoice(src, new ChoiceCircle() {
        @Override
        public boolean getCircle(Circle circle) {
            return circle.getRadius() == 2;
        }
    });

    List<Circle> reds = getCircleByChoice(src, new ChoiceCircle() {
        @Override
        public boolean getCircle(Circle circle) {
            return "Red".equals(circle.getColor());
        }
    });
}

```

匿名内部类占用代码空间较多，而且存在着模版代码，这种情况下，Lambda 表达式就可以派上用场了

```

List<Circle> radiusTwos2 = getCircleByChoice(src,
    (Circle circle) -> circle.getRadius() == 2);

List<Circle> reds2 = getCircleByChoice(src
    , (Circle circle) -> "Red".equals(circle.getColor()));

```

所以可以把 Lambda 表达式看成匿名内部类的一个简洁写法

Lambda

在语法上，Lambda 表达式包含三个部分，参数列表，箭头，主体，比如：

(parameters) -> expression

或

(parameters) -> { statements; }

Lambda 表达式用在函数式接口上，所谓函数式接口，是只定义了一个抽象方法的接口（Interface），接口中是否有默认方法，不影响。

注解 `@FunctionalInterface` 可以帮助我们在设计函数式接口时防止出错。

我们常用的 `Runnable`, `Callable` 都是函数式接口，JDK8 中新增了几个函数式接口：

Predicate<T> :

包含 `test` 方法，接受泛型的 T，返回 `boolean`，可以视为断言（检查）接口

Consumer<T> :

包含 `accept` 方法，接受泛型的 T，无返回，可以视为数据消费接口

Function<T, R> :

包含 `apply` 方法，接受泛型的 T，返回 R，可以视为映射转换接口

Supplier<T>

包含 `get` 方法，无输入，返回 T，可以视为创建一个新对象接口

UnaryOperator<T>

扩展至 `Function<T, T>`，所以这个本质上也是一个映射转换接口，只不过映射转换后的类型保持不变

`BiFunction<T, U, R>`

包含 `apply` 方法，接受泛型的 `T`、`U`，返回 `R`，可以视为复合型映射转换接口

`BinaryOperator<T>`

扩展至 `Function BiFunction<T,T,T>`，所以这个本质上也是一个复合型映射转换接口，只不过映射转换后的类型保持不变

`BiPredicate <T, U>`

包含 `test` 方法，接受泛型的 `T`，`U`，返回 `boolean`，可以视为复合型断言（检查）接口

`BiConsumer<T, U>`

包含 `accept` 方法，接受泛型的 `T`，`U`，无返回，可以视为复合型数据消费接口

同时还提供了一些为了防止自动装箱机制，而特意声明的原始类型特化的函数式接口，比如，

```
@FunctionalInterface
public interface IntPredicate {

    /**
     * Evaluates this predicate on the given argument.
     *
     * @param value the input argument
     * @return {@code true} if the predicate evaluates to true;
     *         {@code false} otherwise
     */
    boolean test(int value);
}
```

在意义上，和对应的 `Predicate` 接口并没有差别。

函数描述符

函数式接口的抽象方法的签名基本上就是 `Lambda` 表达式的签名。我们将这种抽象方法叫作函数描述符。

`Runnable` 接口可以看作一个什么也不接受什么也不返回（`void`）的函数的签名，因为它只有一个叫作 `run` 的抽象方法，这个方法什么也不接受，什么也不返回（`void`）。

我们可以用 `() -> void` 代表参数列表为空，且返回 `void` 的函数。这正是 `Runnable` 接口所代表的。我们于是可以称 `() -> void` 是 `Runnable` 接口的函数描述符。

```

public interface Runnable {
    /**
     * When an object implementing
     * to create a thread, starti
     * <code>run</code> method to
     * thread.
     * <p>
     * The general contract of th
     * take any action whatsoever
     *
     * @see      java.lang.Thread#
     */
    public abstract void run();
}

```

再考察 Callable 接口和 Supplier 接口

```

public interface Callable<V> {
    /**
     * Computes a result, or throw
     *
     * @return computed result
     * @throws Exception if unabl
     */
    V call() throws Exception;
}

public interface Supplier<T> {
    /**
     * Gets a result.
     *
     * @return a result
     */
    T get();
}

```

从函数描述符来看，Callable 接口和 Supplier 接口是一样的，都是
 () -> X

所以同一个 Lambda 可以同时用在这两个函数式接口上，比如：

Callable<Integer> = () -> 33;

Supplier<><Integer> = () -> 33;

扩充知识点- Disruptor

应用背景和介绍

Disruptor 是英国外汇交易公司 LMAX 开发的一个高性能队列，研发的初衷是解决内部的内存队列的延迟问题，而不是分布式队列。基于 Disruptor 开发的系统单线程能支撑每秒 600 万订单，2010 年在 QCon 演讲后，获得了业界关注。

据目前资料显示：应用 Disruptor 的知名项目有如下的一些：Storm, Camel, Log4j2, 还有目前的美团点评技术团队也有很多不少的应用，或者说有一些借鉴了它的设计机制。

Disruptor 是一个高性能的线程间异步通信的框架，即在同一个 JVM 进程中的多线程间消息传递。

传统队列问题

在 JDK 中，Java 内部的队列 BlockQueue 的各种实现，仔细分析可以得知，队列的底层数据结构一般分成三种：数组、链表和堆，堆这里是为了实现带有优先级特性的队列暂且不考虑。

在稳定性和性能要求特别高的系统中，为了防止生产者速度过快，导致内存溢出，只能选择有界队列；同时，为了减少 Java 的垃圾回收对系统性能的影响，会尽量选择 Array 格式的数据结构。这样筛选下来，符合条件的队列就只有 ArrayBlockingQueue。但是 ArrayBlockingQueue 是通过加锁的方式保证线程安全，而且 ArrayBlockingQueue 还存在伪共享问题，这两个问题严重影响了性能。

ArrayBlockingQueue 的这个伪共享问题存在于哪里呢，分析下核心的部分源码，其中最核心的三个成员变量为

```
/** items index for next take, poll, peek or remove */
int takeIndex;

/** items index for next put, offer, or add */
int putIndex;

/** Number of elements in the queue */
int count;
```

是在 ArrayBlockingQueue 的核心 enqueue 和 dequeue 方法中经常会用到的，这三个变量很容易放到同一个缓存行中，进而产生伪共享问题。

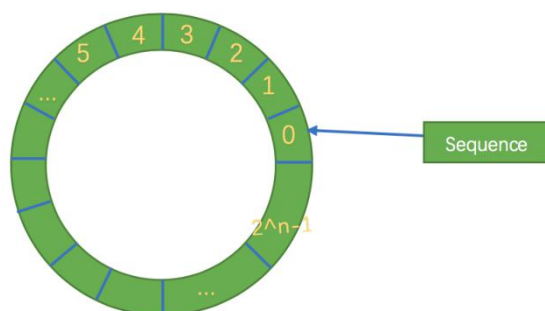
高性能的原理

引入环形的数组结构：数组元素不会被回收，避免频繁的 GC，

无锁的设计：采用 CAS 无锁方式，保证线程的安全性

属性填充：通过添加额外的无用信息，避免伪共享问题

环形数组结构是整个 Disruptor 的核心所在。



首先因为是数组，所以要比链表快，而且根据我们对上面缓存行的解释知道，数组中的一个元素加载，相邻的数组元素也是会被预加载的，因此在这样的结构中，cpu 无需时不时去主存加载数组中的下一个元素。而且，你可以为数组预先分配内存，使得数组对象一直存在（除非程序终止）。这就意味着不需要花大量的时间用于垃圾回收。此外，不像链表那样，需要为每一个添加到其上面的对象创建节点对象一对应的，当删除节点时，需要执行相应的内存清理操作。环形数组中的元素采用覆盖方式，避免了 jvm 的 GC。

其次结构作为环形，数组的大小为 2 的 n 次方，这样元素定位可以通过位运算效率会更高，这个跟一致性哈希中的环形策略有点像。在 `disruptor` 中，这个牛逼的环形结构就是 `RingBuffer`，既然是数组，那么就有大小，而且这个大小必须是 2 的 n 次方

其实质只是一个普通的数组，只是当放置数据填满队列（即到达 $2^n - 1$ 位置）之后，再填充数据，就会从 0 开始，覆盖之前的数据，于是就相当于一个环。

每个生产者首先通过 CAS 竞争获取可以写的空间，然后再进行慢慢往里放数据，如果正好这个时候消费者要消费数据，那么每个消费者都需要获取最大可消费的下标。

同时，`Disruptor` 不像传统的队列，分为一个队头指针和一个队尾指针，而是只有一个角标（上图的 `seq`），它属于一个 `volatile` 变量，同时也是我们能够不用锁操作就能实现 `Disruptor` 的原因之一，而且通过缓存行补充，避免伪共享问题。该指针是通过一直自增的方式来获取下一个可写或者可读数据。

总复习和常见并发面试题

谈面试

1. 面试主要分为两块：一块是考查工程师对基础知识（包括了技术广度、深度、对技术的热情度等）的掌握程度，因为**基础知识决定了一个技术人员发展的上限**；另一块是考察工程师的工程能力，比如：做过哪些项目？遇到最难的问题怎样解决的？说说最有成就感的一项任务？**工程能力是考察工程师当下能为公司带来的利益**。当然还有其它考核方面：抗压性、合作能力。

2. `Java` 只是一门语言，即使是 `Java` 工程师也不能局限于 `Java`，要从面向对象语言本身，甚至从整个计算机体系，从工程实际出发看 `Java`。

3. 很多知识在一般公司的开发中是用不到的，常有人戏称：“面试造火箭，工作拧螺丝”，但这只是通常情况下公司对程序员的标准——迅速产出，完成任务。所以，工程师为了自己职业的发展不能局限于公司对自己的要求，不能停留在应用层面，要能够很好地掌握基础知识，要多看源码，自己多实践，学成记得产出，比如多为开源社区贡献代码，帮助初学者指路等。

其实“面试造火箭，工作拧螺丝”的背后其实是大家都普遍认可基础知识的重要性。

常见面试题

在 java 中守护线程和用户线程的区别？

java 中的线程分为两种：守护线程（Daemon）和用户线程（User）。

任何线程都可以设置为守护线程和用户线程，通过方法 `Thread.setDaemon(bool on)`；true 则把该线程设置为守护线程，反之则为用户线程。`Thread.setDaemon()` 必须在 `Thread.start()` 之前调用，否则运行时抛出异常。

两者的区别：

唯一的区别是判断虚拟机(JVM)何时离开，Daemon 是为其他线程提供服务，如果全部的 User Thread 已经结束，Daemon 没有可服务的线程，JVM 关闭。

扩展：Thread Dump 打印出来的线程信息，含有 daemon 字样的线程即为守护进程

线程与进程的区别

进程是操作系统分配资源的最小单元，线程是操作系统调度的最小单元。

一个程序至少有一个进程，一个进程至少有一个线程。

什么是多线程中的上下文切换

多线程会共同使用一组计算机上的 CPU，而线程数大于给程序分配的 CPU 数量时，为了让各个线程都有执行的机会，就需要轮转使用 CPU。不同的线程切换使用 CPU 发生的切换数据等就是上下文切换。

死锁与活锁的区别，死锁与饥饿的区别？

死锁：是指两个或两个以上的进程（或线程）在执行过程中，因争夺资源而造成的一种互相等待的现象，若无外力作用，它们都将无法推进下去。

产生死锁的必要条件：

互斥条件：所谓互斥就是进程在某一时间内独占资源。

请求与保持条件：一个进程因请求资源而阻塞时，对已获得的资源保持不放。

不剥夺条件：进程已获得资源，在未使用完之前，不能强行剥夺。

循环等待条件：若干进程之间形成一种头尾相接的循环等待资源关系。

活锁：任务或者执行者没有被阻塞，由于某些条件没有满足，导致一直重复尝试，失败，尝试，失败。

活锁和死锁的区别在于，处于活锁的实体是在不断的改变状态，所谓的“活”，而处于死锁的实体表现为等待；活锁有可能自行解开，死锁则不能。

饥饿：一个或者多个线程因为种种原因无法获得所需要的资源，导致一直无法执行的状态。

synchronized 底层实现原理

synchronized (this) 原理：涉及两条指令：monitorenter, monitorexit；再说同步方法，从同步方法反编译的结果来看，方法的同步并没有通过指令 monitorenter 和 monitorexit 来实现，相对于普通方法，其常量池中多了 ACC_SYNCHRONIZED 标示符。

JVM 就是根据该标示符来实现方法的同步的：当方法被调用时，调用指令将会检查方法的 `ACC_SYNCHRONIZED` 访问标志是否被设置，如果设置了，执行线程将先获取 `monitor`，获取成功之后才能执行方法体，方法执行完后再释放 `monitor`。在方法执行期间，其他任何线程都无法再获得同一个 `monitor` 对象。

注意，这个问题可能会接着追问，java 对象头信息，偏向锁，轻量锁，重量级锁及其他相互间转化。

什么是线程组，为什么在 Java 中不推荐使用？

`ThreadGroup` 类，可以把线程归属到某一个线程组中，线程组中可以有线程对象，也可以有线程组，组中还可以有线程，这样的组织结构有点类似于树的形式。

1.线程组 `ThreadGroup` 对象中比较有用的方法是 `stop`、`resume`、`suspend` 等方法，由于这几个方法会导致线程的安全问题（主要是死锁问题），已经被官方废弃掉了，所以线程组本身的应用价值就大打折扣了。

2.线程组 `ThreadGroup` 不是线程安全的，这在使用过程中获取的信息并不全是及时有效的，这就降低了它的统计使用价值。

什么是 Executors 框架？为什么使用 Executor 框架？

`Executor` 框架是一个根据一组执行策略调用，调度，执行和控制的异步任务的框架。

每次执行任务创建线程 `new Thread()` 比较消耗性能，创建一个线程是比较耗时、耗资源的。

调用 `new Thread()` 创建的线程缺乏管理，而且可以无限制的创建，线程之间的相互竞争会导致过多占用系统资源而导致系统瘫痪，还有线程之间的频繁交替也会消耗很多系统资源。

接使用 `new Thread()` 启动的线程不利于扩展，比如定时执行、定期执行、定时定期执行、线程中断等都不便实现。

在 Java 中 Executor 和 Executors 的区别？

`Executors` 工具类的不同方法按照我们的需求创建了不同的线程池，来满足业务的需求。

`Executor` 接口对象能执行我们的线程任务。

`ExecutorService` 接口继承了 `Executor` 接口并进行了扩展，提供了更多的方法我们能获得任务执行的状态并且可以获取任务的返回值。

使用 `ThreadPoolExecutor` 可以创建自定义线程池。

什么是原子操作？在 Java Concurrency API 中有哪些原子类(atomic classes)？

原子操作（atomic operation）意为”不可被中断的一个或一系列操作”。

处理器使用基于对缓存加锁或总线加锁的方式来实现多处理器之间的原子操作。

在 Java 中可以通过锁和循环 CAS 的方式来实现原子操作。CAS 操作——Compare & Set, 或是 Compare & Swap, 现在几乎所有的 CPU 指令都支持 CAS 的原子操作。java.util.concurrent.atomic 下提供了大量的原子操作类, 比如原子类: AtomicBoolean, AtomicInteger, AtomicLong, AtomicReference, 原子数组: AtomicIntegerArray, AtomicLongArray, AtomicReferenceArray, 原子属性更新器: AtomicLongFieldUpdater, AtomicIntegerFieldUpdater, AtomicReferenceFieldUpdater

Java Concurrency API 中的 Lock 接口 (Lock interface) 是什么? 对比 synchronized 它有什么优势?

Lock 接口比同步方法和同步块提供了更具扩展性的锁操作。

他们允许更灵活的结构, 可以具有完全不同的性质, 并且可以支持多个相关类的条件对象。

它的优势有: 可以使锁更公平, 可以使线程在等待锁的时候响应中断, 可以让线程尝试获取锁, 并在无法获取锁的时候立即返回或者等待一段时间, 可以在不同的范围, 以不同的顺序获取和释放锁。

整体上来说 Lock 是 synchronized 的扩展版, Lock 提供了无条件的、可轮询的(tryLock 方法)、定时的(tryLock 带参方法)、可中断的(lockInterruptibly)、可多条件队列的(newCondition 方法)锁操作。另外 Lock 的实现类基本都支持非公平锁(默认)和公平锁, synchronized 只支持非公平锁, 当然, 在大部分情况下, 非公平锁是高效的选择。

什么是阻塞队列? 阻塞队列的实现原理是什么? 如何使用阻塞队列来实现生产者-消费者模型?

阻塞队列 (BlockingQueue) 是一个支持两个附加操作的队列。

这两个附加的操作是: 在队列为空时, 获取元素的线程会等待队列变为非空。当队列满时, 存储元素的线程会等待队列可用。

阻塞队列常用于生产者和消费者的场景, 生产者是往队列里添加元素的线程, 消费者是从队列里拿元素的线程。阻塞队列就是生产者存放元素的容器, 而消费者也只从容器里拿元素。

JDK7 提供了 7 个阻塞队列。在实现上, 主要是利用了 Condition 和 Lock 的等待通知模式。

什么是 Callable 和 Future?

Callable 接口类似于 Runnable, 从名字就可以看出来, 但是 Runnable 不会返回结果, 并且无法抛出返回结果的异常, 而 Callable 功能更强大一些, 被线程执行后, 可以返回值, 这个返回值可以被 Future 拿到, 也就是说, Future 可以拿到异步执行任务的返回值。

可以认为是带有回调的 Runnable。

Future 接口表示异步任务, 是还没有完成的任务给出的未来结果。所以说 Callable 用于产生结果, Future 用于获取结果。

什么是 FutureTask?

在 Java 并发程序中 FutureTask 表示一个可以取消的异步运算。它有启动和取消运算、查询运算是否完成和取回运算结果等方法。只有当运算完成的时候结

果才能取回，如果运算尚未完成 `get` 方法将会阻塞。一个 `FutureTask` 对象可以对调用了 `Callable` 和 `Runnable` 的对象进行包装，由于 `FutureTask` 也是调用了 `Runnable` 接口所以它可以提交给 `Executor` 来执行。

什么是并发容器的实现？

何为同步容器：可以简单地理解为通过 `synchronized` 来实现同步的容器，如果有多个线程调用同步容器的方法，它们将会串行执行。比如 `Vector`，`Hashtable`，以及 `Collections.synchronizedSet`，`synchronizedList` 等方法返回的容器。

并发容器使用了与同步容器完全不同的加锁策略来提供更高的并发性和伸缩性，例如在 `ConcurrentHashMap` 中采用了一种粒度更细的加锁机制，可以称为分段锁，在这种锁机制下，允许任意数量的读线程并发地访问 `map`，并且执行读操作的线程和写操作的线程也可以并发的访问 `map`，同时允许一定数量的写操作线程并发地修改 `map`，所以它可以在并发环境下实现更高的吞吐量。

多线程同步和互斥有几种实现方法，都是什么？

线程同步是指线程之间所具有的一种制约关系，一个线程的执行依赖另一个线程的消息，当它没有得到另一个线程的消息时应等待，直到消息到达时才被唤醒。

线程互斥是指对于共享的进程系统资源，在各单个线程访问时的排它性。当有若干个线程都要使用某一共享资源时，任何时刻最多只允许一个线程去使用，其它要使用该资源的线程必须等待，直到占用资源者释放该资源。线程互斥可以看成是一种特殊的线程同步。

线程间的同步方法大体可分为两类：用户模式和内核模式。顾名思义，内核模式就是指利用系统内核对象的单一性来进行同步，使用时需要切换内核态与用户态，而用户模式就是不需要切换到内核态，只在用户态完成操作。

用户模式下的方法有：原子操作（例如一个单一的全局变量），临界区。内核模式下的方法有：事件，信号量，互斥量。

什么是竞争条件？

当多个进程都企图对共享数据进行某种处理，而最后的结果又取决于进程运行的顺序时，则我们认为这发生了竞争条件（`race condition`）。

为什么我们调用 `start()` 方法时会执行 `run()` 方法，为什么我们不能直接调用 `run()` 方法？

当你调用 `start()` 方法时你将创建新的线程，并且执行在 `run()` 方法里的代码。

但是如果你直接调用 `run()` 方法，它不会创建新的线程也不会执行调用线程的代码，只会把 `run` 方法当作普通方法去执行。

在 Java 中 `CyclicBarrier` 和 `CountdownLatch` 有什么区别？

`CyclicBarrier` 可以重复使用，而 `CountdownLatch` 不能重复使用。

什么是不可变对象，它对写并发应用有什么帮助？

不可变对象(`Immutable Objects`)即对象一旦被创建它的状态（对象的数据，也即对象属性值）就不能改变，反之即为可变对象(`Mutable Objects`)。

不可变对象的类即为不可变类(`Immutable Class`)。Java 平台类库中包含许多不可变类，如 `String`、基本类型的包装类、`BigInteger` 和 `BigDecimal` 等。

不可变对象天生是线程安全的。它们的常量（域）是在构造函数中创建的。既然它们的状态无法修改，这些常量永远不会变。

不可变对象永远是线程安全的。

只有满足如下状态，一个对象才是不可变的；

它的状态不能在创建后再被修改；

所有域都是 `final` 类型；并且，

它被正确创建

notify() 和 notifyAll() 有什么区别？

当一个线程进入 `wait` 之后，就必须等其他线程 `notify/notifyall`，使用 `notifyall`，可以唤醒所有处于 `wait` 状态的线程，使其重新进入锁的争夺队列中，而 `notify` 只能唤醒一个。

如果没把握，建议 `notifyAll`，防止 `notigy` 因为信号丢失而造成程序异常。

什么是可重入锁（ReentrantLock）？谈谈它的实现。

线程可以重复进入任何一个它已经拥有的锁所同步着的代码块，`synchronized`、`ReentrantLock` 都是可重入的锁。在实现上，就是线程每次获取锁时判定如果获得锁的线程是它自己时，简单将计数器累积即可，每 释放一次锁，进行计数器累减，直到计算器归零，表示线程已经彻底释放锁。

当一个线程进入某个对象的一个 synchronized 的实例方法后，其它线程是否可进入此对象的其它方法？

如果其他方法没有 `synchronized` 的话，其他线程是可以进入的。

所以要开放一个线程安全的对象时，得保证每个方法都是线程安全的。

乐观锁和悲观锁的理解及如何实现，有哪些实现方式？

悲观锁：总是假设最坏的情况，每次去拿数据的时候都认为别人会修改，所以每次在拿数据的时候都会上锁，这样别人想拿这个数据就会阻塞直到它拿到锁。Java 里面的同步原语 `synchronized` 关键字的实现是悲观锁。

乐观锁：顾名思义，就是很乐观，每次去拿数据的时候都认为别人不会修改，所以不会上锁，但是在更新的时候会判断一下在此期间别人有没有去更新这个数据，可以使用版本号等机制。在 Java 中 `j` 原子变量类就是使用了乐观锁的一种实现方式 `CAS` 实现的。

乐观锁的实现方式：

- 使用版本标识来确定读到的数据与提交时的数据是否一致。提交后修改版本标识，不一致时可以采取丢弃和再次尝试的策略。
- java 中的 `Compare and Swap` 即 `CAS`，当多个线程尝试使用 `CAS` 同时更新同一个变量时，只有其中一个线程能更新变量的值，而其它线程都失败，失败的线程并不会被挂起，而是被告知这次竞争中失败，并可以再次尝试。

什么是 CAS 操作，缺点是什么？

`CAS` 的基本思路就是，如果这个地址上的值和期望的值相等，则给予其赋予新值，否则不做任何事儿，但是要返回原值是多少。每一个 `CAS` 操作过程都包含三

个运算符：一个内存地址 **V**，一个期望的值 **A** 和一个新值 **B**，操作的时候如果这个地址上存放的值等于这个期望的值 **A**，则将地址上的值赋为新值 **B**，否则不做任何操作。

CAS 缺点：

ABA 问题：

比如说一个线程 **one** 从内存位置 **V** 中取出 **A**，这时候另一个线程 **two** 也从内存中取出 **A**，并且 **two** 进行了一些操作变成了 **B**，然后 **two** 又将 **V** 位置的数据变成 **A**，这时候线程 **one** 进行 CAS 操作发现内存中仍然是 **A**，然后 **one** 操作成功。尽管线程 **one** 的 CAS 操作成功，但可能存在潜藏的问题。从 Java1.5 开始 JDK 的 **atomic** 包里提供了一个类 **AtomicStampedReference** 来解决 ABA 问题。

循环时间长开销大：

对于资源竞争严重（线程冲突严重）的情况，CAS 自旋的概率会比较大，从而浪费更多的 CPU 资源，效率低于 **synchronized**。

只能保证一个共享变量的原子操作：

当对一个共享变量执行操作时，我们可以使用循环 CAS 的方式来保证原子操作，但是对多个共享变量操作时，循环 CAS 就无法保证操作的原子性，这个时候就可以用锁。

SynchronizedMap 和 ConcurrentHashMap 有什么区别？

SynchronizedMap 一次锁住整张表来保证线程安全，所以每次只能有一个线程来访问 **map**。

ConcurrentHashMap 使用分段锁来保证在多线程下的性能。

写时复制容器可以用于什么应用场景？

CopyOnWrite 并发容器用于对于绝大部分访问都是读，且只是偶尔写的并发场景。比如白名单，黑名单，商品类目的访问和更新场景。

透露的思想

读写分离，读和写分开

最终一致性

使用另外开辟空间的思路，来解决并发冲突

volatile 有什么用？能否用一句话说明下 volatile 的应用场景？

volatile 保证内存可见性和禁止指令重排。

volatile 用于多线程环境下的一写多读，或者无关联的多写。

为什么代码会重排序？

在执行程序时，为了提高性能，处理器和编译器常常会对指令进行重排序，但是不能随意重排序，不是你想怎么排序就怎么排序，它需要满足以下两个条件：

在单线程环境下不能改变程序运行的结果；

存在数据依赖关系的不允许重排序

在 java 中 wait 和 sleep 方法的不同？

最大的不同是在等待时 `wait` 会释放锁，而 `sleep` 一直持有锁。`Wait` 通常被用于线程间交互，`sleep` 通常被用于暂停执行。

一个线程运行时发生异常会怎样？

如果异常没有被捕获该线程将会停止执行。`Thread.UncaughtExceptionHandler` 是用于处理未捕获异常造成线程突然中断情况的一个内嵌接口。当一个未捕获异常将造成线程中断的时候 JVM 会使用 `Thread.getUncaughtExceptionHandler()` 来查询线程的 `UncaughtExceptionHandler` 并将线程和异常作为参数传递给 handler 的 `uncaughtException()` 方法进行处理。

为什么 `wait`、`notify` 和 `notifyAll` 这些方法不在 `Thread` 类里面？

JAVA 提供的锁是对象级的而不是线程级的，每个对象都有锁，通过线程获得。如果线程需要等待某些锁那么调用对象中的 `wait()` 方法就有意义了。如果 `wait()` 方法定义在 `Thread` 类中，线程正在等待的是哪个锁就不明显了。简单的说，由于 `wait`、`notify` 和 `notifyAll` 都是锁级别的操作，所以把他们定义在 `Object` 类中因为锁属于对象。

什么是 `ThreadLocal` 变量？

`ThreadLocal` 是 Java 里一种特殊的变量。每个线程都有一个 `ThreadLocal` 就是每个线程都拥有了自己独立的一个变量，竞争条件被彻底消除了。

Java 中 `interrupted` 和 `isInterrupted` 方法的区别？

`interrupted()` 和 `isInterrupted()` 的主要区别是前者会将中断状态清除而后者不会。Java 多线程的中断机制是用内部标识来实现的，调用 `Thread.interrupt()` 来中断一个线程就会设置中断标识为 `true`。当中断线程调用静态方法 `Thread.interrupted()` 来检查中断状态时，中断状态会被清零。而非静态方法 `isInterrupted()` 用来查询其它线程的中断状态且不会改变中断状态标识。

为什么 `wait` 和 `notify` 方法要在同步块中调用？

主要是因为 Java API 强制要求这样做，如果你不这么做，你的代码会抛出 `IllegalMonitorStateException` 异常。

为什么你应该在循环中检查等待条件？

处于等待状态的线程可能会收到错误警报和伪唤醒，如果不在循环中检查等待条件，程序就会在没有满足结束条件的情况下退出。因此，当一个等待线程醒来时，不能认为它原来的等待状态仍然是有效的，在 `notify()` 方法调用之后和等待线程醒来之前这段时间它可能会改变。这就是在循环中使用 `wait()` 方法效果更好的原因

怎么检测一个线程是否拥有锁？

在 `java.lang.Thread` 中有一个方法叫 `holdsLock()`，它返回 `true` 如果当且仅当当前线程拥有某个具体对象的锁。

你如何在 Java 中获取线程堆栈？

`kill -3 [java pid]`

不会在当前终端输出，它会输出到代码执行的或指定的地方去。比如，`kill -3 tomcat pid`，输出堆栈到 `log` 目录下。

`Jstack [java pid]`

这个比较简单，在当前终端显示，也可以重定向到指定文件中。

或者使用 Java 提供的虚拟机线程系统的管理接口

ManagementFactory.getThreadMXBean()。

Java 线程池中 submit() 和 execute() 方法有什么区别？

两个方法都可以向线程池提交任务，execute()方法的返回类型是 void，它定义在 Executor 接口中。

而 submit()方法可以返回持有计算结果的 Future 对象，它定义在 ExecutorService 接口中，它扩展了 Executor 接口

你对线程优先级的理解是什么？

每一个线程都是有优先级的，一般来说，高优先级的线程在运行时会有优先权，但这依赖于线程调度的实现，这个实现是和操作系统相关的(OS dependent)。我们可以定义线程的优先级，但是这并不能保证高优先级的线程会在低优先级的线程前执行。线程优先级是一个 int 变量(从 1-10)，1 代表最低优先级，10 代表最高优先级。

java 的线程优先级调度会委托给操作系统去处理，所以与具体的操作系统优先级有关，如非特别需要，一般无需设置线程优先级。

你如何确保 main() 方法所在的线程是 Java 程序最后结束的线程？

可以使用 Thread 类的 join()方法（或者 CountdownLatch 工具类）来确保所有程序创建的线程在 main()方法退出前结束。

为什么 Thread 类的 sleep() 和 yield() 方法是静态的？

Thread 类的 sleep()和 yield()方法将在当前正在执行的线程上运行。所以在其他处于等待状态的线程上调用这些方法是没有意义的。这就是为什么这些方法是静态的。它们可以在当前正在执行的线程中工作，并避免程序员错误的认为可以在其他非运行线程调用这些方法。

现在有 T1、T2、T3 三个线程，你怎样保证 T2 在 T1 执行完后执行，T3 在 T2 执行完后执行？

可以用 join 方法实现。

你需要实现一个高效的缓存，它允许多个用户读，但只允许一个用户写，以此来保持它的完整性，你会怎样去实现它？

volatile 关键字，读写锁，写时复制等等都可以实现。

用 Java 实现阻塞队列

见作业答案：包 cn.enjoyedu.ch5.answer 下

用 Java 写代码来解决生产者——消费者问题。

阻塞队列实现即可，也可以用 wait 和 notify 来解决这个问题，或者用 Semaphore

用 Java 编程一个会导致死锁的程序，你将怎么解决？

参见代码 cn.enjoyedu.ch7. NormalDeadLock，如何解决死锁，参见笔记。

Java 中如何停止一个线程？

使用共享变量的方式

在这种方式中，之所以引入共享变量，是因为该变量可以被多个执行相同任务的线程用来作为是否中断的信号，通知中断线程的执行。

使用 `interrupt` 方法终止线程

如果一个线程由于等待某些事件的发生而被阻塞，又该怎样停止该线程呢？比如当一个线程由于需要等候键盘输入而被阻塞，或者调用 `Thread.join()` 方法，或者 `Thread.sleep()` 方法，在网络中调用 `ServerSocket.accept()` 方法，或者调用了 `DatagramSocket.receive()` 方法时，都有可能导致线程阻塞，使线程处于不可运行状态时，即使主程序中将该线程的共享变量设置为 `true`，但该线程此时根本无法检查循环标志，当然也就无法立即中断。所以应该尽量使用 `Thread` 提供的 `interrupt()` 方法，因为该方法虽然不会中断一个正在运行的线程，但是它可以使一个被阻塞的线程抛出一个中断异常，从而使线程提前结束阻塞状态。

JVM 中哪个参数是用来控制线程的栈堆栈大小的

-Xss

如果同步块内的线程抛出异常锁会释放吗？

会

单例模式的双重检查实现是什么？为什么并不安全？如何在 Java 中创建线程安全的 Singleton？

实现参见 `cn.enjoyedu.ch7.dcl.SingleDcl`，不安全的根本原因是重排序会导致未初始化完成的对象可以被其他线程看见而导致错误。创建安全的单例模式有：延迟占位模式、在声明的时候就 `new` 这个类的实例、枚举

写出 3 条你遵循的多线程最佳实践

给你的线程起个有意义的名字。这样可以方便找 bug 或追踪。

`OrderProcessor`, `QuoteProcessor` or `TradeProcessor` 这种名字比 `Thread-1`.

`Thread-2` and `Thread-3` 好多了，给线程起一个和它要完成的任务相关的名字，所有的主要框架甚至 JDK 都遵循这个最佳实践。

避免锁定和缩小同步的范围 锁花费的代价高昂且上下文切换更耗费时间空间，试试最低限度的使用同步和锁，缩小临界区。因此相对于同步方法我更喜欢同步块，它给我拥有对锁的绝对控制权。

多用同步类少用 `wait` 和 `notify` 首先，`CountDownLatch`, `Semaphore`, `CyclicBarrier` 和 `Exchanger` 这些同步类简化了编码操作，而用 `wait` 和 `notify` 很难实现对复杂控制流的控制。其次，这些类是由最好的企业编写和维护在后续的 JDK 中它们还会不断优化和完善，使用这些更高等级的同步工具你的程序可以不费吹灰之力获得优化。

多用并发集合少用同步集合 这是另外一个容易遵循且受益巨大的最佳实践，并发集合比同步集合的可扩展性更好，所以在并发编程时使用并发集合效果更好。

比如并发编程的黄金原则，尽量无锁化编程等等.....

请概述线程池的创建参数，怎么样合理配置一个线程池的参数？

参见笔记中[线程池](#)一章的内容

请概述锁的公平和非公平，JDK 内部是如何实现的。

公平锁是指所有试图获得锁的线程按照获取锁的顺序依次获得锁,而非公平锁则是当前的锁状态没有被占用时,当前线程可以直接占用,而不需要等待。在实现上,非公平锁逻辑基本跟公平锁一致,唯一的区别是,当前线程不需要判断同步队列中是否有等待线程。

非公平锁性能高于公平锁性能。首先,在恢复一个被挂起的线程与该线程真正运行之间存在着严重的延迟。而且,非公平锁能更充分的利用 `cpu` 的时间片,尽量的减少 `cpu` 空闲的状态时间。

使用场景的话呢,其实还是和他们的属性一一相关,比如:如果业务中线程占用(处理)时间要远长于线程等待,那用非公平锁其实效率并不明显,但是用公平锁可以保证不会有线程被饿死。

请概述 AQS

是用来构建锁或者其他同步组件的基础框架,比如 `ReentrantLock`、`ReentrantReadWriteLock` 和 `CountDownLatch` 就是基于 AQS 实现的。它使用了一个 `int` 成员变量表示同步状态,通过内置的 FIFO 队列来完成资源获取线程的排队工作。它是 CLH 队列锁的一种变体实现。它可以实现 2 种同步方式:独占式,共享式。

AQS 的主要使用方式是继承,子类通过继承 AQS 并实现它的抽象方法来管理同步状态,同步器的设计基于模板方法模式,所以如果我们要实现我们自己的同步工具类就需要覆盖其中几个可重写的方法,如 `tryAcquire`、`tryReleaseShared` 等等。

这样设计的目的是同步组件(比如锁)是面向使用者的,它定义了使用者与同步组件交互的接口(比如可以允许两个线程并行访问),隐藏了实现细节;同步器面向的是锁的实现者,它简化了锁的实现方式,屏蔽了同步状态管理、线程的排队、等待与唤醒等底层操作。这样就很好地隔离了使用者和实现者所需关注的领域。

在内部,AQS 维护一个共享资源 `state`,通过内置的 FIFO 来完成获取资源线程的排队工作。该队列由一个一个的 `Node` 结点组成,每个 `Node` 结点维护一个 `prev` 引用和 `next` 引用,分别指向自己的前驱和后继结点,构成一个双端双向链表。

同时与 `Condition` 相关的等待队列,节点类型也是 `Node`,构成一个单向链表。

请概述 volatile

`volatile` 关键字的作用主要有两点:

多线程主要围绕可见性和原子性两个特性而展开,使用 `volatile` 关键字修饰的变量,保证了其在多线程之间的可见性,即每次读取到 `volatile` 变量,一定是最新的数据。但是 `volatile` 不能保证操作的原子,对任意单个 `volatile` 变量的读/写具有原子性,但类似于++这种复合操作不具有原子性。。

代码底层在执行时为了获取更好的性能会对指令进行重排序,多线程下可能会出现一些意想不到的问题。使用 `volatile` 则会对禁止重排序,当然这也一定程度上降低了代码执行效率。

同时在内存语义上,当写一个 `volatile` 变量时,JMM 会把该线程对应的本地内存中的共享变量值刷新到主内存,当读一个 `volatile` 变量时,JMM 会把该线程对应的本地内存置为无效。线程接下来将从主内存中读取共享变量。

在 Java 中对于 `volatile` 修饰的变量，编译器在生成字节码时，会在指令序列中插入内存屏障来禁止特定类型的处理器重排序问题、强制刷新和读取。

在具体实现上，`volatile` 关键字修饰的变量会存在一个“`lock:`”的前缀。它不是一种内存屏障，但是它能完成类似内存屏障的功能。`Lock` 会对 CPU 总线和高速缓存加锁，可以理解为 CPU 指令级的一种锁。

同时该指令会将当前处理器缓存行的数据直接写会到系统内存中，且这个写回内存的操作会使在其他 CPU 里缓存了该地址的数据无效。

首席课程导师